

Lemma Supplement: A Lean-Verified All-Density Exclusion for CMV Four-Arc Minimizers

Proof built, Lean-verified, and semantically accepted by
Agentic Lean Math Assistant · github.com/RosaRojacr/agentic-lean-math-assistant

Technical supplement and declaration guide

The supplement records the mathematical meanings of every declaration in the frozen publication closure. The assembled ledger is ordered alphabetically by declaration name within two classification blocks: the first contains the 192 declarations classified as main, and the second contains the 272 declarations classified as supplement. Each entry separates its mathematical statement and explanation from the declaration name, source location, and Lean code supplied as metadata by the renderer.

The notation is as follows. The density is f_λ , weighted area and complete-frontier weighted perimeter are A_λ and P_λ , and $B, D, q, A_3, P_3, A_4, P_4, F_3, F_4$ have the meanings stated in the main proof. Unless an entry states a larger domain, analytic comparisons use $\lambda > 1$ and regular curvatures in $(0, 1)$; endpoint identities are stated separately when they are valid. A cap has chord $L > 0$, half-central angle $0 < \theta < \pi$, midpoint (m, b) , and radius $R = L/(2\sin\theta)$. Write $\alpha(\theta) = (\theta - \sin\theta\cos\theta)/(4\sin^2\theta)$ and $\ell(\theta) = \theta/\sin\theta$. A regular type-iii assembly uses $R = 1/h$, $q = 2h - 1$, $\beta = \arccos q$, $\alpha = \arccos(q/\lambda)$, $\gamma = \pi - \beta$, and $w = R(\sin\alpha - \sin\beta)$. All Hausdorff measures in the ledger are Euclidean; coordinate carriers and their Euclidean images are distinguished explicitly.

No generated declaration families are configured in this frozen package, and no inventoried declaration is marked generated. Accordingly the generated-family list is empty. Repetitive algebra, endpoint arithmetic, parametrization plumbing, and integrability transport remain in the declaration-level ledger; the main proof retains the conceptual geometric and analytic argument. The supplement does not add source normalization, reduced-boundary/complete-frontier compatibility, or a classification of arbitrary minimizers beyond the modeled root.

Conceptual declaration ledger

AdmissibleCompetitor

Classification: main

Source: CMVGeometry.lean:3952-3960

Declaration SHA-256: 52c3c4597213ca5cc7d4e12b21ad852ff148bcb8179b3f52a9c03b7d3665311b

Mathematical statement. An admissible competitor is empty, a general integrable measurable region, a four-arc assembly, or a replacement assembly.

The general constructor allows comparison with the constructed type-iii carrier; minimality is not restricted to scalar or four-arc alternatives.

```
inductive AdmissibleCompetitor (lam : ℝ) where
  | empty
  | general (region : FinitePerimeterRegion lam)
  | fourArc (a : FourArcAssembly)
  | replacement (a : ReplacementAssembly)

namespace AdmissibleCompetitor

/-- Coordinate carrier of an admissible regular competitor. -/
```

AdmissibleCompetitor.WeightedArea

Classification: main

Source: CMVGeometry.lean:3968-3975

Declaration SHA-256: d58b51a28690771d1b404e9048905bb6eae43f08f6798b27166b1234f3251f5e

Mathematical statement. Competitor weighted area is zero for the empty set and the carrier-derived weighted area for each nonempty constructor.

The later carrier identity unifies these cases into a single integral formula.

```
def WeightedArea {lam : ℝ} : AdmissibleCompetitor lam → ℝ
| .empty => 0
| .general region => region.weightedArea
| .fourArc a => a.weightedArea lam
| .replacement a => a.weightedArea lam

/-- Weighted perimeter is canonically determined by the complete topological
frontier of the competitor's coordinate carrier. -/
```

AdmissibleCompetitor.WeightedPerimeter

Classification: main

Source: CMVGeometry.lean:3976-3980

Declaration SHA-256: 17b5c5914d38bc0479d502c48cde9715c303aa7070585ade87ba56d307321437

Mathematical statement. Every competitor's weighted perimeter is the density integral against the frontier measure of its carrier.

Thus all competitor constructors use the same complete-frontier convention.

```
def WeightedPerimeter {lam : ℝ} (r : AdmissibleCompetitor lam) : ℝ :=
  _root_.WeightedPerimeter lam (FrontierMeasure r.carrier)

/-- The constructor-level weighted area is always the canonical density
integral over the constructor's coordinate carrier. -/
```

AdmissibleCompetitor.weightedArea_eq_carrier

Classification: main

Source: CMVGeometry.lean:3981-3990

Declaration SHA-256: bf49ad5d7ddb4e090057dafec5dba684f6e36be017290e55451b4a9c6cfabce1

Mathematical statement. For every admissible competitor, its weighted area equals A_λ of its carrier.

This identity prevents independently selectable scalar areas in the geometric minimality comparison.

```
theorem weightedArea_eq_carrier {lam : ℝ} (r : AdmissibleCompetitor lam) :
  r.WeightedArea = _root_.WeightedArea lam r.carrier := by
  cases r with
| empty =>
  simp [AdmissibleCompetitor.WeightedArea,
        AdmissibleCompetitor.carrier, _root_.WeightedArea]
| general region => rfl
| fourArc a => rfl
| replacement a => rfl
```

CMVSuffixModel.FourArcCandidate.endpoint_weightedArea_eq_typeFourArea

Classification: main

Source: CMVSuffixModel.lean:159-166

Declaration SHA-256: 1d3c440482f713868366732790a87ef12add8058ecc71af2e6fb07df04bb3b83

Mathematical statement. For a CMV candidate with $h = 1$, its weighted area equals $A_4(1)$.

The equality uses the finite geometric formula and does not evaluate or differentiate the singular fold at the endpoint.

```
theorem endpoint_weightedArea_eq_typeFourArea
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  (_hendpoint : candidate.h = 1) :
  candidate.WeightedArea =
    LeanSuffixAnalytic.typeFourArea lam candidate.h :=
  weightedArea_eq_typeFourArea_all candidate hcandidate

/-- Exact type-(iv) perimeter identification at `h_4 = 1`. -/
```

CMVSuffixModel.FourArcCandidate.endpoint_weightedPerimeter_eq_typeFourPerimeter

Classification: main

Source: CMVSuffixModel.lean:167-176

Declaration SHA-256: 9a9144c9e0869ee21b16e09b6aad1d31570ea82a7d5d876263c80c006e5e5661

Mathematical statement. For a CMV candidate with $h = 1$, its weighted perimeter equals $P_4(1)$.

This separates endpoint formula identification from the regular derivative arguments.

```
theorem endpoint_weightedPerimeter_eq_typeFourPerimeter
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  (_hendpoint : candidate.h = 1) :
  candidate.WeightedPerimeter =
    LeanSuffixAnalytic.typeFourPerimeter lam candidate.h :=
  weightedPerimeter_eq_typeFourPerimeter_all candidate hcandidate

/-- Branch-complete formula identification. The proof deliberately splits
the regular curvature from `h_4 = 1`, so later arguments cannot silently apply
a regular fold hypothesis at the endpoint. -/
```

CMVSuffixModel.FourArcCandidate.not_isWeightedPerimeterMinimizer_of_analytic_typeThree

Classification: main

Source: CMVSuffixModel.lean:201-224

Declaration SHA-256: 99c923b45cbc09e9ba012e383efbf5a498fe0a194180526cbe54c89d50a0f151

Mathematical statement. A CMV candidate is not minimizing if some $r \in (0, 1)$ satisfies $A_3(r) = A_4(h)$ and $P_3(r) < P_4(h)$.

Constructing the type-iii assembly at r turns the scalar witness into a measurable, integrable, equal-area competitor with smaller canonical perimeter.

```
theorem not_isWeightedPerimeterMinimizer_of_analytic_typeThree
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  {h_3 : ℝ} (hh_3 : 0 < h_3) (hh_3_one : h_3 < 1)
  (equalArea : LeanSuffixAnalytic.typeThreeArea lam h_3 =
    LeanSuffixAnalytic.typeFourArea lam candidate.h)
  (strictPerimeter : LeanSuffixAnalytic.typeThreePerimeter lam h_3 <
    LeanSuffixAnalytic.typeFourPerimeter lam candidate.h) :
```

```

    ¬ candidate.IsWeightedPerimeterMinimizer := by
  let a : _root_.TypeThreeAssembly lam :=
    { h := h3
      density_jump := hcandidate.density_jump
      h_pos := hh3
      h_lt_one := hh3_one }
  rcases a.hasModeledCompetitor with ⟨realization⟩
  apply candidate.not_isWeightedPerimeterMinimizer_of_typeThree a realization
  · rw [TypeThreeAssembly.scalarWeightedArea_eq_typeThreeArea a,
    weightedArea_eq_typeFourArea candidate hcandidate]
    exact equalArea
  · rw [TypeThreeAssembly.scalarWeightedPerimeter_eq_typeThreePerimeter a,
    weightedPerimeter_eq_typeFourPerimeter candidate hcandidate]
    exact strictPerimeter

/-- A single certified stationary descending pair excludes every modeled
type-(iv) candidate at the same density, including the closure endpoint. -/

```

CMVSuffixModel.FourArcCandidate.not_isWeightedPerimeterMinimizer_of_density_gt_one

Classification: main

Source: UniversalStationaryPair.lean:146-158

Declaration SHA-256: 2bcc5fee077e8fcf68df34eb9a91b0e0249285ccf8a646ec28b5a30fbd597c99

Mathematical statement. For every $\lambda > 1$, a modeled four-arc candidate satisfying the formal CMV hypotheses is not a weighted-perimeter minimizer.

The candidate has $0 < h \leq 1$, $0 < \alpha < \pi/2$, and $\lambda \cos \alpha = h$. The universal stationary pair and envelope provide an admissible regular type-iii carrier with equal weighted area and smaller complete-frontier weighted perimeter; source normalization and reduced-boundary compatibility are not conclusions.

```

theorem not_isWeightedPerimeterMinimizer_of_density_gt_one
  (hlam : 1 < lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  rcases
    LeanSuffixAnalytic.stationaryEqualAreaPair_strictImprovement_exists hlam with
    ⟨pair, hfoldThree, hperimeter, -⟩
  exact not_isWeightedPerimeterMinimizer_of_stationaryEqualAreaPair candidate
    hcandidate pair hfoldThree hperimeter

/-- Explicit contiguous modeled exclusion through the previously retained
near-one endpoint. The proof is a direct specialization of the stronger
all-density theorem. -/

```

CMVSuffixModel.FourArcCandidate.not_isWeightedPerimeterMinimizer_of_stationaryEqualAreaPair

Classification: main

Source: CMVSuffixModel.lean:225-248

Declaration SHA-256: d6670f69ae0dd1a579233b44583ff3b5ab90f13ac210ff1ddaabbb1088c5b5e2

Mathematical statement. A stationary equal-area pair with $F_3(r) < 0$ and $P_3(r) < P_4(s)$ excludes every CMV candidate at the same density.

The pair has $1/2 < r < 1$, $0 < s < 1$, $A_3(r) = A_4(s)$, and $F_4(s) = 0$. The envelope transfers its strict comparison to all candidate curvatures.

```

theorem not_isWeightedPerimeterMinimizer_of_stationaryEqualAreaPair
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  (pair : LeanSuffixAnalytic.StationaryEqualAreaPair lam)

```

```

(hfoldThree :
  LeanSuffixAnalytic.typeThreeFold lam pair.h3 < 0)
(hperimeter :
  LeanSuffixAnalytic.typeThreePerimeter lam pair.h3 <
  LeanSuffixAnalytic.typeFourPerimeter lam pair.h4) :
¬ candidate.IsWeightedPerimeterMinimizer := by
rcases pair.allCurvature_typeThreeImprovement
  hcandidate.density_jump hfoldThree hperimeter
  candidate.h_pos candidate.h_le_one with
  ⟨h3, hh3, hh3_one, hequal, hstrict⟩
exact not_isWeightedPerimeterMinimizer_of_analytic_typeThree
  candidate hcandidate hh3 hh3_one hequal hstrict

end FourArcCandidate

/-- Honest candidate-level consumer for the retained suffix. The two analytic
inputs are the explicit root-envelope existence theorem and the genuine
stationary fold-gap theorem; neither is an alias for candidate exclusion.
Given those separately proved inputs, the conclusion compares the candidate
with the canonical admissible type-(iii) region. -/

```

CMVSuffixModel.FourArcCandidate.weightedArea_eq_typeFourArea

Classification: main

Source: CMVSuffixModel.lean:177-186

Declaration SHA-256: e8473928799276d8ba026ded04a5483fef2171ee9679b0cfa48b5117289324b3

Mathematical statement. Every candidate satisfying the CMV hypotheses has weighted area $A_4(h)$ on $0 < h \leq 1$.

The proof combines the regular and endpoint identities; there is no extra area-formula hypothesis in the candidate predicate.

```

theorem weightedArea_eq_typeFourArea
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  candidate.WeightedArea =
    LeanSuffixAnalytic.typeFourArea lam candidate.h := by
rcases lt_or_eq_of_le candidate.h_le_one with hregular | hendpoint
· exact regular_weightedArea_eq_typeFourArea candidate hcandidate hregular
· exact endpoint_weightedArea_eq_typeFourArea candidate hcandidate hendpoint

/-- Branch-complete perimeter identification, with the same explicit regular
versus endpoint split as the area theorem. -/

```

CMVSuffixModel.FourArcCandidate.weightedPerimeter_eq_typeFourPerimeter

Classification: main

Source: CMVSuffixModel.lean:187-200

Declaration SHA-256: c47649486a6e39478b2697659276a1423a248517e94449da31be1eb9b63be5ce

Mathematical statement. Every candidate satisfying the CMV hypotheses has weighted perimeter $P_4(h)$ on $0 < h \leq 1$.

The identity is with complete-frontier weighted perimeter and includes the finite closure endpoint.

```

theorem weightedPerimeter_eq_typeFourPerimeter
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  candidate.WeightedPerimeter =
    LeanSuffixAnalytic.typeFourPerimeter lam candidate.h := by
rcases lt_or_eq_of_le candidate.h_le_one with hregular | hendpoint
· exact regular_weightedPerimeter_eq_typeFourPerimeter candidate
  hcandidate hregular
· exact endpoint_weightedPerimeter_eq_typeFourPerimeter candidate
  hcandidate hendpoint

/-- A scalar type-(iii) equal-area and strict-perimeter witness becomes a

```

```
counterexample to minimality through the canonical finite-perimeter
realization of its coordinate carrier. The scalar `h3` is required to lie in
the honest regular source domain. -/
```

CMVSuffixModel.TypeThreeAssembly.scalarWeightedArea_eq_typeThreeArea

Classification: main

Source: CMVSuffixModel.lean:78-86

Declaration SHA-256: 09eff0e6af5f2cf4916a82b585443109aad4f3e8bb020859b5a5f6526f9c2f0d

Mathematical statement. The scalar area of every regular type-iii assembly is $A_3(h)$.

Substitution of the angle-term and sine-gap identities identifies the independently derived geometric and analytic area formulas throughout $0 < h < 1$.

```
theorem scalarWeightedArea_eq_typeThreeArea :
  a.scalarWeightedArea = LeanSuffixAnalytic.typeThreeArea lam a.h := by
  rw [_root_.TypeThreeAssembly.scalarWeightedArea,
    LeanSuffixAnalytic.typeThreeArea,
    angleTerm_eq_typeThreeAngle a, sineGap_eq_typeThreeDelta a]
  rfl

/-- Exact identification of CMV equation (26)'s perimeter with the analytic
quantity used by the suffix theorem, on the full regular type-(iii) domain. -/
```

CMVSuffixModel.TypeThreeAssembly.scalarWeightedPerimeter_eq_typeThreePerimeter

Classification: main

Source: CMVSuffixModel.lean:87-99

Declaration SHA-256: 1cbaa2764b23e5bf11163fb498e9653b454e16f1f8f6b84d752362e7a842718b

Mathematical statement. The scalar perimeter of every regular type-iii assembly is $P_3(h)$.

The identification holds on the entire regular curvature interval, not only on one exterior-cap branch.

```
theorem scalarWeightedPerimeter_eq_typeThreePerimeter :
  a.scalarWeightedPerimeter =
    LeanSuffixAnalytic.typeThreePerimeter lam a.h := by
  rw [_root_.TypeThreeAssembly.scalarWeightedPerimeter,
    LeanSuffixAnalytic.typeThreePerimeter,
    angleTerm_eq_typeThreeAngle a, sineGap_eq_typeThreeDelta a]

end TypeThreeAssembly

namespace FourArcCandidate

variable {lam : ℝ} (candidate : _root_.FourArcCandidate lam)
```

FinitePerimeterRegion

Classification: main

Source: CMVGeometry.lean:74-82

Declaration SHA-256: 2efb7176d17b946809f4cd07b78237ddd03cd4197c405c38d08e97342106a1a0

Mathematical statement. A general modeled region consists of a measurable carrier with integrable density over its area and against its frontier measure.

These are precisely the two finiteness obligations stored with the carrier. The name does not assert a separate bounded-variation or reduced-boundary definition.

```

structure FinitePerimeterRegion (lam : ℝ) where
  carrier : Set PlanePoint
  measurable_carrier : MeasurableSet carrier
  finite_weighted_area : IntegrableOn (StripDensity lam) carrier
  finite_weighted_perimeter :
    Integrable (StripDensity lam) (FrontierMeasure carrier)

namespace FinitePerimeterRegion

```

FourArcAssembly

Classification: main

Source: CMVGeometry.lean:2491-2498

Declaration SHA-256: 1ac9185bddaf1e6a0074fb39d1e29417262cc431ccce861d3c65ec2e5216fdb1

Mathematical statement. A four-arc assembly consists of a strip core and an exterior half-angle $0 < \alpha < \pi/2$.

The two minor exterior caps share the core chord. The assembly alone does not assume Snell incidence or equality of exterior and core radii.

```

structure FourArcAssembly where
  core : StripCore
  outerAngle : ℝ
  outerAngle_pos : 0 < outerAngle
  outerAngle_lt_pi_div_two : outerAngle < π / 2

namespace FourArcAssembly

```

FourArcAssembly.boundaryTrace

Classification: main

Source: CMVGeometry.lean:2625-2630

Declaration SHA-256: d8944f364e38eaa5c7b77ce182d417bec91ac425efc229739d886be00ad658c6

Mathematical statement. The four-arc assembly boundary trace is the union of the left and right core arc traces and the upper and lower cap arc traces.

No full interface chord is included: those chord interiors are covered on both sides in the assembled carrier.

```

def boundaryTrace (a : FourArcAssembly) : Set PlanePoint :=
  StripCore.leftArcTrace a.core ∪
  (StripCore.rightArcTrace a.core ∪
   (OneSidedCircularCap.arcTrace a.upperCap ∪
    OneSidedCircularCap.arcTrace a.lowerCap))

```

FourArcAssembly.carrier

Classification: main

Source: CMVGeometry.lean:2519-2524

Declaration SHA-256: a55616796db22264c4fe11d48e20ab173ecb661c3bdced26752617ea90d1f816

Mathematical statement. The four-arc assembly carrier is the union of the strip core and both exterior caps.

This union removes the interface chord interiors from the exposed boundary.

```

def carrier (a : FourArcAssembly) : Set PlanePoint :=
  a.core.carrier ∪ a.upperCap.carrier ∪ a.lowerCap.carrier

/-- Explicit arc-length pushforward measure of the complete regular boundary.

```

The attachment chords are omitted because they are internal; endpoint atoms are null for every summand. -/

FourArcAssembly.frontier_carrier

Classification: main

Source: CMVGeometry.lean:3252-3259

Declaration SHA-256: 7da383d241cb3cc30fc9da784ef7200d9bb87d82e25006745acf6f7f7d86a78f

Mathematical statement. The complete frontier of a four-arc assembly equals its boundary trace.

Together with Euclidean length and overlap-null calculations, this justifies the geometric perimeter formula.

```
theorem frontier_carrier (a : FourArcAssembly) :
  frontier a.carrier = a.boundaryTrace :=
  Set.Subset.antisymm a.frontier_carrier_subset_boundaryTrace
    a.boundaryTrace_subset_frontier_carrier

/-- All four boundary arcs have the same radius when the exterior chord has the
source incidence length `2 * sin alpha / h`. -/
```

FourArcAssembly.weightedArea_formula

Classification: main

Source: CMVGeometry.lean:2554-2602

Declaration SHA-256: 839e3c1d7a83b5801ac118ef77feb57b415f9e5a71dfa154b8987ba3b91118a0

Mathematical statement. For every real λ , assembly area equals core area plus λ times the sum of the two cap areas.

The union intersections have planar measure zero, and the density is one in the core and λ almost everywhere in the caps.

```
theorem weightedArea_formula (lam : ℝ) (a : FourArcAssembly) :
  a.weightedArea lam =
    a.core.euclideanArea + lam *
      (a.upperCap.euclideanArea + a.lowerCap.euclideanArea) := by
  have hcoreUpper : AEDisjoint volume a.core.carrier a.upperCap.carrier := by
    refine measure_mono_null ?_ (volume_horizontalline 1)
    rintro p ⟨hcore, hupper⟩
    have hyCore := a.core.carrier_y_bounds hcore
    have hyUpper : (1 : ℝ) ≤ p.2 := by
      simpa [upperCap] using hupper.2
    exact le_antisymm (le_trans (le_abs_self p.2) hyCore) hyUpper
  have hfirstLower : AEDisjoint volume
    (a.core.carrier ∪ a.upperCap.carrier) a.lowerCap.carrier := by
    refine measure_mono_null ?_ (volume_horizontalline (-1))
    rintro p ⟨hfirst, hlower⟩
    rcases hfirst with hcore | hupper
    · have hyCore := a.core.carrier_y_bounds hcore
      have hyLower : p.2 ≤ (-1 : ℝ) := by
        simpa [lowerCap] using hlower.2
      have hneg : -p.2 ≤ 1 := le_trans (neg_le_abs p.2) hyCore
      exact le_antisymm hyLower (by linarith)
    · have hyUpper : (1 : ℝ) ≤ p.2 := by
        simpa [upperCap] using hupper.2
      have hyLower : p.2 ≤ (-1 : ℝ) := by
        simpa [lowerCap] using hlower.2
      linarith
  have hcore := core_integrableOn lam a.core
  have hupper := cap_integrableOn lam a.upperCap
  have hlower := cap_integrableOn lam a.lowerCap
  rw [weightedArea, WeightedArea, carrier,
    setIntegral_union, hfirstLower
    a.lowerCap.measurableSet_carrier.nullMeasurableSet
    (hcore.union hupper) hlower,
```

```

setIntegral_union, hcoreUpper
a.upperCap.measurableSet_carrier.nullMeasurableSet hcore hupper]
rw [show (f p in a.core.carrier, StripDensity lam p) =
a.core.euclideanArea by
simpa [WeightedArea] using a.core.weightedArea_formula lam,
show (f p in a.upperCap.carrier, StripDensity lam p) =
lam * a.upperCap.euclideanArea by
simpa [WeightedArea] using
cap_upper_weightedArea lam a.upperCap rfl rfl,
show (f p in a.lowerCap.carrier, StripDensity lam p) =
lam * a.lowerCap.euclideanArea by
simpa [WeightedArea] using
cap_lower_weightedArea lam a.lowerCap rfl rfl]
ring

/-- The boundary integral equals the four-arc component formula. -/

```

FourArcAssembly.weightedPerimeter_formula

Classification: main

Source: CMVGeometry.lean:2603-2624

Declaration SHA-256: 43ff4cdbd6774032399c0e7d7f5767a18500986a522e0ace1e470e2035290acd

Mathematical statement. For every real λ , assembly component perimeter equals core curved-side length plus λ times both cap arc lengths.

The statement evaluates the explicit four-measure sum before its canonical frontier identification.

```

theorem weightedPerimeter_formula (lam : ℝ) (a : FourArcAssembly) :
a.weightedPerimeter lam =
a.core.boundaryArcLength + lam *
(a.upperCap.arcLength + a.lowerCap.arcLength) := by
rw [weightedPerimeter, WeightedPerimeter, boundaryMeasure]
rw [integral_add_measure
(((leftCoreBoundaryMeasure_integrable lam a.core).add_measure
(rightCoreBoundaryMeasure_integrable lam a.core)).add_measure
(capBoundaryMeasure_integrable lam a.upperCap))
(capBoundaryMeasure_integrable lam a.lowerCap)]
rw [integral_add_measure
((leftCoreBoundaryMeasure_integrable lam a.core).add_measure
(rightCoreBoundaryMeasure_integrable lam a.core))
(capBoundaryMeasure_integrable lam a.upperCap)]
rw [integral_add_measure
(leftCoreBoundaryMeasure_integrable lam a.core)
(rightCoreBoundaryMeasure_integrable lam a.core)]
rw [leftCore_weightedIntegral, rightCore_weightedIntegral,
cap_upper_weightedIntegral lam a.upperCap rfl rfl,
cap_lower_weightedIntegral lam a.lowerCap rfl rfl,
StripCore.boundaryArcLength]
ring

```

FourArcCandidate

Classification: main

Source: FourArcCandidate.lean:23-35

Declaration SHA-256: 24dc0ff22e0918bdb2c7f16fab0da52bc08326d88e5cb8e1d2e8a8f7ec2bee62

Mathematical statement. A four-arc candidate stores h, α with $0 < h \leq 1$ and $0 < \alpha < \pi/2$.

The density parameter indexes the family, but incidence is not part of this constructor. The region and its area and perimeter are derived from these geometric data.

```

structure FourArcCandidate (lam : ℝ) where
h : ℝ

```

```

alpha : ℝ
h_pos : 0 < h
h_le_one : h ≤ 1
alpha_pos : 0 < alpha
alpha_lt_pi_div_two : alpha < π / 2

namespace FourArcCandidate

variable {lam : ℝ} (candidate : FourArcCandidate lam)

/-- Radius shared by the four source arcs. -/

```

FourArcCandidate.IsWeightedPerimeterMinimizer

Classification: main

Source: FourArcCandidate.lean:90-94

Declaration SHA-256: f04b5c158c1a510cd5a8af38df8f99424aead5e376f986e1d437f95e6a4cbe17

Mathematical statement. A candidate is a modeled minimizer when every admissible equal-area competitor has perimeter at least that of the candidate.

Formally the comparison quantifies over the whole admissible class. One strictly improving equal-area carrier refutes it.

```

def IsWeightedPerimeterMinimizer : Prop :=
  ∀ competitor : AdmissibleCompetitor lam,
    competitor.WeightedArea = candidate.WeightedArea →
      candidate.WeightedPerimeter ≤ competitor.WeightedPerimeter

```

FourArcCandidate.SatisfiesCMVTypeIVHypotheses

Classification: main

Source: FourArcCandidate.lean:84-89

Declaration SHA-256: c09b9c7180b6131040333c0b3c2623f3c89af7f54fe8ab0b9f6a41be873ca0dc

Mathematical statement. The CMV hypothesis predicate is exactly $\lambda > 1$ and $\lambda \cos \alpha = h$.

It adds the density jump and Snell incidence to the constructor's geometric bounds; it assumes no area identity, competitor, or nonminimality.

```

structure SatisfiesCMVTypeIVHypotheses : Prop where
  density_jump : 1 < lam
  snell_incidence : lam * cos candidate.alpha = candidate.h

/-- General minimization semantics: comparison is against every admissible
regular competitor, not merely another four-arc candidate. -/

```

FourArcCandidate.WeightedArea

Classification: main

Source: FourArcCandidate.lean:76-78

Declaration SHA-256: 8256dd516e32d853aaed6a85d09fdb0b62411bd0bf67f1df7fffd123e47baed99

Mathematical statement. The candidate weighted area is the weighted area of its admissible region.

It is computed from the carrier, not supplied as a free scalar field.

```

def WeightedArea : ℝ := candidate.region.WeightedArea

/-- Candidate weighted perimeter, computed from its four boundary arcs. -/

```

FourArcCandidate.WeightedPerimeter

Classification: main

Source: FourArcCandidate.lean:79-83

Declaration SHA-256: d649676f2fd717389a6e9a17a385fe854bc399e17ecfdad24bd8a7bcc9a36521

Mathematical statement. The candidate weighted perimeter is the canonical weighted perimeter of its admissible region.

The definition uses the complete frontier through the common competitor semantics.

```
def WeightedPerimeter : ℝ := candidate.region.WeightedPerimeter

/-- Source-specific hypotheses not already enforced by the geometric
constructor. The equation is CMV (27), equivalently
`alpha = arccos (h / lam)` on the recorded angle domain. -/
```

FourArcCandidate.alpha_eq_arccos

Classification: main

Source: FourArcCandidate.lean:276-292

Declaration SHA-256: 46219b8ad0fc6b679968f8972242a6ad511174ce969ce84539a4054206de6595

Mathematical statement. For a candidate satisfying CMV incidence, $\alpha = \arccos(h/\lambda)$.

The positive density and stored angle bounds select the principal inverse cosine, so incidence determines the intended exterior angle uniquely.

```
theorem alpha_eq_arccos (h : candidate.SatisfiesCMVTypeIVHypotheses) :
  candidate.alpha = arccos (candidate.h / lam) := by
  have hlam_pos : 0 < lam := lt_trans zero_lt_one h.density_jump
  have hratio_nonneg : 0 ≤ candidate.h / lam :=
    (div_pos candidate.h_pos hlam_pos).le
  have hratio_le_one : candidate.h / lam ≤ 1 := by
    apply (div_le_one hlam_pos).2
    exact le_trans candidate.h_le_one h.density_jump.le
  symm
  apply Real.arccos_eq_of_eq_cos candidate.alpha_pos.le
    (le_trans candidate.alpha_lt_pi_div_two.le
      (by linarith [Real.pi_pos]))
  exact (div_eq_iff (ne_of_gt hlam_pos)).2 (by
    simp [mul_comm] using h.snell_incidence.symm)

/-- The literal closed-curvature candidate at `h = 1` and principal exterior
angle `arccos (1 / lam)`. -/
```

FourArcCandidate.assembly

Classification: main

Source: FourArcCandidate.lean:66-72

Declaration SHA-256: ca67fddb2d167afab63544438387902bb645349ebfde9d83c7a2719a2277c941

Mathematical statement. The candidate assembly combines its strip core with exterior half-angle α .

The shared chord fixes two reflected caps; the constructed four arcs have the common radius $1/h$.

```
def assembly : FourArcAssembly where
  core := candidate.stripCore
  outerAngle := candidate.alpha
  outerAngle_pos := candidate.alpha_pos
  outerAngle_lt_pi_div_two := candidate.alpha_lt_pi_div_two
```

```
-- The candidate as an admissible regular coordinate region. -/
```

FourArcCandidate.candidate_area_formula

Classification: main

Source: FourArcCandidate.lean:410-430

Declaration SHA-256: b6d944ecc5d4bab77cadf263520f194142a9d6855088a9b98e5873be144ab8cc

Mathematical statement. For a CMV candidate, its area is $4\sin\alpha/h + 2(\lambda\alpha - \sin\alpha\cos\beta)/h^2 + 2(\operatorname{arcsinh} h - h\sqrt{1-h^2})/h^2$.

The terms come from the central rectangle, the two exterior caps after Snell substitution, and the two side segments. Here $\beta = \pi/2 - \operatorname{arcsinh} h$.

```
theorem candidate_area_formula
  (h : candidate.SatisfiesCMVTypeIVHypotheses) :
  candidate.WeightedArea =
    4 * sin candidate.alpha / candidate.h +
    2 * (lam * candidate.alpha -
      sin candidate.alpha * cos candidate.beta) / candidate.h ^ 2 +
    2 * (arcsin candidate.h -
      candidate.h * sqrt(1 - candidate.h ^ 2)) / candidate.h ^ 2 := by
  rw [candidate.weightedArea_components, StripCore.euclideanArea,
    candidate.side_caps_area_formula]
  simp only [stripCore, capChord]
  have hcosBeta : cos candidate.beta = candidate.h := by
    rw [beta, Real.cos_pi_div_two_sub]
    exact Real.sin_arcsin (by linarith [candidate.h_pos]) candidate.h_le_one
  rw [hcosBeta, capArea]
  field_simp [ne_of_gt candidate.h_pos]
  have hsnell_mul := congrArg (fun z : ℝ => sin candidate.alpha * z)
    h.snell_incidence
  nlinarith [hsnell_mul]

end FourArcCandidate
```

FourArcCandidate.candidate_perimeter_formula

Classification: main

Source: FourArcCandidate.lean:391-409

Declaration SHA-256: b4949bdc7019f429151f2faf734db8466461943a33be006f4c104caebb493fe0

Mathematical statement. For a CMV candidate, $P_\lambda(E) = 4\lambda\alpha/h + 4\operatorname{arcsinh} h/h$.

The two exterior arcs contribute $4\lambda\alpha/h$, and the two strip-side arcs contribute $4\operatorname{arcsinh} h/h$.

```
theorem candidate_perimeter_formula
  (_h : candidate.SatisfiesCMVTypeIVHypotheses) :
  candidate.WeightedPerimeter =
    4 * lam * candidate.alpha / candidate.h +
    4 * arcsin candidate.h / candidate.h := by
  rw [candidate.weightedPerimeter_components,
    StripCore.boundaryArcLength]
  change 4 * (arcsin candidate.h / sin (arcsin candidate.h)) +
    2 * lam * (2 * sin candidate.alpha / candidate.h) *
    (candidate.alpha / sin candidate.alpha) =
    4 * lam * candidate.alpha / candidate.h +
    4 * arcsin candidate.h / candidate.h
  rw [Real.sin_arcsin (by linarith [candidate.h_pos])
    candidate.h_le_one]
  field_simp [ne_of_gt candidate.h_pos, ne_of_gt candidate.sin_alpha_pos]
  ring
```

```

/-- CMV's area display following equation (27), derived from the rectangle and
three cap pieces. -/

```

FourArcCandidate.capArea

Classification: main

Source: FourArcCandidate.lean:45-49

Declaration SHA-256: eb6c25ab8d5e1e4d2ce589b98fc3af2d04e58bd682ad273d1b07834426ee23d0

Mathematical statement. The Euclidean area of one original candidate cap is $(\alpha - \sin\alpha\cos\alpha)/h^2$.

It is the circular-segment area with radius $1/h$; it is unweighted until multiplied by the exterior density.

```

def capArea : ℝ :=
  (candidate.alpha - sin candidate.alpha * cos candidate.alpha) /
    candidate.h ^ 2

/-- Scale-free cap area passed to Morgan's arc function. -/

```

FourArcCandidate.capChord

Classification: main

Source: FourArcCandidate.lean:42-44

Declaration SHA-256: c09aa141e8bbabbe7279bce2473dd47aa931f875442fe1fc79d759685536fb1e

Mathematical statement. The exterior cap chord of a candidate is $L = 2\sin\alpha/h$.

This forces the cap radius $L/(2\sin\alpha)$ to equal the core radius $1/h$.

```

def capChord : ℝ := 2 * sin candidate.alpha / candidate.h

/-- Euclidean area of either original exterior cap. -/

```

FourArcCandidate.not_isWeightedPerimeterMinimizer_of_typeThree

Classification: main

Source: TypeThreeAssembly.lean:3101-3118

Declaration SHA-256: 6807359794d5b2629c85ed715f93a0b06f1e0f5dc3b504c99a54401226562f55

Mathematical statement. An equal-area regular type-iii assembly with smaller scalar perimeter refutes minimality of the four-arc candidate.

The theorem instantiates the candidate's universal admissible-competitor quantifier with the realized type-iii carrier.

```

theorem not_isWeightedPerimeterMinimizer_of_typeThree
  (a : TypeThreeAssembly lam)
  (realization : TypeThreeAssembly.ModeledCompetitorRealization a)
  (equalArea :
    a.scalarWeightedArea = candidate.WeightedArea)
  (strictPerimeter :
    a.scalarWeightedPerimeter < candidate.WeightedPerimeter) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  intro hmin
  have harea :
    realization.asCompetitor.WeightedArea =
      candidate.WeightedArea := by
    rw [realization.asCompetitor_weightedArea, equalArea]
  have hle := hmin realization.asCompetitor harea
  rw [realization.asCompetitor_weightedPerimeter] at hle
  exact (not_le_of_gt strictPerimeter) hle

```

```
end FourArcCandidate
```

FourArcCandidate.region

Classification: main

Source: FourArcCandidate.lean:73-75

Declaration SHA-256: 8ab9f84c197a1a6dbe90236469e9ff5cc2696f04e42133db44cdf34546558b75

Mathematical statement. The candidate region is its four-arc assembly regarded as an admissible competitor.

This embeds the geometric candidate into the same carrier-based comparison class used for alternatives.

```
def region : AdmissibleCompetitor lam := .fourArc candidate.assembly

/-- Candidate weighted area, computed from its region constructor. -/
```

FourArcCandidate.stripCore

Classification: main

Source: FourArcCandidate.lean:53-65

Declaration SHA-256: 66aff9e1eb934bbbeeb8dd09a1a68a4c23aef3ee09735d3ba9bc12eadb4664ee

Mathematical statement. The candidate strip core uses chord $2\sin\alpha/h$ and curvature h .

The candidate angle and curvature bounds establish positive chord and admissible core curvature without adding any comparison premise.

```
def stripCore : StripCore where
  chord := candidate.capChord
  curvature := candidate.h
  chord_pos := by
    exact div_pos (mul_pos (by norm_num)
      (sin_pos_of_pos_of_lt_pi candidate.alpha_pos
        (lt_trans candidate.alpha_lt_pi_div_two
          (by linarith [Real.pi_pos])))) candidate.h_pos
  curvature_pos := candidate.h_pos
  curvature_le_one := candidate.h_le_one

/-- Candidate assembled from the retained strip core and reflected upper/lower
caps. -/
```

FourArcCandidate.weightedArea_components

Classification: main

Source: FourArcCandidate.lean:364-373

Declaration SHA-256: 654554fba4e2f4847eae4f853550061afa29cf158da02de52109cffffacefd702e

Mathematical statement. Candidate weighted area equals core Euclidean area plus 2λ times one cap's Euclidean area.

The two exterior caps have equal area and their interface intersections contribute no planar area.

```
theorem weightedArea_components :
  candidate.WeightedArea = candidate.stripCore.euclideanArea +
    2 * lam * candidate.capArea := by
  change candidate.assembly.weightedArea lam =
    candidate.stripCore.euclideanArea + 2 * lam * candidate.capArea
  rw [FourArcAssembly.weightedArea_formula, assembly_core,
    candidate.upperCap_euclideanArea, candidate.lowerCap_euclideanArea]
  ring
```

```
/-- Candidate perimeter as the sum of its four actual arc constructors. -/
```

FourArcCandidate.weightedPerimeter_components

Classification: main

Source: FourArcCandidate.lean:374-390

Declaration SHA-256: d02953cec6a51d470b1b940fa2318bbd5c86354d0e420d179322c86356c9434a

Mathematical statement. Candidate weighted perimeter equals core curved-side length plus $2\lambda L\ell(\alpha)$.

The frontier theorem identifies the four boundary components with their canonical weighted integral; the top and bottom chords are internal.

```
theorem weightedPerimeter_components :
  candidate.WeightedPerimeter = candidate.stripCore.boundaryArcLength +
    2 * lam * candidate.capChord * ell candidate.alpha := by
rw [FourArcCandidate.WeightedPerimeter, region,
  AdmissibleCompetitor.WeightedPerimeter]
change _root_.WeightedPerimeter lam
  (FrontierMeasure candidate.assembly.carrier) =
  candidate.stripCore.boundaryArcLength +
    2 * lam * candidate.capChord * ell candidate.alpha
rw [← fourArc_frontier_weightedPerimeter_eq,
  FourArcAssembly.weightedPerimeter_formula]
simp only [StripCore.boundaryArcLength,
  OneSidedCircularCap.arcLength, FourArcAssembly.upperCap,
  FourArcAssembly.lowerCap, assembly, stripCore]
ring

/-- CMV equation (27), derived from the boundary pieces. -/
```

FrontierMeasure

Classification: main

Source: CMVGeometry.lean:61-66

Declaration SHA-256: 7323bb80c0a565daf410d27d199e2143ee33a0ecca0460c52504f2e7e50b48e2

Mathematical statement. The frontier measure of E is the pullback under J of $\mathcal{H}^1$ restricted to $\partial J(E)$.

Explicitly, the measure is $(J^{-1})_*(\mathcal{H}^1|_{\partial J(E)})$. The complete Euclidean frontier, not the reduced boundary, is used.

```
def FrontierMeasure (carrier : Set PlanePoint) : Measure PlanePoint :=
  Measure.map planeEuclideanHomeomorph.symm
    ((μH[1] : Measure EuclideanPlane).restrict
      (frontier (planeEuclideanHomeomorph '' carrier)))

/-- Weighted perimeter is the density integral against a boundary measure. -/
```

LeanSuffixAnalytic.DescendingEqualAreaEnvelope

Classification: main

Source: EqualAreaEnvelope.lean:279-300

Declaration SHA-256: ee2db888cf8e5e37251e350d2425fb86808e5c1f7d808208572e20c0e3ea40a9

Mathematical statement. A descending envelope consists of $\lambda > 1$, a stationary $s \in (0, 1)$, and regular roots $\rho(h)$ on $0 < h \leq 1$ with equal area and $F_3(\rho(h)) < 0$.

Precisely, $F_4(s) = 0$, $0 < \rho(h) < 1$, and $A_3(\rho(h)) = A_4(h)$. Continuity, strict root order, and differentiability are consequences, not fields assumed in this structure.

```

structure DescendingEqualAreaEnvelope (lam : ℝ) where
  /-- The unique type-(iv) fold selected by the retained fold-gap statement. -/
  fold : ℝ
  /-- The descending type-(iii) equal-area root for every modeled curvature. -/
  root : ℝ → ℝ
  density_jump : 1 < lam
  fold_pos : 0 < fold
  fold_lt_one : fold < 1
  fold_stationary : typeFourFold lam fold = 0
  root_pos : ∀ {h₄ : ℝ}, 0 < h₄ → h₄ ≤ 1 → 0 < root h₄
  root_lt_one : ∀ {h₄ : ℝ}, 0 < h₄ → h₄ ≤ 1 → root h₄ < 1
  equalArea : ∀ {h₄ : ℝ}, 0 < h₄ → h₄ ≤ 1 →
    typeThreeArea lam (root h₄) = typeFourArea lam h₄
  /-- Every selected type-(iii) root lies on the strictly descending branch.
  This identifies it globally as the least equal-area root and implies the
  simple-root condition used by the inverse-function argument. -/
  root_fold_neg : ∀ {h₄ : ℝ}, 0 < h₄ → h₄ ≤ 1 →
    typeThreeFold lam (root h₄) < 0

  /-- Every selected descending root is bounded above by the root at the
  stationary type-(iv) fold. This fixed regular anchor supplies the compactness
  needed for branch continuity. -/

```

LeanSuffixAnalytic.FoldGapNegative

Classification: main

Source: EqualAreaEnvelope.lean:577-581

Declaration SHA-256: e0a51358d0910c8fb2dd4ffa9d95125b8210328ea93cd0771ca9e9c1138b2ced

Mathematical statement. Fold-gap negativity for an envelope means $P_3(\rho(s)) - P_4(s) < 0$.

This is a strict comparison only at the stationary source curvature; it contains no all-curvature conclusion.

```

def FoldGapNegative {lam : ℝ} (env : DescendingEqualAreaEnvelope lam) : Prop :=
  equalAreaPerimeterGap lam env.root env.fold < 0

  /-- L12, including the closure endpoint: the stationary fold is the strict
  maximum of the genuine perimeter gap along the selected equal-area branch. -/

```

LeanSuffixAnalytic.StationaryEqualAreaPair

Classification: main

Source: LeanSuffixAnalytic.lean:1177-1188

Declaration SHA-256: 865710967f267b9ec6707c5d925cd799ff4862bc66ec64cf0a22afce00862eef

Mathematical statement. A stationary equal-area pair stores $1/2 < r < 1$, $0 < s < 1$, $A_3(r) = A_4(s)$, and $F_4(s) = 0$.

The structure does not by itself assume $\lambda > 1$, root order, negative type-iii fold, or a perimeter improvement; those are supplied or proved separately.

```

structure StationaryEqualAreaPair (lam : ℝ) where
  h₃ : ℝ
  h₄ : ℝ
  h₃_gt_half : 1 / 2 < h₃
  h₃_lt_one : h₃ < 1
  h₄_pos : 0 < h₄
  h₄_lt_one : h₄ < 1
  equalArea : typeThreeArea lam h₃ = typeFourArea lam h₄
  stationary : typeFourFold lam h₄ = 0

  /-- Concrete universal counter-obligation left by the direct reduction. It
  contains the exact cutoff, root equations, branch domains, and radical gap. -/

```

LeanSuffixAnalytic.allCurvature_typeThreeImprovement_of_envelope

Classification: main

Source: EqualAreaEnvelope.lean:645-658

Declaration SHA-256: c4f7d83c3e3a4e728eb9be863cb4e20b5d04c133005f9a44bb6548808441f1ae

Mathematical statement. A descending envelope with negative fold gap supplies, for every $0 < h \leq 1$, a regular r with $A_3(r) = A_4(h)$ and $P_3(r) < P_4(h)$.

Take $r = \rho(h)$; the global maximum bound for the gap transfers the stationary strict inequality to the entire modeled range.

```
theorem allCurvature_typeThreeImprovement_of_envelope {lam h4 : ℝ}
  (env : DescendingEqualAreaEnvelope lam) (hfoldGap : FoldGapNegative env)
  (hh4 : 0 < h4) (hh4_one : h4 ≤ 1) :
  ∃ h3 : ℝ, 0 < h3 ∧ h3 < 1 ∧
    typeThreeArea lam h3 = typeFourArea lam h4 ∧
    typeThreePerimeter lam h3 < typeFourPerimeter lam h4 := by
  refine ⟨env.root h4, env.root_pos hh4 hh4_one,
    env.root_lt_one hh4 hh4_one, env.equalArea hh4 hh4_one, ?_⟩
  rw [← sub_lt_zero]
  exact lt_of_le_of_lt (equalAreaPerimeterGap_le_fold env hh4 hh4_one) hfoldGap

/-- One certified stationary pair with negative type-(iii) fold and strict
perimeter improvement supplies an improving equal-area type-(iii) scalar at
every regular or endpoint type-(iv) curvature. -/
```

LeanSuffixAnalytic.equalAreaPerimeterGap

Classification: main

Source: EqualAreaEnvelope.lean:34-39

Declaration SHA-256: e8643a1549a791c31f1d0a91f7cb9b302d6832ce9c0d1d8e354d9c3e22351fcb

Mathematical statement. For a selected root function ρ , the equal-area perimeter gap is $\Delta(h) = P_3(\rho(h)) - P_4(h)$.

The definition itself does not assume equal area; that constraint is part of the hypotheses of the comparison theorems.

```
def equalAreaPerimeterGap (lam : ℝ) (root : ℝ → ℝ) (h4 : ℝ) : ℝ :=
  typeThreePerimeter lam (root h4) - typeFourPerimeter lam h4

/-- A function selected as the unique zero in a compact branch is continuous.
Compactness is used only for the branch: the zero graph is closed, and its
projection along the compact factor is closed. -/
```

LeanSuffixAnalytic.equalAreaPerimeterGap_hasDerivAt

Classification: main

Source: EqualAreaEnvelope.lean:184-225

Declaration SHA-256: 42f42f4f683c9b8ff6e876958c9ae3b87d6bfd1ebf63072c7e612401d7b5ee93

Mathematical statement. For $\lambda > 1$, a differentiable regular equal-area selection satisfies $\Delta'(h) = (\rho(h) - h)A_4'(h)$ at every regular source height where it is differentiable.

Assume $0 < h < 1$, $0 < \rho(h) < 1$, and $A_3(\rho(x)) = A_4(x)$ for $0 < x \leq 1$. Differentiate this equality and use $P_i' = hA_i'$.

```
theorem equalAreaPerimeterGap_hasDerivAt
  {lam h : ℝ} {root : ℝ → ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1)
  (hroot_pos : 0 < root h) (hroot_lt_one : root h < 1)
  (hroot_differentiable : DifferentiableAt ℝ root h)
  (hequalArea : ∀ {x : ℝ}, 0 < x → x ≤ 1 →
    typeThreeArea lam (root x) = typeFourArea lam x) :
  HasDerivAt (equalAreaPerimeterGap lam root)
```

```

    ((root h - h) * deriv (typeFourArea lam) h) h := by
rcases typeThree_variational_hasDerivAt hlam hroot_pos hroot_lt_one with
  (hareaThree, hperimeterThree)
rcases typeFour_variational_hasDerivAt hlam hh hh1 with
  (hareaFour, hperimeterFour)
have hroot := hroot_differentiable.hasDerivAt
have harea_comp := hareaThree.comp h hroot
have hequalArea_eventually :
  (typeThreeArea lam ◦ root) =f[nhds h] typeFourArea lam := by
  filter_upwards [Ioo_mem_nhds hh hh1] with x hx
  exact hequalArea hx.1 hx.2.le
have harea_comp_from_equalArea :
  HasDerivAt (typeThreeArea lam ◦ root)
    (deriv (typeFourArea lam) h) h :=
  hareaFour.congr_of_eventuallyEq hequalArea_eventually
have hchain :
  deriv (typeThreeArea lam) (root h) * deriv root h =
  deriv (typeFourArea lam) h :=
  harea_comp.unique harea_comp_from_equalArea
have hgap :=
  (hperimeterThree.comp h hroot).sub hperimeterFour
have hgap' : HasDerivAt
  (typeThreePerimeter lam ◦ root - typeFourPerimeter lam)
  ((root h - h) * deriv (typeFourArea lam) h) h :=
  hgap.congr_deriv (by
    rw [mul_assoc, hchain]
    ring)
apply hgap'.congr_of_eventuallyEq
filter_upwards
intro x
rfl

/-- The canonical descending equal-area root selected from one certified
stationary pair. Values outside `0 < h₄ ≤ 1` are deliberately irrelevant. -/

```

LeanSuffixAnalytic.equalAreaPerimeterGap_le_fold

Classification: main

Source: EqualAreaEnvelope.lean:632-644

Declaration SHA-256: 0bde810fe503972e30290f54b6517ae65288b7e84131b161bebac58ed0796e33

Mathematical statement. For every descending envelope, $\Delta(h) \leq \Delta(s)$ for all $0 < h \leq 1$.

Equality at the stationary point is combined with the strict comparison away from it.

```

theorem equalAreaPerimeterGap_le_fold {lam : ℝ}
  (env : DescendingEqualAreaEnvelope lam) {h₄ : ℝ}
  (hh₄ : 0 < h₄) (hh₄_one : h₄ ≤ 1) :
  equalAreaPerimeterGap lam env.root h₄ ≤
  equalAreaPerimeterGap lam env.root env.fold := by
by_cases h : h₄ = env.fold
· subst h₄
  exact le_rfl
· exact (equalAreaPerimeterGap_lt_fold env hh₄ hh₄_one h).le

/-- L14 at the modeled scalar level. A negative genuine fold gap yields an
improving equal-area type-(iii) scalar for every `0 < h₄ ≤ 1`, including the
modeled endpoint `h₄ = 1`. -/

```

LeanSuffixAnalytic.equalAreaPerimeterGap_lt_fold

Classification: main

Source: EqualAreaEnvelope.lean:582-631

Declaration SHA-256: 1a0b26ad0e4af7fdd933118930486f792fc517471c75bd1495b190e7f97f3420

Mathematical statement. For every descending envelope and $h \in (0, 1]$ with $h \neq s$, the perimeter gap satisfies $\Delta(h) < \Delta(s)$.

The derivative is positive before the stationary curvature and negative after it; relative continuity handles the source endpoint.

```
theorem equalAreaPerimeterGap_lt_fold {lam : ℝ}
  (env : DescendingEqualAreaEnvelope lam) {h₄ : ℝ}
  (hh₄ : 0 < h₄) (hh₄_one : h₄ ≤ 1) (hne : h₄ ≠ env.fold) :
  equalAreaPerimeterGap lam env.root h₄ <
    equalAreaPerimeterGap lam env.root env.fold := by
  have hfold_mem_right : env.fold ∈ Icc env.fold (1 : ℝ) :=
    ⟨le_refl, env.fold_lt_one.le⟩
  by_cases hside : h₄ < env.fold
  · have hleft : StrictMonoOn (equalAreaPerimeterGap lam env.root)
      (Icc h₄ env.fold) := by
    apply strictMonoOn_of_deriv_pos (convex_Icc h₄ env.fold)
  · intro h hh
    exact (env.gap_hasDerivAt (hh₄.trans_le hh.1)
      (hh.2.trans_lt env.fold_lt_one)).continuousAt.continuousWithinAt
  · intro h hh
    rw [interior_Icc] at hh
    rw [(env.gap_hasDerivAt (hh₄.trans hh.1)
      (hh.2.trans env.fold_lt_one)).deriv]
    exact mul_pos_of_neg_of_neg
      (sub_neg.mpr (env.root_lt_curvature (hh₄.trans hh.1)
        (hh.2.trans env.fold_lt_one).le))
      (typeFourArea_deriv_neg_before_zero env.density_jump
        ⟨env.fold_pos, env.fold_lt_one⟩ env.fold_stationary
        ⟨hh₄.trans hh.1, hh.2.trans env.fold_lt_one⟩ hh.2)
  exact hleft ⟨le_refl, hside.le⟩ ⟨hside.le, le_refl⟩ hside
  · have hfold_lt : env.fold < h₄ := lt_of_le_of_ne
      (le_of_not_gt hside) (Ne.symm hne)
  have hright : StrictAntiOn (equalAreaPerimeterGap lam env.root)
      (Icc env.fold (1 : ℝ)) := by
    apply strictAntiOn_of_deriv_neg (convex_Icc env.fold 1)
  · intro h hh
    by_cases hh_one : h = 1
    · simpa [hh_one] using env.gap_continuousWithinAt_one.mono
      (fun x hx => ⟨env.fold_pos.trans_le hx.1, hx.2⟩)
    · exact (env.gap_hasDerivAt
      (env.fold_pos.trans_le hh.1) (lt_of_le_of_ne hh.2 hh_one)
      ).continuousAt.continuousWithinAt
  · intro h hh
    rw [interior_Icc] at hh
    rw [(env.gap_hasDerivAt (env.fold_pos.trans hh.1) hh.2).deriv]
    exact mul_neg_of_neg_of_pos
      (sub_neg.mpr (env.root_lt_curvature (env.fold_pos.trans hh.1)
        hh.2.le))
      (typeFourArea_deriv_pos_after_zero env.density_jump
        ⟨env.fold_pos, env.fold_lt_one⟩ env.fold_stationary
        ⟨env.fold_pos.trans hh.1, hh.2⟩ hh.1)
  exact hright hfold_mem_right ⟨hfold_lt.le, hh₄_one⟩ hfold_lt

/-- Non-strict form of the global fold maximum, convenient when only fold-gap
negativity is needed downstream. -/
```

LeanSuffixAnalytic.equalAreaRoot_continuousOn_of_compact_unique

Classification: main

Source: EqualAreaEnvelope.lean:71-107

Declaration SHA-256: 58b73f54ea82a27e264112e64332402373c869c542bc7277160adca9dcc99fdd

Mathematical statement. A selected equal-area root is continuous on $S \subset (0, 1]$ if it lies in a compact $K \subset (0, 1)$ and is the unique root in K for each source height.

Assume $\lambda > 1$, $\rho(S) \subset K$, and $A_3(\rho(h)) = A_4(h)$ on S . Continuity of the area formulas gives a closed zero graph; compactness and uniqueness imply continuity of the selection, including relative endpoint continuity.

```
theorem equalAreaRoot_continuousOn_of_compact_unique
  {lam : ℝ} {s K : Set ℝ} {root : ℝ → ℝ}
  (hlam : 1 < lam)
  (hK : IsCompact K)
  (hs_domain : s ⊆ Ioc 0 1)
  (hK_regular : K ⊆ Ioo 0 1)
  (hroot_mem : MapsTo root s K)
  (hequalArea : ∀ h ∈ s,
    typeThreeArea lam (root h) = typeFourArea lam h)
  (huniqueness : ∀ h ∈ s, ∀ y ∈ K,
    typeThreeArea lam y = typeFourArea lam h → y = root h) :
  ContinuousOn root s := by
  let _ : CompactSpace K := isCompact_iff_compactSpace.mp hK
  let r : s → K := hroot_mem.restrict root s K
  have hA3 : ContinuousOn (typeThreeArea lam) K := by
    intro y hy
    exact (typeThree_variational_hasDerivAt hlam
      (hK_regular hy).1 (hK_regular hy).2).1.continuousAt.continuousWithinAt
  have hA4 : ContinuousOn (typeFourArea lam) s := by
    intro h hh
    have hh0 : h ≠ 0 := ne_of_gt (hs_domain hh).1
    unfold typeFourArea typeFourAngle typeFourDelta
    fun_prop (disch := simp_all)
  have hr : Continuous r := continuous_of_compact_unique_eq
  (F := fun h : s => fun y : K =>
    typeThreeArea lam y - typeFourArea lam h)
  (z := 0)
  ((hA3.domRestrict.comp continuous_snd).sub
    (hA4.domRestrict.comp continuous_fst))
  (fun h => sub_eq_zero.mpr (hequalArea h h.property))
  (fun h y hy => Subtype.ext
    (huniqueness h h.property y y.property (sub_eq_zero.mp hy)))
  rw [continuousOn_iff_continuous_domRestrict]
  exact continuous_subtype_val.comp hr

/-- The type-(iii) area is strictly decreasing on a compact regular interval
when its derivative is negative throughout the interval's interior. -/
```

LeanSuffixAnalytic.equalAreaRoot_hasDerivAt

Classification: main

Source: EqualAreaEnvelope.lean:165-183

Declaration SHA-256: 4240cf112f1c537233001e48d7956d1d7de811e741deeb8b94332f375fdbb1f0

Mathematical statement. A continuous regular equal-area root at $h \in (0, 1)$ with $A_3'(\rho(h)) \neq 0$ has derivative $\rho'(h) = A_4'(h)/A_3'(\rho(h))$.

Assume $\lambda > 1$, $0 < \rho(h) < 1$, and equal area on all $(0, 1]$. The local inverse-function theorem differentiates the constraint; root continuity is an explicit hypothesis of this lemma.

```
theorem equalAreaRoot_hasDerivAt
  {lam h : ℝ} {root : ℝ → ℝ}
```

```

(hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1)
(hroot_pos : 0 < root h) (hroot_lt_one : root h < 1)
(hroot_continuous : ContinuousAt root h)
(hareaThree_ne : deriv (typeThreeArea lam) (root h) ≠ 0)
(hequalArea : ∀ {x : ℝ}, 0 < x → x ≤ 1 →
  typeThreeArea lam (root x) = typeFourArea lam x) :
HasDerivAt root
  (deriv (typeFourArea lam) h /
    deriv (typeThreeArea lam) (root h)) h := by
have hareaThree :=
  (typeThree_variational_hasDerivAt hlam hroot_pos hroot_lt_one).1
have hareaFour := (typeFour_variational_hasDerivAt hlam hh hh1).1
apply hareaThree.of_comp_left hroot_continuous hareaFour hareaThree_ne
filter_upwards [Ioo_mem_nhds hh hh1] with x hx
exact hequalArea hx.1 hx.2.le
/-- Differentiating the equal-area constraint and applying the exact
type-(iii) and type-(iv) variational identities gives the envelope identity. -/

```

LeanSuffixAnalytic.hasDerivAt_typeFourAngle

Classification: main

Source: LeanSuffixAnalytic.lean:241-278

Declaration SHA-256: ed3b0f3804a218b0cd68be6333d68c33832835819c8ca12dd83eb68bae07439d

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, $B'(h) = 1/\sqrt{1-h^2} - \lambda/\sqrt{\lambda^2-h^2}$.

The principal inverse-trigonometric derivatives and positive density simplify to this expression.

```

theorem hasDerivAt_typeFourAngle {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeFourAngle lam)
    (1 / sqrt (1 - h ^ 2) - lam / sqrt (lam ^ 2 - h ^ 2)) h := by
have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
have hu_arg : 0 < 1 - h ^ 2 := by nlinarith
have hd_arg : 0 < lam ^ 2 - h ^ 2 := by nlinarith
have hd_ne : sqrt (lam ^ 2 - h ^ 2) ≠ 0 :=
  ne_of_gt (Real.sqrt_pos.2 hd_arg)
have hratio_pos : 0 < h / lam := div_pos hh hlam_pos
have hratio_lt_one : h / lam < 1 :=
  (div_lt_one hlam_pos).2 (lt_trans hh1 hlam)
have hratio_ne_neg_one : h / lam ≠ -1 := by linarith
have hratio_ne_one : h / lam ≠ 1 := ne_of_lt hratio_lt_one
have hh_ne_neg_one : h ≠ -1 := by linarith
have hh_ne_one : h ≠ 1 := ne_of_lt hh1
have hsqrt_scale :
  sqrt (1 - (h / lam) ^ 2) = sqrt (lam ^ 2 - h ^ 2) / lam := by
have hratio_identity :
  1 - (h / lam) ^ 2 = (lam ^ 2 - h ^ 2) / lam ^ 2 := by
  field_simp [hlam_ne]
rw [hratio_identity, Real.sqrt_div hd_arg.le, Real.sqrt_sq_eq_abs,
  abs_of_pos hlam_pos]
unfold typeFourAngle
have harccos :=
  (Real.hasDerivAt_arccos (x := h / lam) hratio_ne_neg_one
    hratio_ne_one).comp h ((hasDerivAt_id h).div_const lam)
have harcsin :=
  Real.hasDerivAt_arcsin (x := h) hh_ne_neg_one hh_ne_one
have hraw := (harccos.const_mul lam).add harcsin
simp only [Function.comp_apply, id_eq] using! hraw.congr_deriv (by
  rw [hsqrt_scale]
  field_simp [hlam_ne, hd_ne]
  ring)

/-- Exact derivative of the regular type-(iv) radial displacement on its
natural open domain. -/

```

LeanSuffixAnalytic.hasDerivAt_typeFourArea

Classification: main

Source: LeanSuffixAnalytic.lean:496-553

Declaration SHA-256: 406477265a826e873557d3950f02d636becf1dd68c773947e69154fec9631e9e

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, $A_4'(h) = F_4(h)/h^3$.

The denominator is positive, so the fold and area derivative have exactly the same sign.

```
theorem hasDerivAt_typeFourArea {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeFourArea lam) (typeFourFold lam h / h ^ 3) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hh_ne : h ≠ 0 := ne_of_gt hh
  have hu_arg : 0 < 1 - h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - h ^ 2 := by nlinarith
  have hu_ne : sqrt (1 - h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hu_arg)
  have hd_ne : sqrt (lam ^ 2 - h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  have hangle := hasDerivAt_typeFourAngle hlam hh hh1
  have hdelta := hasDerivAt_typeFourDelta hlam hh hh1
  have hdelta_key :
    h * (h / sqrt (1 - h ^ 2) -
      h / (lam * sqrt (lam ^ 2 - h ^ 2))) -
    typeFourDelta lam h =
    1 / sqrt (1 - h ^ 2) -
    lam / sqrt (lam ^ 2 - h ^ 2) := by
  unfold typeFourDelta
  calc
    h * (h / sqrt (1 - h ^ 2) -
      h / (lam * sqrt (lam ^ 2 - h ^ 2))) -
      (sqrt (lam ^ 2 - h ^ 2) / lam - sqrt (1 - h ^ 2)) =
      (h ^ 2 + sqrt (1 - h ^ 2) ^ 2) / sqrt (1 - h ^ 2) -
      (h ^ 2 + sqrt (lam ^ 2 - h ^ 2) ^ 2) /
        (lam * sqrt (lam ^ 2 - h ^ 2)) := by
    field_simp [hlam_ne, hu_ne, hd_ne]
    ring
  _ = 1 / sqrt (1 - h ^ 2) -
    lam / sqrt (lam ^ 2 - h ^ 2) := by
    rw [Real.sq_sqrt hu_arg.le, Real.sq_sqrt hd_arg.le]
    field_simp [hlam_ne, hu_ne, hd_ne]
    ring
  have hquotient_identity (B D p dp : ℝ) (hk : h * dp - D = p) :
    (2 * (p + (D + h * dp)) * h ^ 2 -
      2 * (B + h * D) * (2 * h)) / (h ^ 2) ^ 2 =
      4 * (h * p - B) / h ^ 3 := by
    rw [- hk]
    field_simp [hh_ne]
    ring
  have hnumerator :=
    (hangle.add ((hasDerivAt_id h).mul hdelta)).const_mul 2
  have hraw := hnumerator.div ((hasDerivAt_id h).pow 2)
    (pow_ne_zero 2 hh_ne)
  simp only [typeFourArea, typeFourFold, id_eq] using! hraw.congr_deriv (by
    norm_num [id_eq, Pi.add_apply, Pi.mul_apply, Pi.pow_apply]
  exact hquotient_identity
    (typeFourAngle lam h) (typeFourDelta lam h)
    ((sqrt (1 - h ^ 2))⁻¹ -
      lam / sqrt (lam ^ 2 - h ^ 2))
    (h / sqrt (1 - h ^ 2) -
      h / (lam * sqrt (lam ^ 2 - h ^ 2)))
    (by simp only [one_div] using hdelta_key))

/-- The regular type-(iii) area and perimeter are differentiable, and their
derivatives satisfy the exact variational identity `P₃' = h A₃'`. -/
```

LeanSuffixAnalytic.hasDerivAt_typeFourDelta

Classification: main

Source: LeanSuffixAnalytic.lean:279-305

Declaration SHA-256: 71475be52cec7e059c20046cff749d550fce2a4275677924c383af205dd192c8

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, $D'(h) = h/\sqrt{1-h^2} - h/(\lambda\sqrt{\lambda^2-h^2})$.

The formula implies strict positivity for positive regular curvature and supports the monotonicity and concavity comparisons.

```
theorem hasDerivAt_typeFourDelta {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeFourDelta lam)
    (h / sqrt (1 - h ^ 2) -
     h / (lam * sqrt (lam ^ 2 - h ^ 2))) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hu_arg : 0 < 1 - h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - h ^ 2 := by nlinarith
  have hu_ne : sqrt (1 - h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hu_arg)
  have hd_ne : sqrt (lam ^ 2 - h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  unfold typeFourDelta
  have houter := (((hasDerivAt_const h (lam ^ 2)).sub
    ((hasDerivAt_id h).pow 2)).sqrt (ne_of_gt hd_arg)).div_const lam)
  have hinner := (((hasDerivAt_const h 1).sub
    ((hasDerivAt_id h).pow 2)).sqrt (ne_of_gt hu_arg))
  have hraw := houter.sub hinner
  simpa only [id_eq] using! hraw.congr_deriv (by
    norm_num [id_eq, Pi.sub_apply, Pi.pow_apply]
    field_simp [hlam_ne, hu_ne, hd_ne]
    ring)

/-- The positive radical-cube gap on the positive half of the regular domain.
Its square witness is the exact factorization
`(lam^2-x^2) - lam^2(1-x^2) = x^2(lam^2-1)`. -/
```

LeanSuffixAnalytic.hasDerivAt_typeFourFold

Classification: main

Source: LeanSuffixAnalytic.lean:457-495

Declaration SHA-256: df765bf2b71951f20ce86d7897c71befd43183478a79e7706b6061a82ade97f0

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, $F_4(h) = 4h^2K(h)$.

Cancellation of the first-order angle terms leaves the strictly positive radical-cube expression.

```
theorem hasDerivAt_typeFourFold {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeFourFold lam)
    (4 * h ^ 2 * (1 / sqrt (1 - h ^ 2) ^ 3 -
     lam / sqrt (lam ^ 2 - h ^ 2) ^ 3)) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hu_arg : 0 < 1 - h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - h ^ 2 := by nlinarith
  have hu_ne : sqrt (1 - h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hu_arg)
  have hd_ne : sqrt (lam ^ 2 - h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  have hunit :
    HasDerivAt (fun x : ℝ => 1 / sqrt (1 - x ^ 2))
      (h / sqrt (1 - h ^ 2) ^ 3) h := by
  have hraw := (((hasDerivAt_const h 1).sub
    ((hasDerivAt_id h).pow 2)).sqrt (ne_of_gt hu_arg)).inv hu_ne)
```

```

simpa only [one_div] using! hraw.congr_deriv (by
  simp only [id_eq, Pi.sub_apply, Pi.pow_apply]
  field_simp [hu_ne]
  ring)
have hscaled :
  HasDerivAt (fun x : ℝ => lam / sqrt (lam ^ 2 - x ^ 2))
    (lam * h / sqrt (lam ^ 2 - h ^ 2) ^ 3) h := by
  have hraw := (((hasDerivAt_const h (lam ^ 2)).sub
    ((hasDerivAt_id h).pow 2)).sqrt (ne_of_gt hd_arg)).inv hd_ne).const_mul lam)
simpa only [div_eq_mul_inv] using! hraw.congr_deriv (by
  simp only [id_eq, Pi.sub_apply, Pi.pow_apply]
  field_simp [hd_ne]
  ring)
have hangle := hasDerivAt_typeFourAngle hlam hh hh1
have hraw :=
  (((hasDerivAt_id h).mul (hunit.sub hscaled)).sub hangle).const_mul 4
simpa only [typeFourFold, id_eq] using! hraw.congr_deriv (by
  simp only [id_eq, Pi.sub_apply]
  ring)

/-- Exact derivative of the regular type-(iv) weighted area on its natural
open domain. Its numerator is precisely the stationary-fold function. -/

```

LeanSuffixAnalytic.hasDerivAt_typeThreeArea

Classification: main

Source: LeanSuffixAnalytic.lean:700-724

Declaration SHA-256: 7e0c0da90901c546a8edb152b4ed9a40769e6bd51ef8318f50390135bcace455

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, $A_3'(h) = F_3(h)/h^3$.

The statement gives the derivative itself, not merely a sign implication, and licenses the mean value arguments.

```

theorem hasDerivAt_typeThreeArea {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeArea lam) (typeThreeFold lam h / h ^ 3) h := by
  refine (typeThree_variational_hasDerivAt hlam hh hh1).1.congr_deriv ?_
  rw [typeThreeArea_deriv_eq_explicit hlam hh hh1]
  have hlam_ne : lam ≠ 0 := ne_of_gt (lt_trans (by norm_num) hlam)
  have hq_arg : 0 < 1 - typeThreeShape h ^ 2 := by
    unfold typeThreeShape
    nlinarith
  have hd_arg : 0 < lam ^ 2 - typeThreeShape h ^ 2 := by
    have hlam_sq : 1 < lam ^ 2 := by nlinarith
    have hq_sq : typeThreeShape h ^ 2 < 1 := by
      unfold typeThreeShape
      nlinarith
    linarith
  have hu_ne : sqrt (1 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hq_arg)
  have hd_ne : sqrt (lam ^ 2 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  unfold typeThreeFold
  field_simp [hlam_ne, hu_ne, hd_ne]
  ring

/-- The same type-(iii) fold numerator controls the perimeter derivative,
with one fewer power of the shared height. -/

```

LeanSuffixAnalytic.hasDerivAt_typeThreeFold

Classification: main

Source: LeanSuffixAnalytic.lean:375-456

Declaration SHA-256: c8c79d60a2ba5a7f202a48aa5b5851e2b132f448f609c9f5925a2a8edab62ee5

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, $F_3'(h) = 16h^2K(q(h))$.

The exact derivative is positive by the radical-cube comparison, even when $q(h)$ is negative.

```

theorem hasDerivAt_typeThreeFold {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeFold lam)
    (16 * h ^ 2 * (1 / sqrt (1 - typeThreeShape h ^ 2) ^ 3 -
      lam / sqrt (lam ^ 2 - typeThreeShape h ^ 2) ^ 3)) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hq_low : -1 < typeThreeShape h := by
    unfold typeThreeShape
    linarith
  have hq_high : typeThreeShape h < 1 := by
    unfold typeThreeShape
    linarith
  have hu_arg : 0 < 1 - typeThreeShape h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - typeThreeShape h ^ 2 := by nlinarith
  have hu_ne : sqrt (1 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hu_arg)
  have hd_ne : sqrt (lam ^ 2 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  let U := fun x : ℝ => 1 / sqrt (1 - typeThreeShape x ^ 2)
  let V := fun x : ℝ => (1 / sqrt (lam ^ 2 - typeThreeShape x ^ 2)) / lam
  have hshape := hasDerivAt_typeThreeShape h
  have hunitBase :
    HasDerivAt (fun x : ℝ => 1 / sqrt (1 - x ^ 2))
      (typeThreeShape h / sqrt (1 - typeThreeShape h ^ 2) ^ 3)
      (typeThreeShape h) := by
    have hraw := (((hasDerivAt_const (typeThreeShape h) 1).sub
      ((hasDerivAt_id (typeThreeShape h)).pow 2)).sqrt
      (ne_of_gt hu_arg)).inv hu_ne)
    simpa only [one_div] using! hraw.congr_deriv (by
      simp only [id_eq, Pi.sub_apply, Pi.pow_apply]
      field_simp [hu_ne]
      ring)
  have houterBase :
    HasDerivAt (fun x : ℝ => 1 / sqrt (lam ^ 2 - x ^ 2))
      (typeThreeShape h / sqrt (lam ^ 2 - typeThreeShape h ^ 2) ^ 3)
      (typeThreeShape h) := by
    have hraw := (((hasDerivAt_const (typeThreeShape h) (lam ^ 2)).sub
      ((hasDerivAt_id (typeThreeShape h)).pow 2)).sqrt
      (ne_of_gt hd_arg)).inv hd_ne)
    simpa only [one_div] using! hraw.congr_deriv (by
      simp only [id_eq, Pi.sub_apply, Pi.pow_apply]
      field_simp [hd_ne]
      ring)
  have hunit :
    HasDerivAt U
      (2 * typeThreeShape h / sqrt (1 - typeThreeShape h ^ 2) ^ 3) h := by
    simpa only [U, Function.comp_apply] using!
      (hunitBase.comp h hshape).congr_deriv (by ring)
  have hscaled :
    HasDerivAt V
      (2 * typeThreeShape h /
        (lam * sqrt (lam ^ 2 - typeThreeShape h ^ 2) ^ 3)) h := by
    simpa only [V, Function.comp_apply] using!
      ((houterBase.comp h hshape).div_const lam).congr_deriv (by
        field_simp [hlam_ne])
  have hA :=
    (((hasDerivAt_const h 1).add hshape).mul hunit).sub
      (((hasDerivAt_const h (lam ^ 2)).add hshape).mul hscaled))
  have hangle := hasDerivAt_typeThreeAngle hlam hh hh1
  have hdelta := hasDerivAt_typeThreeDelta hlam hh hh1
  have hraw :=
    (((hasDerivAt_id h).mul hA).const_mul 4).sub
      (((hangle.add_const (π / 2)).add hdelta).const_mul 2)
  have hderiv : HasDerivAt _
    (16 * h ^ 2 * (1 / sqrt (1 - typeThreeShape h ^ 2) ^ 3 -
      lam / sqrt (lam ^ 2 - typeThreeShape h ^ 2) ^ 3)) h :=
    hraw.congr_deriv (by
      simp only [Pi.add_apply, Pi.sub_apply, Pi.mul_apply, zero_add])

```

```

dsimp only [U, V]
simp only [id_eq]
field_simp [hlam_ne, hu_ne, hd_ne]
rw [Real.sq_sqrt hu_arg.le, Real.sq_sqrt hd_arg.le]
unfold typeThreeShape
ring)
apply hderiv.congr_of_eventuallyEq
filter_upwards [] with x
simp only [typeThreeFold, U, V, id_eq, Pi.add_apply, Pi.sub_apply,
  Pi.mul_apply]
field_simp [hlam_ne]
/-- Exact derivative of the regular type-(iv) stationary-fold function on its
natural open domain. -/

```

LeanSuffixAnalytic.radicalCubeGap_pos

Classification: main

Source: LeanSuffixAnalytic.lean:349-374

Declaration SHA-256: ed04fbfe0d2a5c8c68ae2429d8da8593a273f2471a8fcc329e85c97a6780176d

Mathematical statement. For $\lambda > 1$ and $-1 < x < 1$, $K(x) = (1 - x^2)^{-3/2} - \lambda(\lambda^2 - x^2)^{-3/2} > 0$.

For nonzero shape, compare squared radicals; at zero the expression is $1 - \lambda^{-2}$. This supplies fold monotonicity on both shape halves.

```

theorem radicalCubeGap_pos {lam x : ℝ}
  (hlam : 1 < lam) (hx : -1 < x) (hx1 : x < 1) :
  0 < 1 / sqrt (1 - x ^ 2) ^ 3 -
    lam / sqrt (lam ^ 2 - x ^ 2) ^ 3 := by
rcases lt_trichotomy x 0 with hxneg | rfl | hxpos
· have habs_pos : 0 < |x| := abs_pos.mpr (ne_of_lt hxneg)
  have habs_lt_one : |x| < 1 := (abs_lt).2 ⟨hx, hx1⟩
  simpa only [sq_abs] using
    radicalCubeGap_pos_of_pos hlam habs_pos habs_lt_one
· have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hlam_sq : 1 < lam ^ 2 := by nlinarith
  have hsqrt : sqrt (lam ^ 2) = lam := by
    rw [Real.sq_sqrt_eq_abs, abs_of_pos hlam_pos]
  simp only [zero_pow (show (2 : ℕ) ≠ 0 by norm_num), sub_zero,
    Real.sq_sqrt_one, one_pow]
  rw [hsqrt]
  have hquot : lam / lam ^ 3 = 1 / lam ^ 2 := by
    field_simp [hlam_ne]
  rw [hquot, sub_pos]
  simpa only [div_one] using
    (div_lt_one (sq_pos_of_pos hlam_pos)).2 hlam_sq
· exact radicalCubeGap_pos_of_pos hlam hxpos hx1

/-- Exact derivative of the regular type-(iii) stationary-fold function on its
natural open domain. -/

```

LeanSuffixAnalytic.sameHeight_support_identity

Classification: main

Source: SameCurvatureArea.lean:348-361

Declaration SHA-256: dfaacd20b555a53066a164daf8f9b4ebb1dbcc86f6d564ed5188ef405330e787

Mathematical statement. For every real λ and $h \neq 0$, $h[(P_3(h) - hA_3(h)) - (P_4(h) - hA_4(h))] = S(2h - 1) + \pi/2 - 2S(h)$.

This is a direct algebraic identity of the scalar definitions; density and regularity enter only when its sign is evaluated.

```

theorem sameHeight_support_identity (lam h : ℝ) (hh : h ≠ 0) :
  h * ((typeThreePerimeter lam h - h * typeThreeArea lam h) -
    (typeFourPerimeter lam h - h * typeFourArea lam h)) =

```

```

    typeFourSupport lam (2 * h - 1) +  $\pi$  / 2 -
      2 * typeFourSupport lam h := by
    unfold typeThreePerimeter typeThreeArea typeThreeAngle typeThreeShape
    unfold typeThreeDelta typeFourPerimeter typeFourArea typeFourSupport
    unfold typeFourAngle typeFourDelta
    unfold typeThreeShape
    field_simp [hh]
    ring

/-- At a common regular curvature above one half, the type-(iii) value of
`P-hA` is strictly smaller than the type-(iv) value. -/

```

LeanSuffixAnalytic.stationaryEqualAreaPair_exists

Classification: main

Source: UniversalStationaryPair.lean:107-115

Declaration SHA-256: 2bd42d07c65a61210061f6c5531a6ba1c7c3d14d12463bb14fd552e96e5e36b6

Mathematical statement. For every $\lambda > 1$, there exists a stationary equal-area pair with $r < s$.

Choose the regular fold zero in $(9/10, 1)$, then apply the equal-area bracketing theorem.

```

theorem stationaryEqualAreaPair_exists {lam :  $\mathbb{R}$ } (hlam : 1 < lam) :
   $\exists$  pair : StationaryEqualAreaPair lam, pair.h3 < pair.h4 := by
  rcases typeFourFold_zero_exists hlam with ⟨h4, hh4_left, hh4_one, hfold⟩
  rcases stationaryEqualAreaPair_of_typeFourFold_zero hlam
    ⟨by linarith, hh4_one⟩ (by linarith) hfold with ⟨pair, horder, -⟩
  exact ⟨pair, horder⟩

/-- Universal source-scalar improvement at a stationary equal-area pair:
negative type-(iii) fold, strict perimeter decrease, and negative reduced gap. -/

```

LeanSuffixAnalytic.stationaryEqualAreaPair_of_typeFourFold_zero

Classification: main

Source: UniversalStationaryPair.lean:62-106

Declaration SHA-256: 87c6601c0e4b1d865832811725296f245fb9e4dc0b97693804d443361bf40752

Mathematical statement. Every stationary $s \in (1/2, 1)$ at density $\lambda > 1$ admits a stationary equal-area pair with type-iv member s and type-iii member $r < s$.

The inequalities $A_3(s) < A_4(s) \leq A_4(1) < A_3(1/2)$ and continuity give $1/2 < r < s$ with exact equal area.

```

theorem stationaryEqualAreaPair_of_typeFourFold_zero
  {lam h4 :  $\mathbb{R}$ } (hlam : 1 < lam)
  (hh4 : h4 ∈ Set.Ioo (0 :  $\mathbb{R}$ ) 1) (hh4_half : 1 / 2 < h4)
  (hfold : typeFourFold lam h4 = 0) :
   $\exists$  pair : StationaryEqualAreaPair lam,
    pair.h3 < pair.h4  $\wedge$  pair.h4 = h4 := by
  have hfold_le_one : typeFourArea lam h4  $\leq$  typeFourArea lam 1 :=
    typeFourArea_fold_le hlam hh4 hfold ⟨by norm_num, le_refl⟩
  have hleft : typeFourArea lam h4 < typeThreeArea lam (1 / 2 :  $\mathbb{R}$ ) :=
    hfold_le_one.trans_lt (typeFourArea_one_lt_typeThreeArea_half hlam)
  have hright : typeThreeArea lam h4 < typeFourArea lam h4 :=
    typeThreeArea_lt_typeFourArea hlam hh4.1 hh4.2.le
  have hhalf_le : (1 / 2 :  $\mathbb{R}$ )  $\leq$  h4 := hh4_half.le
  have hcont : ContinuousOn (typeThreeArea lam) (Set.Icc (1 / 2 :  $\mathbb{R}$ ) h4) := by
    intro x hx
    have hx0 : x  $\neq$  0 := ne_of_gt (lt_of_lt_of_le (by norm_num) hx.1)
    unfold typeThreeArea typeThreeAngle typeThreeDelta typeThreeShape
    fun_prop (disch := simp_all)
  have htarget : typeFourArea lam h4 ∈ Set.Icc
    (typeThreeArea lam h4) (typeThreeArea lam (1 / 2 :  $\mathbb{R}$ )) :=
    ⟨hright.le, hleft.le⟩
  rcases intermediate_value_Icc' hhalf_le hcont htarget with

```

```

(h₃, hh₃, hequal)
have hh₃_half : (1 / 2 : ℝ) < h₃ := by
  apply lt_of_le_of_ne hh₃.1
  intro heq
  subst h₃
  linarith
have hh₃_h₄ : h₃ < h₄ := by
  apply lt_of_le_of_ne hh₃.2
  intro heq
  subst h₃
  linarith
let pair : StationaryEqualAreaPair lam :=
{ h₃ := h₃
  h₄ := h₄
  h₃_gt_half := hh₃_half
  h₃_lt_one := hh₃_h₄.trans hh₄.2
  h₄_pos := hh₄.1
  h₄_lt_one := hh₄.2
  equalArea := hequal
  stationary := hfold }
exact ⟨pair, hh₃_h₄, rfl⟩

/-- Every density above one has an ordered stationary equal-area pair. -/

```

LeanSuffixAnalytic.stationaryEqualAreaPair_strictImprovement_exists

Classification: main

Source: UniversalStationaryPair.lean:116-145

Declaration SHA-256: 1134a3b056794ea4a2511e4b1ea810f5acd87b2a3db5089871809393e30f50ab

Mathematical statement. For every $\lambda > 1$, some stationary equal-area pair satisfies $F_3(r) < 0$, $P_3(r) < P_4(s)$, and $G(r, s) < 0$.

Root order gives the negative fold and strict perimeter comparison; the exact stationary gap identity gives the reduced-gap sign. No numerical density subdivision is required.

```

theorem stationaryEqualAreaPair_strictImprovement_exists
{lam : ℝ} (hlam : 1 < lam) :
  ∃ pair : StationaryEqualAreaPair lam,
    typeThreeFold lam pair.h₃ < 0 ∧
    typeThreePerimeter lam pair.h₃ < typeFourPerimeter lam pair.h₄ ∧
    reducedFoldGap lam pair.h₃ pair.h₄ < 0 := by
  rcases stationaryEqualAreaPair_exists hlam with ⟨pair, horder⟩
  have hfoldThree : typeThreeFold lam pair.h₃ < 0 :=
    typeThreeFold_neg_of_equalArea_of_lt hlam
    ⟨lt_trans (by norm_num) pair.h₃_gt_half, pair.h₃_lt_one⟩
    ⟨pair.h₄_pos, pair.h₄_lt_one⟩ horder pair.equalArea
  have hperimeter :
    typeThreePerimeter lam pair.h₃ < typeFourPerimeter lam pair.h₄ :=
    typeThreePerimeter_lt_typeFourPerimeter_of_equalArea hlam
    pair.h₃_gt_half horder pair.h₄_lt_one pair.equalArea
  have hgap : reducedFoldGap lam pair.h₃ pair.h₄ < 0 :=
    (stationaryEqualArea_typeThree_improves_iff
      (ne_of_gt (lt_trans (by norm_num) pair.h₃_gt_half))
      (ne_of_gt pair.h₄_pos) pair.equalArea pair.stationary).1 hperimeter
  exact ⟨pair, hfoldThree, hperimeter, hgap⟩

end LeanSuffixAnalytic

namespace CMVSuffixModel.FourArcCandidate

variable {lam : ℝ} (candidate : _root_.FourArcCandidate lam)

/-- The universal stationary pair excludes every modeled type-(iv) candidate
at every density above one. Source classification and model compatibility
remain separate obligations. -/

```

LeanSuffixAnalytic.typeFourAngle

Classification: main

Source: LeanSuffixAnalytic.lean:53-56

Declaration SHA-256: 94e6a9f5b491cb7ca4802d89fba1fa7ccb864bbe777d6e41d89b1d6bf892bdd0

Mathematical statement. The type-iv angle contribution is $B(h) = \lambda \arccos(h/\lambda) + \arcsin h$.

The exterior and interior arc-angle contributions are combined with their respective density weights.

```
def typeFourAngle (lam h : ℝ) : ℝ :=
  lam * arccos (h / lam) + arcsin h

/-- The radial displacement in the exact regular type-(iv) formulas. -/
```

LeanSuffixAnalytic.typeFourArea

Classification: main

Source: LeanSuffixAnalytic.lean:61-64

Declaration SHA-256: 775f7921b66716563dd5e0e4c35562d880f5cef2a71b40cef616cd10eb9488d2

Mathematical statement. The type-iv scalar area is $A_4(h) = 2[B(h) + hD(h)]/h^2$.

For a CMV candidate, this is its weighted coordinate area on the complete modeled range $0 < h \leq 1$.

```
def typeFourArea (lam h : ℝ) : ℝ :=
  2 * (typeFourAngle lam h + h * typeFourDelta lam h) / h ^ 2

/-- Exact regular type-(iv) weighted perimeter used by the suffix comparison. -/
```

LeanSuffixAnalytic.typeFourArea_deriv_neg_before_zero

Classification: main

Source: LeanSuffixAnalytic.lean:1071-1082

Declaration SHA-256: e504f043a30eb739ae5f3ba9174d0ca2cd6aa51665b8ee6815ad3b1b3ea9932b

Mathematical statement. If $s \in (0, 1)$ satisfies $F_4(s) = 0$, then $A_4'(h) < 0$ for $0 < h < s$, assuming $\lambda > 1$.

Strict fold monotonicity gives $F_4(h) < 0$, and division by $h^3 > 0$ preserves the sign.

```
theorem typeFourArea_deriv_neg_before_zero {lam fold h : ℝ}
  (hlam : 1 < lam) (hfold : fold ∈ Set.Ioo (0 : ℝ) 1)
  (hzero : typeFourFold lam fold = 0)
  (hh : h ∈ Set.Ioo (0 : ℝ) 1) (hlt : h < fold) :
  deriv (typeFourArea lam) h < 0 := by
  rw [(hasDerivAt_typeFourArea hlam hh.1 hh.2).deriv]
  exact div_neg_of_neg_of_pos
    (by simpa [hzero] using typeFourFold_strictMonoOn hlam hh hfold hlt)
    (pow_pos hh.1 3)

/-- After a regular zero of the fold, the type-(iv) weighted area has
strictly positive derivative. -/
```

LeanSuffixAnalytic.typeFourArea_deriv_pos_after_zero

Classification: main

Source: LeanSuffixAnalytic.lean:1083-1094

Declaration SHA-256: 574cf814295ee9aa4cb9406582955f50506dd66296979a4f903f27b9ae8e2667

Mathematical statement. If $s \in (0, 1)$ satisfies $F_4(s) = 0$, then $A_4'(h) > 0$ for $s < h < 1$, assuming $\lambda > 1$.

This is the increasing side of the area profile after its unique fold.

```
theorem typeFourArea_deriv_pos_after_zero {lam fold h : ℝ}
  (hlam : 1 < lam) (hfold : fold ∈ Set.Ioo (0 : ℝ) 1)
  (hzero : typeFourFold lam fold = 0)
  (hh : h ∈ Set.Ioo (0 : ℝ) 1) (hlt : fold < h) :
  0 < deriv (typeFourArea lam) h := by
  rw [(hasDerivAt_typeFourArea hlam hh.1 hh.2).deriv]
  exact div_pos
    (by simpa [hzero] using typeFourFold_strictMonoOn hlam hfold hh hlt)
    (pow_pos hh.1 3)

/-- A regular zero of the type-(iv) fold is a global minimum of the
type-(iv) area on `0 < h ≤ 1`, including the closure endpoint. -/
```

LeanSuffixAnalytic.typeFourArea_fold_le

Classification: main

Source: LeanSuffixAnalytic.lean:1095-1129

Declaration SHA-256: 00808b809e3e67ecf06ff39be8e712a367908c043a9c526b98df7d97ff33901d

Mathematical statement. If $\lambda > 1$, $s \in (0, 1)$, and $F_4(s) = 0$, then $A_4(s) \leq A_4(h)$ for every $0 < h \leq 1$.

The regular derivative signs prove the minimum property; continuity of the finite area formula covers $h = 1$.

```
theorem typeFourArea_fold_le {lam fold h : ℝ} (hlam : 1 < lam)
  (hfold : fold ∈ Set.Ioo (0 : ℝ) 1)
  (hzero : typeFourFold lam fold = 0)
  (hh : h ∈ Set.Ioc (0 : ℝ) 1) :
  typeFourArea lam fold ≤ typeFourArea lam h := by
  rcases lt_trichotomy h fold with hlt | rfl | hgt
  · have hanti : StrictAntiOn (typeFourArea lam) (Set.Icc h fold) := by
    apply strictAntiOn_of_deriv_neg (convex_Icc h fold)
    · intro x hx
      exact (hasDerivAt_typeFourArea hlam
        (hh.1.trans_le hx.1) (hx.2.trans_lt hfold.2)
        ).continuousAt.continuousWithinAt
    · intro x hx
      rw [interior_Icc] at hx
      exact typeFourArea_deriv_neg_before_zero
        hlam hfold hzero
        (hh.1.trans hx.1, hx.2.trans hfold.2) hx.2
  exact (hanti ⟨le_rfl, hlt.le⟩ ⟨hlt.le, le_rfl⟩ hlt).le
  · exact le_rfl
  · have hmono : StrictMonoOn (typeFourArea lam) (Set.Icc fold h) := by
    apply strictMonoOn_of_deriv_pos (convex_Icc fold h)
    · intro x hx
      have hx0 : x ≠ 0 := ne_of_gt (hfold.1.trans_le hx.1)
      unfold typeFourArea typeFourAngle typeFourDelta
      fun_prop (disch := simp_all)
    · intro x hx
      rw [interior_Icc] at hx
      exact typeFourArea_deriv_pos_after_zero
        hlam hfold hzero
        (hfold.1.trans hx.1, hx.2.trans_le hh.2) hx.1
  exact (hmono ⟨le_rfl, hgt.le⟩ ⟨hgt.le, le_rfl⟩ hgt).le

/-- A stationary equal-area pair with negative type-(iii) fold constructs the
unique descending equal-area root for every regular or endpoint type-(iv)
curvature. -/
```

LeanSuffixAnalytic.typeFourArea_one_lt_typeThreeArea_half

Classification: main

Source: UniversalStationaryPair.lean:38-61

Declaration SHA-256: 10ba0ee6421be27f2e3c7dd8f7b1b5dc586c66d90118dd94301a5307188face6

Mathematical statement. For every $\lambda > 1$, $A_4(1) < A_3(1/2)$.

Bounding the outer angle by π and the displacement by one gives $A_4(1) \leq 2\lambda\pi + \pi + 2 < 2\pi(\lambda + 1)$.

```
theorem typeFourArea_one_lt_typeThreeArea_half {lam : ℝ} (hlam : 1 < lam) :
  typeFourArea lam 1 < typeThreeArea lam (1 / 2 : ℝ) := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hrad : 0 ≤ lam ^ 2 - 1 := by nlinarith
  have hsqrt_le_lam : sqrt (lam ^ 2 - 1) ≤ lam := by
    apply (Real.sqrt_le_iff).2
    exact ⟨hlam_pos.le, by nlinarith⟩
  have hscaled_le : lam * arccos (1 / lam) ≤ lam * π :=
    mul_le_mul_of_nonneg_left (Real.arccos_le_pi _) hlam_pos.le
  have hangle_le : typeFourAngle lam 1 ≤ lam * π + π / 2 := by
    unfold typeFourAngle
    rw [Real.arcsin_one]
    linarith
  have hdelta_le : typeFourDelta lam 1 ≤ 1 := by
    unfold typeFourDelta
    norm_num
    exact (div_le_one hlam_pos).2 hsqrt_le_lam
  rw [typeThreeArea_half hlam]
  unfold typeFourArea
  norm_num
  nlinarith [Real.pi_gt_three]
```

```
/-- Every regular stationary type-(iv) height admits a strictly lower,
equal-area type-(iii) height above `1 / 2`. -/
```

LeanSuffixAnalytic.typeFourClearedFold

Classification: main

Source: TypeFourStationaryRoot.lean:24-28

Declaration SHA-256: 1266cefaf66b4dab37ed6ce0451a3ec8c8e11c97af24ed298db2b491c2b5e620

Mathematical statement. The cleared type-iv fold is $C(h) = h[\sqrt{\lambda^2 - h^2} - \lambda\sqrt{1 - h^2}] - B(h)\sqrt{1 - h^2}\sqrt{\lambda^2 - h^2}$.

Unlike the raw fold, this formula has no radical denominators and is continuous at the closure endpoint.

```
def typeFourClearedFold (lam h : ℝ) : ℝ :=
  h * (sqrt (lam ^ 2 - h ^ 2) - lam * sqrt (1 - h ^ 2)) -
  typeFourAngle lam h * sqrt (1 - h ^ 2) * sqrt (lam ^ 2 - h ^ 2)

/-- The cleared fold is continuous even at the singular endpoint `h = 1`. -/
```

LeanSuffixAnalytic.typeFourClearedFold_continuous

Classification: main

Source: TypeFourStationaryRoot.lean:29-34

Declaration SHA-256: c2810ce70b601a04bf730b7d74e237dfa7c31c5a16c6857177a5231549ec1ba2

Mathematical statement. For every real λ , the cleared fold C is continuous on the real line.

The total real square-root and inverse-trigonometric functions are continuous; no division by a height-dependent radical remains.

```

theorem typeFourClearedFold_continuous (lam : ℝ) :
  Continuous (typeFourClearedFold lam) := by
  unfold typeFourClearedFold typeFourAngle
  fun_prop

/-- Closed-interval continuity used by the endpoint-sign IVT. -/

```

LeanSuffixAnalytic.typeFourClearedFold_eq_fold

Classification: main

Source: TypeFourStationaryRoot.lean:41-54

Declaration SHA-256: 45ea307682f4ec267ee71b675163dd61e82ecf847d6e7f8bf8dbc13d2cb15e17

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, $C(h) = \sqrt{1-h^2} \sqrt{\lambda^2-h^2} F_4(h)/4$.

Clearing the two nonzero radical denominators gives the exact relation between the continuous extension and the actual fold.

```

theorem typeFourClearedFold_eq_fold {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  typeFourClearedFold lam h =
    sqrt (1 - h ^ 2) * sqrt (lam ^ 2 - h ^ 2) *
    typeFourFold lam h / 4 := by
  have hu_arg : 0 < 1 - h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - h ^ 2 := by nlinarith
  have hu_ne : sqrt (1 - h ^ 2) ≠ 0 := ne_of_gt (Real.sqrt_pos.2 hu_arg)
  have hd_ne : sqrt (lam ^ 2 - h ^ 2) ≠ 0 := ne_of_gt (Real.sqrt_pos.2 hd_arg)
  unfold typeFourClearedFold typeFourFold
  field_simp [hu_ne, hd_ne]

/-- The multiplier relating the cleared expression to the source fold is
strictly positive throughout the regular domain. -/

```

LeanSuffixAnalytic.typeFourClearedFold_multiplier_pos

Classification: main

Source: TypeFourStationaryRoot.lean:55-61

Declaration SHA-256: da4a3633659e62178714586a00f2c1bd42b925fbeatbc151bdee7d46c4c36171

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, the multiplier $\sqrt{1-h^2} \sqrt{\lambda^2-h^2} / 4$ is strictly positive.

Consequently zeros and signs of the cleared and raw folds agree throughout the regular domain.

```

theorem typeFourClearedFold_multiplier_pos {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  0 < sqrt (1 - h ^ 2) * sqrt (lam ^ 2 - h ^ 2) / 4 := by
  have hu_arg : 0 < 1 - h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - h ^ 2 := by nlinarith
  positivity

```

LeanSuffixAnalytic.typeFourClearedFold_nine_tenths_neg

Classification: main

Source: TypeFourStationaryRoot.lean:112-124

Declaration SHA-256: 08d09dc1047004d87702d12abfa5bdbcdcdf35bfb88c89bf559151153a6e642e

Mathematical statement. For every $\lambda > 1$, $C(9/10) < 0$.

At this fixed height, the positive term in the raw fold is less than 27/20, while $B(9/10) \geq \pi/2 > 27/20$; the multiplier preserves strict negativity.

```

theorem typeFourClearedFold_nine_tenths_neg {lam : ℝ} (hlam : 1 < lam) :
  typeFourClearedFold lam (9 / 10 : ℝ) < 0 := by
  rw [typeFourClearedFold_eq_fold hlam (by norm_num) (by norm_num)]
  calc
  _ = (sqrt (1 - (9 / 10 : ℝ) ^ 2) *
      sqrt (lam ^ 2 - (9 / 10 : ℝ) ^ 2) / 4) *
      typeFourFold lam (9 / 10 : ℝ) := by ring
  _ < 0 := mul_neg_of_pos_of_neg
      (typeFourClearedFold_multiplier_pos hlam (by norm_num) (by norm_num))
      (typeFourFold_nine_tenths_neg hlam)

/-- The endpoint value is the surviving outer radical and is strictly positive.
No value of the singular raw fold at `h = 1` is used. -/

```

LeanSuffixAnalytic.typeFourClearedFold_one_pos

Classification: main

Source: TypeFourStationaryRoot.lean:125-131

Declaration SHA-256: 5b0cd6de2bd6df7214891200ea0e61cc8d389c1b1edf8e7a99120d747111a4bc

Mathematical statement. For every $\lambda > 1$, $C(1) > 0$.

The endpoint simplifies exactly to $\sqrt{\lambda^2 - 1}$, which is positive without evaluating the raw fold there.

```

theorem typeFourClearedFold_one_pos {lam : ℝ} (hlam : 1 < lam) :
  0 < typeFourClearedFold lam 1 := by
  have hrad : 0 < lam ^ 2 - 1 := by nlinarith
  simpa [typeFourClearedFold] using Real.sqrt_pos.2 hrad

/-- Every density `lam > 1` has a regular stationary type-(iv) height strictly
between `9/10` and `1`. -/

```

LeanSuffixAnalytic.typeFourDelta

Classification: main

Source: LeanSuffixAnalytic.lean:57-60

Declaration SHA-256: a4c8b30971f59f8f04626dbddf49af92c98dbfa9126bf3766c0be8b6472cbfb3

Mathematical statement. The type-iv radical displacement is $D(h) = \sqrt{\lambda^2 - h^2} / \lambda - \sqrt{1 - h^2}$.

Both square roots are nonnegative; their derivative formulas are invoked only in the regular open domain.

```

def typeFourDelta (lam h : ℝ) : ℝ :=
  sqrt (lam ^ 2 - h ^ 2) / lam - sqrt (1 - h ^ 2)

/-- Exact regular type-(iv) weighted area used by the suffix comparison. -/

```

LeanSuffixAnalytic.typeFourFold

Classification: main

Source: LeanSuffixAnalytic.lean:78-82

Declaration SHA-256: e492724315b735482cab607f768694fcd028864cb336800b7469d841099293ee

Mathematical statement. The type-iv fold is $F_4(h) = 4[h(1/\sqrt{1 - h^2} - \lambda/\sqrt{\lambda^2 - h^2}) - B(h)]$.

Its zero on the regular interval is the stationary area curvature. The proof never interprets its totalized endpoint value as a geometric derivative.

```

def typeFourFold (lam h : ℝ) : ℝ :=
  4 * (h * (1 / sqrt (1 - h ^ 2) -

```

```
lam / sqrt (lam ^ 2 - h ^ 2)) - typeFourAngle lam h)

/-- The radical expression left after equal-area and fold elimination. -/
```

LeanSuffixAnalytic.typeFourFold_strictMonoOn

Classification: main

Source: LeanSuffixAnalytic.lean:1056-1070

Declaration SHA-256: 16af12c38f3bb1c629d7204f781995879e693ae4c65a0b5d7c3475b2043d0222

Mathematical statement. For every $\lambda > 1$, F_4 is strictly increasing on $(0, 1)$.

Its positive derivative gives uniqueness of any regular stationary curvature and the opposite derivative signs on either side.

```
theorem typeFourFold_strictMonoOn {lam : ℝ} (hlam : 1 < lam) :
  StrictMonoOn (typeFourFold lam) (Set.Ioo 0 1) := by
  apply strictMonoOn_of_hasDerivWithinAt_pos (convex_Ioo (0 : ℝ) 1)
  · intro h hh
    exact (hasDerivAt_typeFourFold hlam hh.1 hh.2).continuousAt.continuousWithinAt
  · intro h hh
    rw [interior_Ioo] at hh
    exact (hasDerivAt_typeFourFold hlam hh.1 hh.2).hasDerivWithinAt
  · intro h hh
    rw [interior_Ioo] at hh
    exact mul_pos (mul_pos (by norm_num) (sq_pos_of_pos hh.1))
      (radicalCubeGap_pos hlam (by linarith [hh.1]) hh.2)

/-- Before a regular zero of the fold, the type-(iv) weighted area has
strictly negative derivative. -/
```

LeanSuffixAnalytic.typeFourFold_zero_exists

Classification: main

Source: TypeFourStationaryRoot.lean:132-167

Declaration SHA-256: d603264a772d81bf55a0dad7fb9b6a0704e412d433e0051ac8f104d2554f1f15

Mathematical statement. For every $\lambda > 1$, there exists $s \in (9/10, 1)$ with $F_4(s) = 0$.

The intermediate value theorem applies to the cleared fold with opposite endpoint signs; positivity of its multiplier recovers a regular zero of the original fold.

```
theorem typeFourFold_zero_exists {lam : ℝ} (hlam : 1 < lam) :
  ∃ h : ℝ, 9 / 10 < h ∧ h < 1 ∧ typeFourFold lam h = 0 := by
  have hcont : ContinuousOn (typeFourClearedFold lam)
    (Set.Icc (9 / 10 : ℝ) 1) :=
    typeFourClearedFold_continuousOn lam
  have htarget : (0 : ℝ) ∈ Set.Icc
    (typeFourClearedFold lam (9 / 10 : ℝ))
    (typeFourClearedFold lam 1) :=
    ((typeFourClearedFold_nine_tenths_neg hlam).le,
     (typeFourClearedFold_one_pos hlam).le)
  rcases intermediate_value_Icc (show (9 / 10 : ℝ) ≤ 1 by norm_num)
    hcont htarget with ⟨h, hh, hzero⟩
  have hh_left : (9 / 10 : ℝ) < h := by
    apply lt_of_le_of_ne hh.1
    intro heq
    subst h
    linarith [typeFourClearedFold_nine_tenths_neg hlam]
  have hh_right : h < 1 := by
    apply lt_of_le_of_ne hh.2
    intro heq
    subst h
    linarith [typeFourClearedFold_one_pos hlam]
  have hfactor_pos := typeFourClearedFold_multiplier_pos
    hlam (by linarith) hh_right
```

```

have hproduct :
  (sqrt (1 - h ^ 2) * sqrt (lam ^ 2 - h ^ 2) / 4) *
  typeFourFold lam h = 0 := by
  calc
  _ = typeFourClearedFold lam h := by
    rw [typeFourClearedFold_eq_fold hlam (by linarith) hh_right]
    ring
  _ = 0 := hzero
exact ⟨h, hh_left, hh_right,
  (mul_eq_zero.mp hproduct).resolve_left (ne_of_gt hfactor_pos)⟩
end LeanSuffixAnalytic

```

LeanSuffixAnalytic.typeFourPerimeter

Classification: main

Source: LeanSuffixAnalytic.lean:65-70

Declaration SHA-256: ad0bebbd79c17351c0f12ce6a7fdae3e930dba33de0077415a6116df1d1e6e25

Mathematical statement. The type-iv scalar perimeter is $P_4(h) = 4B(h)/h$.

The formula remains finite at $h = 1$, unlike the regular fold expression.

```

def typeFourPerimeter (lam h : ℝ) : ℝ :=
  4 * typeFourAngle lam h / h

/-- The checker numerator for the derivative of the regular type-(iii) area.
It is kept as one expression in the shared height argument so interval
evaluation does not decorrelate its repeated occurrences. -/

```

LeanSuffixAnalytic.typeFourSupport

Classification: main

Source: SameCurvatureArea.lean:266-269

Declaration SHA-256: fdeb0489b7e6f5e26dfa807a57579da3ecea22f44d0bba90cd5fa78c0744aa33

Mathematical statement. The type-iv support term is $S(x) = B(x) - xD(x)$.

This is a scalar function, related by an exact identity to the difference of the two families' $P - hA$ values.

```

def typeFourSupport (lam x : ℝ) : ℝ :=
  typeFourAngle lam x - x * typeFourDelta lam x

/-- Exact derivative of the type-(iv) support term on the regular domain. -/

```

LeanSuffixAnalytic.typeFourSupport_concave0n

Classification: main

Source: SameCurvatureArea.lean:293-311

Declaration SHA-256: 8ef0a051daef9d1f2bbdf8d2a9a39d0e0a16c808a777adfdd62560b57626cb9b

Mathematical statement. For $\lambda > 1$, S is concave on $[0, 1]$.

The displacement is increasing, so $S' = -2D$ is nonincreasing; continuity extends the conclusion to both endpoints.

```

theorem typeFourSupport_concave0n {lam : ℝ} (hlam : 1 < lam) :
  ConcaveOn ℝ (Set.Icc 0 1) (typeFourSupport lam) := by
  apply AntitoneOn.concaveOn_of_deriv (convex_Icc (0 : ℝ) 1)
  · intro x hx
    unfold typeFourSupport typeFourAngle typeFourDelta
    fun_prop
  · intro x hx

```

```

    rw [interior_Icc] at hx
    exact (typeFourSupport_hasDerivAt
      hlam hx.1 hx.2).differentiableAt.differentiableWithinAt
  · intro x hx y hy hxy
    rw [interior_Icc] at hx hy
    rw [(typeFourSupport_hasDerivAt hlam hx.1 hx.2).deriv,
      (typeFourSupport_hasDerivAt hlam hy.1 hy.2).deriv]
    have hdelta := (typeFourDelta_strictMonoOn hlam).monotoneOn
      ⟨hx.1.le, hx.2.le⟩ ⟨hy.1.le, hy.2.le⟩ hxy
    nlinarith

```

```

/-- Strict endpoint margin for the support term. -/

```

LeanSuffixAnalytic.typeFourSupport_hasDerivAt

Classification: main

Source: SameCurvatureArea.lean:270-292

Declaration SHA-256: 7ec9ad85e9c92ae246dffaf1ad5a0c048a08bd6754c8a63a3be2760ea37a093f

Mathematical statement. For $\lambda > 1$ and $0 < x < 1$, $S'(x) = -2D(x)$.

Differentiating $B - xD$ and simplifying the radicals cancels the other terms.

```

theorem typeFourSupport_hasDerivAt {lam x : ℝ}
  (hlam : 1 < lam) (hx : 0 < x) (hx1 : x < 1) :
  HasDerivAt (typeFourSupport lam) (-2 * typeFourDelta lam x) x := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hu_arg : 0 < 1 - x ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - x ^ 2 := by nlinarith
  have hu_ne : sqrt (1 - x ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hu_arg)
  have hd_ne : sqrt (lam ^ 2 - x ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  have hraw := (hasDerivAt_typeFourAngle hlam hx hx1).sub
    ((hasDerivAt_id x).mul (hasDerivAt_typeFourDelta hlam hx hx1))
  simpa only [typeFourSupport, id_eq] using! hraw.congr_deriv (by
    simp only [id_eq]
    unfold typeFourDelta
    field_simp [hlam_ne, hu_ne, hd_ne]
    ring_nf
    rw [Real.sq_sqrt hu_arg.le, Real.sq_sqrt hd_arg.le]
    ring)

/-- The type-(iv) support term is concave on the complete closed regular
height interval. Only interior derivatives are used. -/

```

LeanSuffixAnalytic.typeFourSupport_midpoint

Classification: main

Source: SameCurvatureArea.lean:321-347

Declaration SHA-256: fc169235755eca8de1ba9408de035c749430eb621ef56b64e6fdcdc4766386f

Mathematical statement. For $\lambda > 1$ and $1/2 < h < 1$, $S(2h - 1) + S(1) \leq 2S(h)$.

The curvature h is the midpoint of $2h - 1$ and one, both in the concavity interval.

```

theorem typeFourSupport_midpoint {lam h : ℝ}
  (hlam : 1 < lam) (hh : 1 / 2 < h) (hh1 : h < 1) :
  typeFourSupport lam (2 * h - 1) + typeFourSupport lam 1 ≤
    2 * typeFourSupport lam h := by
  let q := 2 * h - 1
  have hq0 : 0 < q := by
    dsimp only [q]
    nlinarith

```

```

have hq1 : q < 1 := by
  dsimp only [q]
  linarith
have hqh : q < h := by
  dsimp only [q]
  linarith
have hslope := (typeFourSupport_concaveOn hlam).slope_anti_adjacent
  (show q ∈ Set.Icc (0 : ℝ) 1 from ⟨hq0.le, hq1.le⟩)
  (show (1 : ℝ) ∈ Set.Icc 0 1 by norm_num) hqh hh1
have hden : 0 < 1 - h := sub_pos.mpr hh1
have hqden : h - q = 1 - h := by
  dsimp only [q]
  ring
rw [hqden] at hslope
have hnum := (div_le_div_iff_of_pos_right hden).mp hslope
dsimp only [q] at hnum ⊢
linarith

/-- Exact same-height identity relating the support terms to `P-hA`. -/

```

LeanSuffixAnalytic.typeFourSupport_one_gt_halfPi

Classification: main

Source: SameCurvatureArea.lean:312-320

Declaration SHA-256: 0c5ddb44fb627a6e8b09e47012b025857f4fe02fa3598a05502fda2e5e2e8921

Mathematical statement. For $\lambda > 1$, $S(1) > \pi/2$.

The excess is $\lambda \arccos(1/\lambda) - \sqrt{\lambda^2 - 1} / \lambda$, positive by $\sin \theta < \theta$ at $\theta = \arccos(1/\lambda) > 0$.

```

theorem typeFourSupport_one_gt_halfPi {lam : ℝ} (hlam : 1 < lam) :
  π / 2 < typeFourSupport lam 1 := by
  have hmargin := sameCurvatureMargin_one_pos hlam
  unfold sameCurvatureMargin angleGap at hmargin
  unfold typeFourSupport
  norm_num at hmargin ⊢
  linarith

/-- Concavity at the exact midpoint `h = ((2h-1)+1)/2`. -/

```

LeanSuffixAnalytic.typeFour_variational_hasDerivAt

Classification: main

Source: LeanSuffixAnalytic.lean:733-753

Declaration SHA-256: be216789e3804b1c530f202d3470319a386e5c80dc03952cfaf3ebc685f27297

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, both type-iv scalar functions are differentiable and $P_4'(h) = hA_4'(h)$.

Together with its type-iii counterpart, this produces the equal-area envelope derivative.

```

theorem typeFour_variational_hasDerivAt {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeFourArea lam) (deriv (typeFourArea lam) h) h ∧
  HasDerivAt (typeFourPerimeter lam)
    (h * deriv (typeFourArea lam) h) h := by
  have hh_ne : h ≠ 0 := ne_of_gt hh
  have hangle := hasDerivAt_typeFourAngle hlam hh hh1
  have hraw := (hangle.const_mul (4 : ℝ)).div (hasDerivAt_id h) hh_ne
  have hperimeter : HasDerivAt (typeFourPerimeter lam)
    ((4 * (1 / sqrt (1 - h ^ 2)) - lam / sqrt (lam ^ 2 - h ^ 2)) * h -
     4 * typeFourAngle lam h) / h ^ 2 h := by
    simpa only [typeFourPerimeter, id_eq] using! hraw.congr_deriv (by
      simp only [id_eq]
      ring)
  have harea := hasDerivAt_typeFourArea hlam hh hh1

```

```

refine {harea.congr_deriv harea.deriv.symm, hperimeter.congr_deriv ?_}
rw [harea.deriv]
unfold typeFourFold
field_simp [hh_ne]

/-- Exact type-(iv) variational identity on the regular domain. -/

```

LeanSuffixAnalytic.typeThreeAngle

Classification: main

Source: LeanSuffixAnalytic.lean:35-38

Declaration SHA-256: fb7617b8b745792c6f019f3cbcf3c26d5e54f1fc7dd47bb69cec919c712d110a

Mathematical statement. The type-iii angle contribution is $B(q(h)) = \lambda \arccos(q(h)/\lambda) + \arcsin q(h)$.

Its additional $\pi/2$ in the area and perimeter formulas converts it to the geometric term $\lambda\alpha + \pi - \beta$.

```

def typeThreeAngle (lam h : ℝ) : ℝ :=
  lam * arccos (typeThreeShape h / lam) + arcsin (typeThreeShape h)

/-- The radial displacement in the exact type-(iii) formulas. -/

```

LeanSuffixAnalytic.typeThreeArea

Classification: main

Source: LeanSuffixAnalytic.lean:44-48

Declaration SHA-256: 63d13d5879252e16ac6f53a2d4449b4f511e353d6016e98158793484dcfaa8f7

Mathematical statement. The type-iii scalar area is $A_3(h) = [B(q(h)) + \pi/2 + (q(h) + 2)D(q(h))]/h^2$.

The geometric theorem identifies this real function with weighted carrier area when $\lambda > 1$ and $0 < h < 1$; closed formula values are used only where justified.

```

def typeThreeArea (lam h : ℝ) : ℝ :=
  (typeThreeAngle lam h + π / 2 +
   (typeThreeShape h + 2) * typeThreeDelta lam h) / h ^ 2

/-- Exact regular type-(iii) weighted perimeter used by the suffix comparison. -/

```

LeanSuffixAnalytic.typeThreeAreaSupport

Classification: main

Source: SameCurvatureArea.lean:464-466

Declaration SHA-256: 59983bd94c5b2c86b0a792e561e5d858c16e3c1a5d5652441772674daa8dcfc8

Mathematical statement. The type-iii area-support function at target V is $H_V(x) = P_3(x) + x(V - A_3(x))$.

At an equal-area root its value is exactly the type-iii perimeter.

```

def typeThreeAreaSupport (lam V x : ℝ) : ℝ :=
  typeThreePerimeter lam x + x * (V - typeThreeArea lam x)

```

LeanSuffixAnalytic.typeThreeAreaSupport_hasDerivAt

Classification: main

Source: SameCurvatureArea.lean:467-480

Declaration SHA-256: 84f5f511ccfd2c66897b9ce22e970b7b5810edf0c56f08f711aad8e54224205e

Mathematical statement. For $\lambda > 1$, real V , and $0 < x < 1$, $H_V(x) = V - A_3(x)$.

The derivative cancellation follows from $P_3'(x) = xA_3'(x)$, turning an area sublevel bound into support monotonicity.

```
theorem typeThreeAreaSupport_hasDerivAt {lam V x : ℝ}
  (hlam : 1 < lam) (hx : 0 < x) (hx1 : x < 1) :
  HasDerivAt (typeThreeAreaSupport lam V)
    (V - typeThreeArea lam x) x := by
  have hvar := typeThreeVariational_hasDerivAt hlam hx hx1
  have hraw := hvar.2.add
    ((hasDerivAt_id x).mul ((hasDerivAt_const x V).sub hvar.1))
  simpa only [typeThreeAreaSupport, id_eq, Pi.sub_apply] using!
  hraw.congr_deriv (by
    simp only [id_eq, Pi.sub_apply]
    ring)

/-- The shared type-(iii) area-support function is continuous through the
closed endpoint on every interval bounded away from zero. -/
```

LeanSuffixAnalytic.typeThreeArea_centered_meanValue

Classification: main

Source: LeanSuffixAnalytic.lean:778-789

Declaration SHA-256: 0495f3e7854574a7754a0b63ac7fd008229aa9ba79f907c1f5c180c5d5dea354

Mathematical statement. For $\lambda > 1$ and $c, h \in (0, 1)$, some ξ between them satisfies $A_3(h) = A_3(c) + F_3(\xi)(h - c)/\xi^3$.

This mean value identity locates a negative fold when the area strictly drops between two ordered regular heights.

```
theorem typeThreeArea_centered_meanValue {lam c h : ℝ}
  (hlam : 1 < lam) (hc : c ∈ Set.Ioo (0 : ℝ) 1)
  (hh : h ∈ Set.Ioo (0 : ℝ) 1) :
  ∃ ξ ∈ Set.uIcc c h,
    typeThreeArea lam h = typeThreeArea lam c +
      typeThreeFold lam ξ / ξ ^ 3 * (h - c) := by
  apply centered_meanValue_exists
  intro x hx
  have hxreg := uIcc_subset_regular hc hh hx
  exact hasDerivAt_typeThreeArea hlam hxreg.1 hxreg.2

/-- Exact centered enclosure witness for the type-(iii) perimeter. -/
```

LeanSuffixAnalytic.typeThreeArea_continuousAt_one

Classification: main

Source: SameCurvatureArea.lean:384-393

Declaration SHA-256: b82ff69676ab90ec4fb13d857723ab9b2dd80ce0482e1e3d55da8f2320d7eef0

Mathematical statement. For $\lambda > 1$, the scalar function A_3 is continuous at one.

Only the finite formula is asserted at this endpoint; no regular type-iii assembly or endpoint fold derivative is introduced.

```
theorem typeThreeArea_continuousAt_one {lam : ℝ} (hlam : 1 < lam) :
  ContinuousAt (typeThreeArea lam) 1 := by
  have hlam_ne : lam ≠ 0 := ne_of_gt (lt_trans (by norm_num) hlam)
  unfold typeThreeArea typeThreeAngle typeThreeDelta typeThreeShape
  fun_prop (disch := simp_all)

/-- A type-(iii) area profile with increasing fold cannot exceed both endpoint
areas on a regular interval whose right endpoint may be the closed height
`b = 1`. The endpoint case uses continuity of the source formula, never the
totalized fold at one. -/
```

LeanSuffixAnalytic.typeThreeArea_descending_root_existsUnique

Classification: main

Source: LeanSuffixAnalytic.lean:1013-1039

Declaration SHA-256: 15660b04d9615f11056408ab4ae99cb03c7fde3d8d26f5056c89e742987b7edb

Mathematical statement. For $\lambda > 1$, a regular anchor a with $F_3(a) < 0$, and $V \geq A_3(a)$, there is exactly one regular r with $A_3(r) = V$ and $F_3(r) < 0$.

Divergence at zero and strict descent produce a root at or below the anchor; the least-root property proves uniqueness among all negative-fold roots.

```
theorem typeThreeArea_descending_root_existsUnique
  {lam anchor target : ℝ}
  (hlam : 1 < lam) (hanchor : anchor ∈ Set.Ioo (0 : ℝ) 1)
  (hfold : typeThreeFold lam anchor < 0)
  (htarget : typeThreeArea lam anchor ≤ target) :
  ∃! root : ℝ, root ∈ Set.Ioo (0 : ℝ) 1 ∧
    typeThreeArea lam root = target ∧ typeThreeFold lam root < 0 := by
  rcases typeThreeArea_descending_root_exists_below
    hlam hanchor hfold htarget with
    ⟨root, hroot_mem, hroot_area, hroot_fold⟩
  have hroot_regular : root ∈ Set.Ioo (0 : ℝ) 1 :=
    ⟨hroot_mem.1, hroot_mem.2.trans_lt hanchor.2⟩
  refine ⟨root, ⟨hroot_regular, hroot_area, hroot_fold⟩, ?_⟩
  intro y hy
  apply le_antisymm
  · exact typeThreeArea_root_le_of_fold_neg
    (root := y) (y := root) hlam hy.1
    ⟨hroot_regular.1, hroot_regular.2.le⟩ hy.2.2
    (hy.2.1.trans hroot_area.symm)
  · exact typeThreeArea_root_le_of_fold_neg
    (root := root) (y := y) hlam hroot_regular
    ⟨hy.1.1, hy.1.2.le⟩ hroot_fold
    (hroot_area.trans hy.2.1.symm)

/-- Above a fixed equal-area root there is at most one further root in
`0 < h ≤ 1`. Combined with `typeThreeArea_root_le_of_fold_neg`, this classifies
all roots as the certified descending root and at most one upper root. -/
```

LeanSuffixAnalytic.typeThreeArea_descending_root_existsUnique_of_stationary_pair

Classification: main

Source: LeanSuffixAnalytic.lean:1130-1149

Declaration SHA-256: f368bc947db4d0b119fefc4c63c0de6d0d14cb14db7e5a28faa26c535eb880b8

Mathematical statement. A regular stationary equal-area pair with negative type-iii fold yields a unique descending type-iii equal-area root for each $0 < h \leq 1$.

Assume $\lambda > 1$, $r, s \in (0, 1)$, $A_3(r) = A_4(s)$, $F_3(r) < 0$, and $F_4(s) = 0$. The type-iv minimum ensures every target area is at least the anchor area.

```
theorem typeThreeArea_descending_root_existsUnique_of_stationary_pair
  {lam h3fold h4fold h4 : ℝ}
  (hlam : 1 < lam)
  (hh3fold : h3fold ∈ Set.Ioo (0 : ℝ) 1)
  (hh4fold : h4fold ∈ Set.Ioo (0 : ℝ) 1)
  (hh4 : h4 ∈ Set.Ioc (0 : ℝ) 1)
  (hfoldThree : typeThreeFold lam h3fold < 0)
  (hfoldFour : typeFourFold lam h4fold = 0)
  (hequalFold :
    typeThreeArea lam h3fold = typeFourArea lam h4fold) :
  ∃! root : ℝ, root ∈ Set.Ioo (0 : ℝ) 1 ∧
```

```

    typeThreeArea lam root = typeFourArea lam h4 ^
    typeThreeFold lam root < 0 := by
  apply typeThreeArea_descending_root_existsUnique
    hlam hh3fold hfoldThree
  rw [hequalFold]
  exact typeFourArea_fold_le hlam hh4fold hfoldFour hh4

/-- The regular type-(iv) fold has at most one zero in its admissible height
interval. -/

```

LeanSuffixAnalytic.typeThreeArea_descending_root_lt

Classification: main

Source: SameCurvatureArea.lean:586-622

Declaration SHA-256: f0218743e3c66d2b3ba1e5bdde4d44d6f7c58ed79dc9416fb8fb18d852a14824

Mathematical statement. If $\lambda > 1$, $r \in (0, 1)$, $0 < h \leq 1$, $F_3(r) < 0$, and $A_3(r) = A_4(h)$, then $r < h$.

If the source height were no larger than the root, descent of A_3 before the root would contradict the strict same-curvature area comparison.

```

theorem typeThreeArea_descending_root_lt
  {lam root h : ℝ}
  (hlam : 1 < lam)
  (hroot : root ∈ Set.Ioo (0 : ℝ) 1)
  (hh : 0 < h) (hh1 : h ≤ 1)
  (hfold : typeThreeFold lam root < 0)
  (harea : typeThreeArea lam root = typeFourArea lam h) :
  root < h := by
  have hsame := typeThreeArea_lt_typeFourArea hlam hh hh1
  apply lt_of_not_ge
  intro hle
  have hne : h ≠ root := by
    intro heq
    subst root
    linarith
  have hlt : h < root := lt_of_le_of_ne hle hne
  have hanti : StrictAntiOn (typeThreeArea lam) (Set.Icc h root) := by
    apply strictAntiOn_of_deriv_neg (convex_Icc h root)
    · intro y hy
      exact (hasDerivAt_typeThreeArea hlam
        (hh.trans_le hy.1) (hy.2.trans_lt hroot.2)
      ).continuousAt.continuousWithinAt
    · intro y hy
      rw [interior_Icc] at hy
      have hyreg : y ∈ Set.Ioo (0 : ℝ) 1 :=
        ⟨hh.trans hy.1, hy.2.trans hroot.2⟩
      rw [(hasDerivAt_typeThreeArea hlam hyreg.1 hyreg.2).deriv]
      exact div_neg_of_neg_of_pos
        (lt_trans (typeThreeFold_strictMonoOn hlam hyreg hroot hy.2) hfold)
        (pow_pos hyreg.1 3)
  have hc := hanti
  (show h ∈ Set.Icc h root from ⟨le_rfl, hle⟩)
  (show root ∈ Set.Icc h root from ⟨hle, le_rfl⟩) hlt
  rw [harea] at hc
  linarith
end LeanSuffixAnalytic

```

LeanSuffixAnalytic.typeThreeArea_half

Classification: main

Source: UniversalStationaryPair.lean:27-37

Declaration SHA-256: 315087e4cfd2ae0e85b582525152b7d3311b9e9d14e2dc9c72058614a85c134f

Mathematical statement. For every $\lambda > 1$, $A_3(1/2) = 2\pi(\lambda + 1)$.

The shape is zero, both relevant angles are semicircular, and the displacement vanishes, giving a finite exact endpoint for the bracketing argument.

```
theorem typeThreeArea_half {lam : ℝ} (hlam : 1 < lam) :
  typeThreeArea lam (1 / 2 : ℝ) = 2 * π * (lam + 1) := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hsqrt : sqrt (lam ^ 2) = lam := by
    rw [Real.sqrt_sq_eq_abs, abs_of_pos hlam_pos]
  simp [typeThreeArea, typeThreeAngle, typeThreeShape, typeThreeDelta,
    hsqrt, ne_of_gt hlam_pos, Real.arccos_zero]
  ring

/-- The finite type-(iv) closure area lies strictly below the type-(iii)
half-height area for every density above one. -/
```

LeanSuffixAnalytic.typeThreeArea_le_max_endpoints

Classification: main

Source: SameCurvatureArea.lean:394-463

Declaration SHA-256: f271402f8f772a214053257391493497c3abc162da11014d954aaf909e4fe1d5

Mathematical statement. For $\lambda > 1$ and $0 < a \leq x \leq b \leq 1$, $A_3(x) \leq \max\{A_3(a), A_3(b)\}$.

Increasing fold means the derivative can switch sign only from negative to positive; continuity permits the upper endpoint to be one.

```
theorem typeThreeArea_le_max_endpoints {lam a b x : ℝ}
  (hlam : 1 < lam) (ha : 0 < a) (hax : a ≤ x)
  (hxb : x ≤ b) (hb : b ≤ 1) :
  typeThreeArea lam x ≤
    max (typeThreeArea lam a) (typeThreeArea lam b) := by
  by_cases hx_one : x = 1
  · subst x
    have hb_one : b = 1 := le_antisymm hb hxb
    subst b
    exact le_max_right _ _
  have hx1 : x < 1 := lt_of_le_of_ne (hxb.trans hb) hx_one
  have hxreg : x ∈ Set.Ioo (0 : ℝ) 1 :=
    ⟨ha.trans_le hax, hx1⟩
  by_cases hxFold : typeThreeFold lam x ≤ 0
  · have hanti : AntitoneOn (typeThreeArea lam) (Set.Icc a x) := by
      apply antitoneOn_of_hasDerivWithinAt_nonpos
      (f' := fun y => typeThreeFold lam y / y ^ 3)
      (convex_Icc a x)
    · intro y hy
      have hyreg : y ∈ Set.Ioo (0 : ℝ) 1 :=
        ⟨ha.trans_le hy.1, hy.2.trans_lt hx1⟩
      exact (hasDerivAt_typeThreeArea
        hlam hyreg.1 hyreg.2).continuousAt.continuousWithinAt
    · intro y hy
      rw [interior_Icc] at hy
      have hyreg : y ∈ Set.Ioo (0 : ℝ) 1 :=
        ⟨ha.trans hy.1, hy.2.trans hx1⟩
      exact (hasDerivAt_typeThreeArea
        hlam hyreg.1 hyreg.2).hasDerivWithinAt
    · intro y hy
      rw [interior_Icc] at hy
```

```

have hyreg : y ∈ Set.Ioo (0 : ℝ) 1 :=
  ⟨ha.trans hy.1, hy.2.trans hx1⟩
have hfold := (typeThreeFold_strictMonoOn hlam).monotoneOn
  hyreg hxreg hy.2.le
exact div_nonpos_of_nonpos_of_nonneg
  (hfold.trans hxFold) (pow_pos hyreg.1 3).le
exact (hanti ⟨le_rfl, hax⟩ ⟨hax, le_rfl⟩ hax).trans
  (le_max_left _ _)
· have hxFoldPos : 0 < typeThreeFold lam x := lt_of_not_ge hxFold
have hmono : MonotoneOn (typeThreeArea lam) (Set.Icc x b) := by
  apply monotoneOn_of_hasDerivWithinAt_nonneg
    (f' := fun y => typeThreeFold lam y / y ^ 3)
    (convex_Icc x b)
· intro y hy
  by_cases hy_one : y = 1
  · subst y
    exact (typeThreeArea_continuousAt_one hlam).continuousWithinAt
  · have hyreg : y ∈ Set.Ioo (0 : ℝ) 1 :=
      ⟨hxreg.1.trans_le hy.1,
        lt_of_le_of_ne (hy.2.trans hb) hy_one⟩
    exact (hasDerivAt_typeThreeArea
      hlam hyreg.1 hyreg.2).continuousAt.continuousWithinAt
· intro y hy
  rw [interior_Icc] at hy
  have hyreg : y ∈ Set.Ioo (0 : ℝ) 1 :=
    ⟨hxreg.1.trans hy.1, hy.2.trans_le hb⟩
  exact (hasDerivAt_typeThreeArea
    hlam hyreg.1 hyreg.2).hasDerivWithinAt
· intro y hy
  rw [interior_Icc] at hy
  have hyreg : y ∈ Set.Ioo (0 : ℝ) 1 :=
    ⟨hxreg.1.trans hy.1, hy.2.trans_le hb⟩
  have hfold := (typeThreeFold_strictMonoOn hlam).monotoneOn
    hxreg hyreg hy.1.le
  exact div_nonneg (hxFoldPos.le.trans hfold)
    (pow_pos hyreg.1 3).le
exact (hmono ⟨le_rfl, hxb⟩ ⟨hxb, le_rfl⟩ hxb).trans
  (le_max_right _ _)

```

LeanSuffixAnalytic.typeThreeArea_lt_typeFourArea

Classification: main

Source: SameCurvatureArea.lean:225-265

Declaration SHA-256: 292616e61180882b74b7405ba97fd168e8ed5d3653115a72ad0fa3e78e338f65

Mathematical statement. For $\lambda > 1$ and $0 < h \leq 1$, $A_3(h) < A_4(h)$.

The proof compares the area numerators using increasing angle and displacement functions and a positive endpoint-controlled margin; the negative shape case uses reflection identities.

```

theorem typeThreeArea_lt_typeFourArea {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h ≤ 1) :
  typeThreeArea lam h < typeFourArea lam h := by
  rw [(area_normal_forms lam h).1, (area_normal_forms lam h).2]
  apply (div_lt_div_iff_of_pos_right (sq_pos_of_pos hh)).2
  let q := 2 * h - 1
  have hmem : h ∈ Set.Icc (0 : ℝ) 1 := ⟨hh.le, hh1⟩
  have hqb : -1 < q ∧ q ≤ h := by
    dsimp [q]
    constructor <|> linarith
  rcases le_total 0 q with hq | hq
  · have hqmem : q ∈ Set.Icc (0 : ℝ) 1 :=
      ⟨hq, hqb.2.trans hh1⟩
  have hF : angleGap lam q ≤ angleGap lam h :=
    (angleGap_strictMonoOn hlam).monotoneOn hqmem hmem hqb.2
  have hd : typeFourDelta lam q ≤ typeFourDelta lam h :=
    (typeFourDelta_strictMonoOn hlam).monotoneOn hqmem hmem hqb.2
  have hd0 := typeFourDelta_nonneg hlam hmem

```

```

have hk := angleGap_gt_mul_delta hlam hmem
change  $\pi + \text{angleGap } \text{lam } q + (2 * h + 1) * \text{typeFourDelta } \text{lam } q <$ 
 $\pi + 2 * \text{angleGap } \text{lam } h + 2 * h * \text{typeFourDelta } \text{lam } h$ 
nlinarith
· have hxmem :  $-q \in \text{Set.Icc } (0 : \mathbb{R}) 1 := \text{by}$ 
  constructor <=> dsimp [q] at hqb hq ⊢ <=> linarith
have hz :  $(0 : \mathbb{R}) \in \text{Set.Icc } (0 : \mathbb{R}) 1 := \text{by norm\_num}$ 
have hF :  $\text{angleGap } \text{lam } 0 < \text{angleGap } \text{lam } h :=$ 
   $\text{angleGap\_strictMonoOn } \text{lam } hz \text{ hmem } hh$ 
have hd0 :=  $\text{typeFourDelta\_nonneg } \text{lam } hz \text{ hmem } hh$ 
have hk :=  $\text{angleGap\_gt\_mul\_delta } \text{lam } hz \text{ hxmem}$ 
have hsF :=  $\text{angleGap\_neg } \text{lam } (-q)$ 
have hsd :=  $\text{typeFourDelta\_neg } \text{lam } (-q)$ 
have hF0 :=  $\text{angleGap\_neg } \text{lam } 0$ 
simp only [neg_neg] at hsF hsd
simp only [neg_zero] at hF0
change  $\pi + \text{angleGap } \text{lam } q + (2 * h + 1) * \text{typeFourDelta } \text{lam } q <$ 
 $\pi + 2 * \text{angleGap } \text{lam } h + 2 * h * \text{typeFourDelta } \text{lam } h$ 
dsimp [q] at hsF hsd hk ⊢
nlinarith

```

-- The support term whose concavity compares the two regular topologies at the same curvature. -/

LeanSuffixAnalytic.typeThreeArea_root_le_of_fold_neg

Classification: main

Source: LeanSuffixAnalytic.lean:954-973

Declaration SHA-256: a9a71e3bf8960b5c96a8fc59bf3fe2a372f70aa365bd91817e7ca128cb8520b9

Mathematical statement. A regular negative-fold root is the least root of its area level in $(0, 1]$.

Precisely, if $\lambda > 1$, $r \in (0, 1)$, $y \in (0, 1]$, $F_3(r) < 0$, and $A_3(y) = A_3(r)$, then $r \leq y$; a smaller root would force an earlier fold zero.

```

theorem typeThreeArea_root_le_of_fold_neg {lam root y : ℝ}
  (hlam : 1 < lam) (hroot : root ∈ Set.Ioo (0 : ℝ) 1)
  (hy : y ∈ Set.Ioc (0 : ℝ) 1) (hfold : typeThreeFold lam root < 0)
  (harea : typeThreeArea lam root = typeThreeArea lam y) :
  root ≤ y := by
  apply le_of_not_gt
  intro hyroot
  have hyreg : y ∈ Set.Ioo (0 : ℝ) 1 :=
    ⟨hy.1, hyroot.trans hroot.2⟩
  rcases typeThreeFold_zero_between_of_area_eq
    hlam hyreg ⟨hroot.1, hroot.2.le⟩ hyroot harea.symm with
    ⟨c, hc, hfoldC⟩
  have hcreg : c ∈ Set.Ioo (0 : ℝ) 1 :=
    ⟨hyreg.1.trans hc.1, hc.2.trans hroot.2⟩
  have hmono := typeThreeFold_strictMonoOn hlam hcreg hroot hc.2
  rw [hfoldC] at hmono
  linarith

```

-- If one regular point on the descending type-(iii) branch lies below a target area, the target is attained before that point. -/

LeanSuffixAnalytic.typeThreeArea_strictAntiOn_Icc

Classification: main

Source: EqualAreaEnvelope.lean:108-123

Declaration SHA-256: fd210f8a463759cd45da345eae2826fb3a7bdf59d29239656d68c8efc64fb7fc

Mathematical statement. If $\lambda > 1$, $a > 0$, $b < 1$, and $A_3' < 0$ on (a, b) , then A_3 is strictly decreasing on $[a, b]$.

The assertion also accommodates empty or singleton intervals; on a nontrivial regular interval it follows from continuity and the mean value theorem.

```
theorem typeThreeArea_strictAntiOn_Icc
  {lam a b : ℝ}
  (hlam : 1 < lam) (ha : 0 < a) (hb : b < 1)
  (hderiv : ∀ y ∈ Ioo a b,
    deriv (typeThreeArea lam) y < 0) :
  StrictAntiOn (typeThreeArea lam) (Icc a b) := by
  apply strictAntiOn_of_deriv_neg (convex_Icc a b)
  · intro y hy
    exact (typeThree_variational_hasDerivAt hlam
      (ha.trans_le hy.1) (hy.2.trans_lt hb)).1.continuousAt.continuousWithinAt
  · intro y hy
    rw [interior_Icc] at hy
    exact hderiv y hy

/-- A negative fold at the right endpoint forces strict decrease of the
type-(iii) area on the entire preceding regular interval. -/
```

LeanSuffixAnalytic.typeThreeArea_strictAntiOn_Icc_of_fold_neg_right

Classification: main

Source: EqualAreaEnvelope.lean:124-138

Declaration SHA-256: 1b1d7870499bdaf74cbcd036814fc1cce139b4cf2463cce4ba39b71ae55024b7

Mathematical statement. If $\lambda > 1$, $a > 0$, $0 < b < 1$, and $F_3(b) < 0$, then A_3 is strictly decreasing on $[a, b]$.

Increasing fold and the derivative identity give $A_3'(x) < 0$ throughout the preceding interval.

```
theorem typeThreeArea_strictAntiOn_Icc_of_fold_neg_right
  {lam a b : ℝ} (hlam : 1 < lam) (ha : 0 < a) (hb : b < 1)
  (hb_pos : 0 < b) (hfoldb : typeThreeFold lam b < 0) :
  StrictAntiOn (typeThreeArea lam) (Icc a b) := by
  apply typeThreeArea_strictAntiOn_Icc hlam ha hb
  intro y hy
  rw [(hasDerivAt_typeThreeArea hlam
    (ha.trans hy.1) (hy.2.trans hb)).deriv]
  apply div_neg_of_neg_of_pos _ (pow_pos (ha.trans hy.1) 3)
  exact (typeThreeFold_strictMonoOn hlam
    (ha.trans hy.1, hy.2.trans hb) (hb_pos, hb) hy.2).trans hfoldb

/-- Certificate-facing specialization for a root contained in a compact
curvature interval. Uniform derivative negativity supplies both uniqueness
and continuity of the selected equal-area branch. -/
```

LeanSuffixAnalytic.typeThreeArea_tendsto_atRight_zero

Classification: main

Source: LeanSuffixAnalytic.lean:828-871

Declaration SHA-256: e82ffa97f750ea7c64d3d22eebba58e03fe2fc0f731297b9787ae711ded5a49a

Mathematical statement. For every $\lambda > 1$, $A_3(h) \rightarrow +\infty$ as $h \downarrow 0$.

The numerator tends to $\lambda \arccos(-1/\lambda) + \sqrt{\lambda^2 - 1} / \lambda > 0$, while the denominator tends to zero through positive values.

```
theorem typeThreeArea_tendsto_atRight_zero {lam : ℝ} (hlam : 1 < lam) :
  Tendsto (typeThreeArea lam) (ℳ[>] 0) atTop := by
  let numerator : ℝ → ℝ := fun h =>
    typeThreeAngle lam h + π / 2 +
    (typeThreeShape h + 2) * typeThreeDelta lam h
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hrad : 0 < lam ^ 2 - 1 := by nlinarith
  have hnum_cont : ContinuousAt numerator 0 := by
```

```

dsimp [numerator, typeThreeAngle, typeThreeDelta, typeThreeShape]
fun_prop
have hnum_zero : numerator 0 =
  lam * arccos (-1 / lam) + sqrt (lam ^ 2 - 1) / lam := by
  simp [numerator, typeThreeAngle, typeThreeDelta, typeThreeShape]
  ring
have hC : 0 < numerator 0 := by
  rw [hnum_zero]
  exact add_pos_of_nonneg_of_pos
    (mul_nonneg hlam_pos.le (Real.arccos_nonneg _))
    (div_pos (Real.sqrt_pos.2 hrad) hlam_pos)
have hnum_tendsto : Tendsto numerator (ℕ[>] (0 : ℝ)) (ℕ (numerator 0)) :=
  hnum_cont.tendsto.mono_left inf_le_left
have hnum_lb : ∀f h in ℕ[>] (0 : ℝ), numerator 0 / 2 < numerator h :=
  hnum_tendsto.eventually_const_lt (by linarith)
have hsmall : ∀f h in ℕ[>] (0 : ℝ), h < 1 :=
  (eventually_lt_nhds (show (0 : ℝ) < 1 by norm_num)).filter_mono inf_le_left
have hlower : Tendsto (fun h : ℝ => numerator 0 / 2 * h-1)
  (ℕ[>] 0) atTop :=
  tendsto_inv_nhdsGT_zero.const_mul_atTop (by linarith)
apply tendsto_atTop_mono' (ℕ[>] (0 : ℝ)) _ hlower
filter_upwards [hnum_lb, self_mem_nhdsWithin, hsmall] with h hnum hh hlt
have hhpos : 0 < h := hh
have hsq_pos : 0 < h ^ 2 := sq_pos_of_pos hhpos
have hsquare_le : h ^ 2 ≤ h := by nlinarith
calc
  numerator 0 / 2 * h-1 = numerator 0 / 2 / h := by
    simp only [div_eq_mul_inv]
  _ ≤ numerator 0 / 2 / h ^ 2 :=
    div_le_div_of_nonneg_left (by linarith) hsq_pos hsquare_le
  _ ≤ numerator h / h ^ 2 :=
    div_le_div₀ (by linarith) hnum.le hsq_pos le_rfl
  _ = typeThreeArea lam h := by rfl

/-- For `lam > 1`, the regular type-(iii) fold is strictly increasing
throughout the admissible height interval. -/

```

LeanSuffixAnalytic.typeThreeDelta

Classification: main

Source: LeanSuffixAnalytic.lean:39-43

Declaration SHA-256: f96722ed089f4e5911c6013a0aa601eb4080090036491702cd0fa78e3e31113f

Mathematical statement. The type-iii radical displacement is $D(q(h)) = \sqrt{\lambda^2 - q(h)^2} / \lambda - \sqrt{1 - q(h)^2}$.

On the regular geometric domain this is $\sin \alpha - \sin \beta$.

```

def typeThreeDelta (lam h : ℝ) : ℝ :=
  sqrt (lam ^ 2 - typeThreeShape h ^ 2) / lam -
  sqrt (1 - typeThreeShape h ^ 2)

/-- Exact regular type-(iii) weighted area used by the suffix comparison. -/

```

LeanSuffixAnalytic.typeThreeFold

Classification: main

Source: LeanSuffixAnalytic.lean:71-77

Declaration SHA-256: de466d8508a0def3484419e5a98402f15940017ff9d116432fb67c21c1d22c86

Mathematical statement. The type-iii fold is

$$F_3(h) = 4h[(1+q)/\sqrt{1-q^2} - (\lambda^2+q)/(\lambda\sqrt{\lambda^2-q^2})] - 2[B(q) + \pi/2 + D(q)], \text{ with } q = 2h - 1.$$

On $\lambda > 1, 0 < h < 1$, the identity $A_3'(h) = F_3(h)/h^3$ makes this the numerator controlling the sign of the area derivative.

```

def typeThreeFold (lam h : ℝ) : ℝ :=
  4 * h * ((1 + typeThreeShape h) / sqrt (1 - typeThreeShape h ^ 2) -
    (lam ^ 2 + typeThreeShape h) /
    (lam * sqrt (lam ^ 2 - typeThreeShape h ^ 2))) -
  2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h)

/-- The stationary-fold equation for the regular type-(iv) profile. -/

```

LeanSuffixAnalytic.typeThreeFold_neg_of_equalArea_of_lt

Classification: main

Source: SameCurvatureArea.lean:554-585

Declaration SHA-256: b45c2e12676c7119b05c3219fa4f0e77bf807a7780d13de5a358f0b41a3fa64d

Mathematical statement. If $\lambda > 1$, $0 < r < h < 1$, and $A_3(r) = A_4(h)$, then $F_3(r) < 0$.

The same-curvature area inequality gives $A_3(h) < A_3(r)$; the mean value theorem and increasing fold force negativity already at r .

```

theorem typeThreeFold_neg_of_equalArea_of_lt
  {lam root h : ℝ}
  (hlam : 1 < lam)
  (hroot : root ∈ Set.Ioo (0 : ℝ) 1)
  (hh : h ∈ Set.Ioo (0 : ℝ) 1)
  (hroot_lt : root < h)
  (harea : typeThreeArea lam root = typeFourArea lam h) :
  typeThreeFold lam root < 0 := by
  have hsame := typeThreeArea_lt_typeFourArea hlam hh.1 hh.2.le
  have harea_lt : typeThreeArea lam h < typeThreeArea lam root := by
    rw [harea]
    exact hsame
  rcases typeThreeArea_centered_meanValue hlam hroot hh with
    ⟨ξ, hξ, hmean⟩
  have hξIcc : ξ ∈ Set.Icc root h := by
    simpa [Set.uIcc_of_le hroot_lt.le] using hξ
  have hξreg : ξ ∈ Set.Ioo (0 : ℝ) 1 :=
    ⟨hroot.1.trans_le hξIcc.1, hξIcc.2.trans_lt hh.2⟩
  have hfoldξ : typeThreeFold lam ξ < 0 := by
    apply lt_of_not_ge
    intro hnonneg
    have hquot : 0 ≤ typeThreeFold lam ξ / ξ ^ 3 :=
      div_nonneg hnonneg (pow_nonneg hξreg.1.le 3)
    have hterm : 0 ≤ typeThreeFold lam ξ / ξ ^ 3 * (h - root) :=
      mul_nonneg hquot (sub_nonneg.mpr hroot_lt.le)
    linarith
  exact lt_of_le_of_lt
    ((typeThreeFold_strictMonoOn hlam).monotoneOn
      hroot hξreg hξIcc.1) hfoldξ
  /-- A regular negative-fold type-(iii) root at the type-(iv) area lies strictly
  below the type-(iv) curvature. This supplies the order invariant required by
  the descending equal-area envelope. -/

```

LeanSuffixAnalytic.typeThreeFold_strictMonoOn

Classification: main

Source: LeanSuffixAnalytic.lean:872-893

Declaration SHA-256: 548db5145f9bcf2011e54d3648e735633b97c51f2514a21b4368c3304bb06a32

Mathematical statement. For every $\lambda > 1$, F_3 is strictly increasing on $(0, 1)$.

Its derivative is $16h^2K(q(h)) > 0$; this controls the entire descending branch, not a chosen numerical slab.

```

theorem typeThreeFold_strictMonoOn {lam : ℝ} (hlam : 1 < lam) :
  StrictMonoOn (typeThreeFold lam) (Set.Ioo 0 1) := by

```

```

apply strictMonoOn_of_hasDerivWithinAt_pos (convex_Ioo (0 : ℝ) 1)
· intro h hh
  exact
    (hasDerivAt_typeThreeFold hlam hh.1 hh.2).continuousAt.continuousWithinAt
· intro h hh
  rw [interior_Ioo] at hh
  exact (hasDerivAt_typeThreeFold hlam hh.1 hh.2).hasDerivWithinAt
· intro h hh
  rw [interior_Ioo] at hh
  have hq_low : -1 < typeThreeShape h := by
    unfold typeThreeShape
    linarith [hh.1]
  have hq_high : typeThreeShape h < 1 := by
    unfold typeThreeShape
    linarith [hh.2]
  exact mul_pos (mul_pos (by norm_num) (sq_pos_of_pos hh.1))
    (radicalCubeGap_pos hlam hq_low hq_high)

/-- The regular type-(iii) fold has at most one zero in its admissible height
interval. -/

```

LeanSuffixAnalytic.typeThreeFold_zero_between_of_area_eq

Classification: main

Source: LeanSuffixAnalytic.lean:905-925

Declaration SHA-256: 314dbeb5db22bb018c3bb2b871eb2f76a4711df985ead7c117e1cc5036bed173

Mathematical statement. If $\lambda > 1$, $0 < a < b \leq 1$, and $A_3(a) = A_3(b)$, then some $c \in (a, b)$ satisfies $F_3(c) = 0$.

Continuity of the area at the possible right endpoint and Rolle's theorem give an interior derivative zero, where the positive denominator can be cleared.

```

theorem typeThreeFold_zero_between_of_area_eq {lam a b : ℝ}
  (hlam : 1 < lam) (ha : a ∈ Set.Ioo (0 : ℝ) 1)
  (hb : b ∈ Set.Ioc (0 : ℝ) 1) (hab : a < b)
  (harea : typeThreeArea lam a = typeThreeArea lam b) :
  ∃ c ∈ Set.Ioo a b, typeThreeFold lam c = 0 := by
  have hcont : ContinuousOn (typeThreeArea lam) (Set.Icc a b) := by
    intro x hx
    have hx0 : x ≠ 0 := ne_of_gt (ha.1.trans_le hx.1)
    unfold typeThreeArea typeThreeAngle typeThreeDelta typeThreeShape
    fun_prop (disch := simp_all)
  rcases exists_deriv_eq_zero hab hcont harea with ⟨c, hc, hzero⟩
  have hcreg : c ∈ Set.Ioo (0 : ℝ) 1 :=
    ⟨ha.1.trans hc.1, hc.2.trans_le hb.2⟩
  rw [(hasDerivAt_typeThreeArea hlam hcreg.1 hcreg.2).deriv] at hzero
  refine ⟨c, hc, ?_⟩
  exact (div_eq_zero_iff.mp hzero).resolve_right
    (pow_ne_zero 3 (ne_of_gt hcreg.1))

/-- A horizontal level meets the type-(iii) area profile at no more than two
heights in `0 < h ≤ 1`. This includes a possible third height at the closure
endpoint. -/

```

LeanSuffixAnalytic.typeThreePerimeter

Classification: main

Source: LeanSuffixAnalytic.lean:49-52

Declaration SHA-256: 22e35e36f9c0930c9f9233f0004c686c48f6f282298a578def5a6e6bba11f34e

Mathematical statement. The type-iii scalar perimeter is $P_3(h) = 2[B(q(h)) + \pi/2 + D(q(h))]/h$.

Its geometric realization includes the bottom density-one segment as well as the three arcs.

```
def typeThreePerimeter (lam h : ℝ) : ℝ :=
  2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h) / h

/-- The angle contribution in the exact regular type-(iv) formulas. -/
```

LeanSuffixAnalytic.typeThreePerimeter_deriv_eq_height_mul_area_deriv

Classification: main

Source: LeanSuffixAnalytic.lean:640-648

Declaration SHA-256: 7607f8124dad06702d520d2b8e20b8eab5117f85a2007d935f98d72479aad4e2

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, $P_3'(h) = hA_3'(h)$.

This derivative equality is extracted from the stronger differentiability statement; it expresses curvature as the marginal perimeter cost along the family.

```
theorem typeThreePerimeter_deriv_eq_height_mul_area_deriv {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  deriv (typeThreePerimeter lam) h =
    h * deriv (typeThreeArea lam) h :=
  (typeThree_variational_hasDerivAt hlam hh hh1).2.deriv

/-- Explicit numerator for the type-(iii) area derivative. This form is
designed for interval certificates: after replacing the two radicals by their
angle parametrizations, the numerator is exactly the checker quantity `K3`. -/
```

LeanSuffixAnalytic.typeThreePerimeter_lt_typeFourPerimeter_of_equalArea

Classification: main

Source: SameCurvatureArea.lean:495-553

Declaration SHA-256: e7b6d671984a991fd2b92d8c073a4e4e44f40679b3766f18a0ac3843895845e6

Mathematical statement. If $\lambda > 1$, $1/2 < r < h < 1$, and $A_3(r) = A_4(h)$, then $P_3(r) < P_4(h)$.

The area endpoint bound makes H_V nondecreasing on $[r, h]$, while the same-height support comparison makes its final value strictly less than $P_4(h)$.

```
theorem typeThreePerimeter_lt_typeFourPerimeter_of_equalArea
  {lam r h : ℝ}
  (hlam : 1 < lam) (hr : 1 / 2 < r) (hrh : r < h) (hh1 : h < 1)
  (harea : typeThreeArea lam r = typeFourArea lam h) :
  typeThreePerimeter lam r < typeFourPerimeter lam h := by
  have hrpos : 0 < r := lt_trans (by norm_num) hr
  have hhpos : 0 < h := hrpos.trans hrh
  have hsame := typeThreeArea_lt_typeFourArea hlam hhpos hh1.le
  have hareaHlt : typeThreeArea lam h < typeThreeArea lam r := by
    rw [harea]
    exact hsame
  have hsublevel : ∀ x ∈ Set.Icc r h,
    typeThreeArea lam x ≤ typeThreeArea lam r := by
    intro x hx
    have hbound := typeThreeArea_le_max_endpoints
      hlam hrpos hx.1 hx.2 hh1.le
    simpa [max_eq_left hareaHlt.le] using hbound
  have hsupport := typeThree_support_lt_typeFour_support
    hlam (hr.trans hrh) hh1
  have hmono : MonotoneOn
    (typeThreeAreaSupport lam (typeThreeArea lam r))
    (Set.Icc r h) := by
    apply monotoneOn_of_hasDerivWithinAt_nonneg
    (f' := fun x => typeThreeArea lam r - typeThreeArea lam x)
    (convex_Icc r h)
  · intro x hx
    have hxreg : x ∈ Set.Ioo (0 : ℝ) 1 :=
```

```

    (hrpos.trans_le hx.1, hx.2.trans_lt hh1)
    exact (typeThreeAreaSupport_hasDerivAt
      hlam hxreg.1 hxreg.2).continuousAt.continuousWithinAt
  · intro x hx
    rw [interior_Icc] at hx
    have hxreg : x ∈ Set.Ioo (0 : ℝ) 1 :=
      (hrpos.trans hx.1, hx.2.trans hh1)
    exact (typeThreeAreaSupport_hasDerivAt
      hlam hxreg.1 hxreg.2).hasDerivWithinAt
  · intro x hx
    rw [interior_Icc] at hx
    exact sub_nonneg.mpr (hsublevel x (hx.1.le, hx.2.le))
have hends := hmono
  (show r ∈ Set.Icc r h from (le_rfl, hrh.le))
  (show h ∈ Set.Icc r h from (hrh.le, le_rfl)) hrh.le
have hright :
  typeThreeAreaSupport lam (typeThreeArea lam r) h <
    typeFourPerimeter lam h := by
    unfold typeThreeAreaSupport
    rw [harea]
    linarith
calc
  typeThreePerimeter lam r =
    typeThreeAreaSupport lam (typeThreeArea lam r) r := by
      unfold typeThreeAreaSupport
      ring
  _ ≤ typeThreeAreaSupport lam (typeThreeArea lam r) h := hends
  _ < typeFourPerimeter lam h := hright

```

*-- Conversely, an equal-area type-(iii) curvature below the compared
type-(iv) curvature lies on the negative-fold branch. -/*

LeanSuffixAnalytic.typeThreeShape

Classification: main

Source: LeanSuffixAnalytic.lean:32-34

Declaration SHA-256: 9ef91461acc972465b98decfe31d601788f1354eb0ffbb8d094be6f5c46ca593

Mathematical statement. The type-iii shape parameter is $q(h) = 2h - 1$.

It maps regular curvature $0 < h < 1$ to the principal trigonometric domain $-1 < q < 1$.

```

def typeThreeShape (h : ℝ) : ℝ := 2 * h - 1

-- The angle contribution in the exact type-(iii) formulas. -/

```

LeanSuffixAnalytic.typeThree_support_lt_typeFour_support

Classification: main

Source: SameCurvatureArea.lean:362-383

Declaration SHA-256: 3b4307e15a93ff15bacde69d80f20dc52b17ba9ab4d81965f114554585e3a373

Mathematical statement. For $\lambda > 1$ and $1/2 < h < 1$, $P_3(h) - hA_3(h) < P_4(h) - hA_4(h)$.

Concavity at the exact midpoint and the strict endpoint margin make the support identity negative, and $h > 0$ preserves the sign.

```

theorem typeThree_support_lt_typeFour_support {lam h : ℝ}
  (hlam : 1 < lam) (hh : 1 / 2 < h) (hh1 : h < 1) :
  typeThreePerimeter lam h - h * typeThreeArea lam h <
    typeFourPerimeter lam h - h * typeFourArea lam h := by
  have hhpos : 0 < h := lt_trans (by norm_num) hh
  have hmid := typeFourSupport_midpoint hlam hh hh1
  have hend := typeFourSupport_one_gt_halfPi hlam

```

```

have hnegative :
  typeFourSupport lam (2 * h - 1) +  $\pi$  / 2 -
    2 * typeFourSupport lam h < 0 := by
  nlinarith
rw [← sameHeight_support_identity lam h (ne_of_gt hhpos)] at hnegative
apply lt_of_not_ge
intro hnonneg
have hdiff : 0 ≤
  (typeThreePerimeter lam h - h * typeThreeArea lam h) -
  (typeFourPerimeter lam h - h * typeFourArea lam h) := by
  linarith
exact (not_lt_of_ge (mul_nonneg hhpos.le hdiff)) hnegative

/-- The actual type-(iii) area is continuous at the closed height endpoint.
No endpoint fold value is used. -/

```

LeanSuffixAnalytic.typeThree_variational_hasDerivAt

Classification: main

Source: LeanSuffixAnalytic.lean:554-639

Declaration SHA-256: 727f5f60431aed129e76617edde1e3ccdf8d18a1ae71fbf80ca33147d09b9cfa

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, both type-iii scalar functions are differentiable and $P_3'(h) = hA_3'(h)$.

The statement records actual differentiability as well as the variational derivative identity used in the support and envelope arguments.

```

theorem typeThree_variational_hasDerivAt {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeArea lam) (deriv (typeThreeArea lam) h) h ^
  HasDerivAt (typeThreePerimeter lam)
    (h * deriv (typeThreeArea lam) h) h := by
  let q := typeThreeShape h
  let D := typeThreeDelta lam h
  let p := 2 * (1 / sqrt (1 - q ^ 2) -
    lam / sqrt (lam ^ 2 - q ^ 2))
  let dp := 2 * q * (1 / sqrt (1 - q ^ 2) -
    1 / (lam * sqrt (lam ^ 2 - q ^ 2)))
  have hh_ne : h ≠ 0 := ne_of_gt hh
  have hshape := hasDerivAt_typeThreeShape h
  have hangle : HasDerivAt (typeThreeAngle lam) p h := by
    simpa only [p, q] using hasDerivAt_typeThreeAngle hlam hh hh1
  have hdelta : HasDerivAt (typeThreeDelta lam) dp h := by
    simpa only [dp, q] using hasDerivAt_typeThreeDelta hlam hh hh1
  have hq_arg : 0 < 1 - q ^ 2 := by
    dsimp only [q, typeThreeShape]
    nlinarith
  have hd_arg : 0 < lam ^ 2 - q ^ 2 := by
    have hlam_sq : 1 < lam ^ 2 := by nlinarith
    have hq_sq : q ^ 2 < 1 := by
      dsimp only [q, typeThreeShape]
      nlinarith
    linarith
  have hlam_ne : lam ≠ 0 := ne_of_gt (lt_trans (by norm_num) hlam)
  have hq_ne : sqrt (1 - q ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hq_arg)
  have hd_ne : sqrt (lam ^ 2 - q ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  have hkey : p - q * dp + 2 * D = 0 := by
    dsimp only [p, dp, D, typeThreeDelta]
    change
      2 * (1 / sqrt (1 - q ^ 2) - lam / sqrt (lam ^ 2 - q ^ 2)) -
      q * (2 * q * (1 / sqrt (1 - q ^ 2) -
        1 / (lam * sqrt (lam ^ 2 - q ^ 2)))) +
      2 * (sqrt (lam ^ 2 - q ^ 2) / lam - sqrt (1 - q ^ 2)) = 0
  field_simp [hlam_ne, hq_ne, hd_ne]
  ring_nf
  rw [Real.sq_sqrt hq_arg.le, Real.sq_sqrt hd_arg.le]

```

```

ring
have hB := hangle.add_const (π / 2)
have hareaRaw :=
  (hB.add ((hshape.add_const 2).mul hdelta)).div
  ((hasDerivAt_id h).pow 2) (pow_ne_zero 2 hh_ne)
have harea : HasDerivAt (typeThreeArea lam)
  (((p + (2 * D + (q + 2) * dp)) * h ^ 2 -
    (typeThreeAngle lam h + π / 2 + (q + 2) * D) * (2 * h)) /
    (h ^ 2) ^ 2) h := by
  simpa only [typeThreeArea, q, D, id_eq] using!
  hareaRaw.congr_deriv (by
    norm_num [id_eq, Pi.add_apply, Pi.mul_apply, Pi.pow_apply])
have hperimeterRaw :=
  ((hB.add hdelta).const_mul (2 : ℝ)).div (hasDerivAt_id h) hh_ne
have hperimeter : HasDerivAt (typeThreePerimeter lam)
  (((2 * (p + dp)) * h -
    2 * (typeThreeAngle lam h + π / 2 + D)) / h ^ 2) h := by
  simpa only [typeThreePerimeter, D, id_eq] using!
  hperimeterRaw.congr_deriv (by
    norm_num [id_eq, Pi.add_apply, Pi.mul_apply, Pi.pow_apply])
refine ⟨harea.congr_deriv harea.deriv.symm, hperimeter.congr_deriv ?_⟩
rw [harea.deriv]
have halgebra : ∀ (q D p dp B h : ℝ), h ≠ 0 → q = 2 * h - 1 →
  p - q * dp + 2 * D = 0 →
  ((2 * (p + dp)) * h - 2 * (B + D)) / h ^ 2 =
  h * (((p + (2 * D + (q + 2) * dp)) * h ^ 2 -
    (B + (q + 2) * D) * (2 * h)) / (h ^ 2) ^ 2) := by
  intro q D p dp B h hh_ne hq hkey
  calc
    ((2 * (p + dp)) * h - 2 * (B + D)) / h ^ 2 =
    h * (((p + (2 * D + (q + 2) * dp)) * h ^ 2 -
      (B + (q + 2) * D) * (2 * h)) / (h ^ 2) ^ 2) +
    (p - q * dp + 2 * D) / h := by
      rw [hq]
      field_simp [hh_ne]
      ring
    _ = h * (((p + (2 * D + (q + 2) * dp)) * h ^ 2 -
      (B + (q + 2) * D) * (2 * h)) / (h ^ 2) ^ 2) := by
      rw [hkey]
      ring
exact halgebra
q D p dp (typeThreeAngle lam h + π / 2) h hh_ne (by
  simp only [q, typeThreeShape]) hkey

/-- Exact type-(iii) variational identity on the regular domain. -/

```

OneSidedCircularCap

Classification: main

Source: CMVGeometry.lean:100-112

Declaration SHA-256: a9dd2ba7ee653be367b6a2072de1384c3362b0214117e3aecceae763bc6f8403

Mathematical statement. A one-sided circular cap is specified by $L > 0$, $0 < \theta < \pi$, a chord midpoint (m, b) , and a choice of side.

The radius, center, carrier, and boundary arc are derived. The interval includes minor, semicircular, and major caps.

```

structure OneSidedCircularCap where
  chord : ℝ
  theta : ℝ
  midpointX : ℝ
  baseY : ℝ
  side : CapSide
  chord_pos : 0 < chord
  theta_pos : 0 < theta
  theta_lt_pi : theta < π

namespace OneSidedCircularCap

```

```
-- Radius determined by chord length and half-central-angle. -/
```

OneSidedCircularCap.arcLength

Classification: main

Source: CMVGeometry.lean:167-170

Declaration SHA-256: 60daa6fa20eb20410aecf4ceebd5a41e50c48a0eb958e6b55f1bcb20cd85ff25

Mathematical statement. The cap's scalar arc length is $L\ell(\theta)$.

The chord-radius relation turns this into $2R\theta$; later results identify it with Euclidean Hausdorff length.

```
def arcLength (c : OneSidedCircularCap) : ℝ := c.chord * ell c.theta

/-- The branch-correct cap with prescribed positive normalized area. This is
the reusable geometric constructor behind the candidate-specific replacement. -/
```

OneSidedCircularCap.arcTrace

Classification: main

Source: CMVGeometry.lean:656-661

Declaration SHA-256: 5e3edb81f4f16ef8dbff7c15d5756a176b1f99c3bd5fb86ce703134a0a727a89

Mathematical statement. The cap arc trace is the circle $\|p - c\|_2^2 = R^2$ restricted to the chosen closed side of the base line.

It excludes the interior of the chord but includes its endpoints; it is a set-theoretic boundary piece.

```
def arcTrace (c : OneSidedCircularCap) : Set PlanePoint :=
  {p | c.radiusSquaredAt p = c.radius ^ 2 ^
    match c.side with
    | .upper => c.baseY ≤ p.2
    | .lower => p.2 ≤ c.baseY}
```

OneSidedCircularCap.carrier

Classification: main

Source: CMVGeometry.lean:146-152

Declaration SHA-256: e0644dd0b5f330958b79efbb3810eb1cc2f4356e5db251a95736b532b75bfcf1

Mathematical statement. The cap carrier is its closed disk intersected with $y \geq b$ for an upper cap or $y \leq b$ for a lower cap.

The disk is centered as specified above and has radius R . This is the actual planar set whose area and arc trace are computed.

```
def carrier (c : OneSidedCircularCap) : Set PlanePoint :=
  {p | c.radiusSquaredAt p ≤ c.radius ^ 2 ^
    match c.side with
    | .upper => c.baseY ≤ p.2
    | .lower => p.2 ≤ c.baseY}

/-- The open cap interior. Endpoints and the chord are deliberately absent. -/
```

OneSidedCircularCap.euclideanArea

Classification: main

Source: CMVGeometry.lean:164-166

Declaration SHA-256: 5c476b88f35c9e9f6afc55ed7ed9ef6be1395a7b5a3003452ff8691443c49da

Mathematical statement. The cap's scalar Euclidean area is $L^2a(\theta)$.

Since $L = 2R\sin\theta$, this equals $R^2(\theta - \sin\theta\cos\theta)$; equality with carrier volume is a theorem.

```
def euclideanArea (c : OneSidedCircularCap) : ℝ := c.chord ^ 2 * area c.theta

/-- Euclidean length of the selected circular arc. -/
```

OneSidedCircularCap.frontier_carrier

Classification: main

Source: CMVGeometry.lean:697-761

Declaration SHA-256: d9cb2c749721b51457faae17a95033cf908527d3fdc652ebedacfbfb2292d579d

Mathematical statement. The complete cap frontier is the union of its arc trace and chord carrier.

No other boundary components occur; the endpoints may belong to both pieces.

```
theorem frontier_carrier (c : OneSidedCircularCap) :
  frontier c.carrier = arcTrace c ∪ c.chordCarrier := by
  apply Subset.antisymm
  · intro p hp
    have hpc : p ∈ c.carrier :=
      c.isClosed_carrier.frontier_subset hp
    by_cases heq : c.radiusSquaredAt p = c.radius ^ 2
    · exact Or.inl (heq, hpc.2)
    have hlt : c.radiusSquaredAt p < c.radius ^ 2 :=
      lt_of_le_of_ne hpc.1 heq
    cases hside : c.side with
    | upper =>
      have hsidep : c.baseY ≤ p.2 := by
        simpa [OneSidedCircularCap.carrier, hside] using hpc.2
      by_cases hy : p.2 = c.baseY
      · exact Or.inr
        (c.mem_chordCarrier_of_mem_carrier_of_eq_base hpc hy)
      · exfalso
        have hi : p ∈ c.interiorCarrier := by
          simpa [OneSidedCircularCap.interiorCarrier, hside] using
            (hlt, lt_of_le_of_ne hsidep (Ne.symm hy))
        exact (mem_frontier_iff_notMem_interior hpc).mp hp
          (interiorCarrier_subset_interior c hi)
    | lower =>
      have hsidep : p.2 ≤ c.baseY := by
        simpa [OneSidedCircularCap.carrier, hside] using hpc.2
      by_cases hy : p.2 = c.baseY
      · exact Or.inr
        (c.mem_chordCarrier_of_mem_carrier_of_eq_base hpc hy)
      · exfalso
        have hi : p ∈ c.interiorCarrier := by
          simpa [OneSidedCircularCap.interiorCarrier, hside] using
            (hlt, lt_of_le_of_ne hsidep hy)
        exact (mem_frontier_iff_notMem_interior hpc).mp hp
          (interiorCarrier_subset_interior c hi)
  · rintro p (hp | hp)
    · have hpc : p ∈ c.carrier := (hp.1.le, hp.2)
      rw [mem_frontier_iff_notMem_interior hpc]
      intro hi
      apply circle_eq_not_mem_interior_disk
        c.center.1 c.center.2 c.radius c.radius_pos p
        (by simpa [OneSidedCircularCap.radiusSquaredAt] using hp.1)
```

```

    apply interior_mono _ hi
    intro q hq
    exact hq.1
  · have hpc := c.chordCarrier_subset_carrier hp
    rw [mem_frontier_iff_notMem_interior hpc]
    intro hi
    cases hside : c.side with
    | upper =>
      apply second_eq_not_mem_interior_ge c.baseY p hp.1
      apply interior_mono _ hi
      intro q hq
      simpa [OneSidedCircularCap.carrier, hside] using hq.2
    | lower =>
      apply second_eq_not_mem_interior_le c.baseY p hp.1
      apply interior_mono _ hi
      intro q hq
      simpa [OneSidedCircularCap.carrier, hside] using hq.2

end OneSidedCircularCap

namespace StripCore

/-- The exact left circular trace of a strip core. -/

```

OneSidedCircularCap.volume_carrier

Classification: main

Source: CMVGeometry.lean:1996-2028

Declaration SHA-256: 0d51f9d06b45af9eac50b2062d616aee51a260200ad5a2e96d2ecbe8e98be85c

Mathematical statement. The planar volume of every cap carrier equals its scalar Euclidean area.

Thus the coordinate disk-half-plane intersection has volume $L^2a(\theta) = R^2(\theta - \sin\theta\cos\theta)$, not merely an assigned numerical label.

```

theorem volume_carrier (c : OneSidedCircularCap) :
  volume c.carrier = ENNReal.ofReal c.euclideanArea := by
  have hreal :
    c.radius ^ 2 * (c.theta - sin c.theta * cos c.theta) =
      c.euclideanArea := by
    rw [euclideanArea, area, radius]
    field_simp [ne_of_gt c.sin_theta_pos]
    ring
  cases hside : c.side with
  | upper =>
    calc
      volume c.carrier =
        ENNReal.ofReal
          (c.radius ^ 2 *
            (c.theta - sin c.theta * cos c.theta)) := by
      simpa [carrier, radiusSquaredAt, center, hside] using
        volume_upper_circular_segment c.midpointX
          (c.baseY - c.radius * cos c.theta) c.radius c.theta
          c.radius_pos c.theta_pos.le c.theta_lt_pi.le
    _ = ENNReal.ofReal c.euclideanArea := congrArg ENNReal.ofReal hreal
  | lower =>
    calc
      volume c.carrier =
        ENNReal.ofReal
          (c.radius ^ 2 *
            (c.theta - sin c.theta * cos c.theta)) := by
      simpa [carrier, radiusSquaredAt, center, hside] using
        volume_lower_circular_segment c.midpointX
          (c.baseY + c.radius * cos c.theta) c.radius c.theta
          c.radius_pos c.theta_pos.le c.theta_lt_pi.le
    _ = ENNReal.ofReal c.euclideanArea := congrArg ENNReal.ofReal hreal

end OneSidedCircularCap

```

StripCore

Classification: main

Source: CMVGeometry.lean:416-425

Declaration SHA-256: a32ba501747725a0a1307cf7bd2abbffd387c8bc9d810f24cf348c7c04513aad

Mathematical statement. A strip core is specified by $L > 0$ and $0 < \kappa \leq 1$.

Its rectangle and circular side pieces are derived from the chord and curvature, including the semicircular endpoint $\kappa = 1$.

```
structure StripCore where
  chord : ℝ
  curvature : ℝ
  chord_pos : 0 < chord
  curvature_pos : 0 < curvature
  curvature_le_one : curvature ≤ 1

namespace StripCore

/-- Common radius of both strip arcs. -/
```

StripCore.boundaryArcLength

Classification: main

Source: CMVGeometry.lean:457-459

Declaration SHA-256: 74e634700740d7651d4ed773b1cb3cb71b11c7eedacd0c71b6a84a0e6ec43665

Mathematical statement. The total length of the core's two curved sides is $4\ell(\phi)$.

Each side arc has chord two and length $2\ell(\phi) = 2R\phi$; the interface chords are not counted here.

```
def boundaryArcLength (c : StripCore) : ℝ :=
  4 * ell c.sideAngle
```

StripCore.carrier

Classification: main

Source: CMVGeometry.lean:449-452

Declaration SHA-256: 05f714bedd5ca3f09fd1dac168964dc190e4830b8099d994e590458d8167e259

Mathematical statement. The strip-core carrier is the union of its rectangle and its two side pieces.

The construction defines an actual closed coordinate set within the strip rather than an assigned area-perimeter pair.

```
def carrier (c : StripCore) : Set PlanePoint :=
  c.rectangleCarrier ∪ c.leftCapCarrier ∪ c.rightCapCarrier

/-- Rectangle plus two side circular segments. -/
```

StripCore.euclideanArea

Classification: main

Source: CMVGeometry.lean:453-456

Declaration SHA-256: d05ac0be663f49bfb5425917ab3d2dcdd3367e8ef25ec467ccc3d1ef4ccf07a7

Mathematical statement. The strip core's scalar area is $2L + 8a(\phi)$.

Each side segment has chord length two and hence area $4a(\phi)$; the rectangle contributes $2L$.

```

def euclideanArea (c : StripCore) : ℝ :=
  2 * c.chord + 8 * area c.sideAngle

/-- The two side arcs, each of chord length two. -/

```

StripCore.frontier_carrier

Classification: main

Source: CMVGeometry.lean:1088-1199

Declaration SHA-256: 34bdf246812f32ba3d6167bf3b3ff5fea490b0816170740d97a7d77e703585bf

Mathematical statement. The strip-core frontier is the union of the two side arc traces and the upper and lower chord traces.

This exact decomposition distinguishes exposed chords from chords later hidden by attaching exterior caps.

```

theorem frontier_carrier (c : StripCore) :
  frontier c.carrier =
    leftArcTrace c ∪
      (rightArcTrace c ∪ (upperChordTrace c ∪ lowerChordTrace c)) := by
  apply Subset.antisymm
  · intro p hp
    have hpc : p ∈ c.carrier := (isClosed_carrier c).frontier_subset hp
    have hnint : p ∉ interior c.carrier :=
      (mem_frontier_iff_notMem_interior hpc).mp hp
    rcases hpc with (hrect | hleft) | hright
    · change -c.chord / 2 ≤ p.1 ∧ p.1 ≤ c.chord / 2 ∧
      |p.2| ≤ 1 at hrect
      by_cases htop : p.2 = 1
      · exact Or.inr (Or.inr (Or.inl
        ⟨htop, abs_le.mpr
          ⟨by linarith [hrect.1], hrect.2.1⟩⟩))
      by_cases hbottom : p.2 = -1
      · exact Or.inr (Or.inr (Or.inr
        ⟨hbottom, abs_le.mpr
          ⟨by linarith [hrect.1], hrect.2.1⟩⟩))
      have hybounds := abs_le.mp hrect.2.2
      have hylt : |p.2| < 1 := abs_lt.mpr
        ⟨lt_of_le_of_ne hybounds.1 (Ne.symm hbottom),
          lt_of_le_of_ne hybounds.2 htop)
      by_cases hxl : p.1 = -c.chord / 2
      · exfalso
        exact hnint (leftInterface_subset_interior c ⟨hxl, hylt⟩)
      by_cases hxr : p.1 = c.chord / 2
      · exfalso
        exact hnint (rightInterface_subset_interior c ⟨hxr, hylt⟩)
      · exfalso
        apply hnint
        exact strictRectangle_subset_interior c
          ⟨lt_of_le_of_ne hrect.1 (Ne.symm hxl),
            lt_of_le_of_ne hrect.2.1 hxr, hylt)
    · change
      ((p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2) ∧
      p.1 ≤ -c.chord / 2 ∧ |p.2| ≤ 1 at hleft
      by_cases heq :
        (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2
      · exact Or.inl ⟨heq, hleft.2⟩
      have hdisk :
        (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 < c.radius ^ 2 :=
          lt_of_le_of_ne hleft.1 heq
      have hylt : |p.2| < 1 := by
        apply lt_of_le_of_ne hleft.2.2
        intro hone
        exact heq (leftCap_circle_eq_of_abs_eq_one c hleft hone)
      by_cases hx : p.1 = -c.chord / 2
      · exfalso
        exact hnint (leftInterface_subset_interior c ⟨hx, hylt⟩)
      · exfalso

```

```

    exact hrint (strictLeftCap_subset_interior c
      (hdisk, lt_of_le_of_ne hleft.2.1 hx, hylt))
· change
  ((p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2) ^
  c.chord / 2 ≤ p.1 ^ |p.2| ≤ 1 at hright
by_cases heq :
  (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2
· exact Or.inr (Or.inl (heq, hright.2))
have hdisk :
  (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 < c.radius ^ 2 :=
  lt_of_le_of_ne hright.1 heq
have hylt : |p.2| < 1 := by
  apply lt_of_le_of_ne hright.2.2
intro hone
exact heq (rightCap_circle_eq_of_abs_eq_one c hright hone)
by_cases hx : p.1 = c.chord / 2
· ex falso
  exact hrint (rightInterface_subset_interior c (hx, hylt))
· ex falso
  exact hrint (strictRightCap_subset_interior c
    (hdisk, lt_of_le_of_ne hright.2.1 (Ne.symm hx), hylt))
· rintro p (hleft | hright | hupper | hlower)
· have hpc : p ∈ c.carrier :=
  Or.inl (Or.inr (hleft.1.le, hleft.2))
exact (mem_frontier_iff_notMem_interior hpc).2
  (leftArc_not_mem_interior c hleft)
· have hpc : p ∈ c.carrier :=
  Or.inr (hright.1.le, hright.2)
exact (mem_frontier_iff_notMem_interior hpc).2
  (rightArc_not_mem_interior c hright)
· have hpc : p ∈ c.carrier := by
  apply Or.inl
  apply Or.inl
  have hxbounds := abs_le.mp hupper.2
  exact (by linarith [hxbounds.1], hxbounds.2,
    by simp [hupper.1])
rw [mem_frontier_iff_notMem_interior hpc]
intro hi
apply second_eq_not_mem_interior_le 1 p hupper.1
exact interior_mono (by
  intro q hq
  exact le_trans (le_abs_self q.2) (c.carrier_y_bounds hq)) hi
· have hpc : p ∈ c.carrier := by
  apply Or.inl
  apply Or.inl
  have hxbounds := abs_le.mp hlower.2
  exact (by linarith [hxbounds.1], hxbounds.2,
    by simp [hlower.1])
rw [mem_frontier_iff_notMem_interior hpc]
intro hi
apply second_eq_not_mem_interior_ge (-1) p hlower.1
exact interior_mono (by
  intro q hq
  have hneg : -q.2 ≤ 1 :=
    le_trans (neg_le_abs q.2) (c.carrier_y_bounds hq)
  show (-1 : ℝ) ≤ q.2
  linarith) hi

end StripCore
/-- Push one-dimensional Lebesgue measure on a closed interval through a curve. -/

```

StripCore.volume_carrier

Classification: main

Source: CMVGeometry.lean:2202-2242

Declaration SHA-256: a426f0908aec7b73455f4a0093170232e841cf46ab8f580ee2fe442ff66a7997

Mathematical statement. The planar volume of the whole strip-core carrier equals $2L + 8a(\phi)$.

The pieces overlap only on planar-null line segments, so their volumes add to the stored scalar area.

```

lemma volume_carrier (c : StripCore) :
  volume c.carrier = ENNReal.ofReal c.euclideanArea := by
  have hrectLeft : AEDisjoint volume c.rectangleCarrier c.leftCapCarrier := by
    refine measure_mono_null ?_ (volume_verticalline (-c.chord / 2))
    rintro p ⟨hrect, hleft⟩
    exact le_antisymm hleft.2.1 hrect.1
  have hfirstRight : AEDisjoint volume
    (c.rectangleCarrier ∪ c.leftCapCarrier) c.rightCapCarrier := by
    refine measure_mono_null ?_ (volume_verticalline (c.chord / 2))
    rintro p ⟨hfirst, hright⟩
    rcases hfirst with hrect | hleft
    · exact le_antisymm hrect.2.1 hright.2.1
    · ex falso
      nlinarith [hleft.2.1, hright.2.1, c.chord_pos]
  have hleftMeas : MeasurableSet c.leftCapCarrier := by
    rw [leftCapCarrier]
    exact ((isClosed_le
      (((continuous_fst.sub continuous_const).pow 2).add
        (continuous_snd.pow 2)) continuous_const).inter
      ((isClosed_le continuous_fst continuous_const).inter
        (isClosed_le continuous_snd.abs continuous_const))).measurableSet
  have hrightMeas : MeasurableSet c.rightCapCarrier := by
    rw [rightCapCarrier]
    exact ((isClosed_le
      (((continuous_fst.sub continuous_const).pow 2).add
        (continuous_snd.pow 2)) continuous_const).inter
      ((isClosed_le continuous_const continuous_fst).inter
        (isClosed_le continuous_snd.abs continuous_const))).measurableSet
  rw [carrier, measure_union, hrightMeas.nullMeasurableSet hfirstRight,
    measure_union, hleftMeas.nullMeasurableSet hrectLeft,
    volume_rectangleCarrier, volume_leftCapCarrier, volume_rightCapCarrier]
  have hq : 0 ≤ 2 * c.chord := mul_nonneg (by norm_num) c.chord_pos.le
  have ha : 0 ≤ 4 * area c.sideAngle := by
    exact mul_nonneg (by norm_num) (area_pos c.sideAngle_pos c.sideAngle_lt_pi).le
  rw [← ENNReal.ofReal_add hq ha, ← ENNReal.ofReal_add (add_nonneg hq ha) ha]
  congr 1
  rw [euclideanArea]
  ring

end StripCore

```

StripCore.weightedArea_formula

Classification: main

Source: CMVGeometry.lean:2315-2333

Declaration SHA-256: 22039501f3884648956188fd011ab629e4d948f733db9c19194b8b310134e20a

Mathematical statement. The weighted area of a strip-core carrier equals its Euclidean scalar area, for every real λ .

The density is one at every core point, so the weighted integral equals planar volume.

```

lemma StripCore.weightedArea_formula (lam : ℝ) (c : StripCore) :
  WeightedArea lam c.carrier = c.euclideanArea := by
  rw [WeightedArea]
  calc
    (∫ p in c.carrier, StripDensity lam p) =
      ∫ _p in c.carrier, (1 : ℝ) := by
        apply setIntegral_congr_fun c.measurableSet_carrier
        intro p hp
        rw [StripDensity, if_pos (c.carrier_y_bounds hp)]
  _ = c.euclideanArea := by
    have ha : 0 ≤ c.euclideanArea := by
      rw [StripCore.euclideanArea]
      exact add_nonneg (mul_nonneg (by norm_num) c.chord_pos.le)
        (mul_nonneg (by norm_num)
          (area_pos c.sideAngle_pos c.sideAngle_lt_pi).le)

```

```
rw [integral_const, measureReal_restrict_apply_univ,
  Measure.real, c.volume_carrier, ENNReal.toReal_ofReal ha]
simp
```

StripDensity

Classification: main

Source: CMVGeometry.lean:43-54

Declaration SHA-256: 51e629a8115629e4652f417b9dcfc46beea6a584d8ebcb351e746df181f97ab1

Mathematical statement. The strip density is $f_\lambda(x, y) = 1$ for $|y| \leq 1$, and $f_\lambda(x, y) = \lambda$ otherwise.

Both interfaces have density one. The definition permits any real density parameter; the main theorem requires $\lambda > 1$.

```
def StripDensity (lam : ℝ) (p : PlanePoint) : ℝ :=
  if |p.2| ≤ 1 then 1 else lam

@[simp] theorem stripDensity_upper_interface (lam x : ℝ) :
  StripDensity lam (x, 1) = 1 := by
  simp [StripDensity]

@[simp] theorem stripDensity_lower_interface (lam x : ℝ) :
  StripDensity lam (x, -1) = 1 := by
  simp [StripDensity]

/-- Lebesgue weighted area for a general measurable planar carrier. -/
```

TypeThreeAssembly

Classification: main

Source: TypeThreeAssembly.lean:29-35

Declaration SHA-256: ba80fb031079587a301c08eeb69ef793b83ab9e2bcde30a061c829264b7e1959

Mathematical statement. A regular type-iii assembly stores $\lambda > 1$ and a curvature $0 < h < 1$.

The strict curvature range gives radius $R = 1/h > 1$, so the carrier is the regular branch-complete source geometry rather than the closure endpoint.

```
structure TypeThreeAssembly (lam : ℝ) where
  h : ℝ
  density_jump : 1 < lam
  h_pos : 0 < h
  h_lt_one : h < 1

/-- The three geometric branches of the source type-(iii) upper arc. -/
```

TypeThreeAssembly.HasModeledCompetitor

Classification: main

Source: TypeThreeAssembly.lean:3076-3080

Declaration SHA-256: 6f62c793e9d706d4618e398ebe39871915cbb898f90f26305f1f8dad0a496a66

Mathematical statement. The type-iii carrier has an admissible competitor realization with the same carrier and the stated scalar area and perimeter.

This is the bridge from the constructed coordinate set to the common admissible-competitor type used by the candidate minimality predicate.

```
def HasModeledCompetitor : Prop :=
  Nonempty (ModeledCompetitorRealization a)
```

```
-- Every regular type-(iii) coordinate carrier is a genuine admissible
finite-perimeter region; no external realization hypothesis is needed. -/
```

TypeThreeAssembly.ModeledCompetitorRealization

Classification: main

Source: TypeThreeAssembly.lean:3043-3052

Declaration SHA-256: 7461e543281be1bed3a7575ad4267ecf1122dd3b3edd989aadf4754226aea410

Mathematical statement. A modeled competitor realization pairs an admissible competitor with proof that its carrier is exactly the type-iii carrier.

This structure prevents the scalar comparison from bypassing the carrier-based semantics of minimality.

```
structure ModeledCompetitorRealization where
  competitor : AdmissibleCompetitor lam
  carrier_eq : competitor.carrier = a.carrier

namespace ModeledCompetitorRealization

variable {a : TypeThreeAssembly lam}
  (realization : ModeledCompetitorRealization a)

/-- The genuine admissible competitor selected by a geometric realization. -/
```

TypeThreeAssembly.ModeledCompetitorRealization.asCompetitor_weightedArea

Classification: main

Source: TypeThreeAssembly.lean:3059-3064

Declaration SHA-256: ac19f7d26c088671b4dd0552d598cd1124637e2815ea1238d0d5daf917f4ca9b

Mathematical statement. The realized admissible competitor has weighted area equal to the type-iii scalar area.

The carrier equality lets the common competitor definition use the geometric carrier integral, after which the area formula theorem applies.

```
theorem asCompetitor_weightedArea :
  realization.asCompetitor.WeightedArea = a.scalarWeightedArea := by
  rw [asCompetitor, realization.competitor.weightedArea_eq_carrier,
    realization.carrier_eq]
  exact a.weightedArea_formula
```

TypeThreeAssembly.ModeledCompetitorRealization.asCompetitor_weightedPerimeter

Classification: main

Source: TypeThreeAssembly.lean:3065-3075

Declaration SHA-256: c73f289c45ecd5870b19789b66c484f95d8c8aad15bb42c8e172d87d107523d8

Mathematical statement. The realized admissible competitor has weighted perimeter equal to the type-iii scalar perimeter.

The same carrier equality and complete-frontier perimeter theorem transfer the geometric perimeter value to the admissible competitor.

```
theorem asCompetitor_weightedPerimeter :
  realization.asCompetitor.WeightedPerimeter =
  a.scalarWeightedPerimeter := by
  change _root_.WeightedPerimeter lam
  (FrontierMeasure realization.competitor.carrier) =
```

```

    a.scalarWeightedPerimeter
  rw [realization.carrier_eq]
  exact a.weightedPerimeter_formula
end ModeledCompetitorRealization

```

TypeThreeAssembly.WeightedArea

Classification: main

Source: TypeThreeAssembly.lean:265-268

Declaration SHA-256: 5d054747c494e793da416a19e2d895420eb9e121366a5645e79b7ab53a0c3c7f

Mathematical statement. For a regular type-iii assembly with density $\lambda > 1$, curvature $0 < h < 1$, and coordinate carrier E_3 , the weighted area is the ambient Lebesgue integral

$$A_\lambda(E_3) = \int_{E_3} f_\lambda d\mathcal{L}^2.$$

The integration domain is the carrier E_3 , and the measure is two-dimensional Lebesgue measure restricted to that carrier. No frontier measure enters this definition. No scalar value is chosen independently of the carrier; the later area formula identifies this canonical quantity with $A_3(h)$.

```

def WeightedArea : ℝ := _root_.WeightedArea lam a.carrier

/-- Density integral against the complete topological frontier of the
coordinate carrier. -/

```

TypeThreeAssembly.WeightedPerimeter

Classification: main

Source: TypeThreeAssembly.lean:269-272

Declaration SHA-256: ac2df4f2daf804e358c901d1a6b496eaadc35076745e389cd399733fc5dead10

Mathematical statement. For a regular type-iii assembly with density $\lambda > 1$, curvature $0 < h < 1$, and coordinate carrier E_3 , let J be the coordinate-preserving homeomorphism to the Euclidean realization. The weighted perimeter is the complete-frontier integral

$$P_\lambda(E_3) = \int_{\partial J(E_3)} (f_\lambda \circ J^{-1}) d\mathcal{H}^1.$$

The boundary measure is Euclidean one-dimensional Hausdorff measure restricted to the complete topological frontier of $J(E_3)$, transported back to the coordinate plane. No scalar value is chosen independently of the carrier; the later perimeter formula identifies this canonical quantity with $P_3(h)$.

```

def WeightedPerimeter : ℝ :=
  _root_.WeightedPerimeter lam (FrontierMeasure a.carrier)

/-- The horizontal expansion between the two translated side half-caps. -/

```

TypeThreeAssembly.angleTerm

Classification: main

Source: TypeThreeAssembly.lean:289-291

Declaration SHA-256: 8089feb5fd033f48e23e1ab4579cce22afbbbfde128400123f3b8b2b2a245332

Mathematical statement. The declaration defines the type-iii angle contribution $\lambda\alpha + \pi - \beta$ or sine displacement $\sin\alpha - \sin\beta$.

These are the two geometric combinations that enter the area and perimeter formulas.

```
def angleTerm : ℝ := lam * a.outerAngle + π - a.innerAngle

/-- The horizontal sine displacement between the upper and side arcs. -/
```

TypeThreeAssembly.bottomChord

Classification: main

Source: TypeThreeAssembly.lean:136-139

Declaration SHA-256: 08cc63d3aedc396b5efa9b27fd1bac931beb8b7d14cbce558e5fde942be277c2

Mathematical statement. The bottom-segment length parameter is $2w$.

It is nonnegative and vanishes exactly at the semicircular transition.

```
def bottomChord : ℝ := 2 * a.sideHalfWidth

/-- Finite branch selection; it does not assert membership in a later
area-fold branch. -/
```

TypeThreeAssembly.bottomChord_eq_zero_iff

Classification: main

Source: TypeThreeAssembly.lean:355-358

Declaration SHA-256: 1b82dffbe772075bcb5095c9dc075644068e9232dc927ae0cd23919b863062dd

Mathematical statement. $2w = 0$ if and only if $h = 1/2$.

The positive factor two preserves the exact vanishing condition for w .

```
theorem bottomChord_eq_zero_iff : a.bottomChord = 0 ↔ a.h = 1 / 2 := by
  rw [bottomChord, mul_eq_zero]
  norm_num
  exact a.sideHalfWidth_eq_zero_iff
```

TypeThreeAssembly.bottomSegmentCarrier

Classification: main

Source: TypeThreeAssembly.lean:132-135

Declaration SHA-256: 18e70891a1386b33220d27cc96af45d0ffdcc88b970fd467c64035c04da589f5

Mathematical statement. The bottom segment carrier is $\{(x, -1) : |x| \leq w\}$.

The definition includes the singleton $\{(0, -1)\}$ when $w = 0$; it does not discard the degenerate boundary component.

```
def bottomSegmentCarrier : Set PlanePoint :=
  {p | p.2 = -1} ∩ {p | |p.1| ≤ a.sideHalfWidth}

/-- Bottom-segment length, including the zero-length semicircular transition. -/
```

TypeThreeAssembly.bottomSegmentCarrier_subset_frontier

Classification: main

Source: TypeThreeAssembly.lean:1058-1079

Declaration SHA-256: 82d466e4484f4553cdf0fe29df6958b297f06ff314377fa09237775f676763e1

Mathematical statement. The entire bottom segment carrier is contained in the complete frontier of the type-iii carrier:

$$\{(x, -1) : |x| \leq w\} \subseteq \partial E_3.$$

This is a frontier-inclusion theorem, including the singleton degenerate case at $w = 0$; it is not merely the definition of the bottom set.

```
theorem bottomSegmentCarrier_subset_frontier :
  a.bottomSegmentCarrier ⊆ frontier a.carrier := by
  intro p hp
  rw [mem_frontier_iff_notMem_interior
    (a.bottomSegmentCarrier_subset_carrier hp)]
  intro hinterior
  have hnhds : interior a.carrier ∈ nhds p :=
    isOpen_interior.mem_nhds hinterior
  rcases Metric.mem_nhds_iff.mp hnhds with
    (epsilon, hepsilon, hball)
  have hhalf : 0 < epsilon / 2 := div_pos hepsilon (by norm_num)
  have hnear :
    (p.1, p.2 - epsilon / 2) ∈ Metric.ball p epsilon := by
    rw [Metric.mem_ball, Prod.dist_eq, dist_self, Real.dist_eq,
      show p.2 - epsilon / 2 - p.2 = -(epsilon / 2) by ring,
      abs_neg, abs_of_pos hhalf, max_eq_right hhalf.le]
    linarith
  have hmember :
    (p.1, p.2 - epsilon / 2) ∈ a.carrier :=
    interior_subset (hball hnear)
  exact a.below_bottomSegmentCarrier_not_mem hp hhalf hmember
```

TypeThreeAssembly.boundaryTrace

Classification: main

Source: TypeThreeAssembly.lean:1595-1599

Declaration SHA-256: 77ded3af26dffe034b843162e20202cd8e868c69330a0cb95819f9e3b2a0bbf5

Mathematical statement. The declared boundary trace is the union of the two strip-side arc traces, the upper cap arc trace, and the bottom segment.

It is the candidate set for the complete frontier; the two inclusion theorems prove that it is exactly the topological frontier.

```
def boundaryTrace : Set PlanePoint :=
  a.leftArcTrace ∪
  (a.rightArcTrace ∪
    (OneSidedCircularCap.arcTrace a.outerCap ∪ a.bottomSegmentCarrier))
```

TypeThreeAssembly.carrier

Classification: main

Source: TypeThreeAssembly.lean:262-264

Declaration SHA-256: 0849c97e398a59f917657f18205fab9fffd6aa8e524ee751befd78f1c4735a4ff

Mathematical statement. The complete type-iii carrier is the union of its core carrier and its upper exterior cap carrier.

This is the actual coordinate set whose area and complete frontier are used in the admissible competitor.

```
def carrier : Set PlanePoint := a.coreCarrier ∪ a.outerCap.carrier

/-- Ambient weighted-area integral of the coordinate carrier. -/
```

TypeThreeAssembly.coreCarrier

Classification: main

Source: TypeThreeAssembly.lean:99-103

Declaration SHA-256: 5e82c9d38bf8a70ad9c2ae143c293bbf0238ee48148c3dd2d922c95471160878

Mathematical statement. The type-iii core carrier is the union of the central rectangle, left clipped disk segment, and right clipped disk segment.

The definition forms the actual coordinate set inside the closed strip before the upper exterior cap is attached.

```
def coreCarrier : Set PlanePoint :=
  a.rectangleCarrier ∪ a.leftSegmentCarrier ∪ a.rightSegmentCarrier

/-- The exact left circular trace of the type-(iii) strip-side boundary. -/
```

TypeThreeAssembly.core_weightedArea_formula

Classification: main

Source: TypeThreeAssembly.lean:1530-1550

Declaration SHA-256: 5a80fac760bbad4e64452fa4d575b54c924b85ffbad6442cb8801547dabcbba7

Mathematical statement. For a regular type-iii assembly,

$$A_\lambda(E_{\text{core}}) = 4w + \text{Area}(\text{side cap}).$$

The core lies in the closed unit strip, where the density is one, so its weighted area is the rectangle contribution plus the two side-segment contribution.

```
theorem core_weightedArea_formula :
  _root_.WeightedArea lam a.coreCarrier =
    a.rectangleArea + a.sideCap.euclideanArea := by
  rw [_root_.WeightedArea]
  calc
    (∫ p in a.coreCarrier, StripDensity lam p) =
      ∫ _p in a.coreCarrier, (1 : ℝ) := by
        apply setIntegral_congr_fun a.isClosed_coreCarrier.measurableSet
        intro p hp
        rw [StripDensity, if_pos (a.coreCarrier_y_bounds hp)]
    _ = a.rectangleArea + a.sideCap.euclideanArea := by
      have hnonneg :
        0 ≤ a.rectangleArea + a.sideCap.euclideanArea :=
          add_nonneg
          (mul_nonneg (by norm_num) a.sideHalfWidth_nonneg)
          a.sideCap.euclideanArea_pos.le
      rw [integral_const, measureReal_restrict_apply_univ,
        Measure.real, a.volume_coreCarrier,
        ENNReal.toReal_ofReal hnonneg]
      simp
```

TypeThreeAssembly.cos_innerAngle

Classification: main

Source: TypeThreeAssembly.lean:178-183

Declaration SHA-256: e774d0a2578a71402d098d5ac027f2d6d1dbb90425ca111832168cfb9f013412

Mathematical statement. The principal inner angle satisfies $\cos\beta = q$.

This is the principal arccosine identity on $[-1, 1]$.

```

theorem cos_innerAngle : cos a.innerAngle = a.shapeParameter := by
  rw [innerAngle, Real.cos_arccos a.shapeParameter_mem.1.le
    a.shapeParameter_mem.2.le]

/-- The principal-angle definitions recover the Snell-adjusted exterior
argument exactly. -/

```

TypeThreeAssembly.cos_outerAngle

Classification: main

Source: TypeThreeAssembly.lean:184-188

Declaration SHA-256: 3eb066bbadef1d5d0722d2c322b7cfe8bce8e1eb62204f2872b1f8f1bbb41a24

Mathematical statement. The principal exterior angle satisfies $\cos \alpha = q/\lambda$.

This is the corresponding principal arccosine identity for the Snell-adjusted argument.

```

theorem cos_outerAngle : cos a.outerAngle = a.outerArgument := by
  rw [outerAngle, Real.cos_arccos a.outerArgument_mem.1.le
    a.outerArgument_mem.2.le]

/-- CMV's type-(iii) incidence equation, derived rather than assumed. -/

```

TypeThreeAssembly.euclidean_frontier

Classification: main

Source: TypeThreeAssembly.lean:2937-2949

Declaration SHA-256: 5e03c2ded7f93ebce1b77460290c9d90f4e5be41990287351d94b69c624d60ea

Mathematical statement. The Euclidean frontier of the realized type-iii carrier is the realization of those four boundary components.

The coordinate frontier equality transports through the homeomorphism and identifies the Euclidean Hausdorff-measure domain.

```

theorem euclidean_frontier :
  frontier (planeEuclideanHomeomorph '' a.carrier) =
    (planeEuclideanHomeomorph '' a.leftArcTrace) ∪
    ((planeEuclideanHomeomorph '' a.rightArcTrace) ∪
      ((planeEuclideanHomeomorph ''
        OneSidedCircularCap.arcTrace a.outerCap) ∪
        (planeEuclideanHomeomorph '' a.bottomSegmentCarrier))) := by
  rw [← planeEuclideanHomeomorph.image_frontier, a.frontier_carrier]
  simp only [boundaryTrace, image_union]

/-- The component perimeter is the canonical weighted  $\mathcal{H}^1$  integral over the
complete topological frontier. This includes the exposed bottom segment and
is valid at its zero-length semicircular transition. -/

```

TypeThreeAssembly.exact_euclidean_bottomSegmentCarrier_hausdorffMeasure

Classification: main

Source: TypeThreeAssembly.lean:1101-1129

Declaration SHA-256: 56949577898a4e9f1638a1a8bab2c43a8263ea9e6cf0976fce47fbff56b3fe00

Mathematical statement. The Euclidean $\mathcal{H}^1$ -measure of the type-iii bottom segment equals its chord length.

This includes the zero-length value at $h = 1/2$ and supplies the density-one boundary contribution in the perimeter formula.

```

theorem exact_euclidean_bottomSegmentCarrier_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' a.bottomSegmentCarrier) =
  ENNReal.ofReal a.bottomChord := by
  rcases eq_or_ne a.h (1 / 2) with hh | hh
  · have hw : a.sideHalfWidth = 0 :=
    a.sideHalfWidth_eq_zero_iff.mpr hh
  have hc : a.bottomChord = 0 :=
    a.bottomChord_eq_zero_iff.mpr hh
  have hcarrier : a.bottomSegmentCarrier = {(0, -1)} := by
    ext p
    simp only [bottomSegmentCarrier, mem_inter_iff, mem_ofPred_eq,
      hw, abs_nonpos_iff, mem_singleton_iff]
  constructor
  · rintro ⟨hy, hx⟩
    exact Prod.ext hx hy
  · intro hp
    rw [hp]
    exact ⟨rfl, rfl⟩
  rw [hcarrier, image_singleton, hc, ENNReal.ofReal_zero]
  let _ : NullSingletonClass (μH[1] : Measure EuclideanPlane) :=
    Measure.nullSingletonClass_hausdorff EuclideanPlane (by norm_num)
  exact measure_singleton _
  · rw [- a.bottomHorizontalSegment_carrier hh,
    exact_euclidean_segmentCarrier_hausdorffMeasure]
    rfl

/-- The four primitive boundary joins. The two bottom points coincide at the
semicircular transition. -/

```

TypeThreeAssembly.frontier_carrier

Classification: main

Source: TypeThreeAssembly.lean:2848-2854

Declaration SHA-256: 0ce2b73ef2aba31ce763d170404d962fb774d3c494bf5afe10757388adbfbdb30

Mathematical statement. The complete frontier of the type-iii carrier equals its boundary trace.

Combining both inclusions gives the exact set-theoretic frontier needed for canonical perimeter.

```

theorem frontier_carrier :
  frontier a.carrier = a.boundaryTrace :=
  Set.Subset.antisymm a.frontier_carrier_subset_boundaryTrace
  a.boundaryTrace_subset_frontier_carrier

/-- The Euclidean realization of the bottom segment is measurable, including
the singleton occurring at the semicircular transition. -/

```

TypeThreeAssembly.hasModeledCompetitor

Classification: main

Source: TypeThreeAssembly.lean:3081-3100

Declaration SHA-256: 6010bc1b92d07ee7af37566e925a969f227d520e7144371bb334d772b94b8386

Mathematical statement. Every regular type-iii assembly has an admissible competitor realization.

The carrier is measurable, the density is integrable over its area and frontier, and its carrier can therefore be packaged in the common admissible-competitor type.

```

theorem hasModeledCompetitor : a.HasModeledCompetitor := by
  refine {
    competitor := .general {
      carrier := a.carrier
      measurable_carrier := a.measurableSet_carrier
    }
  }

```

```

    finite_weighted_area := a.integrableOn_carrier
    finite_weighted_perimeter := a.integrable_frontierMeasure }
    carrier_eq := rfl })

end TypeThreeAssembly

namespace FourArcCandidate

variable {lam : ℝ} (candidate : FourArcCandidate lam)

/-- Observable modeled-geometry bridge. Once the coordinate type-(iii)
carrier is realized as an admissible competitor, the scalar certificate needs
only equal weighted area and strict scalar perimeter improvement. Minimality
then fails against that genuine competitor. -/

```

TypeThreeAssembly.integrableOn_carrier

Classification: main

Source: TypeThreeAssembly.lean:3000-3011

Declaration SHA-256: f50bce5009ef96b4081db29960a7e72186fb76edbfbef9fa6fd538bc59ab8c22

Mathematical statement. For every regular type-iii assembly with density $\lambda > 1$, curvature $0 < h < 1$, and coordinate carrier E_3 , the strip density is Lebesgue integrable on the carrier:

$$f_\lambda \in L^1(E_3, \mathcal{L}^2).$$

The carrier is the union of the strip core and the upper cap. The core has finite Lebesgue measure, so the bounded strip density is integrable there; cap integrability gives integrability on the union. This establishes that the carrier-derived weighted area is a genuine finite integral, not a formal scalar assignment. Integrability against the complete-frontier measure, needed for weighted perimeter, is a separate assertion.

```

theorem integrableOn_carrier :
  IntegrableOn (StripDensity lam) a.carrier := by
  rw [carrier]
  have hcore : IntegrableOn (StripDensity lam) a.coreCarrier := by
    apply stripDensity_integrableOn_of_volume_ne_top
    rw [a.volume_coreCarrier]
    exact ENNReal.ofReal_ne_top
  exact hcore.union (cap_integrableOn lam a.outerCap)

/-- The density is integrable against the canonical frontier measure. This is
the finite-perimeter obligation for treating the coordinate carrier as a
general admissible competitor. -/

```

TypeThreeAssembly.integrable_frontierMeasure

Classification: main

Source: TypeThreeAssembly.lean:3012-3042

Declaration SHA-256: 242285e9e93302048576853114a4bc7f6ca16cdc917f22c33a2fe025def880a8

Mathematical statement. The strip density is integrable against the complete-frontier measure of the type-iii carrier.

The frontier is a finite union of measurable circular traces and a horizontal segment with finite Euclidean $\mathcal{H}^1$ -measure, so the bounded two-valued density has a finite integral.

```

theorem integrable_frontierMeasure :
  Integrable (StripDensity lam) (FrontierMeasure a.carrier) := by
  have hfrontier :
    (μH[1] : Measure EuclideanPlane)
    (frontier (planeEuclideanHomeomorph '' a.carrier)) ≠ ⊤ := by
  rw [a.euclidean_frontier]
  apply measure_union_ne_top

```

```

· rw [a.exact_euclidean_leftArcTrace_hausdorffMeasure]
  exact ENNReal.ofReal_ne_top
apply measure_union_ne_top
· rw [a.exact_euclidean_rightArcTrace_hausdorffMeasure]
  exact ENNReal.ofReal_ne_top
apply measure_union_ne_top
· rw [exact_euclidean_capTrace_hausdorffMeasure]
  exact ENNReal.ofReal_ne_top
· rw [a.exact_euclidean_bottomSegmentCarrier_hausdorffMeasure]
  exact ENNReal.ofReal_ne_top
unfold FrontierMeasure
rw [integrable_map_measure
  (measurable_stripDensity lam).aestronglyMeasurable
  planeEuclideanHomeomorph.symm.continuous.measurable.aemeasurable]
change IntegrableOn (euclideanStripDensity lam)
  (frontier (planeEuclideanHomeomorph '' a.carrier))
  (μH[1] : Measure EuclideanPlane)
exact euclideanStripDensity_integrableOn_of_measure_ne_top lam hfrontier

```

```

/-- Geometric realization contract for the coordinate carrier. Both weighted
measure formulas follow from `carrier_eq`; neither is independently selectable
data. -/

```

TypeThreeAssembly.leftUpperJunction

Classification: main

Source: TypeThreeAssembly.lean:946-957

Declaration SHA-256: a660a79d351317f739116ce079a065c0c8bced856fae9b73442ad8d1339dd7ec

Mathematical statement. The left side-arc upper junction equals the outer-cap left endpoint:

$$(-w - R\sin\beta, 1) = (-R\sin\alpha, 1).$$

The identity expresses the common interface junction using $w = R(\sin\alpha - \sin\beta)$.

```

theorem leftUpperJunction :
  (a.leftCenter.1 - a.radius * sin a.innerAngle, 1) =
    a.outerCap.leftEndpoint := by
  rw [a.outerCap_leftEndpoint]
  change
    (-a.sideHalfWidth - a.radius * sin a.innerAngle, 1) =
      (-a.radius * sin a.outerAngle, 1)
  rw [sideHalfWidth]
  ring_nf

/-- The right strip arc and exterior cap use the same upper junction
coordinate. -/

```

TypeThreeAssembly.modeledWeightedArea

Classification: main

Source: TypeThreeAssembly.lean:278-284

Declaration SHA-256: 35499fdca0a3bbbed11909d6dec0bbb1c83123d6ea5cb0599717ee289f567e87e

Mathematical statement. The modeled type-iii weighted area is the rectangle area plus the lower-side cap area plus λ times the upper exterior-cap area.

This is an explicit scalar expression derived from the actual carrier components; the formula theorem later identifies it with the canonical integral.

```

def modeledWeightedArea : ℝ :=
  a.rectangleArea + a.sideCap.euclideanArea +

```

```
lam * a.outerCap.euclideanArea
```

```
/-- Component perimeter of the modeled type-(iii) profile. The side-cap arc
is the sum of the two translated side arcs; `bottomChord` also covers the
zero-length transition at `h = 1 / 2`. -/
```

TypeThreeAssembly.modeledWeightedArea_formula

Classification: main

Source: TypeThreeAssembly.lean:885-895

Declaration SHA-256: b0e70e8aeaefad1e18dbaff1d3d6364f544c6042f48ce97bebd3fa5c6cc8e74e

Mathematical statement. The component modeled weighted area equals the scalar formula: $A_{\text{modeled}} = A_{\text{scalar}}$.

The equality is the algebraic reduction of the rectangle, side-cap, and density-weighted outer-cap contributions.

```
theorem modeledWeightedArea_formula :
  a.modeledWeightedArea = a.scalarWeightedArea := by
  rw [modeledWeightedArea, rectangleArea, a.sideCap_euclideanArea,
    a.outerCap_euclideanArea, scalarWeightedArea, angleTerm, sineGap,
    sideHalfWidth, a.sin_sideArcAngle, a.cos_sideArcAngle, a.cos_outerAngle,
    outerArgument, sideArcAngle, radius, shapeParameter]
  field_simp [ne_of_gt a.h_pos,
    ne_of_gt (lt_trans (by norm_num : (0 : ℝ) < 1) a.density_jump)]
  ring

/-- CMV equation (26), including the degenerate bottom-chord branch. -/
```

TypeThreeAssembly.modeledWeightedPerimeter

Classification: main

Source: TypeThreeAssembly.lean:285-288

Declaration SHA-256: 76198be6eb5b8790a4af34449bf29c4b963314e1e4bd268b4d3c9cdb2e69ea0d

Mathematical statement. The modeled type-iii perimeter is the lower side-cap arc length, λ times the upper cap arc length, and the bottom chord length.

The bottom chord has density one and the exterior cap arc has density λ ; the two strip-side arcs are represented by the side-cap contribution.

```
def modeledWeightedPerimeter : ℝ :=
  a.sideCap.arcLength + lam * a.outerCap.arcLength + a.bottomChord

/-- The angle term `lambda * a_3 + pi - b_3` in CMV equation (26). -/
```

TypeThreeAssembly.modeledWeightedPerimeter_formula

Classification: main

Source: TypeThreeAssembly.lean:896-903

Declaration SHA-256: bb4c05a2ca2fd9d5b63be1b6ad16da716cc43497abacad228054765e6eaa2e39e

Mathematical statement. The component modeled weighted perimeter equals the scalar formula: $P_{\text{modeled}} = P_{\text{scalar}}$.

The equality combines the two side traces, the density-weighted outer arc, and the bottom segment.

```
theorem modeledWeightedPerimeter_formula :
  a.modeledWeightedPerimeter = a.scalarWeightedPerimeter := by
  rw [modeledWeightedPerimeter, a.sideCap_arcLength,
    a.outerCap_arcLength, scalarWeightedPerimeter, angleTerm, sineGap,
    bottomChord, sideHalfWidth, sideArcAngle, radius]
```

```
field_simp [ne_of_gt a.h_pos]
ring
```

TypeThreeAssembly.outerArgument_mem

Classification: main

Source: TypeThreeAssembly.lean:156-164

Declaration SHA-256: fc2a8b1032448c01ece41eff0bcd5d748f70ee556d5721867bb36b59d56ce63a

Mathematical statement. For $\lambda > 1$ and $-1 < q < 1$, the exterior argument q/λ satisfies $-1 < q/\lambda < 1$.

Positivity and strict largeness of the density parameter preserve the interior arccosine domain after division.

```
theorem outerArgument_mem : -1 < a.outerArgument ^ a.outerArgument < 1 := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) a.density_jump
  have hx := a.shapeParameter_mem
  constructor
  · apply (lt_div_iff0 hlam_pos).2
    nlinarith [a.density_jump, hx.1]
  · apply (div_lt_iff0 hlam_pos).2
    nlinarith [a.density_jump, hx.2]
```

TypeThreeAssembly.outerCap

Classification: main

Source: TypeThreeAssembly.lean:251-261

Declaration SHA-256: 5482055613f2ec0454ddd9c71d722bea04494543bc659e447732870c662eb86f

Mathematical statement. The exterior cap is the upper one-sided cap with chord $2R\sin\alpha$, half-central angle α , midpoint $(0, 1)$, and upper side.

The open angle range $0 < \alpha < \pi$ permits minor, semicircular, and major branches of the upper cap.

```
def outerCap : OneSidedCircularCap where
  chord := 2 * a.radius * sin a.outerAngle
  theta := a.outerAngle
  midpointX := 0
  baseY := 1
  side := .upper
  chord_pos := mul_pos (mul_pos (by norm_num) a.radius_pos) a.outerSin_pos
  theta_pos := a.outerAngle_pos
  theta_lt_pi := a.outerAngle_lt_pi

/-- Genuine closed coordinate carrier  $E_3 = K_3 \cup \text{upper\_cap}$ . -/
```

TypeThreeAssembly.outerCap_arcLength

Classification: main

Source: TypeThreeAssembly.lean:875-884

Declaration SHA-256: e4c6a09e492a4c9395262e1c277f2cd0b185e72ae34042a7f14138b12c6b4c19

Mathematical statement. The outer cap arc length is $2R\alpha$.

The same identity applies to the exterior cap.

```
theorem outerCap_arcLength :
  a.outerCap.arcLength = 2 * a.radius * a.outerAngle := by
  rw [OneSidedCircularCap.arcLength, ell]
  change
    (2 * a.radius * sin a.outerAngle) *
    (a.outerAngle / sin a.outerAngle) =
```

```

2 * a.radius * a.outerAngle
field_simp [ne_of_gt a.outerSin_pos]

/-- CMV equation (26), derived from the branch-complete component geometry. -/

```

TypeThreeAssembly.outerCap_center

Classification: main

Source: TypeThreeAssembly.lean:929-933

Declaration SHA-256: 1057a7a30ad6e7558c1c0e586fc0147d20f1fc01e6c3e508acda901e55191c4c

Mathematical statement. The outer cap center equals the upper center $(0, 1 - R\cos\alpha)$.

The cap constructor and the assembly center definition use the same radius, base line, and exterior angle.

```

theorem outerCap_center : a.outerCap.center = a.upperCenter := by
  change (0, 1 - a.outerCap.radius * cos a.outerAngle) =
    (0, 1 - a.radius * cos a.outerAngle)
  rw [a.outerCap_radius]

```

TypeThreeAssembly.outerCap_euclideanArea

Classification: main

Source: TypeThreeAssembly.lean:851-864

Declaration SHA-256: 3c7709918bd2e08d7c7ea431b339d946f2eb6c3c9d1138f4304b6afe64a153c2

Mathematical statement. The outer cap has Euclidean area

$$R^2(\alpha - \sin\alpha\cos\alpha).$$

This is the circular-segment area for the upper exterior cap.

```

theorem outerCap_euclideanArea :
  a.outerCap.euclideanArea =
    a.radius ^ 2 *
      (a.outerAngle - sin a.outerAngle * cos a.outerAngle) := by
  rw [OneSidedCircularCap.euclideanArea, area]
  change
    (2 * a.radius * sin a.outerAngle) ^ 2 *
      ((a.outerAngle - sin a.outerAngle * cos a.outerAngle) /
        (4 * sin a.outerAngle ^ 2)) =
    a.radius ^ 2 *
      (a.outerAngle - sin a.outerAngle * cos a.outerAngle)
  field_simp [ne_of_gt a.outerSin_pos]
  ring

```

TypeThreeAssembly.outerCap_leftEndpoint

Classification: main

Source: TypeThreeAssembly.lean:934-938

Declaration SHA-256: 369e532ae6bbe742ee13471a1a8cc638a185e8c74c4f29cdde432417763b0fdb

Mathematical statement. The outer cap left endpoint is $(-R\sin\alpha, 1)$.

The chord-radius relation converts the left endpoint formula to the displayed coordinate.

```

theorem outerCap_leftEndpoint :
  a.outerCap.leftEndpoint = (-a.radius * sin a.outerAngle, 1) := by
  simp [OneSidedCircularCap.leftEndpoint, outerCap]
  ring

```

TypeThreeAssembly.outerCap_rightEndpoint

Classification: main

Source: TypeThreeAssembly.lean:939-945

Declaration SHA-256: 0bf0b00b58f0396d9b9b8bc9e08617bcd76cf3f0b76937e23b52dec1f9b5fe12

Mathematical statement. The outer cap right endpoint is $(R\sin\alpha, 1)$.

This is the reflected right endpoint on the upper interface.

```
theorem outerCap_rightEndpoint :  
  a.outerCap.rightEndpoint = (a.radius * sin a.outerAngle, 1) := by  
  simp [OneSidedCircularCap.rightEndpoint, outerCap]  
  ring  
  
/-- The left strip arc and exterior cap use the same upper junction  
coordinate. -/
```

TypeThreeAssembly.radius_mul_h

Classification: main

Source: TypeThreeAssembly.lean:197-200

Declaration SHA-256: e90d4095fef5e1e089527a179a53736b5dbf462b6149abecb5375e96b7380078

Mathematical statement. The reciprocal-radius normalization satisfies $Rh = 1$.

This identity is valid on the regular domain $h > 0$ and links curvature to the common radius.

```
theorem radius_mul_h : a.radius * a.h = 1 := by  
  rw [radius]  
  field_simp [ne_of_gt a.h_pos]
```

TypeThreeAssembly.rectangleArea

Classification: main

Source: TypeThreeAssembly.lean:273-277

Declaration SHA-256: 3314187b7b48ab958a7d8415bcf10afdb2dd85317d354dfd049f5344c2569e90

Mathematical statement. The rectangle contribution is $4w$.

The rectangle has width $2w$ and height 2.

```
def rectangleArea : ℝ := 4 * a.sideHalfWidth  
  
/-- Component area of the modeled type-(iii) profile. The side-cap term is  
the sum of the two congruent translated half-caps; the outer cap has density  
'lam`. -/
```

TypeThreeAssembly.rightUpperJunction

Classification: main

Source: TypeThreeAssembly.lean:958-968

Declaration SHA-256: ae6c6a14cc5b76c0f3aa60155a1fb16c629e4d67ae173c7593aa69c135bbff39

Mathematical statement. The right side-arc upper junction equals the outer-cap right endpoint:

$$(w + R\sin\beta, 1) = (R\sin\alpha, 1).$$

This is the symmetric right-hand junction identity.

```

theorem rightUpperJunction :
  (a.rightCenter.1 + a.radius * sin a.innerAngle, 1) =
    a.outerCap.rightEndpoint := by
  rw [a.outerCap_rightEndpoint]
  change
  (a.sideHalfWidth + a.radius * sin a.innerAngle, 1) =
    (a.radius * sin a.outerAngle, 1)
  rw [sideHalfWidth]
  ring_nf

/-- The core and upper cap overlap exactly along the upper cap's chord. -/

```

TypeThreeAssembly.scalarWeightedArea

Classification: main

Source: TypeThreeAssembly.lean:295-299

Declaration SHA-256: 195fdb382725116907cc721bab689c5c6f6f34b356a442f900ba649d89b62b9c

Mathematical statement. The scalar type-iii weighted-area formula is

$$\frac{\lambda\alpha + \pi - \beta + (q + 2)(\sin\alpha - \sin\beta)}{h^2}.$$

This definition is the geometric angle-and-displacement expression after substituting the assembly parameters.

```

def scalarWeightedArea : ℝ :=
  (a.angleTerm + (a.shapeParameter + 2) * a.sineGap) / a.h ^ 2

/-- Scalar type-(iii) weighted-perimeter formula used by the suffix
certificate. -/

```

TypeThreeAssembly.scalarWeightedPerimeter

Classification: main

Source: TypeThreeAssembly.lean:300-302

Declaration SHA-256: fcbac6602a2b133a8e714c37e1ef51bfb698ab0ae754dcd7eca0cb6e9c49204d

Mathematical statement. The scalar type-iii weighted-perimeter formula is

$$\frac{2(\lambda\alpha + \pi - \beta + \sin\alpha - \sin\beta)}{h}.$$

The formula sums the two side traces, the density-weighted upper cap, and the bottom segment in the modeled normalization.

```

def scalarWeightedPerimeter : ℝ :=
  2 * (a.angleTerm + a.sineGap) / a.h

```

TypeThreeAssembly.shapeParameter_mem

Classification: main

Source: TypeThreeAssembly.lean:153-155

Declaration SHA-256: 018fe2d07fb99b9420ec19a75327dc616f0119968b1c22fb83265de47b1261df

Mathematical statement. For $0 < h < 1$, the shape parameter $q = 2h - 1$ satisfies $-1 < q < 1$.

This places the principal inner angle in its nondegenerate arccosine domain.

```

theorem shapeParameter_mem : -1 < a.shapeParameter ∧ a.shapeParameter < 1 := by
  constructor <=> simp only [shapeParameter] <=> linarith [a.h_pos, a.h_lt_one]

```

TypeThreeAssembly.sideCap

Classification: main

Source: TypeThreeAssembly.lean:238-250

Declaration SHA-256: 62a5fb2c6003ebc71ae1cea156d819929dd2061bdfd35d03ac33c3bf896e8fd0

Mathematical statement. The auxiliary side cap is the lower one-sided cap with chord $2R\sin\beta$, half-central angle γ , midpoint $(0, 1)$, and lower side.

Its major-cap allowance packages the two congruent translated strip-side arcs into one cap whose area and arc length equal their sums.

```
def sideCap : OneSidedCircularCap where
  chord := 2 * a.radius * sin a.innerAngle
  theta := a.sideArcAngle
  midpointX := 0
  baseY := 1
  side := .lower
  chord_pos := mul_pos (mul_pos (by norm_num) a.radius_pos) a.innerSin_pos
  theta_pos := a.sideArcAngle_pos
  theta_lt_pi := a.sideArcAngle_lt_pi

/-- The upper one-sided cap. `OneSidedCircularCap` deliberately permits a
half-central-angle above  $\pi / 2$ , so this definition covers the major branch.
-/
```

TypeThreeAssembly.sideCap_arcLength

Classification: main

Source: TypeThreeAssembly.lean:865-874

Declaration SHA-256: 0735084a84605f457abd29f126bfc3b41f13fe2e361355b34befa71a89121ae9

Mathematical statement. The side cap arc length is $2R\gamma$.

The exact arc-length identity converts the side cap parameter to radius times central angle.

```
theorem sideCap_arcLength :
  a.sideCap.arcLength = 2 * a.radius * a.sideArcAngle := by
  rw [OneSidedCircularCap.arcLength, ell]
  change
    (2 * a.radius * sin a.innerAngle) *
      (a.sideArcAngle / sin a.sideArcAngle) =
    2 * a.radius * a.sideArcAngle
  rw [a.sin_sideArcAngle]
  field_simp [ne_of_gt a.innerSin_pos]
```

TypeThreeAssembly.sideCap_euclideanArea

Classification: main

Source: TypeThreeAssembly.lean:833-850

Declaration SHA-256: b6b94a03ab653ee46e011e7fa757125fdbdfe26f1473cd6911d9100fe92583e4

Mathematical statement. The side cap has Euclidean area

$$R^2(\gamma - \sin\gamma\cos\gamma).$$

The circular-segment area formula applies to the auxiliary cap with radius R and half-central angle γ .

```
theorem sideCap_euclideanArea :
  a.sideCap.euclideanArea =
    a.radius ^ 2 *
      (a.sideArcAngle -
```

```

    sin a.sideArcAngle * cos a.sideArcAngle) := by
rw [OneSidedCircularCap.euclideanArea, area]
change
  (2 * a.radius * sin a.innerAngle) ^ 2 *
    ((a.sideArcAngle -
      sin a.sideArcAngle * cos a.sideArcAngle) /
      (4 * sin a.sideArcAngle ^ 2)) =
a.radius ^ 2 *
  (a.sideArcAngle -
    sin a.sideArcAngle * cos a.sideArcAngle)
rw [a.sin_sideArcAngle]
field_simp [ne_of_gt a.innerSin_pos]
ring

```

TypeThreeAssembly.sideHalfWidth_eq_zero_iff

Classification: main

Source: TypeThreeAssembly.lean:344-351

Declaration SHA-256: aa31042682af8c9fc65e3a19a4a9ce7dc71ed7e2ad3e807732dcfcc2da35644f

Mathematical statement. $w = 0$ if and only if $h = 1/2$.

The positive radius reduces vanishing of the half-width to equality of the two principal sines, which occurs exactly at zero shape.

```

theorem sideHalfWidth_eq_zero_iff : a.sideHalfWidth = 0 ↔ a.h = 1 / 2 := by
  constructor
  · intro hzero
    by_contra hh
    linarith [a.sideHalfWidth_pos hh]
  · intro hh
    simp [sideHalfWidth, innerAngle, outerAngle, outerArgument, shapeParameter, hh]

```

TypeThreeAssembly.sideHalfWidth_nonneg

Classification: main

Source: TypeThreeAssembly.lean:338-340

Declaration SHA-256: be70cb71b25ed2f43754461915097af9183cc7aac3504655dda98d8a67b605ec

Mathematical statement. The side half-width satisfies $w \geq 0$.

It is the positive radius multiplied by $\sin\alpha - \sin\beta \geq 0$.

```

theorem sideHalfWidth_nonneg : 0 ≤ a.sideHalfWidth := by
  exact mul_nonneg a.radius_pos.le (sub_nonneg.mpr a.sin_inner_le_sin_outer)

```

TypeThreeAssembly.sin_inner_le_sin_outer

Classification: main

Source: TypeThreeAssembly.lean:303-311

Declaration SHA-256: def468198882d033570072298a83ae1d41fc003f0ecb9d0b4ee3a6b94e6a1d10

Mathematical statement. For every regular assembly, $\sin\beta \leq \sin\alpha$.

The density factor $\lambda > 1$ contracts the arccosine argument in absolute value, and the principal sine formula gives the non-strict comparison.

```

theorem sin_inner_le_sin_outer : sin a.innerAngle ≤ sin a.outerAngle := by
  rw [innerAngle, outerAngle, Real.sin_arccos, Real.sin_arccos]
  apply Real.sqrt_le_sqrt
  have hlam_pos : 0 < lam := lt_trans (by norm_num) a.density_jump

```

```

have habs : |a.outerArgument| ≤ |a.shapeParameter| := by
  rw [outerArgument, abs_div, abs_of_pos hlam_pos]
  exact div_le_self (abs_nonneg _) (le_of_lt a.density_jump)
exact sub_le_sub_left (sq_le_sq.mpr habs) 1

```

TypeThreeAssembly.sin_inner_lt_sin_outer

Classification: main

Source: TypeThreeAssembly.lean:312-337

Declaration SHA-256: b3a6348544f17baf0cf9a8d51663f5b8b6187359294117163fc5d2afba78dd58

Mathematical statement. If $h \neq 1/2$, then $\sin\beta < \sin\alpha$.

At the only equality case the shape parameter is zero; excluding $h = 1/2$ makes the squared argument comparison strict.

```

theorem sin_inner_lt_sin_outer (hh : a.h ≠ 1 / 2) :
  sin a.innerAngle < sin a.outerAngle := by
  rw [innerAngle, outerAngle, Real.sin_arccos, Real.sin_arccos]
  have hlam_pos : 0 < lam := lt_trans (by norm_num) a.density_jump
  have hx : a.shapeParameter ≠ 0 := by
    intro hzero
    apply hh
    simp only [shapeParameter] at hzero
    linarith
  have hx_sq : 0 < a.shapeParameter ^ 2 := sq_pos_of_ne_zero hx
  have hlam_sq : 1 < lam ^ 2 := by nlinarith [a.density_jump]
  have harg_sq : a.outerArgument ^ 2 < a.shapeParameter ^ 2 := by
    rw [outerArgument, div_pow]
    apply (div_lt_iff0 (sq_pos_of_pos hlam_pos)).2
    nlinarith
  have habs : |a.shapeParameter| < 1 := abs_lt.mpr a.shapeParameter_mem
  have hshape_sq : a.shapeParameter ^ 2 ≤ 1 := by
    have habs_le : |a.shapeParameter| ≤ |(1 : ℝ)| := by
      simpa using le_of_lt habs
    have hsq : a.shapeParameter ^ 2 ≤ (1 : ℝ) ^ 2 :=
      sq_le_sq.mpr habs_le
    simpa using hsq
  have hleft : 0 ≤ 1 - a.shapeParameter ^ 2 := by
    linarith
  exact Real.sqrt_lt_sqrt hleft (by nlinarith)

```

TypeThreeAssembly.sineGap

Classification: main

Source: TypeThreeAssembly.lean:292-294

Declaration SHA-256: e9d7a7074e27fe28a23aa0db9c4cf9b0ef4e5f2181f1cfab6098f7cc2e240ecb

Mathematical statement. The declaration defines the type-iii angle contribution $\lambda\alpha + \pi - \beta$ or sine displacement $\sin\alpha - \sin\beta$.

These are the two geometric combinations that enter the area and perimeter formulas.

```

def sineGap : ℝ := sin a.outerAngle - sin a.innerAngle

/-- Scalar type-(iii) weighted-area formula used by the suffix certificate. -/

```

TypeThreeAssembly.volume_coreCarrier

Classification: main

Source: TypeThreeAssembly.lean:1481-1529

Declaration SHA-256: e422839d8a82e578aa8619bf8a6d240c5dc840fd4d4e1880ecc09a0307558380

Mathematical statement. The Euclidean planar volume of the type-iii core equals the rectangle area plus one side-cap area.

The two circular side pieces contribute the same segment area, while their overlaps with the rectangle and each other lie on planar-null joining sets; hence the core volume is $4w + a_{\text{side}}$.

```
theorem volume_coreCarrier :
  volume a.coreCarrier =
    ENNReal.ofReal (a.rectangleArea + a.sideCap.euclideanArea) := by
have hrectLeft :
  AEDisjoint volume a.rectangleCarrier a.leftSegmentCarrier := by
  refine measure_mono_null ?_ (volume_verticalline (-a.sideHalfWidth))
  rintro p ⟨hrect, hleft⟩
  exact le_antisymm hleft.2.1 hrect.1.1
have hfirstRight :
  AEDisjoint volume
    (a.rectangleCarrier ∪ a.leftSegmentCarrier)
    a.rightSegmentCarrier := by
  refine measure_mono_null ?_ (volume_verticalline a.sideHalfWidth)
  rintro p ⟨hfirst, hright⟩
  rcases hfirst with hrect | hleft
  · exact le_antisymm hrect.1.2 hright.2.1
  · have hx1 : p.1 ≤ -a.sideHalfWidth := hleft.2.1
    have hxr : a.sideHalfWidth ≤ p.1 := hright.2.1
    exact le_antisymm (by linarith [a.sideHalfWidth_nonneg]) hxr
have hrectangle : 0 ≤ a.rectangleArea := by
  exact mul_nonneg (by norm_num) a.sideHalfWidth_nonneg
calc
  volume a.coreCarrier =
    volume a.rectangleCarrier + volume a.leftSegmentCarrier +
    volume a.rightSegmentCarrier := by
  rw [coreCarrier,
    measure_union,
    a.isClosed_rightSegmentCarrier.measurableSet.nullMeasurableSet
    hfirstRight,
    measure_union,
    a.isClosed_leftSegmentCarrier.measurableSet.nullMeasurableSet
    hrectLeft]
_ = ENNReal.ofReal a.rectangleArea +
  (volume a.sideCapLeftHalf + volume a.sideCapRightHalf) := by
  rw [a.volume_rectangleCarrier, a.volume_leftSegmentCarrier,
    a.volume_rightSegmentCarrier, add_assoc]
_ = ENNReal.ofReal a.rectangleArea + volume a.sideCap.carrier := by
  rw [a.volume_sideCap_partition]
_ = ENNReal.ofReal a.rectangleArea +
  ENNReal.ofReal a.sideCap.euclideanArea := by
  rw [a.sideCap.volume_carrier]
_ = ENNReal.ofReal
  (a.rectangleArea + a.sideCap.euclideanArea) := by
  exact
    (ENNReal.ofReal_add hrectangle
      a.sideCap.euclideanArea_pos.le).symm

/-- The actual density integral over the strip core is its Euclidean component
area. -/
```

TypeThreeAssembly.weightedArea_eq_modeledWeightedArea

Classification: main

Source: TypeThreeAssembly.lean:1562-1583

Declaration SHA-256: 5f7913f0171c133683a735330062b186a060d0e518987c5488549de7bfd06347

Mathematical statement. The canonical weighted area of the type-iii carrier equals its rectangle-plus-cap modeled area expression.

The core density is one and the exterior cap density is λ away from its interface chord; the carrier decomposition therefore agrees with the stored modeled calculation.

```
theorem weightedArea_eq_modeledWeightedArea :
  a.WeightedArea = a.modeledWeightedArea := by
  have hcore : IntegrableOn (StripDensity lam) a.coreCarrier := by
    apply stripDensity_integrableOn_of_volume_ne_top
    rw [a.volume_coreCarrier]
    exact ENNReal.ofReal_ne_top
  have hcap := cap_integrableOn lam a.outerCap
  unfold TypeThreeAssembly.WeightedArea _root_.WeightedArea
  rw [carrier,
    setIntegral_union, a.carrier_core_outer_aedisjoint
    a.outerCap.measurableSet_carrier.nullMeasurableSet hcore hcap]
  rw [show (∫ p in a.coreCarrier, StripDensity lam p) =
    a.rectangleArea + a.sideCap.euclideanArea by
    simp [a._root_.WeightedArea] using a.core_weightedArea_formula,
    show (∫ p in a.outerCap.carrier, StripDensity lam p) =
    lam * a.outerCap.euclideanArea by
    simp [a._root_.WeightedArea] using
    cap_upper_weightedArea lam a.outerCap rfl rfl,
    modeledWeightedArea]

/-- CMV equation (26) is the exact canonical weighted area of the coordinate
carrier, not only a sum assigned to its modeled components. -/
```

TypeThreeAssembly.weightedArea_formula

Classification: main

Source: TypeThreeAssembly.lean:1584-1588

Declaration SHA-256: 53616c2c25615a95257aa4773c87d4c69464dc905650e6ed5b53696dc5f08570

Mathematical statement. The canonical weighted area of the type-iii carrier equals its scalar area formula.

Substituting the angle term and sine gap into the geometric decomposition gives the analytic expression used as $A_3(h)$.

```
theorem weightedArea_formula :
  a.WeightedArea = a.scalarWeightedArea := by
  rw [a.weightedArea_eq_modeledWeightedArea,
    a.modeledWeightedArea_formula]
```

TypeThreeAssembly.weightedPerimeter_eq_modeledWeightedPerimeter

Classification: main

Source: TypeThreeAssembly.lean:2950-2993

Declaration SHA-256: 3dd334c8f3cff9146f23a5d90e6da2520151c567d49c8604446b90aa7a9f4d2b

Mathematical statement. The canonical complete-frontier weighted perimeter of the type-iii carrier equals its modeled component perimeter.

The frontier decomposition, exact Hausdorff lengths, density integrals, and null finite joins show that no hidden chord or extra boundary is counted.

```

theorem weightedPerimeter_eq_modeledWeightedPerimeter :
  a.WeightedPerimeter = a.modeledWeightedPerimeter := by
  rcases a.boundaryTrace_pairwise with
  (hAB, hAC, hAD, hBC, hBD, hCD)
  have hAint := euclideanStripDensity_integrableOn_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph '' a.leftArcTrace) (by
      rw [a.exact_euclidean_leftArcTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
  have hBint := euclideanStripDensity_integrableOn_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph '' a.rightArcTrace) (by
      rw [a.exact_euclidean_rightArcTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
  have hCint := euclideanStripDensity_integrableOn_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph ''
      OneSidedCircularCap.arcTrace a.outerCap) (by
      rw [exact_euclidean_capTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
  have hDint := euclideanStripDensity_integrableOn_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph '' a.bottomSegmentCarrier) (by
      rw [a.exact_euclidean_bottomSegmentCarrier_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
  unfold TypeThreeAssembly.WeightedPerimeter _root_.WeightedPerimeter
  FrontierMeasure
  rw [integral_map
    planeEuclideanHomeomorph.symm.continuous.measurable.aemeasurable
    (measurable_stripDensity lam).aestronglyMeasurable]
  change (∫ z in frontier (planeEuclideanHomeomorph '' a.carrier),
    euclideanStripDensity lam z
      ∂(μH[1] : Measure EuclideanPlane)) =
    a.modeledWeightedPerimeter
  rw [a.euclidean_frontier]
  rw [integral_four_union hAB hAC hAD hBC hBD hCD
    a.measurableSet_euclidean_rightArcTrace
    (measurableSet_euclidean_capTrace a.outerCap)
    a.measurableSet_euclidean_bottomSegmentCarrier
    hAint hBint hCint hDint]
  rw [a.leftArc_integral, a.rightArc_integral,
    upper_cap_integral lam a.outerCap rfl rfl,
    a.bottomSegment_integral, modeledWeightedPerimeter,
    a.sideCap_arcLength]
  ring

/-- CMV equation (26) is the exact canonical weighted perimeter of the
coordinate carrier, not only a sum assigned to its modeled components. -/

```

TypeThreeAssembly.weightedPerimeter_formula

Classification: main

Source: TypeThreeAssembly.lean:2994-2999

Declaration SHA-256: c6f51109b06f9f61cbbcb63e9d6e1d950e5a79c8decdcf76b0e2fc3f366f8eca

Mathematical statement. The canonical weighted perimeter of the type-iii carrier equals its scalar perimeter formula.

The geometric expression becomes $P_3(h)$ after the principal-angle identities and the common-radius relation are substituted.

```

theorem weightedPerimeter_formula :
  a.WeightedPerimeter = a.scalarWeightedPerimeter := by
  rw [a.weightedPerimeter_eq_modeledWeightedPerimeter,
    a.modeledWeightedPerimeter_formula]

/-- The density is integrable over the complete coordinate carrier. -/

```

WeightedArea

Classification: main

Source: CMVGeometry.lean:55-60

Declaration SHA-256: 959b5ddd7363861d4d0d79c211781b3b19e09df50087472690ae0581725b8c6d

Mathematical statement. The weighted area of a carrier E is $A_\lambda(E) = \int_E f_\lambda d\mathcal{L}^2$.

This is a real integral over the coordinate carrier; the geometric realization proves integrability rather than assuming a scalar area.

```
def WeightedArea (lam : ℝ) (carrier : Set PlanePoint) : ℝ :=
  ∫ p in carrier, StripDensity lam p

/-- Canonical Euclidean one-dimensional Hausdorff measure on the complete
topological frontier of a planar carrier, mapped back to the coordinate plane.
A general competitor cannot choose perimeter data independently of its region. -/
```

WeightedPerimeter

Classification: main

Source: CMVGeometry.lean:67-73

Declaration SHA-256: 8d1f1a60a6d4c5ce03ccda31c1bc34ec4f837b6ce55275e4b0c06fe04e566dba

Mathematical statement. For a boundary measure μ , weighted perimeter is $\int f_\lambda d\mu$.

The definition accepts an arbitrary measure; canonical carrier perimeter is obtained by using the frontier measure.

```
def WeightedPerimeter (lam : ℝ)
  (boundaryMeasure : Measure PlanePoint) : ℝ :=
  ∫ p, StripDensity lam p ∂boundaryMeasure

/-- General finite-perimeter input to minimization. Its perimeter measure is
canonically the one-dimensional Hausdorff measure restricted to the complete
frontier; finiteness is an invariant, not independently supplied data. -/
```

area

Classification: main

Source: MorgansArcFunction.lean:92-93

Declaration SHA-256: 8c44e8dff416b2208afe91aad54bf1678ad6f2299c75592fe1fed4f6cbf55dc2

Mathematical statement. The normalized cap area on a unit chord is $a(\theta) = (\theta - \sin\theta\cos\theta)/(4\sin^2\theta)$.

Scaling the chord by L multiplies this area by L^2 ; the geometric interpretation uses $0 < \theta < \pi$.

```
noncomputable def area (θ : ℝ) : ℝ := (θ - sin θ * cos θ) / (4 * sin θ ^ 2)
```

cap_arcLength_eq_two_radius_theta

Classification: main

Source: FrontierPerimeter.lean:451-456

Declaration SHA-256: edad6e24948a1499ce1a66a5efdc84d9c8f7899f5c52d6551716312e95163fb3

Mathematical statement. For every one-sided cap c ,

$$L \frac{\theta}{\sin\theta} = 2R\theta.$$

The chord-radius relation converts the normalized scalar arc length into the geometric arc length. The cap hypotheses give $L > 0$ and $0 < \theta < \pi$, so division by $\sin\theta$ is valid.

```
lemma cap_arclength_eq_two_radius_theta (c : OneSidedCircularCap) :
  c.arclength = 2 * c.radius * c.theta := by
  rw [OneSidedCircularCap.arclength, OneSidedCircularCap.radius, ell]
  field_simp [ne_of_gt c.sin_theta_pos]

/-- Exact Euclidean  $H^1$  mass for every existing `OneSidedCircularCap`, including major arcs. -/
```

ell

Classification: main

Source: MorgansArcFunction.lean:89-91

Declaration SHA-256: 9adb32c6bf9324acec1e791c0ec46f0e17df835ffee7b3549cab07bdc3a760c9

Mathematical statement. The normalized arc length on a unit chord is $\ell(\theta) = \theta/\sin\theta$.

For the geometric interval $0 < \theta < \pi$, chord-radius scaling gives the usual circular arc length; the real definition is total outside that interval.

```
noncomputable def ell (θ : ℝ) : ℝ := θ / sin θ

/-- Area enclosed by a unit-chord circular segment with half-central-angle `θ`. -/
```

exact_euclidean_capParam_hausdorffMeasure

Classification: main

Source: FrontierPerimeter.lean:457-483

Declaration SHA-256: ce3a2c8ff8b380f7da5859563d117de2848052560e457ef5f88e25aa88808f43

Mathematical statement. The Euclidean $\mathcal{H}^1$ -measure of the indicated realized arc or segment trace equals its declared scalar length.

The relevant parametrization is continuous, injective on its closed parameter interval, and an isometric image of a unit-speed circular or affine path. Endpoints and finite joins do not alter the one-dimensional measure.

```
theorem exact_euclidean_capParam_hausdorffMeasure (c : OneSidedCircularCap) :
  (μH[1] : Measure EuclideanPlane)
  (realizedCapParam c '' Icc (capStart c) (capEnd c)) =
  ENNReal.ofReal c.arclength := by
  have hfun : realizedCapParam c = capComplexIsometryMap c ∘ unitCircleArc c.radius := by
    funext s
    exact realizedCapParam_eq c s
  calc
    (μH[1] : Measure EuclideanPlane)
    (realizedCapParam c '' Icc (capStart c) (capEnd c)) =
    (μH[1] : Measure EuclideanPlane)
    (capComplexIsometryMap c ''
      (unitCircleArc c.radius '' Icc (capStart c) (capEnd c))) := by
      rw [hfun]
      congr 1
      simp only [Function.comp_apply] using (Set.image_image _ _ _).symm
  _ = (μH[1] : Measure ℂ)
    (unitCircleArc c.radius '' Icc (capStart c) (capEnd c)) :=
    (isometry_capComplexIsometryMap c).hausdorffMeasure_image
    (Or.inl (by norm_num)) _
  _ = ENNReal.ofReal (2 * c.radius * c.theta) := by
    simp [capStart, capEnd] using
      hausdorffMeasure_unitCircleArc_image_eq c.radius_pos
      ⟨c.theta_pos, c.theta_lt_pi⟩
  _ = ENNReal.ofReal c.arclength := by rw [cap_arclength_eq_two_radius_theta]
```

exact_euclidean_capTrace_hausdorffMeasure

Classification: main

Source: FrontierPerimeter.lean:1081-1088

Declaration SHA-256: 8a0602f46d010a0d38a019d5e1052cfe4c5d69fc3db5988eaac8c5dbcc5c5e1c

Mathematical statement. The Euclidean $\mathcal{H}^1$ -measure of the indicated realized arc or segment trace equals its declared scalar length.

The relevant parametrization is continuous, injective on its closed parameter interval, and an isometric image of a unit-speed circular or affine path. Endpoints and finite joins do not alter the one-dimensional measure.

```
theorem exact_euclidean_capTrace_hausdorffMeasure
  (c : OneSidedCircularCap) :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' c.arcTrace) =
  ENNReal.ofReal c.arcLength := by
  rw [← realizedCapParam_image_Icc]
  exact exact_euclidean_capParam_hausdorffMeasure c
```

exact_euclidean_leftArcTrace_hausdorffMeasure

Classification: main

Source: FrontierPerimeter.lean:821-827

Declaration SHA-256: f6b756b038d2dd53ff9e214034534494d6216a6823376bb73c389a754a1ad0d9

Mathematical statement. The Euclidean $\mathcal{H}^1$ -measure of the indicated realized arc or segment trace equals its declared scalar length.

The relevant parametrization is continuous, injective on its closed parameter interval, and an isometric image of a unit-speed circular or affine path. Endpoints and finite joins do not alter the one-dimensional measure.

```
theorem exact_euclidean_leftArcTrace_hausdorffMeasure (c : StripCore) :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' c.leftArcTrace) =
  ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  rw [← realizedLeftCoreParam_image_Icc]
  exact exact_euclidean_leftCoreParam_hausdorffMeasure c
```

exact_euclidean_rightArcTrace_hausdorffMeasure

Classification: main

Source: FrontierPerimeter.lean:828-834

Declaration SHA-256: 47e43c0cab0cb015904a2fe672eeec0720a25537d058b25dd2b767b05bcf1d39

Mathematical statement. The Euclidean $\mathcal{H}^1$ -measure of the indicated realized arc or segment trace equals its declared scalar length.

The relevant parametrization is continuous, injective on its closed parameter interval, and an isometric image of a unit-speed circular or affine path. Endpoints and finite joins do not alter the one-dimensional measure.

```
theorem exact_euclidean_rightArcTrace_hausdorffMeasure (c : StripCore) :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' c.rightArcTrace) =
  ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  rw [← realizedRightCoreParam_image_Icc]
  exact exact_euclidean_rightCoreParam_hausdorffMeasure c
```

exact_euclidean_segmentCarrier_hausdorffMeasure

Classification: main

Source: FrontierPerimeter.lean:937-946

Declaration SHA-256: f2e6ec79942c7df8c258fab2922138cb782506dbaa0184c81a94b12e6e9b9896

Mathematical statement. The Euclidean $\mathcal{H}^1$ -measure of the indicated realized arc or segment trace equals its declared scalar length.

The relevant parametrization is continuous, injective on its closed parameter interval, and an isometric image of a unit-speed circular or affine path. Endpoints and finite joins do not alter the one-dimensional measure.

```
theorem exact_euclidean_segmentCarrier_hausdorffMeasure
  (seg : HorizontalSegment) :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' seg.carrier) =
    ENNReal.ofReal seg.euclideanLength := by
  rw [← realizedSegmentParam_image_carrier,
    realizedSegmentParam_image_affineSegment,
    hausdorffMeasure_affineSegment,
    segmentEndpoint_edist]
```

fourArc_euclidean_frontier

Classification: main

Source: FrontierPerimeter.lean:1338-1348

Declaration SHA-256: 748ff0ec3e12377aa952eabaeafb59718df5bc069c8651bdcd45a9c687a1d41d

Mathematical statement. The Euclidean frontier of a four-arc assembly is the union of its two strip-side arcs and its two exterior cap arcs.

The interface chords are internal to the union and therefore do not contribute to the complete frontier.

```
theorem fourArc_euclidean_frontier (a : FourArcAssembly) :
  frontier (planeEuclideanHomeomorph '' a.carrier) =
    (planeEuclideanHomeomorph '' StripCore.leftArcTrace a.core) ∪
    ((planeEuclideanHomeomorph '' StripCore.rightArcTrace a.core) ∪
      (planeEuclideanHomeomorph ''
        OneSidedCircularCap.arcTrace a.upperCap) ∪
      (planeEuclideanHomeomorph ''
        OneSidedCircularCap.arcTrace a.lowerCap))) := by
  rw [← planeEuclideanHomeomorph.image_frontier, a.frontier_carrier]
  simp only [FourArcAssembly.boundaryTrace, image_union]
```

fourArc_frontier_weightedPerimeter_eq

Classification: main

Source: FrontierPerimeter.lean:1794-1844

Declaration SHA-256: e95ee1864c7bc7d9bc3a3ec2659625a510c6cefccdaf0cd2efab0e2d93e1067

Mathematical statement. The explicit four-arc component perimeter equals the canonical weighted perimeter obtained from the complete frontier.

The four exposed arcs are exactly the frontier, their pairwise intersections are $\mathcal{H}^1$ -null, and the density integrals give the component formula.

```
theorem fourArc_frontier_weightedPerimeter_eq
  (lam : ℝ) (assembly : FourArcAssembly) :
  assembly.weightedPerimeter lam =
    WeightedPerimeter lam (FrontierMeasure assembly.carrier) := by
```

```

rcases fourArc_pairwise assembly with
  (hAB, hAC, hAD, hBC, hBD, hCD)
have hAint := euclideanStripDensity_integrableOn_of_measure_ne_top
  lam (s := planeEuclideanHomeomorph '')
    StripCore.leftArcTrace assembly.core) (by
      rw [exact_euclidean_leftArcTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
have hBint := euclideanStripDensity_integrableOn_of_measure_ne_top
  lam (s := planeEuclideanHomeomorph '')
    StripCore.rightArcTrace assembly.core) (by
      rw [exact_euclidean_rightArcTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
have hCint := euclideanStripDensity_integrableOn_of_measure_ne_top
  lam (s := planeEuclideanHomeomorph '')
    OneSidedCircularCap.arcTrace assembly.upperCap) (by
      rw [exact_euclidean_capTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
have hDint := euclideanStripDensity_integrableOn_of_measure_ne_top
  lam (s := planeEuclideanHomeomorph '')
    OneSidedCircularCap.arcTrace assembly.lowerCap) (by
      rw [exact_euclidean_capTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
rw [FourArcAssembly.weightedPerimeter_formula]
symm
unfold WeightedPerimeter FrontierMeasure
rw [integral_map
  planeEuclideanHomeomorph.symm.continuous.measurable.aemeasurable
  (measurable_stripDensity lam).aestronglyMeasurable]
change (f z in frontier (planeEuclideanHomeomorph '' assembly.carrier),
  euclideanStripDensity lam z
  ∂(μH[1] : Measure EuclideanPlane)) =
  assembly.core.boundaryArcLength +
  lam * (assembly.upperCap.arcLength + assembly.lowerCap.arcLength)
rw [fourArc_euclidean_frontier]
rw [integral_four_union hAB hAC hAD hBC hBD hCD
  (measurableSet_euclidean_rightArcTrace assembly.core)
  (measurableSet_euclidean_capTrace assembly.upperCap)
  (measurableSet_euclidean_capTrace assembly.lowerCap)
  hAint hBint hCint hDint]
rw [left_integral, right_integral,
  upper_cap_integral lam assembly.upperCap rfl rfl,
  lower_cap_integral lam assembly.lowerCap rfl rfl,
  core_param_length, StripCore.boundaryArcLength]
ring

/-- The explicit replacement boundary cost equals the canonical weighted H¹
integral on the complete topological frontier of its coordinate carrier. -/

```

hausdorffMeasure_unitCircleArc_image_eq

Classification: main

Source: FrontierPerimeter.lean:294-311

Declaration SHA-256: f4746d8c9b2c6ef1416bccca1f814590cdef08c393e10b7189ff167eb0402ab3

Mathematical statement. If $r > 0$ and $0 < \theta < \pi$, then

$$\mathcal{H}^1(\gamma_r([-r\theta, r\theta])) = \text{ofReal}(2r\theta).$$

The Lipschitz upper bound and polygonal lower bounds meet, giving the exact one-dimensional Hausdorff measure of the injective circular arc. The positive-radius premise is essential to the geometric statement.

```

theorem hausdorffMeasure_unitCircleArc_image_eq {r theta : ℝ} (hr : 0 < r)
  (htheta : theta ∈ Ioo 0 Real.pi) :
  (μH[1] : Measure ℂ)
    (unitCircleArc r '' Icc (-r * theta) (r * theta)) =
    ENNReal.ofReal (2 * r * theta) := by
  apply le_antisymm
  · calc

```

```

    (μH[1] : Measure ℂ)
      (unitCircleArc r '' Icc (-r * theta) (r * theta)) ≤
      ENNReal.ofReal (r * theta - (-r * theta)) :=
      hausdorffMeasure_unitCircleArc_image_le hr
    _ = ENNReal.ofReal (2 * r * theta) := by congr 1 <|> ring
  · apply le_of_tendsto'
    (ENNReal.tendsto_ofReal (tendsto_cap_polygon_length (r := r) htheta.1))
  intro n
  exact cap_partition_polygon_lower hr htheta (n + 1) (Nat.zero_lt_succ n)

/-- Rotation of the standard complex circle by the signed-arclength phase `m`. -/

```

hausdorffMeasure_unitCircleArc_image_eq_of_span_lt_two_pi

Classification: main

Source: FrontierPerimeter.lean:350-389

Declaration SHA-256: a5a68294ffad2a36064599c08079161b28ea03171c66437e00e8f24eba632c26

Mathematical statement. An injective circular arc whose signed-arclength span is less than $2\pi r$ has Euclidean $\mathcal{H}^1$ -measure equal to its parameter span.

The strict span bound prevents wraparound, so the circular image is a single non-self-overlapping arc and its one-dimensional Hausdorff measure is $\text{ofReal}(v - u)$.

```

theorem hausdorffMeasure_unitCircleArc_image_eq_of_span_lt_two_pi
  {r u v : ℝ} (hr : 0 < r) (huv : u < v)
  (hspan : v - u < 2 * Real.pi * r) :
  (μH[1] : Measure ℂ) (unitCircleArc r '' Icc u v) =
    ENNReal.ofReal (v - u) := by
  let theta : ℝ := (v - u) / (2 * r)
  let m : ℝ := (u + v) / 2
  have htheta : theta ∈ Ioo 0 Real.pi := by
    dsimp [theta]
    constructor
  · exact div_pos (sub_pos.mpr huv) (mul_pos (by norm_num) hr)
  · apply (div_lt_iff₀ (mul_pos (by norm_num) hr)).2
    nlinarith [Real.pi_pos]
  have hleft : -r * theta + m = u := by
    dsimp [theta, m]
    field_simp [hr.ne']
    ring
  have hright : r * theta + m = v := by
    dsimp [theta, m]
    field_simp [hr.ne']
    ring
  calc
    (μH[1] : Measure ℂ) (unitCircleArc r '' Icc u v) =
      (μH[1] : Measure ℂ)
        (unitCircleArcRotation r m ''
          (unitCircleArc r '' Icc (-r * theta) (r * theta))) := by
          rw [unitCircleArcRotation_image_Icc hr.ne', hleft, hright]
    _ = (μH[1] : Measure ℂ)
      (unitCircleArc r '' Icc (-r * theta) (r * theta)) :=
      (isometry_unitCircleArcRotation r m).hausdorffMeasure_image
      (Or.inl (by norm_num)) _
    _ = ENNReal.ofReal (2 * r * theta) :=
      hausdorffMeasure_unitCircleArc_image_eq hr htheta
    _ = ENNReal.ofReal (v - u) := by
      congr 1
      dsimp [theta]
      field_simp [hr.ne']

/-- Translation and possible reflection carrying the standard complex circle to a cap. -/

```

AdmissibleCompetitor.carrier

Classification: supplement

Source: CMVGeometry.lean:3961-3967

Declaration SHA-256: b57d4c4c33776a05ffbbbc22f1b363a7f356aa7cd089d7c2b3335ead1bb65644

Mathematical statement. Each admissible competitor has its underlying coordinate carrier, with the empty constructor assigned the empty set.

The other cases return the carrier of the stored region or assembly.

```
def carrier {lam : ℝ} : AdmissibleCompetitor lam → Set PlanePoint
| .empty => ∅
| .general region => region.carrier
| .fourArc a => a.carrier
| .replacement a => a.carrier

/-- Weighted area on the regular constructor model. -/
```

CMVSuffixModel.FourArcCandidate.regular_weightedArea_eq_typeFourArea

Classification: supplement

Source: CMVSuffixModel.lean:142-149

Declaration SHA-256: 48c010f5756d8d71b74c4c11f50c789619d79d3e32a79f62433bedf4290ea164

Mathematical statement. For a CMV candidate with $h < 1$, its weighted area equals $A_4(h)$.

The candidate already supplies $h > 0$, angle bounds, and the incidence hypothesis; the strict upper bound records the regular subcase.

```
theorem regular_weightedArea_eq_typeFourArea
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  (_hregular : candidate.h < 1) :
  candidate.WeightedArea =
    LeanSuffixAnalytic.typeFourArea lam candidate.h :=
  weightedArea_eq_typeFourArea_all candidate hcandidate

/-- Exact type-(iv) perimeter identification on the regular source domain. -/
```

CMVSuffixModel.FourArcCandidate.regular_weightedPerimeter_eq_typeFourPerimeter

Classification: supplement

Source: CMVSuffixModel.lean:150-158

Declaration SHA-256: 7c7abd9d4131a70966d7ee06757ae9000eb6c0370ad6826014e6939680589328

Mathematical statement. For a CMV candidate with $h < 1$, its weighted perimeter equals $P_4(h)$.

This is the regular-domain identification of the complete-frontier perimeter with the analytic formula.

```
theorem regular_weightedPerimeter_eq_typeFourPerimeter
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  (_hregular : candidate.h < 1) :
  candidate.WeightedPerimeter =
    LeanSuffixAnalytic.typeFourPerimeter lam candidate.h :=
  weightedPerimeter_eq_typeFourPerimeter_all candidate hcandidate

/-- Exact type-(iv) area identification at the source endpoint `h_4 = 1`.
This is separate from the regular theorem and invokes no fold expression. -/
```

CMVSuffixModel.TypeThreeAssembly.angleTerm_eq_typeThreeAngle

Classification: supplement

Source: CMVSuffixModel.lean:42-52

Declaration SHA-256: fe9ab761a6d7819237c18cc5b6d5b714cc699cb312b84007d27510fce813ca50

Mathematical statement. For every regular type-iii assembly, its geometric angle term equals $B(q(h)) + \pi/2$.

The identity rewrites $\lambda\alpha + \pi - \beta$ using $\arcsin q + \arccos q = \pi/2$, matching the analytic formula.

```
theorem angleTerm_eq_typeThreeAngle :
  a.angleTerm = LeanSuffixAnalytic.typeThreeAngle lam a.h +  $\pi$  / 2 := by
  rw [_root_.TypeThreeAssembly.angleTerm,
      _root_.TypeThreeAssembly.outerAngle,
      _root_.TypeThreeAssembly.outerArgument,
      _root_.TypeThreeAssembly.innerAngle,
      LeanSuffixAnalytic.typeThreeAngle_add_halfPi]
  rfl

/-- The coordinate carrier's horizontal sine displacement is exactly the
radical displacement in the retained analytic type-(iii) formula. -/
```

CMVSuffixModel.TypeThreeAssembly.sineGap_eq_typeThreeDelta

Classification: supplement

Source: CMVSuffixModel.lean:53-77

Declaration SHA-256: 0b54dd8dd926b7670949326ff7f7fb7d21250d76068f1da2eb207447216f1aac

Mathematical statement. The geometric sine gap of a regular type-iii assembly equals $D(q(h))$.

The principal-angle sine identities turn $\sin\alpha - \sin\beta$ into the difference of the two nonnegative radicals.

```
theorem sineGap_eq_typeThreeDelta :
  a.sineGap = LeanSuffixAnalytic.typeThreeDelta lam a.h := by
  have hlam_pos : 0 < lam :=
  lt_trans (by norm_num : (0 :  $\mathbb{R}$ ) < 1) a.density_jump
  have hshape := a.shapeParameter_mem
  have hminus : 0 < lam - a.shapeParameter := by
    linarith [hshape.2, a.density_jump]
  have hplus : 0 < lam + a.shapeParameter := by
    linarith [hshape.1, a.density_jump]
  have hrad : 0 ≤ lam ^ 2 - a.shapeParameter ^ 2 := by
    nlinarith [mul_pos hminus hplus]
  rw [_root_.TypeThreeAssembly.sineGap,
      _root_.TypeThreeAssembly.outerAngle,
      _root_.TypeThreeAssembly.innerAngle, Real.sin_arccos,
      Real.sin_arccos, _root_.TypeThreeAssembly.outerArgument,
      LeanSuffixAnalytic.typeThreeDelta]
  change sqrt (1 - (a.shapeParameter / lam) ^ 2) -
    sqrt (1 - a.shapeParameter ^ 2) =
    sqrt (lam ^ 2 - a.shapeParameter ^ 2) / lam -
    sqrt (1 - a.shapeParameter ^ 2)
  rw [sqrt_one_sub_div_sq hlam_pos hrad]

/-- Exact identification of CMV equation (26)'s area with the analytic
quantity used by the suffix theorem. Its domain is the full regular
`0 < h3 < 1` type-(iii) domain, across all three arc branches. -/
```

CapSide

Classification: supplement

Source: CMVGeometry.lean:92-99

Declaration SHA-256: 6863ffc025769eb8169e9582a3d673fe180025902f766c2557257f6937cc7a7e

Mathematical statement. A cap side is either upper or lower.

This two-element choice determines which closed half-plane is intersected with the cap's disk.

```
inductive CapSide where
  | upper
  | lower
  deriving DecidableEq

/-- A branch-correct circular segment on a horizontal chord. Its carrier is
`disk ∩ half-plane`; this single definition includes minor, semicircular, and
major caps. -/
```

EuclideanPlane

Classification: supplement

Source: CMVGeometry.lean:34-37

Declaration SHA-256: dfa2db26da624a2b5333b886c ff8c9cd3151218ea8aa3e3cb39c78359b32470e

Mathematical statement. The Euclidean realization of the coordinate plane has the norm $\sqrt{x^2 + y^2}$.

The underlying pairs are unchanged, but the metric for Hausdorff measure is the Euclidean product norm.

```
abbrev EuclideanPlane := WithLp 2 (ℝ × ℝ)

/-- The coordinate-preserving homeomorphism from the existing coordinate plane
to its Euclidean L2 realization. -/
```

FinitePerimeterRegion.weightedArea

Classification: supplement

Source: CMVGeometry.lean:83-85

Declaration SHA-256: 3892310b68bd795c4ddff8f1d276f58be0efcb96c1e8c4fad96ec7d73deea20a

Mathematical statement. The weighted area of a general modeled region is the weighted integral over its stored carrier.

No independent area value is stored: this operation applies A_λ to the carrier.

```
def weightedArea {lam : ℝ} (r : FinitePerimeterRegion lam) : ℝ :=
  WeightedArea lam r.carrier
```

FourArcAssembly.boundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:2525-2529

Declaration SHA-256: a07e8d5b98aab97a737c50d9ee9eae871142e1adb2b58a3132be8c4b6a093cce

Mathematical statement. The assembly boundary measure is the sum of its two core arc measures and two cap arc measures.

It is an explicit component measure; equality of its weighted cost with complete-frontier cost is proved later.

```
def boundaryMeasure (a : FourArcAssembly) : Measure PlanePoint :=
  leftCoreBoundaryMeasure a.core + rightCoreBoundaryMeasure a.core +
  capBoundaryMeasure a.upperCap + capBoundaryMeasure a.lowerCap

/-- The explicit four-arc boundary measure gives every endpoint zero mass. -/
```

FourArcAssembly.boundaryTrace_subset_frontier_carrier

Classification: supplement

Source: CMVGeometry.lean:3244-3251

Declaration SHA-256: 160385ee99b171abc63b7cc7cb5ade687af8d5249c1ab26a5150be49b8c4654b

Mathematical statement. The four-arc assembly boundary trace is contained in its complete frontier.

The converse inclusion confirms that none of the four arcs is hidden inside the assembled region.

```
theorem boundaryTrace_subset_frontier_carrier (a : FourArcAssembly) :
  a.boundaryTrace ⊆ frontier a.carrier := by
  rintro p (hleft | hright | hupper | hlower)
  · exact leftArc_mem_frontier a hleft
  · exact rightArc_mem_frontier a hright
  · exact upperArc_mem_frontier a hupper
  · exact lowerArc_mem_frontier a hlower
```

FourArcAssembly.frontier_carrier_subset_boundaryTrace

Classification: supplement

Source: CMVGeometry.lean:2795-2819

Declaration SHA-256: 0d72e7b21e3528564863313aa2366a2e4806e01c00a4772872911d424abf3cc5

Mathematical statement. The complete frontier of a four-arc assembly is contained in its boundary trace.

This inclusion rules out additional exposed boundary components beyond the four declared arcs.

```
theorem frontier_carrier_subset_boundaryTrace (a : FourArcAssembly) :
  frontier a.carrier ⊆ a.boundaryTrace := by
  intro p hp
  have htop := frontier_union_subset
    (a.core.carrier ∪ a.upperCap.carrier) a.lowerCap.carrier hp
  rcases htop with hfirst | hlower
  · have hmiddle := frontier_union_subset a.core.carrier a.upperCap.carrier hfirst.1
    rcases hmiddle with hcore | hupper
    · rw [a.core.frontier_carrier] at hcore
      rcases hcore.1 with hleft | hright | hupperChord | hlowerChord
      · exact Or.inl hleft
      · exact Or.inr (Or.inl hright)
      · exact chord_to_boundaryTrace a (Or.inl hupperChord.1) hupperChord.2 hp
      · exact chord_to_boundaryTrace a (Or.inr hlowerChord.1) hlowerChord.2 hp
    · rw [a.upperCap.frontier_carrier] at hupper
      rcases hupper.2 with harc | hchord
      · exact Or.inr (Or.inr (Or.inl harc))
      · exact chord_to_boundaryTrace a (Or.inl hchord.1)
        (by simp [upperCap] using hchord.2) hp
  · rw [a.lowerCap.frontier_carrier] at hlower
    rcases hlower.2 with harc | hchord
    · exact Or.inr (Or.inr (Or.inr harc))
    · exact chord_to_boundaryTrace a (Or.inr hchord.1)
      (by simp [lowerCap] using hchord.2) hp
```

FourArcAssembly.lowerCap

Classification: supplement

Source: CMVGeometry.lean:2509-2518

Declaration SHA-256: a60a9393b363872f08f12dbab41380b51afeffa746b9504c5841f28d389091bfe

Mathematical statement. The assembly's lower cap has chord L , half-angle α , midpoint $(0, -1)$, and lower side.

It is the horizontal reflection of the upper cap with identical Euclidean area and arc length.

```
def lowerCap (a : FourArcAssembly) : OneSidedCircularCap where
  chord := a.core.chord
  theta := a.outerAngle
  midpointX := 0
  baseY := -1
  side := .lower
  chord_pos := a.core.chord_pos
  theta_pos := a.outerAngle_pos
  theta_lt_pi := lt_trans a.outerAngle_lt_pi_div_two (by linarith [Real.pi_pos])
```

FourArcAssembly.upperCap

Classification: supplement

Source: CMVGeometry.lean:2499-2508

Declaration SHA-256: 3f084f0dbec6bee8992e556fe2ca304aae5eb8981815636bf15454a3b8e613f9

Mathematical statement. The assembly's upper cap has chord L , half-angle α , midpoint $(0, 1)$, and upper side.

These data fix its carrier above the strip and its shared top chord.

```
def upperCap (a : FourArcAssembly) : OneSidedCircularCap where
  chord := a.core.chord
  theta := a.outerAngle
  midpointX := 0
  baseY := 1
  side := .upper
  chord_pos := a.core.chord_pos
  theta_pos := a.outerAngle_pos
  theta_lt_pi := lt_trans a.outerAngle_lt_pi_div_two (by linarith [Real.pi_pos])
```

FourArcAssembly.weightedArea

Classification: supplement

Source: CMVGeometry.lean:2541-2545

Declaration SHA-256: ad43970e4ac79b935baed41701092f4d901053e35bb88583dcbd5a9003db13a3

Mathematical statement. The assembly weighted area is the integral A_λ of its carrier.

The scalar component formula follows from this definition rather than being part of the assembly data.

```
def weightedArea (lam : ℝ) (a : FourArcAssembly) : ℝ :=
  WeightedArea lam a.carrier

/-- Weighted perimeter is the density integral against the explicit regular
boundary measure. -/
```

FourArcAssembly.weightedPerimeter

Classification: supplement

Source: CMVGeometry.lean:2546-2548

Declaration SHA-256: df8ef218db7076c4681d4506265ae824cfa4b1a2ec872c320aeb4f283b25493b

Mathematical statement. The assembly component perimeter is the density integral against its four-arc boundary measure.

It initially uses the explicit arc-measure sum; the frontier theorem identifies it with canonical carrier perimeter.

```
def weightedPerimeter (lam : ℝ) (a : FourArcAssembly) : ℝ :=  
  WeightedPerimeter lam a.boundaryMeasure
```

FourArcCandidate.beta

Classification: supplement

Source: FourArcCandidate.lean:39-41

Declaration SHA-256: f3520601e601026585d5024a59c3bee3be514d265897b65e66919a9d3ea7fd1e

Mathematical statement. The candidate angle β is $\pi/2 - \arcsin h$.

Thus $\cos\beta = h$, including the endpoint $h = 1$.

```
def beta : ℝ := π / 2 - arcsin candidate.h  
  
/-- Common horizontal chord length of the two exterior caps. -/
```

FourArcCandidate.capArea_eq_chord_sq_mul_area

Classification: supplement

Source: FourArcCandidate.lean:104-109

Declaration SHA-256: 0e9545ae1662d69d55500dc9264e42c4c5b03b78571f6473a85822aca53c6073

Mathematical statement. One original cap area equals $L^2 a(\alpha)$, where $L = 2\sin\alpha/h$.

Substituting the normalized circular-segment area cancels the positive sine factors and yields the radius-based expression.

```
theorem capArea_eq_chord_sq_mul_area :  
  candidate.capArea = candidate.capChord ^ 2 * area candidate.alpha := by  
  rw [capArea, capChord, area]  
  field_simp [ne_of_gt candidate.h_pos, ne_of_gt candidate.sin_alpha_pos]  
  ring
```

FourArcCandidate.lowerCap_euclideanArea

Classification: supplement

Source: FourArcCandidate.lean:159-166

Declaration SHA-256: 14a860cf99daaf759c3fa7838aed86edfb2879de1a5a3f6f7d84f9996c7da155

Mathematical statement. The lower cap of the constructed candidate has Euclidean scalar area equal to the candidate cap-area formula.

Horizontal reflection preserves the same chord, angle, and Euclidean area.

```
theorem lowerCap_euclideanArea :  
  candidate.assembly.lowerCap.euclideanArea = candidate.capArea := by  
  rw [OneSidedCircularCap.euclideanArea,  
    candidate.capArea_eq_chord_sq_mul_area]
```

```
rfl
```

```
-- The endpoint-incidence definition forces the exterior and strip radii to agree. -/
```

FourArcCandidate.side_caps_area_formula

Classification: supplement

Source: FourArcCandidate.lean:351-363

Declaration SHA-256: a639bd8c9eef40f2c4f87fdb62b671cf7db0ce41013e7d923721f6f472bc8cf1

Mathematical statement. The combined side-segment area is $8a(\operatorname{arcsinh}) = 2(\operatorname{arcsinh} - h\sqrt{1-h^2})/h^2$.

The identity is valid for every candidate, including $h = 1$, and uses the principal sine and cosine formulas.

```
theorem side_caps_area_formula :
  8 * area candidate.stripCore.sideAngle =
    2 * (arcsin candidate.h -
      candidate.h * √(1 - candidate.h ^ 2)) / candidate.h ^ 2 := by
change 8 * area (arcsin candidate.h) =
  2 * (arcsin candidate.h -
    candidate.h * √(1 - candidate.h ^ 2)) / candidate.h ^ 2
rw [area, Real.sin_arcsin (by linarith [candidate.h_pos])
  candidate.h_le_one, Real.cos_arcsin]
field_simp [ne_of_gt candidate.h_pos]
ring

/-- Candidate area as the sum of the actual core and cap constructors. -/
```

FourArcCandidate.sin_alpha_pos

Classification: supplement

Source: FourArcCandidate.lean:95-98

Declaration SHA-256: b92e14d3743fcaa1d6c447d96c6334ba3f5f129152aa26ff47b97d6fd8de2e28

Mathematical statement. For every candidate, $\sin \alpha > 0$.

The stored bound $0 < \alpha < \pi/2$ ensures a nondegenerate chord and justifies denominator cancellation.

```
theorem sin_alpha_pos : 0 < sin candidate.alpha :=
  sin_pos_of_pos_of_lt_pi candidate.alpha_pos
  (lt_trans candidate.alpha_lt_pi_div_two (by linarith [Real.pi_pos]))
```

FourArcCandidate.upperCap_euclideanArea

Classification: supplement

Source: FourArcCandidate.lean:153-158

Declaration SHA-256: 63eb34459de3e09e8c932883edd9446c50efc2cc1a148ef60c5c77cbd56d7496

Mathematical statement. The upper cap of the constructed candidate has Euclidean scalar area equal to the candidate cap-area formula.

The cap constructor uses exactly the candidate chord and angle, so its normalized-area scaling agrees.

```
theorem upperCap_euclideanArea :
  candidate.assembly.upperCap.euclideanArea = candidate.capArea := by
rw [OneSidedCircularCap.euclideanArea,
  candidate.capArea_eq_chord_sq_mul_area]
rfl
```

HorizontalSegment

Classification: supplement

Source: CMVGeometry.lean:392-399

Declaration SHA-256: e7bebc91d2e392d80b21e5e8b5487591a5477fe10be16627e5b6180dd120506

Mathematical statement. A horizontal segment is specified by length $L > 0$ and midpoint (m, b) .

This auxiliary constructor is nondegenerate; the type-iii bottom segment has a separate description allowing zero length.

```
structure HorizontalSegment where
  chord : ℝ
  midpointX : ℝ
  baseY : ℝ
  chord_pos : 0 < chord

namespace HorizontalSegment
```

HorizontalSegment.carrier

Classification: supplement

Source: CMVGeometry.lean:406-408

Declaration SHA-256: 094b96b3006204081e6d336250f56e63f6b337cc71255210b9bae8ae3b894041

Mathematical statement. The segment carrier is $\{(x, b) : |x - m| \leq L/2\}$.

This is a closed horizontal interval, including its two endpoints.

```
def carrier (s : HorizontalSegment) : Set PlanePoint :=
  {p | p.2 = s.baseY ∧ |p.1 - s.midpointX| ≤ s.chord / 2}
```

HorizontalSegment.euclideanLength

Classification: supplement

Source: CMVGeometry.lean:409-415

Declaration SHA-256: c965c7e2ff7089171e9318c88e3733eb9629da0051af6a9aad082bf1ad5a5aef

Mathematical statement. The scalar Euclidean length of the horizontal segment is L .

The value is later identified with its Euclidean Hausdorff measure.

```
def euclideanLength (s : HorizontalSegment) : ℝ := s.chord

end HorizontalSegment

/-- The unchanged central part of a type-(iv) candidate. It is the rectangle
of height two and width `chord`, together with the two radius-`1/h` side
segments. -/
```

HorizontalSegment.leftEndpoint

Classification: supplement

Source: CMVGeometry.lean:400-402

Declaration SHA-256: d64f9e0ea48cfbbc1b06bed874068659f0011957eca2b8226c738430113a0615

Mathematical statement. The segment's left endpoint is $(m - L/2, b)$.

The point is obtained by translating the midpoint left by half the chord length.

```
def leftEndpoint (s : HorizontalSegment) : PlanePoint :=
  (s.midpointX - s.chord / 2, s.baseY)
```

HorizontalSegment.rightEndpoint

Classification: supplement

Source: CMVGeometry.lean:403-405

Declaration SHA-256: a14791633639531ab0cb8801c8e0e16383dd7712b7d5730aa7f0d9178c83e5da

Mathematical statement. The segment's right endpoint is $(m + L/2, b)$.

The point is obtained by translating the midpoint right by half the chord length.

```
def rightEndpoint (s : HorizontalSegment) : PlanePoint :=
  (s.midpointX + s.chord / 2, s.baseY)
```

LeanSuffixAnalytic.centered_meanValue_exists

Classification: supplement

Source: LeanSuffixAnalytic.lean:93-129

Declaration SHA-256: 9e16c64246998ba49ad719b4074913f0337c8612efed42e98f32935fcd55a3e5

Mathematical statement. If f has derivative $g(x)$ throughout the closed interval between c and h , some ξ in that interval satisfies $f(h) = f(c) + g(\xi)(h - c)$.

This centered mean value theorem covers either order of the endpoints and also their equality.

```
theorem centered_meanValue_exists
  (f f' : ℝ → ℝ) {c h : ℝ}
  (hderiv : ∀ x ∈ Set.uIcc c h, HasDerivAt f (f' x) x) :
  ∃ ξ ∈ Set.uIcc c h, f h = f c + f' ξ * (h - c) := by
  rcases lt_trichotomy c h with hlt | heq | hgt
  · have hcont : ContinuousOn f (Set.Icc c h) := by
    intro x hx
    exact (hderiv x (Set.mem_uIcc.mpr (Or.inl hx))).continuousAt.continuousWithinAt
  have hdiff : DifferentiableOn ℝ f (Set.Ioo c h) := by
    intro x hx
    exact (hderiv x (Set.mem_uIcc.mpr
      (Or.inl ⟨hx.1.le, hx.2.le⟩))).differentiableAt.differentiableWithinAt
  rcases exists_deriv_eq_slope f hlt hcont hdiff with ⟨ξ, hξ, hslope⟩
  have hξu : ξ ∈ Set.uIcc c h :=
    Set.mem_uIcc.mpr (Or.inl ⟨hξ.1.le, hξ.2.le⟩)
  refine ⟨ξ, hξu, ?_⟩
  rw [(hderiv ξ hξu).deriv] at hslope
  field_simp [ne_of_gt (sub_pos.mpr hlt)] at hslope
  linarith
  · subst h
    exact ⟨c, by simp, by ring⟩
  · have hcont : ContinuousOn f (Set.Icc h c) := by
    intro x hx
    exact (hderiv x (Set.mem_uIcc.mpr (Or.inr hx))).continuousAt.continuousWithinAt
  have hdiff : DifferentiableOn ℝ f (Set.Ioo h c) := by
    intro x hx
    exact (hderiv x (Set.mem_uIcc.mpr
      (Or.inr ⟨hx.1.le, hx.2.le⟩))).differentiableAt.differentiableWithinAt
  rcases exists_deriv_eq_slope f hgt hcont hdiff with ⟨ξ, hξ, hslope⟩
  have hξu : ξ ∈ Set.uIcc c h :=
    Set.mem_uIcc.mpr (Or.inr ⟨hξ.1.le, hξ.2.le⟩)
  refine ⟨ξ, hξu, ?_⟩
  rw [(hderiv ξ hξu).deriv] at hslope
  field_simp [ne_of_gt (sub_pos.mpr hgt)] at hslope
  linarith
/-- The type-(iii) angle term is exactly the checkers' `B3` expression after
the substitution `q = cos w`. -/
```

LeanSuffixAnalytic.equalArea_gap_identity

Classification: supplement

Source: LeanSuffixAnalytic.lean:1208-1220

Declaration SHA-256: 7a449a0d73201ece195743b096dbde5032c8598ab1748985129132f301a8ba9d

Mathematical statement. If $r, s \neq 0$ and $A_3(r) = A_4(s)$, then $P_3(r) - P_4(s) = 4[(r - s)B(s)/s^2 + (r/s)D(s) - D(q(r))]$.

This algebraic identity eliminates the type-iii angle term using equal area. It imposes no density or regularity restriction beyond the stated nonzero denominators.

```
theorem equalArea_gap_identity {lam h3 h4 : ℝ}
  (hh3 : h3 ≠ 0) (hh4 : h4 ≠ 0)
  (harea : typeThreeArea lam h3 = typeFourArea lam h4) :
  typeThreePerimeter lam h3 - typeFourPerimeter lam h4 =
    4 * (((h3 - h4) / h4 ^ 2) * typeFourAngle lam h4 +
      h3 / h4 * typeFourDelta lam h4 - typeThreeDelta lam h3) := by
  apply equalArea_gap_algebra hh3 hh4
  unfold typeThreeArea typeFourArea typeThreeShape at harea
  convert harea using 1
  all_goals ring

/-- At a stationary type-(iv) fold, the exact perimeter gap is four times the
single reduced radical expression. -/
```

LeanSuffixAnalytic.hasDerivAt_typeThreeAngle

Classification: supplement

Source: LeanSuffixAnalytic.lean:154-205

Declaration SHA-256: 6ec918819525fbc766cbbf40969985e480000f0bcbcf23d536b033e9e80b6380

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, the derivative of $B(q(h))$ is $2[1/\sqrt{1 - q(h)^2} - \lambda/\sqrt{\lambda^2 - q(h)^2}]$.

The regular domain ensures both inverse-trigonometric arguments are interior and both radical denominators are nonzero.

```
theorem hasDerivAt_typeThreeAngle {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeAngle lam)
    (2 * (1 / sqrt (1 - typeThreeShape h ^ 2) -
      lam / sqrt (lam ^ 2 - typeThreeShape h ^ 2))) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hshape := hasDerivAt_typeThreeShape h
  have hq_low : -1 < typeThreeShape h := by
    unfold typeThreeShape
    linarith
  have hq_high : typeThreeShape h < 1 := by
    unfold typeThreeShape
    linarith
  have hu_arg : 0 < 1 - typeThreeShape h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - typeThreeShape h ^ 2 := by nlinarith
  have hd_ne : sqrt (lam ^ 2 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  have hrratio_low : -1 < typeThreeShape h / lam :=
    (lt_div_iff0 hlam_pos).2 (by linarith)
  have hrratio_high : typeThreeShape h / lam < 1 :=
    (div_lt_one hlam_pos).2 (by linarith)
  have hrratio_ne_neg_one : typeThreeShape h / lam ≠ -1 :=
    ne_of_gt hrratio_low
  have hrratio_ne_one : typeThreeShape h / lam ≠ 1 :=
    ne_of_lt hrratio_high
  have hq_ne_neg_one : typeThreeShape h ≠ -1 := ne_of_gt hq_low
  have hq_ne_one : typeThreeShape h ≠ 1 := ne_of_lt hq_high
  have hsqrt_scale :
    sqrt (1 - (typeThreeShape h / lam) ^ 2) =
```

```

    sqrt (lam ^ 2 - typeThreeShape h ^ 2) / lam := by
  have hratio_identity :
    1 - (typeThreeShape h / lam) ^ 2 =
      (lam ^ 2 - typeThreeShape h ^ 2) / lam ^ 2 := by
    field_simp [hlam_ne]
  rw [hratio_identity, Real.sqrt_div hd_arg.le, Real.sqrt_sq_eq_abs,
    abs_of_pos hlam_pos]
  unfold typeThreeAngle
  have harccos :=
    (Real.hasDerivAt_arccos (x := typeThreeShape h / lam)
      hratio_ne_neg_one hratio_ne_one).comp h (hshape.div_const lam)
  have harcsin :=
    (Real.hasDerivAt_arcsin (x := typeThreeShape h)
      hq_ne_neg_one hq_ne_one).comp h hshape
  have hraw := (harccos.const_mul lam).add harcsin
  simpa only [Function.comp_apply, id_eq] using! hraw.congr_deriv (by
    rw [hsqrt_scale]
    field_simp [hlam_ne, hd_ne]
    ring)

/-- Exact derivative of the regular type-(iii) radial displacement on its
natural open domain. -/

```

LeanSuffixAnalytic.hasDerivAt_typeThreeDelta

Classification: supplement

Source: LeanSuffixAnalytic.lean:206-240

Declaration SHA-256: 6a259e36542b424c2657b96b59d66f9d38d054281073fe5c590c3110a33522c8

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, the derivative of $D(q(h))$ is $2q(h)[1/\sqrt{1 - q(h)^2} - 1/(\lambda\sqrt{\lambda^2 - q(h)^2})]$.

Differentiate the two square roots and apply $q'(h) = 2$, retaining the common shape argument.

```

theorem hasDerivAt_typeThreeDelta {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeDelta lam)
    (2 * typeThreeShape h *
      (1 / sqrt (1 - typeThreeShape h ^ 2) -
        1 / (lam * sqrt (lam ^ 2 - typeThreeShape h ^ 2)))) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hshape := hasDerivAt_typeThreeShape h
  have hq_low : -1 < typeThreeShape h := by
    unfold typeThreeShape
    linarith
  have hq_high : typeThreeShape h < 1 := by
    unfold typeThreeShape
    linarith
  have hu_arg : 0 < 1 - typeThreeShape h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - typeThreeShape h ^ 2 := by nlinarith
  have hu_ne : sqrt (1 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hu_arg)
  have hd_ne : sqrt (lam ^ 2 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  unfold typeThreeDelta
  have houter :=
    (((hasDerivAt_const h (lam ^ 2)).sub (hshape.pow 2)).sqrt
      (ne_of_gt hd_arg)).div_const lam)
  have hinner :=
    (((hasDerivAt_const h 1).sub (hshape.pow 2)).sqrt (ne_of_gt hu_arg))
  have hraw := houter.sub hinner
  simpa only [id_eq] using! hraw.congr_deriv (by
    norm_num [id_eq, Pi.sub_apply, Pi.pow_apply]
    field_simp [hlam_ne, hu_ne, hd_ne]
    ring)

```

```

/-- Exact derivative of the regular type-(iv) angle contribution on its
natural open domain. -/

```

LeanSuffixAnalytic.hasDerivAt_typeThreeShape

Classification: supplement

Source: LeanSuffixAnalytic.lean:146-153

Declaration SHA-256: fc5b59d714869fde00b69f1950c83bd7a32c47524e01ad8d04455ff346c39d58

Mathematical statement. The shape function has derivative $q'(h) = 2$ for every real h .

This elementary affine derivative supplies the chain-rule factor in the type-iii calculations.

```

theorem hasDerivAt_typeThreeShape (h : ℝ) :
  HasDerivAt typeThreeShape 2 h := by
  unfold typeThreeShape
  simp only [id_eq, mul_one] using
    ((hasDerivAt_id h).const_mul (2 : ℝ)).sub_const 1

/-- Exact derivative of the regular type-(iii) angle contribution on its
natural open domain. -/

```

LeanSuffixAnalytic.reducedFoldGap

Classification: supplement

Source: LeanSuffixAnalytic.lean:83-92

Declaration SHA-256: 90c9f564cc40a2e05814e95c52b8ea113ecc45335e828ff2a54c8e9f2caf08b3

Mathematical statement. The reduced stationary gap is

$$G(r, s) = (r - s)[1/\sqrt{1 - s^2} - \lambda/\sqrt{\lambda^2 - s^2}] / s + (r/s)D(s) - D(q(r)).$$

Equal-area and stationary-fold identities make $P_3(r) - P_4(s) = 4G(r, s)$. This algebraic elimination is retained in the closure but is not needed to prove negativity via concavity.

```

def reducedFoldGap (lam h3 h4 : ℝ) : ℝ :=
  (h3 - h4) / h4 *
    (1 / sqrt (1 - h4 ^ 2) - lam / sqrt (lam ^ 2 - h4 ^ 2)) +
    h3 / h4 * typeFourDelta lam h4 - typeThreeDelta lam h3

/-- Correlation-preserving centered mean-value identity. Unlike a bound
assumed as part of a certificate schema, this produces the intermediate point
from the mean value theorem. Every occurrence in `f` ξ` therefore shares the
same `ξ`, which is the analytic justification for evaluating a derivative
expression once on the whole height interval. -/

```

LeanSuffixAnalytic.stationaryEqualArea_gap_identity

Classification: supplement

Source: LeanSuffixAnalytic.lean:1221-1238

Declaration SHA-256: 7ca190ef8d3e84eb702882d4666162a0576db626556bb2591b0ba437f522132f

Mathematical statement. If $r, s \neq 0$, $A_3(r) = A_4(s)$, and $F_4(s) = 0$, then $P_3(r) - P_4(s) = 4G(r, s)$.

Substitute the stationary relation for $B(s)$ into the equal-area gap identity; the theorem is an exact algebraic reduction.

```

theorem stationaryEqualArea_gap_identity {lam h3 h4 : ℝ}
  (hh3 : h3 ≠ 0) (hh4 : h4 ≠ 0)
  (harea : typeThreeArea lam h3 = typeFourArea lam h4)
  (hfold : typeFourFold lam h4 = 0) :
  typeThreePerimeter lam h3 - typeFourPerimeter lam h4 =
    4 * reducedFoldGap lam h3 h4 := by

```

```

rw [equalArea_gap_identity hh3 hh4 harea]
unfold typeFourFold at hfold
unfold reducedFoldGap
have hangle : typeFourAngle lam h4 =
  h4 * (1 / sqrt (1 - h4 ^ 2) -
    lam / sqrt (lam ^ 2 - h4 ^ 2)) := by
  linarith
rw [hangle]
field_simp [hh4]

/-- Precise remaining scalar obligation for a direct suffix proof: negativity
of the reduced fold gap is equivalent to strict type-(iii) improvement. -/

```

LeanSuffixAnalytic.stationaryEqualArea_typeThree_improves_iff

Classification: supplement

Source: LeanSuffixAnalytic.lean:1239-1252

Declaration SHA-256: d313a9a9b483a140c805abe01de0a03576005db4ae69b2c417e95c5e91300b63

Mathematical statement. Under the same nonzero, equal-area, and stationary hypotheses, $P_3(r) < P_4(s)$ is equivalent to $G(r, s) < 0$.

The exact factor relating the two gaps is four, so their strict signs agree.

```

theorem stationaryEqualArea_typeThree_improves_iff {lam h3 h4 : ℝ}
  (hh3 : h3 ≠ 0) (hh4 : h4 ≠ 0)
  (harea : typeThreeArea lam h3 = typeFourArea lam h4)
  (hfold : typeFourFold lam h4 = 0) :
  typeThreePerimeter lam h3 < typeFourPerimeter lam h4 ↔
  reducedFoldGap lam h3 h4 < 0 := by
  rw [← sub_lt_zero]
  rw [stationaryEqualArea_gap_identity hh3 hh4 harea hfold]
  constructor <=> intro h <=> linarith

/-- Both strict radical inequalities used by the retained fold- and
equal-area-root uniqueness arguments follow uniformly from the suffix domain;
they do not require interval cells. -/

```

LeanSuffixAnalytic.typeFourClearedFold_continuousOn

Classification: supplement

Source: TypeFourStationaryRoot.lean:35-40

Declaration SHA-256: aad0d8aa37abd7a9983d9c3fef33a66d571f8cc007154660095f3a1d473e9d30

Mathematical statement. For every real λ , C is continuous on $[9/10, 1]$.

This is the restriction of its global continuity to the interval used for the stationary-root intermediate value argument.

```

theorem typeFourClearedFold_continuousOn (lam : ℝ) :
  ContinuousOn (typeFourClearedFold lam) (Set.Icc (9 / 10 : ℝ) 1) :=
  (typeFourClearedFold_continuous lam).continuousOn

/-- On the regular domain, the cleared expression is the actual source fold
multiplied by its two positive radical denominators. -/

```

LeanSuffixAnalytic.typeThreeAngle_add_halfPi

Classification: supplement

Source: LeanSuffixAnalytic.lean:130-138

Declaration SHA-256: 309f9c361ad038b7eae75200b2b081130c71a155a43f2f07c9ad2ae282ce68ff

Mathematical statement. For all real parameters, $B(q(h)) + \pi/2 = \lambda \arccos(q(h)/\lambda) + \pi - \arccos(q(h))$.

The identity uses the globally valid relation between principal arcsine and arccosine and aligns the analytic and coordinate angle terms.

```
theorem typeThreeAngle_add_halfPi (lam h : ℝ) :
  typeThreeAngle lam h + π / 2 =
    lam * arccos (typeThreeShape h / lam) + π -
      arccos (typeThreeShape h) := by
  rw [typeThreeAngle, Real.arcsin_eq_pi_div_two_sub_arccos]
  ring

/-- The type-(iv) angle term is exactly the checkers' `B4` expression after
the substitution `h = cos x`. -/
```

LeanSuffixAnalytic.typeThreeArea_deriv_eq_explicit

Classification: supplement

Source: LeanSuffixAnalytic.lean:649-699

Declaration SHA-256: ec96d74eb7c231919df47f4c5a7bcce391a0bf68733b2ba3385beb5c882b9ca6

Mathematical statement. If $\lambda > 1$ and $0 < h < 1$, put $q = 2h - 1$,

$$U = \frac{1}{\sqrt{1-q^2}} - \frac{\lambda}{\sqrt{\lambda^2 - q^2}}, \quad V = \frac{1}{\sqrt{1-q^2}} - \frac{1}{\lambda \sqrt{\lambda^2 - q^2}}.$$

Then

$$A_3'(h) = \frac{(2(2U + 2qV)h - 2(B(q) + \pi/2 + D(q)))}{h^3}.$$

The derivative of the shape displacement contributes $2qV$, so the numerator contains $2U + 2qV$, not $2U + qV$. This is the exact explicit derivative identity on the regular domain.

```
theorem typeThreeArea_deriv_eq_explicit {lam h : ℝ}
  (h1lam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  deriv (typeThreeArea lam) h =
    ((2 * (2 * (1 / sqrt (1 - typeThreeShape h ^ 2)) -
      lam / sqrt (lam ^ 2 - typeThreeShape h ^ 2)) +
      2 * typeThreeShape h *
        (1 / sqrt (1 - typeThreeShape h ^ 2) -
          1 / (lam * sqrt (lam ^ 2 - typeThreeShape h ^ 2)))) * h -
      2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h)) /
      h ^ 3) := by
  let q := typeThreeShape h
  let p := 2 * (1 / sqrt (1 - q ^ 2) -
    lam / sqrt (lam ^ 2 - q ^ 2))
  let dp := 2 * q * (1 / sqrt (1 - q ^ 2) -
    1 / (lam * sqrt (lam ^ 2 - q ^ 2)))
  have hh_ne : h ≠ 0 := ne_of_gt hh
  have hangle : HasDerivAt (typeThreeAngle lam) p h := by
    simpa only [p, q] using hasDerivAt_typeThreeAngle h1lam hh hh1
  have hdelta : HasDerivAt (typeThreeDelta lam) dp h := by
    simpa only [dp, q] using hasDerivAt_typeThreeDelta h1lam hh hh1
  have hperimeterRaw :=
    (((hangle.add_const (π / 2)).add hdelta).const_mul (2 : ℝ)).div
      (hasDerivAt_id h) hh_ne
```

```

have hperimeter : HasDerivAt (typeThreePerimeter lam)
  (((2 * (p + dp)) * h -
    2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h)) /
    h ^ 2) h := by
  simp only [typeThreePerimeter, id_eq] using!
  hperimeterRaw.congr_deriv (by
    norm_num [id_eq, Pi.add_apply, Pi.mul_apply, Pi.pow_apply])
have hvariational :=
  typeThreePerimeter_deriv_eq_height_mul_area_deriv hlam hh hh1
rw [hperimeter.deriv] at hvariational
change deriv (typeThreeArea lam) h =
  (((2 * (p + dp)) * h -
    2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h)) /
    h ^ 3)
apply (eq_div_iff (pow_ne_zero 3 hh_ne)).2
calc
  deriv (typeThreeArea lam) h * h ^ 3 =
    (h * deriv (typeThreeArea lam) h) * h ^ 2 := by ring
_ = (((2 * (p + dp)) * h -
  2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h)) /
  h ^ 2) * h ^ 2 := by rw [hvariational]
_ = (2 * (p + dp)) * h -
  2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h) := by
  field_simp [hh_ne]

/-- The checker quantity `typeThreeFold` is exactly the numerator of the
type-(iii) area derivative. -/

```

OneSidedCircularCap.arcLength_pos

Classification: supplement

Source: CMVGeometry.lean:220-222

Declaration SHA-256: 615464c0ac73967d872be72b9e5d931fb0644d2db70b6e9168ad1dd7c81aa184

Mathematical statement. Every cap's scalar arc length is strictly positive.

It is the product of the positive chord length and positive normalized arc length.

```

theorem arcLength_pos (c : OneSidedCircularCap) : 0 < c.arcLength :=
  mul_pos c.chord_pos c.ell_theta_pos

```

OneSidedCircularCap.arcPoint

Classification: supplement

Source: CMVGeometry.lean:124-133

Declaration SHA-256: 2f12b44987266410d1b21333c27225fca467817287b4028245b465856f5540c3

Mathematical statement. The cap arc is parameterized by $(m + R\sin t, b \pm R(\cos t - \cos \theta))$, with the plus sign for the upper side.

The intended angular interval is $[-\theta, \theta]$; the formula itself is defined for every real t .

```

def arcPoint (c : OneSidedCircularCap) (t : ℝ) : PlanePoint :=
  match c.side with
  | .upper =>
    (c.midpointX + c.radius * sin t,
     c.baseY + c.radius * (cos t - cos c.theta))
  | .lower =>
    (c.midpointX + c.radius * sin t,
     c.baseY - c.radius * (cos t - cos c.theta))

/-- Left endpoint of the oriented chord. -/

```

OneSidedCircularCap.arcPoint_in_closed_side

Classification: supplement

Source: CMVGeometry.lean:240-252

Declaration SHA-256: 4692c99be4beede3a58f1e164acd1cfe21db7a9160da63340de9490acd46c184

Mathematical statement. For $|t| \leq \theta$, the cap arc lies in its chosen closed half-plane.

On this angular interval, $\cos t \geq \cos \theta$; the vertical sign distinguishes the upper and lower statements.

```
theorem arcPoint_in_closed_side (c : OneSidedCircularCap) {t : ℝ}
  (ht : |t| ≤ c.theta) :
  match c.side with
  | .upper => c.baseY ≤ (c.arcPoint t).2
  | .lower => (c.arcPoint t).2 ≤ c.baseY := by
  have hc : cos c.theta ≤ cos t := by
    rw [← Real.cos_abs t]
    exact Real.cos_le_cos_of_nonneg_of_le_pi (abs_nonneg t)
      (le_of_lt c.theta_lt_pi) ht
  have hr := c.radius_pos
  cases h : c.side <|> simp [arcPoint, h] <|> nlinarith

/-- A non-endpoint arc point is strictly on the cap side of the chord. -/
```

OneSidedCircularCap.arcPoint_in_open_side

Classification: supplement

Source: CMVGeometry.lean:253-266

Declaration SHA-256: 3dbf6a865d1d6331f9c3406c05af249acf00d7c487349d982b588394a814d697

Mathematical statement. For $|t| < \theta$, the cap arc lies in its chosen open half-plane.

The strict cosine inequality separates all interior arc points from the chord line.

```
theorem arcPoint_in_open_side (c : OneSidedCircularCap) {t : ℝ}
  (ht : |t| < c.theta) :
  match c.side with
  | .upper => c.baseY < (c.arcPoint t).2
  | .lower => (c.arcPoint t).2 < c.baseY := by
  have hc : cos c.theta < cos t := by
    rw [← Real.cos_abs t]
    exact Real.cos_lt_cos_of_nonneg_of_le_pi (abs_nonneg t)
      (le_of_lt c.theta_lt_pi) ht
  have hr := c.radius_pos
  cases h : c.side <|> simp [arcPoint, h] <|> nlinarith

/-- The selected circular arc has no self-intersections, including on the
major branch. -/
```

OneSidedCircularCap.arcPoint_left

Classification: supplement

Source: CMVGeometry.lean:228-232

Declaration SHA-256: 4e22ae8901c6f895322d9f1591e370b613671c823882adf707a5fac1ae750287

Mathematical statement. At $t = -\theta$, the cap arc reaches the left endpoint.

Oddness of sine and the chord-radius identity give horizontal coordinate $m - L/2$; the vertical displacement vanishes.

```
theorem arcPoint_left (c : OneSidedCircularCap) :
  c.arcPoint (-c.theta) = c.leftEndpoint := by
```

```
cases h : c.side <=>
  simp [arcPoint, leftEndpoint, h, sin_neg, c.radius_mul_sin] <=> ring
```

OneSidedCircularCap.arcPoint_right

Classification: supplement

Source: CMVGeometry.lean:233-239

Declaration SHA-256: ba639553cc75debbcac47cb7f164b0cdee0e14e9586cdc6d1cf857f7fa27f21b

Mathematical statement. At $t = \theta$, the cap arc reaches the right endpoint.

The chord-radius identity gives horizontal coordinate $m + L/2$, and the point lies on the base line.

```
theorem arcPoint_right (c : OneSidedCircularCap) :
  c.arcPoint c.theta = c.rightEndpoint := by
  cases h : c.side <=>
  simp [arcPoint, rightEndpoint, h, c.radius_mul_sin]

/-- Every point of the selected arc lies in the declared closed half-plane,
including on the major branch. -/
```

OneSidedCircularCap.center

Classification: supplement

Source: CMVGeometry.lean:117-123

Declaration SHA-256: 58a018a3af71f1eb6c5aebb7ee395b60aab79f7b55a9d384d43694baace0d346

Mathematical statement. The upper cap center is $(m, b - R\cos\theta)$, and the lower cap center is $(m, b + R\cos\theta)$.

The vertical sign selects which side of the chord contains the cap, including the major-cap case.

```
def center (c : OneSidedCircularCap) : PlanePoint :=
  match c.side with
  | .upper => (c.midpointX, c.baseY - c.radius * cos c.theta)
  | .lower => (c.midpointX, c.baseY + c.radius * cos c.theta)

/-- Coordinate realization of the selected arc, with parameter
`-theta ≤ t ≤ theta`. -/
```

OneSidedCircularCap.chordCarrier

Classification: supplement

Source: CMVGeometry.lean:160-163

Declaration SHA-256: 0705736836c9203b120221cb0770c365bada15ab749e72d0791d55f58eba4c1e

Mathematical statement. The chord carrier is $\{(x, b) : |x - m| \leq L/2\}$.

It includes both endpoints and is shared by the upper and lower cap descriptions.

```
def chordCarrier (c : OneSidedCircularCap) : Set PlanePoint :=
  {p | p.2 = c.baseY ∧ |p.1 - c.midpointX| ≤ c.chord / 2}

/-- Euclidean area computed by the circular-segment constructor. -/
```

OneSidedCircularCap.chordCarrier_subset_carrier

Classification: supplement

Source: CMVGeometry.lean:311-334

Declaration SHA-256: a35dcecd8fd218da748131deaec9b02f5328c1034952ad269b6ba1f5b686d947

Mathematical statement. The chord carrier is contained in the cap carrier.

The chord inequality implies the disk inequality, and the base line satisfies either closed side condition.

```
theorem chordCarrier_subset_carrier (c : OneSidedCircularCap) :
  c.chordCarrier ⊆ c.carrier := by
  intro p hp
  have hxabs : |p.1 - c.midpointX| ≤ c.chord / 2 := hp.2
  have hhalf : 0 < c.chord / 2 := div_pos c.chord_pos (by norm_num)
  have hxsq : (p.1 - c.midpointX) ^ 2 ≤ (c.chord / 2) ^ 2 :=
    sq_le_sq.mpr (by simpa [abs_of_pos hhalf] using hxabs)
  rw [← c.radius_mul_sin] at hxsq
  have htrig := Real.sin_sq_add_cos_sq c.theta
  have htrigScaled := congrArg
    (fun z : ℝ => c.radius ^ 2 * z) htrig
  cases hside : c.side with
  | upper =>
    constructor
    · simp only [radiusSquaredAt, center, hside, hp.1]
      nlinarith
    · simp [hside, hp.1]
  | lower =>
    constructor
    · simp only [radiusSquaredAt, center, hside, hp.1]
      nlinarith
    · simp [hside, hp.1]

/-- The coordinate cap is measurable. -/
```

OneSidedCircularCap.ell_theta_pos

Classification: supplement

Source: CMVGeometry.lean:216-219

Declaration SHA-256: 88c4230a8ce8dbe3ed5e64385c18de0588039e6c242ead1e2df6299d087f0ad8

Mathematical statement. For every cap, $\ell(\theta) > 0$.

The quotient $\theta/\sin\theta$ has positive numerator and denominator.

```
theorem ell_theta_pos (c : OneSidedCircularCap) : 0 < ell c.theta := by
  rw [ell]
  exact div_pos c.theta_pos c.sin_theta_pos
```

OneSidedCircularCap.euclideanArea_pos

Classification: supplement

Source: CMVGeometry.lean:213-215

Declaration SHA-256: 3902ccdc9eb3694014c5a47aad3179ebe09a3fd3cddb066b9c4c1ca933463218

Mathematical statement. Every cap's scalar Euclidean area is strictly positive.

The factor L^2 is positive and $a(\theta) > 0$ throughout the cap-angle domain.

```
theorem euclideanArea_pos (c : OneSidedCircularCap) : 0 < c.euclideanArea := by
  exact mul_pos (sq_pos_of_pos c.chord_pos) (area_pos c.theta_pos c.theta_lt_pi)
```

OneSidedCircularCap.interiorCarrier

Classification: supplement

Source: CMVGeometry.lean:153-159

Declaration SHA-256: d9986ab2dd520dfd41d6ccb99ca9878df085185d6c10c136685b43549b4c4dc1

Mathematical statement. The interior cap carrier uses strict disk and strict side inequalities.

It is the intersection of the open disk with the chosen open half-plane; its inclusion in the carrier interior is proved separately.

```
def interiorCarrier (c : OneSidedCircularCap) : Set PlanePoint :=
  {p | c.radiusSquaredAt p < c.radius ^ 2 ∧
    match c.side with
    | .upper => c.baseY < p.2
    | .lower => p.2 < c.baseY}

/-- The horizontal chord, including both endpoints. -/
```

OneSidedCircularCap.interiorCarrier_subset_interior

Classification: supplement

Source: CMVGeometry.lean:687-696

Declaration SHA-256: e055c7441e3e2230b6bd3c3989fc3043eb631862e9cdc4e7fe3a7963b0242ddc

Mathematical statement. The strict interior cap carrier lies in the topological interior of the closed cap carrier.

An open subset of the carrier is necessarily contained in its interior.

```
theorem interiorCarrier_subset_interior (c : OneSidedCircularCap) :
  c.interiorCarrier ⊆ interior c.carrier := by
  apply interior_maximal
  intro p hp
  exact ⟨hp.1.le, by
    cases hside : c.side <;>
    simp only [OneSidedCircularCap.interiorCarrier, hside] at hp <;>
    simp only [OneSidedCircularCap.carrier, hside] <;> linarith [hp.2]⟩
  exact isOpen_interiorCarrier c
```

OneSidedCircularCap.isClosed_carrier

Classification: supplement

Source: CMVGeometry.lean:364-391

Declaration SHA-256: d757ff8cbf988a97323a1f1c27dc667eab9b6475f2eb7d1c16beffa82e3009a4

Mathematical statement. Every cap carrier is closed.

The disk and the chosen half-plane are closed, so their intersection is closed.

```
theorem isClosed_carrier (c : OneSidedCircularCap) :
  IsClosed c.carrier := by
  have hdist : Continuous (fun p : PlanePoint => c.radiusSquaredAt p) := by
    simp only [radiusSquaredAt]
    fun_prop
  have hdisk : IsClosed {p : PlanePoint |
    c.radiusSquaredAt p ≤ c.radius ^ 2} :=
    isClosed_le hdist continuous_const
  cases hside : c.side with
  | upper =>
    rw [carrier]
    simp only [hside]
    change IsClosed
```

```

      ({p : PlanePoint | c.radiusSquaredAt p ≤ c.radius ^ 2} ∩
       {p : PlanePoint | c.baseY ≤ p.2})
    exact hdisk.inter (isClosed_le continuous_snd continuous_snd)
| lower =>
  rw [carrier]
  simp only [hside]
  change IsClosed
  ({p : PlanePoint | c.radiusSquaredAt p ≤ c.radius ^ 2} ∩
   {p : PlanePoint | p.2 ≤ c.baseY})
  exact hdisk.inter (isClosed_le continuous_snd continuous_snd)

end OneSidedCircularCap

/-- A horizontal segment. Its metric length is its positive chord parameter,
not an independently asserted scalar. -/

```

OneSidedCircularCap.isOpen_interiorCarrier

Classification: supplement

Source: CMVGeometry.lean:662-686

Declaration SHA-256: ec8c11c26c05148d770698c167cdec56869de41dc90864a365cf2514de87bc1

Mathematical statement. The strict interior cap carrier is open.

Both its defining inequalities are strict inequalities of continuous functions.

```

theorem isOpen_interiorCarrier (c : OneSidedCircularCap) :
  IsOpen c.interiorCarrier := by
  have hdisk : IsOpen {p : PlanePoint |
    c.radiusSquaredAt p < c.radius ^ 2} := by
    apply isOpen_lt
    · change Continuous (fun p : PlanePoint =>
      (p.1 - c.center.1) ^ 2 + (p.2 - c.center.2) ^ 2)
      fun_prop
    · fun_prop
  cases hside : c.side with
| upper =>
  rw [OneSidedCircularCap.interiorCarrier]
  simp only [hside]
  change IsOpen
  ({p : PlanePoint | c.radiusSquaredAt p < c.radius ^ 2} ∩
   {p : PlanePoint | c.baseY < p.2})
  exact hdisk.inter (isOpen_lt continuous_snd continuous_snd)
| lower =>
  rw [OneSidedCircularCap.interiorCarrier]
  simp only [hside]
  change IsOpen
  ({p : PlanePoint | c.radiusSquaredAt p < c.radius ^ 2} ∩
   {p : PlanePoint | p.2 < c.baseY})
  exact hdisk.inter (isOpen_lt continuous_snd continuous_snd)

```

OneSidedCircularCap.leftEndpoint

Classification: supplement

Source: CMVGeometry.lean:134-137

Declaration SHA-256: a193f337e9021d7e081f4666c9f784092558b2268ec0e9060f11b3d18dfc3c8d

Mathematical statement. The left endpoint of a cap is $(m - L/2, b)$.

It is the left end of the horizontal chord, independently of the cap side.

```

def leftEndpoint (c : OneSidedCircularCap) : PlanePoint :=
  (c.midpointX - c.chord / 2, c.baseY)

/-- Right endpoint of the oriented chord. -/

```

OneSidedCircularCap.measurableSet_carrier

Classification: supplement

Source: CMVGeometry.lean:335-363

Declaration SHA-256: e0f551f525fbf0409babb736205ec484e0a0c2ff7039ce55de1e7a7fc5595a27

Mathematical statement. Every cap carrier is measurable.

It is described by continuous polynomial inequalities and a closed half-plane condition.

```
theorem measurableSet_carrier (c : OneSidedCircularCap) :
  MeasurableSet c.carrier := by
  have hdist : Continuous (fun p : PlanePoint => c.radiusSquaredAt p) := by
    simp only [radiusSquaredAt]
    fun_prop
  have hdisk : MeasurableSet {p : PlanePoint |
    c.radiusSquaredAt p ≤ c.radius ^ 2} :=
    (isClosed_le hdist continuous_const).measurableSet
  cases h : c.side with
  | upper =>
    have hhalf : MeasurableSet {p : PlanePoint | c.baseY ≤ p.2} :=
      (isClosed_le continuous_const continuous_snd).measurableSet
    rw [carrier]
    simp only [h]
    change MeasurableSet
      ({p : PlanePoint | c.radiusSquaredAt p ≤ c.radius ^ 2} ∩
       {p : PlanePoint | c.baseY ≤ p.2})
    exact hdisk.inter hhalf
  | lower =>
    have hhalf : MeasurableSet {p : PlanePoint | p.2 ≤ c.baseY} :=
      (isClosed_le continuous_snd continuous_const).measurableSet
    rw [carrier]
    simp only [h]
    change MeasurableSet
      ({p : PlanePoint | c.radiusSquaredAt p ≤ c.radius ^ 2} ∩
       {p : PlanePoint | p.2 ≤ c.baseY})
    exact hdisk.inter hhalf

/-- The disk/closed-half-plane cap carrier is closed. -/
```

OneSidedCircularCap.mem_chordCarrier_of_mem_carrier_of_eq_base

Classification: supplement

Source: CMVGeometry.lean:291-310

Declaration SHA-256: 424261710f758c71ebe7e57dd13ebd11ffc59cb51b840ff8fc86319890ddae19

Mathematical statement. A cap-carrier point on the base line belongs to the chord carrier.

Substituting $y = b$ into the disk inequality bounds the horizontal displacement by $L/2$.

```
theorem mem_chordCarrier_of_mem_carrier_of_eq_base
  (c : OneSidedCircularCap) {p : PlanePoint}
  (hp : p ∈ c.carrier) (hy : p.2 = c.baseY) :
  p ∈ c.chordCarrier := by
  refine ⟨hy, ?_⟩
  have hdisk := hp.1
  cases hside : c.side <|>
  simp only [radiusSquaredAt, center, hside, hy] at hdisk
  all_goals
    have htrig := Real.sin_sq_add_cos_sq c.theta
    have htrigScaled := congrArg
      (fun z : ℝ => c.radius ^ 2 * z) htrig
    have hsq : (p.1 - c.midpointX) ^ 2 ≤
      (c.radius * sin c.theta) ^ 2 := by
      nlinarith
    rw [c.radius_mul_sin] at hsq
```

```

exact abs_le_of_sq_le_sq hsq
  (div_nonneg c.chord_pos.le (by norm_num))

/-- Every point of the finite chord belongs to the closed cap. -/

```

OneSidedCircularCap.radius

Classification: supplement

Source: CMVGeometry.lean:113-116

Declaration SHA-256: e898cc7066a75a606fdc3b1d1e563939ad569bdb0f66fd289569e58c7f4c44b8

Mathematical statement. A cap with chord L and half-central angle θ has radius $R = L/(2\sin\theta)$.

The angular bounds make the denominator strictly positive and give the usual chord relation.

```

def radius (c : OneSidedCircularCap) : ℝ := c.chord / (2 * sin c.theta)

/-- The center lies below an upper chord and above a lower chord. For a major
cap `cos θ < 0`, so the center crosses to the cap side, as it should. -/

```

OneSidedCircularCap.radiusSquaredAt

Classification: supplement

Source: CMVGeometry.lean:142-145

Declaration SHA-256: a1c0f970b426c7f0e6a013d8d8bbb613fbba3b5c253445dacecdb9b47cb6785a

Mathematical statement. The squared distance of p from the cap center c is $(p_1 - c_1)^2 + (p_2 - c_2)^2$.

This coordinate polynomial defines the disk inequality without invoking the product-space metric.

```

def radiusSquaredAt (c : OneSidedCircularCap) (p : PlanePoint) : ℝ :=
  (p.1 - c.center.1) ^ 2 + (p.2 - c.center.2) ^ 2

/-- The genuine closed coordinate carrier of a one-sided cap. -/

```

OneSidedCircularCap.radius_mul_sin

Classification: supplement

Source: CMVGeometry.lean:223-227

Declaration SHA-256: 4b800b6287f8bc78c34b40f39fff02ab7d25631ad2fd22f8819095ceda60143e

Mathematical statement. The cap radius satisfies $R\sin\theta = L/2$.

This identity relates the trigonometric parameterization to the geometric chord endpoints.

```

theorem radius_mul_sin (c : OneSidedCircularCap) :
  c.radius * sin c.theta = c.chord / 2 := by
  rw [radius]
  field_simp [ne_of_gt c.sin_theta_pos]

```

OneSidedCircularCap.radius_pos

Classification: supplement

Source: CMVGeometry.lean:210-212

Declaration SHA-256: 7bcdcf3411946f45cb3c854e76728bf4e1477fc16c0bd8673b5d0f6f904d6adf

Mathematical statement. Every cap radius is strictly positive.

Both the chord and the denominator $2\sin\theta$ are positive.

```
theorem radius_pos (c : OneSidedCircularCap) : 0 < c.radius := by
  exact div_pos c.chord_pos (mul_pos (by norm_num) c.sin_theta_pos)
```

OneSidedCircularCap.rightEndpoint

Classification: supplement

Source: CMVGeometry.lean:138-141

Declaration SHA-256: e83d4942af5a95dd72bd914bc15f416e046e239fc23caff7a851807f832f26a3

Mathematical statement. The right endpoint of a cap is $(m + L/2, b)$.

It is the right end of the horizontal chord, independently of the cap side.

```
def rightEndpoint (c : OneSidedCircularCap) : PlanePoint :=
  (c.midpointX + c.chord / 2, c.baseY)

/-- Squared distance from the cap's circle center. -/
```

OneSidedCircularCap.sin_theta_pos

Classification: supplement

Source: CMVGeometry.lean:207-209

Declaration SHA-256: 8c22795350624813eb7b9aec2fdcda7bfc438d144ebb38b607db3cac77688583

Mathematical statement. For every cap, $\sin\theta > 0$.

This follows from $0 < \theta < \pi$ and justifies division by the sine.

```
theorem sin_theta_pos (c : OneSidedCircularCap) : 0 < sin c.theta :=
  sin_pos_of_pos_of_lt_pi c.theta_pos c.theta_lt_pi
```

PlanePoint

Classification: supplement

Source: CMVGeometry.lean:30-33

Declaration SHA-256: b6c33ab7d9e6652b4833188977c0694ee665f31ebd07e572ea7a67ee7a8a0183

Mathematical statement. The coordinate point space is $\mathbb{R} \times \mathbb{R}$.

This abbreviation supplies ordered real coordinates; it does not itself select the Euclidean norm used for perimeter.

```
abbrev PlanePoint := ℝ × ℝ

/-- The Euclidean L2 realization used to form geometric Hausdorff measure.
`PlanePoint` keeps its coordinate topology and all existing carrier definitions. -/
```

ReplacementAssembly

Classification: supplement

Source: CMVGeometry.lean:3281-3288

Declaration SHA-256: e0b73790946058b82b8d6a4bdde47ad7560092e26acf3d58cb0c91dc0cfa445f

Mathematical statement. A replacement assembly consists of a strip core and an upper half-angle $0 < \theta < \pi$.

This is an auxiliary competitor constructor retained in the model's type definition; its separate exclusion results are not publication claims here.

```

structure ReplacementAssembly where
  core : StripCore
  replacementAngle : ℝ
  replacementAngle_pos : 0 < replacementAngle
  replacementAngle_lt_pi : replacementAngle < π

namespace ReplacementAssembly

```

ReplacementAssembly.carrier

Classification: supplement

Source: CMVGeometry.lean:3305-3309

Declaration SHA-256: 36b80491dc0fecbf358d87c6bfe7194faf4bcad4ac91372874e8af60a30fd23e

Mathematical statement. The replacement carrier is the union of the core and its upper cap.

Unlike a four-arc assembly, it has no lower exterior cap.

```

def carrier (a : ReplacementAssembly) : Set PlanePoint :=
  a.core.carrier ∪ a.upperCap.carrier

/-- Explicit pushforward measure of the two retained side arcs, inserted upper
arc, and genuinely exposed lower chord. -/

```

ReplacementAssembly.upperCap

Classification: supplement

Source: CMVGeometry.lean:3289-3298

Declaration SHA-256: b30d86506333a29d0fea0a0c213d5ab5e404eb8902dd90c6019d57dad3b8f56f

Mathematical statement. The replacement upper cap uses the core chord, half-angle θ , and upper base $(0, 1)$.

The angle range allows a minor cap, semicircle, or major cap.

```

def upperCap (a : ReplacementAssembly) : OneSidedCircularCap where
  chord := a.core.chord
  theta := a.replacementAngle
  midpointX := 0
  baseY := 1
  side := .upper
  chord_pos := a.core.chord_pos
  theta_pos := a.replacementAngle_pos
  theta_lt_pi := a.replacementAngle_lt_pi

```

ReplacementAssembly.weightedArea

Classification: supplement

Source: CMVGeometry.lean:3326-3329

Declaration SHA-256: 30517c93164485b2476e45ea547b19187ad53db44a33af2bf70144ca4c93c662

Mathematical statement. The replacement assembly weighted area is A_λ of its carrier.

This definition is present because replacement regions are among the formal competitor constructors.

```

def weightedArea (lam : ℝ) (a : ReplacementAssembly) : ℝ :=
  WeightedArea lam a.carrier

/-- Density integral against the complete explicit replacement boundary. -/

```

StripCore.carrier_y_bounds

Classification: supplement

Source: CMVGeometry.lean:482-490

Declaration SHA-256: dfd3510aaac02f0e3230984bb1d2516c15f954b7d416fa7e30c71ad44efa3ab8

Mathematical statement. Every point of the strip-core carrier satisfies $|y| \leq 1$.

Each constituent set includes the same vertical strip restriction.

```
theorem carrier_y_bounds (c : StripCore) {p : PlanePoint}
  (hp : p ∈ c.carrier) : |p.2| ≤ 1 := by
  rcases hp with (hp | hp) | hp
  · exact hp.2.2
  · exact hp.2.2
  · exact hp.2.2

/-- Every horizontal segment across the core rectangle is contained in the
retained region. -/
```

StripCore.cos_sideAngle

Classification: supplement

Source: CMVGeometry.lean:478-481

Declaration SHA-256: 5c26f39a6dccf790fede705362af68051955e479d2108518ff90a5d9985148ed

Mathematical statement. The identity $\cos\phi = \sqrt{1 - \kappa^2}$ holds for every strip core.

The cosine is nonnegative on the selected principal branch, including the zero endpoint cosine.

```
theorem cos_sideAngle (c : StripCore) :
  cos c.sideAngle = √(1 - c.curvature ^ 2) := by
  simp [sideAngle, Real.cos_arcsin]
```

StripCore.isClosed_carrier

Classification: supplement

Source: CMVGeometry.lean:805-843

Declaration SHA-256: d28fca8c993d5d2b9aa0b2ebe7f3f0372ebdd951a5cda90d26adf748bface0e4

Mathematical statement. Every strip-core carrier is closed.

Its rectangle and circular pieces are closed and there are finitely many pieces.

```
theorem isClosed_carrier (c : StripCore) : IsClosed c.carrier := by
  have hx : Continuous (fun p : PlanePoint => p.1) := continuous_fst
  have hy : Continuous (fun p : PlanePoint => p.2) := continuous_snd
  have habsy : Continuous (fun p : PlanePoint => |p.2|) := hy.abs
  have hrect : IsClosed c.rectangleCarrier := by
    rw [StripCore.rectangleCarrier]
    change IsClosed
    ({p : PlanePoint | -c.chord / 2 ≤ p.1} ∩
     {p : PlanePoint | p.1 ≤ c.chord / 2} ∩
     {p : PlanePoint | |p.2| ≤ 1})
  exact (isClosed_le continuous_const hx).inter
    ((isClosed_le hx continuous_const).inter
     (isClosed_le habsy continuous_const))
  have hleft : IsClosed c.leftCapCarrier := by
    rw [StripCore.leftCapCarrier]
    change IsClosed
    ({p : PlanePoint |
      (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} ∩
     {p : PlanePoint | p.1 ≤ -c.chord / 2} ∩
```

```

    {p : PlanePoint | |p.2| ≤ 1}))
  exact (isClosed_le
    (((hx.sub continuous_const).pow 2).add (hy.pow 2))
    continuous_const).inter
    ((isClosed_le hx continuous_const).inter
    (isClosed_le habsy continuous_const))
  have hright : IsClosed c.rightCapCarrier := by
  rw [StripCore.rightCapCarrier]
  change IsClosed
  ({p : PlanePoint |
    (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} ∩
    {p : PlanePoint | c.chord / 2 ≤ p.1} ∩
    {p : PlanePoint | |p.2| ≤ 1}))
  exact (isClosed_le
    (((hx.sub continuous_const).pow 2).add (hy.pow 2))
    continuous_const).inter
    ((isClosed_le continuous_const hx).inter
    (isClosed_le habsy continuous_const))
  exact (hrect.union hleft).union hright

```

StripCore.leftArcTrace

Classification: supplement

Source: CMVGeometry.lean:762-766

Declaration SHA-256: 702872b40533619867ab566579d52d3f6a44e812910796d96106d1c50acbbfc2

Mathematical statement. The left core arc trace is $(x - c_L)^2 + y^2 = R^2$, with $x \leq -L/2$ and $|y| \leq 1$.

This is the curved outer boundary of the left side piece.

```

def leftArcTrace (c : StripCore) : Set PlanePoint :=
  {p | (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2 ∧
    p.1 ≤ -c.chord / 2 ∧ |p.2| ≤ 1}

/-- The exact right circular trace of a strip core. -/

```

StripCore.leftCapCarrier

Classification: supplement

Source: CMVGeometry.lean:440-443

Declaration SHA-256: 2ca301cf56be587995c60fad814a6021c666b20d0f45e1fa3f2964b23f678cbc

Mathematical statement. The left core piece is the disk $(x - c_L)^2 + y^2 \leq R^2$ restricted to $x \leq -L/2$ and $|y| \leq 1$.

Only the outer left circular segment is retained; the rectangle occupies the remaining central portion.

```

def leftCapCarrier (c : StripCore) : Set PlanePoint :=
  {p | (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2 ∧
    p.1 ≤ -c.chord / 2 ∧ |p.2| ≤ 1}

```

StripCore.leftCenterX

Classification: supplement

Source: CMVGeometry.lean:431-433

Declaration SHA-256: 01fd3b14ca7e8a9d341e71e2fb86b773a9bea051016f7f6499489c3b65bab474

Mathematical statement. The left side-circle center has horizontal coordinate $c_L = -L/2 + R\cos\phi$.

Its vertical coordinate is zero; the circle meets the left chord endpoints at heights ± 1 .

```

def leftCenterX (c : StripCore) : ℝ :=

```

```
-c.chord / 2 + c.radius * cos c.sideAngle
```

StripCore.lowerChordTrace

Classification: supplement

Source: CMVGeometry.lean:774-776

Declaration SHA-256: faa702d9afdf2eed044fad4474692be94c38bb3a9868779993853eb74445596

Mathematical statement. The lower chord trace is $\{(x, -1) : |x| \leq L/2\}$.

It is the bottom boundary of the isolated strip core before the lower cap is attached.

```
def lowerChordTrace (c : StripCore) : Set PlanePoint :=
  {p | p.2 = -1 ∧ |p.1| ≤ c.chord / 2}
```

StripCore.measurableSet_carrier

Classification: supplement

Source: CMVGeometry.lean:500-551

Declaration SHA-256: c4a110b00efc328d9afc36bfb523f064a5cb69170025121ec86c0022fc798171

Mathematical statement. The strip-core carrier is measurable.

Measurability follows from the coordinate inequalities and closure under finite unions.

```
theorem measurableSet_carrier (c : StripCore) : MeasurableSet c.carrier := by
  have hx : Continuous (fun p : PlanePoint => p.1) := continuous_fst
  have hy : Continuous (fun p : PlanePoint => p.2) := continuous_snd
  have hrect : MeasurableSet c.rectangleCarrier := by
    have hxmin : IsClosed {p : PlanePoint | -c.chord / 2 ≤ p.1} :=
      isClosed_le continuous_const hx
    have hxmax : IsClosed {p : PlanePoint | p.1 ≤ c.chord / 2} :=
      isClosed_le hx continuous_const
    have hybound : IsClosed {p : PlanePoint | |p.2| ≤ 1} :=
      isClosed_le hy.abs continuous_const
    rw [rectangleCarrier]
    change MeasurableSet
    ({p : PlanePoint | -c.chord / 2 ≤ p.1} ∩
     {p : PlanePoint | p.1 ≤ c.chord / 2} ∩
     {p : PlanePoint | |p.2| ≤ 1})
  exact (hxmin.inter (hxmax.inter hybound)).measurableSet
  have hleft : MeasurableSet c.leftCapCarrier := by
    have hdisk : IsClosed {p : PlanePoint |
      (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} :=
      isClosed_le (((hx.sub continuous_const).pow 2).add (hy.pow 2))
      continuous_const
    have hxbound : IsClosed {p : PlanePoint | p.1 ≤ -c.chord / 2} :=
      isClosed_le hx continuous_const
    have hybound : IsClosed {p : PlanePoint | |p.2| ≤ 1} :=
      isClosed_le hy.abs continuous_const
    rw [leftCapCarrier]
    change MeasurableSet
    ({p : PlanePoint |
      (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} ∩
     {p : PlanePoint | p.1 ≤ -c.chord / 2} ∩
     {p : PlanePoint | |p.2| ≤ 1})
  exact (hdisk.inter (hxbound.inter hybound)).measurableSet
  have hright : MeasurableSet c.rightCapCarrier := by
    have hdisk : IsClosed {p : PlanePoint |
      (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} :=
      isClosed_le (((hx.sub continuous_const).pow 2).add (hy.pow 2))
      continuous_const
    have hxbound : IsClosed {p : PlanePoint | c.chord / 2 ≤ p.1} :=
      isClosed_le continuous_const hx
    have hybound : IsClosed {p : PlanePoint | |p.2| ≤ 1} :=
```

```

    isClosed_le hy.abs continuous_const
    rw [rightCapCarrier]
    change MeasurableSet
    ({p : PlanePoint |
      (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} ∩
      ({p : PlanePoint | c.chord / 2 ≤ p.1} ∩
      {p : PlanePoint | |p.2| ≤ 1}))
    exact (hdisk.inter (hxbound.inter hybound)).measurableSet
    exact (hrect.union hleft).union hright
  end StripCore

```

StripCore.radius

Classification: supplement

Source: CMVGeometry.lean:426-428

Declaration SHA-256: f23d45618cc8f0a402155f6b0b9c9bed86df54007d8b97c4f4027753ebfcd373

Mathematical statement. The strip-core radius is $R = 1/\kappa$.

The curvature bound implies $R \geq 1$, so the circle can span the strip height.

```

def radius (c : StripCore) : ℝ := 1 / c.curvature

/-- Half-central-angle of either strip arc. -/

```

StripCore.radius_pos

Classification: supplement

Source: CMVGeometry.lean:460-462

Declaration SHA-256: 754f25da7343b067aca1ccf07fea956e09bfd461a16a072435b6b938ea98f462

Mathematical statement. The strip-core radius is positive.

This follows from the strictly positive curvature in the constructor.

```

theorem radius_pos (c : StripCore) : 0 < c.radius :=
  one_div_pos.mpr c.curvature_pos

```

StripCore.rectangleCarrier

Classification: supplement

Source: CMVGeometry.lean:437-439

Declaration SHA-256: 5204ba429abcba2c9a7bfd26b44272d1068d5b57b8a97a88331831661e9408f9

Mathematical statement. The core rectangle is $[-L/2, L/2] \times [-1, 1]$.

This central rectangle joins the two circular side pieces and has area $2L$.

```

def rectangleCarrier (c : StripCore) : Set PlanePoint :=
  {p | -c.chord / 2 ≤ p.1 ∧ p.1 ≤ c.chord / 2 ∧ |p.2| ≤ 1}

```

StripCore.rightArcTrace

Classification: supplement

Source: CMVGeometry.lean:767-770

Declaration SHA-256: 059982ba84229cdd58242cd7f98720be73ad274e207529f914e522d73ba5cbeb

Mathematical statement. The right core arc trace is $(x - c_R)^2 + y^2 = R^2$, with $x \geq L/2$ and $|y| \leq 1$.

This is the reflected curved outer boundary of the right side piece.

```
def rightArcTrace (c : StripCore) : Set PlanePoint :=
  {p | (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2 ∧
    c.chord / 2 ≤ p.1 ∧ |p.2| ≤ 1}
```

StripCore.rightCapCarrier

Classification: supplement

Source: CMVGeometry.lean:444-448

Declaration SHA-256: 0f7231f5e97656f30e8e5131e2afc098206a3ff9953e2cd02ae8207ac4d54b12

Mathematical statement. The right core piece is the disk $(x - c_R)^2 + y^2 \leq R^2$ restricted to $x \geq L/2$ and $|y| \leq 1$.

This is the reflected right circular segment adjoining the rectangle.

```
def rightCapCarrier (c : StripCore) : Set PlanePoint :=
  {p | (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2 ∧
    c.chord / 2 ≤ p.1 ∧ |p.2| ≤ 1}

/-- Genuine coordinate carrier of the retained central region. -/
```

StripCore.rightCenterX

Classification: supplement

Source: CMVGeometry.lean:434-436

Declaration SHA-256: ff1acc9e9b3ef170758d23460b0f614f6eb1c034ebd024b460f189e9f303f923

Mathematical statement. The right side-circle center has horizontal coordinate $c_R = L/2 - R\cos\phi$.

It is the reflection of the left center across the vertical axis.

```
def rightCenterX (c : StripCore) : ℝ :=
  c.chord / 2 - c.radius * cos c.sideAngle
```

StripCore.sideAngle

Classification: supplement

Source: CMVGeometry.lean:429-430

Declaration SHA-256: 4f6b6599037149db7ce42f881c45d0eb7f35fbc43f124df7507d6a83a6d6e15b

Mathematical statement. The strip-core side angle is $\phi = \arcsin\kappa$.

The principal arcsine gives $0 < \phi \leq \pi/2$.

```
def sideAngle (c : StripCore) : ℝ := arcsin c.curvature
```

StripCore.sideAngle_le_pi_div_two

Classification: supplement

Source: CMVGeometry.lean:466-469

Declaration SHA-256: 0c0857ab96c23693b4448efce2e70a6069dda6f460cf0118b7e365f0f1074987

Mathematical statement. The strip-core side angle satisfies $\phi \leq \pi/2$.

The upper curvature bound includes equality, so this assertion is deliberately non-strict.

```
theorem sideAngle_le_pi_div_two (c : StripCore) :
  c.sideAngle ≤ π / 2 := by
  exact Real.arcsin_le_pi_div_two c.curvature
```

StripCore.sideAngle_lt_pi

Classification: supplement

Source: CMVGeometry.lean:470-473

Declaration SHA-256: 4d165924e20daf34cd0cd3a93af12e6637e356347b723990beeb9d1a85af59a9e

Mathematical statement. The strip-core side angle satisfies $\phi < \pi$.

The sharper bound $\phi \leq \pi/2$ places the side segment in the cap-angle domain.

```
theorem sideAngle_lt_pi (c : StripCore) : c.sideAngle < π := by
  have hp := Real.pi_pos
  linarith [c.sideAngle_le_pi_div_two]
```

StripCore.sideAngle_pos

Classification: supplement

Source: CMVGeometry.lean:463-465

Declaration SHA-256: 20886da5ba412dbc0666239218b14d32a1bad68cf7a0f4837b3f6216a46aefbd

Mathematical statement. The strip-core side angle is positive.

The principal arcsine is positive at a positive curvature.

```
theorem sideAngle_pos (c : StripCore) : 0 < c.sideAngle := by
  simpa [sideAngle] using (Real.arcsin_pos.mpr c.curvature_pos)
```

StripCore.sin_sideAngle

Classification: supplement

Source: CMVGeometry.lean:474-477

Declaration SHA-256: e85f9cb3c0c2b22c1ce74fdc60660033205e7a0821e76cfdd7248571352ea184

Mathematical statement. The identity $\sin \phi = \kappa$ holds for every strip core.

The curvature lies in the principal inverse-sine domain.

```
theorem sin_sideAngle (c : StripCore) : sin c.sideAngle = c.curvature := by
  rw [sideAngle]
  exact Real.sin_arcsin (by linarith [c.curvature_pos]) c.curvature_le_one
```

StripCore.upperChordTrace

Classification: supplement

Source: CMVGeometry.lean:771-773

Declaration SHA-256: 71db3f396f38ac0c4ed32b75273f08b0092359d4c0fdd7bc77976000e7791fb0

Mathematical statement. The upper chord trace is $\{(x, 1) : |x| \leq L/2\}$.

It is the top boundary of the isolated strip core before the exterior cap is attached.

```
def upperChordTrace (c : StripCore) : Set PlanePoint :=
  {p | p.2 = 1 ∧ |p.1| ≤ c.chord / 2}
```

StripCore.volume_leftCapCarrier

Classification: supplement

Source: CMVGeometry.lean:2076-2138

Declaration SHA-256: ee7dbe308fa5b9feabca0b7d1f31d2c91cdf7e530a43dcc64a52dce7563fb7b8

Mathematical statement. The left core circular piece has volume $4a(\phi)$.

It is a circular segment on a chord of length two, so normalized area scales by four.

```
lemma volume_leftCapCarrier (c : StripCore) :
  volume c.leftCapCarrier = ENNReal.ofReal (4 * area c.sideAngle) := by
  let U : Set PlanePoint :=
    {p | (p.1 - 0) ^ 2 + (p.2 - c.leftCenterX) ^ 2 ≤ c.radius ^ 2 ∧
      p.2 ≤ c.leftCenterX - c.radius * cos c.sideAngle}
  have hcos : 0 ≤ cos c.sideAngle := by
    rw [c.cos_sideAngle]
    positivity
  have hpre : Prod.swap ⁻¹' U = c.leftCapCarrier := by
    ext p
    change ((p.2 - 0) ^ 2 + (p.1 - c.leftCenterX) ^ 2 ≤ c.radius ^ 2 ∧
      p.1 ≤ c.leftCenterX - c.radius * cos c.sideAngle) ↔
      ((p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2 ∧
      p.1 ≤ -c.chord / 2 ∧ |p.2| ≤ 1)
  have hbase : c.leftCenterX - c.radius * cos c.sideAngle = -c.chord / 2 := by
    rw [leftCenterX]
    ring
  rw [hbase]
  constructor
  · rintro ⟨hdisk, hx⟩
    have hdx : p.1 - c.leftCenterX ≤ -(c.radius * cos c.sideAngle) := by
      rw [leftCenterX]
      linarith
    have hdx0 : p.1 - c.leftCenterX ≤ 0 := by
      nlinarith [c.radius_pos]
    have hsquare : (c.radius * cos c.sideAngle) ^ 2 ≤
      (p.1 - c.leftCenterX) ^ 2 := by
      apply sq_le_sq.mpr
      rw [abs_of_nonneg (mul_nonneg c.radius_pos.le hcos), abs_of_nonpos hdx0]
      linarith
    have hy2 : p.2 ^ 2 ≤ 1 := by
      nlinarith [radius_sq_cos_sq' c]
    have hyabs : |p.2| ≤ |(1 : ℝ)| := sq_le_sq.mp (by simpa using hy2)
    exact ⟨by nlinarith, hx, by simpa using hyabs⟩
  · rintro ⟨hdisk, hx, _hy⟩
    exact ⟨by nlinarith, hx⟩
  have hU : MeasurableSet U := by
    change MeasurableSet
    ({p : PlanePoint |
      (p.1 - 0) ^ 2 + (p.2 - c.leftCenterX) ^ 2 ≤ c.radius ^ 2} ∩
      {p : PlanePoint |
      p.2 ≤ c.leftCenterX - c.radius * cos c.sideAngle})
    exact (isClosed_le (by fun_prop) continuous_const).measurableSet.inter
      (isClosed_le continuous_snd continuous_const).measurableSet
  have hmeasure : volume c.leftCapCarrier = volume U := by
    rw [← hpre]
    exact Measure.measurePreserving_swap.measure_preimage hU.nullMeasurableSet
  have hreal : c.radius ^ 2 *
    (c.sideAngle - sin c.sideAngle * cos c.sideAngle) =
    4 * area c.sideAngle := by
    rw [area]
    have hs : sin c.sideAngle = c.curvature := c.sin_sideAngle
    rw [hs]
    rw [radius]
    field_simp [ne_of_gt c.curvature_pos]
  calc
    volume c.leftCapCarrier = volume U := hmeasure
    _ = ENNReal.ofReal (c.radius ^ 2 *
      (c.sideAngle - sin c.sideAngle * cos c.sideAngle)) := by
      simpa [U] using volume_lower_circular_segment 0 c.leftCenterX
```

```
c.radius c.sideAngle c.radius_pos c.sideAngle_pos.le c.sideAngle_lt_pi.le
_ = ENNReal.ofReal (4 * area c.sideAngle) := congrArg ENNReal.ofReal hreal
```

StripCore.volume_rectangleCarrier

Classification: supplement

Source: CMVGeometry.lean:2060-2075

Declaration SHA-256: b2c1d8716422a2e0169b6aaca489790c14b0594bc75f1e77b6bbec1349d5110

Mathematical statement. The rectangle in a strip core has planar volume $2L$.

Its horizontal width is L and its vertical height is two.

```
lemma volume_rectangleCarrier (c : StripCore) :
  volume c.rectangleCarrier = ENNReal.ofReal (2 * c.chord) := by
  have hset : c.rectangleCarrier =
    Icc (-c.chord / 2) (c.chord / 2) ×ˢ Icc (-1 : ℝ) 1 := by
    ext p
    simp only [rectangleCarrier, mem_setOf_eq, mem_prod, mem_Icc]
  constructor
  · rintro ⟨hx1, hx2, hy⟩
    exact ⟨⟨hx1, hx2⟩, abs_le.mp hy⟩
  · rintro ⟨⟨hx1, hx2⟩, hy1, hy2⟩
    exact ⟨hx1, hx2, abs_le.mpr ⟨hy1, hy2⟩⟩
  rw [hset, Measure.volume_eq_prod, Measure.prod_prod,
    Real.volume_Icc, Real.volume_Icc,
    show c.chord / 2 - -c.chord / 2 = c.chord by ring]
  norm_num [ENNReal.ofReal_mul, c.chord_pos.le, mul_comm]
```

StripCore.volume_rightCapCarrier

Classification: supplement

Source: CMVGeometry.lean:2139-2201

Declaration SHA-256: 7bc214af5d8d445665e416853a1fab80112ca5fe1116500de0e9b2b6285dfdf7

Mathematical statement. The right core circular piece has volume $4a(\phi)$.

Reflection preserves the volume of the left segment.

```
lemma volume_rightCapCarrier (c : StripCore) :
  volume c.rightCapCarrier = ENNReal.ofReal (4 * area c.sideAngle) := by
  let U : Set PlanePoint :=
    {p | (p.1 - 0) ^ 2 + (p.2 - c.rightCenterX) ^ 2 ≤ c.radius ^ 2 ∧
      c.rightCenterX + c.radius * cos c.sideAngle ≤ p.2}
  have hcos : 0 ≤ cos c.sideAngle := by
    rw [c.cos_sideAngle]
    positivity
  have hpre : Prod.swap ⁻¹' U = c.rightCapCarrier := by
    ext p
    change ((p.2 - 0) ^ 2 + (p.1 - c.rightCenterX) ^ 2 ≤ c.radius ^ 2 ∧
      c.rightCenterX + c.radius * cos c.sideAngle ≤ p.1) ↔
      ((p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2 ∧
        c.chord / 2 ≤ p.1 ∧ |p.2| ≤ 1)
  have hbase : c.rightCenterX + c.radius * cos c.sideAngle = c.chord / 2 := by
    rw [rightCenterX]
    ring
  rw [hbase]
  constructor
  · rintro ⟨hdisk, hx⟩
    have hdx : c.radius * cos c.sideAngle ≤ p.1 - c.rightCenterX := by
      rw [rightCenterX]
      linarith
    have hdx0 : 0 ≤ p.1 - c.rightCenterX := by
      nlinarith [c.radius_pos]
    have hsquare : (c.radius * cos c.sideAngle) ^ 2 ≤
```

```

    (p.1 - c.rightCenterX) ^ 2 := by
      apply sq_le_sq.mpr
      rw [abs_of_nonneg (mul_nonneg c.radius_pos.le hcos), abs_of_nonneg hdx0]
      exact hdx
  have hy2 : p.2 ^ 2 ≤ 1 := by
    nlinarith [radius_sq_cos_sq' c]
  have hyabs : |p.2| ≤ |(1 : ℝ)| := sq_le_sq.mp (by simp using hy2)
  exact ⟨by nlinarith, hx, by simp using hyabs⟩
· rintro ⟨hdisk, hx, _hy⟩
  exact ⟨by nlinarith, hx⟩
have hU : MeasurableSet U := by
  change MeasurableSet
  ({p : PlanePoint |
    (p.1 - 0) ^ 2 + (p.2 - c.rightCenterX) ^ 2 ≤ c.radius ^ 2} ∩
  {p : PlanePoint |
    c.rightCenterX + c.radius * cos c.sideAngle ≤ p.2})
  exact (isClosed_le (by fun_prop) continuous_const).measurableSet.inter
    (isClosed_le continuous_const continuous_snd).measurableSet
have hmeasure : volume c.rightCapCarrier = volume U := by
  rw [← hpre]
  exact Measure.measurePreserving_swap.measure_preimage hU.nullMeasurableSet
have hreal : c.radius ^ 2 *
  (c.sideAngle - sin c.sideAngle * cos c.sideAngle) =
  4 * area c.sideAngle := by
  rw [area]
  have hs : sin c.sideAngle = c.curvature := c.sin_sideAngle
  rw [hs]
  rw [radius]
  field_simp [ne_of_gt c.curvature_pos]
calc
  volume c.rightCapCarrier = volume U := hmeasure
  _ = ENNReal.ofReal (c.radius ^ 2 *
    (c.sideAngle - sin c.sideAngle * cos c.sideAngle)) := by
    simp [U] using volume_upper_circular_segment 0 c.rightCenterX
      c.radius c.sideAngle c.radius_pos c.sideAngle_pos.le c.sideAngle_lt_pi.le
  _ = ENNReal.ofReal (4 * area c.sideAngle) := congrArg ENNReal.ofReal hreal

```

TypeThreeAssembly.ModeledCompetitorRealization.asCompetitor

Classification: supplement

Source: TypeThreeAssembly.lean:3053-3054

Declaration SHA-256: 6c62dc13fbd8bcc95bc9815a8c9dace231a72bb0614d3ccc231b26790b97baab

Mathematical statement. Given an existing realization $\mathcal{R}$ containing an admissible competitor F with carrier E_3 , the extracted competitor is $\mathcal{R}.\text{competitor} = F$.

This declaration is an extraction from a supplied realization. It does not assert that a realization exists; existence is a separate proposition.

```
def asCompetitor : AdmissibleCompetitor lam := realization.competitor
```

TypeThreeAssembly.bottomChord_nonneg

Classification: supplement

Source: TypeThreeAssembly.lean:352-354

Declaration SHA-256: bbe5a0b68c27b1eab70889a99796728aff04d58c47eda0333c16191c78a8f12a

Mathematical statement. The bottom chord parameter satisfies $2w \geq 0$.

This follows from nonnegativity of the side half-width.

```
theorem bottomChord_nonneg : 0 ≤ a.bottomChord := by
  exact mul_nonneg (by norm_num) a.sideHalfWidth_nonneg
```

TypeThreeAssembly.bottomChord_pos

Classification: supplement

Source: TypeThreeAssembly.lean:359-362

Declaration SHA-256: 5deb1da8ce95e59329d432d4f1484bad3b915566f05bd8a9e6b37a69ad4de177

Mathematical statement. If $h \neq 1/2$, then $2w > 0$.

The bottom segment is nondegenerate on both open branches.

```
theorem bottomChord_pos (hh : a.h ≠ 1 / 2) : 0 < a.bottomChord := by
  exact mul_pos (by norm_num) (a.sideHalfWidth_pos hh)
```

TypeThreeAssembly.boundaryJoinSet

Classification: supplement

Source: TypeThreeAssembly.lean:1130-1135

Declaration SHA-256: 4d107bd731d106706191e0cc66c1eb6e52b0ad83df797a9379d71e46e9ea68c7

Mathematical statement. The type-iii boundary join set is

$$\{(-R\sin\alpha, 1), (R\sin\alpha, 1), (-w, -1), (w, -1)\}.$$

These four points contain all possible pairwise intersections of the four primitive frontier traces, with the two bottom points coinciding when $w = 0$.

```
def boundaryJoinSet : Set PlanePoint :=
  {(-a.radius * sin a.outerAngle, 1),
   (a.radius * sin a.outerAngle, 1),
   (-a.sideHalfWidth, -1),
   (a.sideHalfWidth, -1)}
```

TypeThreeAssembly.boundaryJoinSet_finite

Classification: supplement

Source: TypeThreeAssembly.lean:1136-1141

Declaration SHA-256: fb8b2d3307f8905b1616f7f8934468dae8dc2d1df098e8fa72f92f5a3759db5d

Mathematical statement. The type-iii boundary join set is finite.

It is explicitly a set of four displayed points, possibly with repetitions.

```
theorem boundaryJoinSet_finite : a.boundaryJoinSet.Finite := by
  simp [boundaryJoinSet]

/-- Every pairwise overlap of the four primitive frontier traces is a boundary
join. In particular, when `h = 1 / 2`, the two side arcs meet only at the
shared bottom point. -/
```

TypeThreeAssembly.boundaryTrace_subset_frontier_carrier

Classification: supplement

Source: TypeThreeAssembly.lean:2840-2847

Declaration SHA-256: daeda01e39ff76764527644f12850660eec76e1d4031dd8579c24a3b55dda6ba

Mathematical statement. Every declared type-iii boundary trace lies in the complete frontier.

Each circular or horizontal component contains points approached from both the carrier and its complement, including the degenerate-join convention.

```
theorem boundaryTrace_subset_frontier_carrier :
  a.boundaryTrace ⊆ frontier a.carrier := by
rintro p (hleft | hright | hupper | hbottom)
· exact a.leftArc_mem_frontier hleft
· exact a.rightArc_mem_frontier hright
· exact a.upperArc_mem_frontier hupper
· exact a.bottomSegmentCarrier_subset_frontier hbottom
```

TypeThreeAssembly.continuous_realizedLeftArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:690-699

Declaration SHA-256: 5aa13a8e0a54ece98427e269f77eeefa81d89485ad9d1f32dde5c0e6f344ef9b4

Mathematical statement. The realized left arc parametrization is continuous on $\mathbb{R}$.

It is the composition of a continuous standard circle map with an isometry.

```
lemma continuous_realizedLeftArcParam :
  Continuous a.realizedLeftArcParam := by
rw [show a.realizedLeftArcParam =
  sideArcComplexIsometryMap a.leftCenter ◦
  unitCircleArc a.radius by
  funext s
  exact a.realizedLeftArcParam_eq s]
exact (isometry_sideArcComplexIsometryMap a.leftCenter).continuous.comp
  (continuous_unitCircleArc a.radius)
```

TypeThreeAssembly.continuous_realizedRightArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:700-709

Declaration SHA-256: 3d9c0740b38c1fa009799b8c35785a0cb767211b94d4ca0bc37637a119a58afd

Mathematical statement. The realized right arc parametrization is continuous on $\mathbb{R}$.

The same composition argument applies to the right side.

```
lemma continuous_realizedRightArcParam :
  Continuous a.realizedRightArcParam := by
rw [show a.realizedRightArcParam =
  sideArcComplexIsometryMap a.rightCenter ◦
  unitCircleArc a.radius by
  funext s
  exact a.realizedRightArcParam_eq s]
exact (isometry_sideArcComplexIsometryMap a.rightCenter).continuous.comp
  (continuous_unitCircleArc a.radius)
```

TypeThreeAssembly.coreCarrier_y_bounds

Classification: supplement

Source: TypeThreeAssembly.lean:1447-1455

Declaration SHA-256: a0a8c181bd45290d862033f8524f84e76d590a21a370fbbc0214aab783d9df85

Mathematical statement. Every point of the type-iii core satisfies $|y| \leq 1$.

The rectangle and both circular side segments are explicitly restricted to the closed strip; this makes their density identically one.

```
theorem coreCarrier_y_bounds {p : PlanePoint}
  (hp : p ∈ a.coreCarrier) : |p.2| ≤ 1 := by
```

```
rcases hp with (hrect | hleft) | hright
· exact abs_le.mpr ⟨hrect.2.1, hrect.2.2⟩
· exact abs_le.mpr hleft.2.2
· exact abs_le.mpr hright.2.2
```

```
-- Above the source strip, the genuine type-(iii) carrier is exactly its
actual exterior cap; none of the three core pieces can occur there. -/
```

TypeThreeAssembly.cos_sideArcAngle

Classification: supplement

Source: TypeThreeAssembly.lean:231-237

Declaration SHA-256: a8f9a8c72dcee4cc4dc59b4da10f2ae6d2741d7865a4b2faca10168fa17cc76a

Mathematical statement. $\cos \gamma = -q$.

This is the cosine identity for $\gamma = \pi - \beta$, using $\cos \beta = q$.

```
theorem cos_sideArcAngle :
  cos a.sideArcAngle = -a.shapeParameter := by
  rw [sideArcAngle, Real.cos_pi_sub, a.cos_innerAngle]

-- A full lower cap with the same radius and angle as the two translated
half-caps forming the strip-side pieces. Its area and arc length are exactly
the sums of those two congruent halves. -/
```

TypeThreeAssembly.exact_euclidean_leftArcParam_hausdorffMeasure

Classification: supplement

Source: TypeThreeAssembly.lean:742-778

Declaration SHA-256: c6dc496bb93e612df391bdcd13229ada6ddb03a05f7a4a85318498972631a8fc

Mathematical statement. The Euclidean left arc image has $\mathcal{H}^1$ -measure of $\text{Real}(R\gamma)$.

The parametrization interval has length $R\gamma$, and the side-arc isometry preserves the exact Hausdorff mass.

```
theorem exact_euclidean_leftArcParam_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
  (a.realizedLeftArcParam ''
    Icc (-a.radius * a.sideArcAngle) 0) =
  ENNReal.ofReal (a.radius * a.sideArcAngle) := by
have hfun : a.realizedLeftArcParam =
  sideArcComplexIsometryMap a.leftCenter ∘
  unitCircleArc a.radius := by
  funext s
  exact a.realizedLeftArcParam_eq s
have huv : -a.radius * a.sideArcAngle < 0 := by
  nlinarith [a.radius_pos, a.sideArcAngle_pos]
have hspan :
  0 - (-a.radius * a.sideArcAngle) < 2 * π * a.radius := by
  have hangle : a.sideArcAngle < 2 * π := by
    nlinarith [a.sideArcAngle_lt_pi, Real.pi_pos]
have hmul :=
  mul_lt_mul_of_pos_left hangle a.radius_pos
nlinarith
calc
  _ = (μH[1] : Measure EuclideanPlane)
    (sideArcComplexIsometryMap a.leftCenter ''
      (unitCircleArc a.radius ''
        Icc (-a.radius * a.sideArcAngle) 0)) := by
  rw [hfun]
  congr 1
  simp only [Function.comp_apply] using (Set.image_image _ _ _).symm
  _ = (μH[1] : Measure ℂ)
    (unitCircleArc a.radius ''
```

```

Icc (-a.radius * a.sideArcAngle) 0) :=
(isometry_sideArcComplexIsometryMap a.leftCenter).hausdorffMeasure_image
(Or.inl (by norm_num)) _
_ = ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  simp using
    (hausdorffMeasure_unitCircleArc_image_eq_of_span_lt_two_pi
     a.radius_pos huv hspan)

```

TypeThreeAssembly.exact_euclidean_leftArcTrace_hausdorffMeasure

Classification: supplement

Source: TypeThreeAssembly.lean:817-824

Declaration SHA-256: 9b484f3710815971495ad59b2a292d470a94ac2f3cad3edb96b8870979733b3

Mathematical statement. The Euclidean image of the left arc trace has $\mathcal{H}^1$ -measure ofReal($R\gamma$).

The image equality transfers the exact parametrized-arc measure to the trace set.

```

theorem exact_euclidean_leftArcTrace_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' a.leftArcTrace) =
  ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  rw [← a.realizedLeftArcParam_image_Icc]
  exact a.exact_euclidean_leftArcParam_hausdorffMeasure

/-- Exact Euclidean H1 mass of the right strip-side trace. -/

```

TypeThreeAssembly.exact_euclidean_rightArcParam_hausdorffMeasure

Classification: supplement

Source: TypeThreeAssembly.lean:779-816

Declaration SHA-256: f1f573c6c931308699d4611129bc237b3c89fe21d3a82fbb640a2bc4f452f054

Mathematical statement. The Euclidean right arc image has $\mathcal{H}^1$ -measure ofReal($R\gamma$).

The right trace has the same radius and angular span as the left trace.

```

theorem exact_euclidean_rightArcParam_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
    (a.realizedRightArcParam ''
      Icc 0 (a.radius * a.sideArcAngle)) =
  ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  have hfun : a.realizedRightArcParam =
    sideArcComplexIsometryMap a.rightCenter ∘
    unitCircleArc a.radius := by
    funext s
    exact a.realizedRightArcParam_eq s
  have huv : 0 < a.radius * a.sideArcAngle :=
    mul_pos a.radius_pos a.sideArcAngle_pos
  have hspan :
    a.radius * a.sideArcAngle - 0 < 2 * π * a.radius := by
    have hangle : a.sideArcAngle < 2 * π := by
      linarith [a.sideArcAngle_lt_pi, Real.pi_pos]
    have hmul :=
      mul_lt_mul_of_pos_left hangle a.radius_pos
    nlinarith
  calc
  _ = (μH[1] : Measure EuclideanPlane)
    (sideArcComplexIsometryMap a.rightCenter ''
      (unitCircleArc a.radius ''
        Icc 0 (a.radius * a.sideArcAngle))) := by
    rw [hfun]
    congr 1
    simp only [Function.comp_apply] using (Set.image_image _ _ _).symm
  _ = (μH[1] : Measure ℂ)

```

```

    (unitCircleArc a.radius ' '
      Icc 0 (a.radius * a.sideArcAngle)) :=
    (isometry_sideArcComplexIsometryMap a.rightCenter).hausdorffMeasure_image
      (Or.inl (by norm_num)) _
  _ = ENNReal.ofReal (a.radius * a.sideArcAngle) := by
    simp using
      (hausdorffMeasure_unitCircleArc_image_eq_of_span_lt_two_pi
        a.radius_pos huv hspan)

/-- Exact Euclidean  $H^1$  mass of the left strip-side trace. -/

```

TypeThreeAssembly.exact_euclidean_rightArcTrace_hausdorffMeasure

Classification: supplement

Source: TypeThreeAssembly.lean:825-832

Declaration SHA-256: 82deac992e3b3149a73b5a9867d35c880f6af8171bd8434cc5a883788eaac184

Mathematical statement. The Euclidean image of the right arc trace has $\mathcal{H}^1$ -measure of $\text{Real}(R\gamma)$.

This is the matching exact measure identity for the reflected trace.

```

theorem exact_euclidean_rightArcTrace_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph ' ' a.rightArcTrace) =
    ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  rw [← a.realizedRightArcParam_image_Icc]
  exact a.exact_euclidean_rightArcParam_hausdorffMeasure

/-- The two translated strip-side half-caps have the area of `sideCap`. -/

```

TypeThreeAssembly.frontier_carrier_subset_boundaryTrace

Classification: supplement

Source: TypeThreeAssembly.lean:2443-2458

Declaration SHA-256: e4a9f2900fa53b9204f0a28ca553872a35240978918b3cf32712160073f759cb

Mathematical statement. The complete frontier of the type-iii carrier is contained in the declared boundary trace.

No boundary component occurs outside the two side arcs, upper cap arc, and bottom segment.

```

theorem frontier_carrier_subset_boundaryTrace :
  frontier a.carrier ⊆ a.boundaryTrace := by
  intro p hp
  have htop := frontier_union_subset a.coreCarrier a.outerCap.carrier hp
  rcases htop with hcore | hcap
  · rcases a.frontier_coreCarrier_subset hcore.1 with
    hleft | hright | hchord | hbottom
    · exact Or.inl hleft
    · exact Or.inr (Or.inl hright)
    · exact a.outerChord_to_boundaryTrace hchord hp
    · exact Or.inr (Or.inr (Or.inr hbottom))
  · rw [a.outerCap.frontier_carrier] at hcap
    rcases hcap.2 with harc | hchord
    · exact Or.inr (Or.inr (Or.inl harc))
    · exact a.outerChord_to_boundaryTrace hchord hp

```

TypeThreeAssembly.innerAngle

Classification: supplement

Source: TypeThreeAssembly.lean:53-55

Declaration SHA-256: bb9acabc2bc9f5427d7041305c401b4e2793720759dd063e3153448db0793dd9

Mathematical statement. The inner strip angle is $\beta = \arccos(2h - 1)$.

It is the principal angle whose cosine is the source shape parameter.

```
def innerAngle : ℝ := arccos a.shapeParameter

/-- Snell-adjusted arccosine argument for the exterior angle `a₃`. -/
```

TypeThreeAssembly.innerAngle_lt_pi

Classification: supplement

Source: TypeThreeAssembly.lean:168-170

Declaration SHA-256: 31385b2dc48409bb1698d05b2b8cedcea6151f9ce4bedc38534228b77f838b5c

Mathematical statement. The principal inner angle satisfies $\beta < \pi$.

The shape parameter is strictly above minus one.

```
theorem innerAngle_lt_pi : a.innerAngle < π :=
  Real.arccos_lt_pi.mpr a.shapeParameter_mem.1
```

TypeThreeAssembly.innerAngle_pos

Classification: supplement

Source: TypeThreeAssembly.lean:165-167

Declaration SHA-256: e78599ffc8793999ca239c142df39a815df5b9220a19bd2bd882d169b85658b5

Mathematical statement. The principal inner angle $\beta = \arccos q$ is positive.

The shape parameter is strictly below one.

```
theorem innerAngle_pos : 0 < a.innerAngle :=
  Real.arccos_pos.mpr a.shapeParameter_mem.2
```

TypeThreeAssembly.innerAngle_semicircular

Classification: supplement

Source: TypeThreeAssembly.lean:370-373

Declaration SHA-256: 136b95db3cfbe6ffba49a9d377637e6fd44e92cfd37653b368ff453e7d5af642

Mathematical statement. If $h = 1/2$, then $\beta = \pi/2$.

At the transition the shape parameter is zero.

```
theorem innerAngle_semicircular (hh : a.h = 1 / 2) :
  a.innerAngle = π / 2 := by
  simp [innerAngle, shapeParameter, hh]
```

TypeThreeAssembly.innerSin_pos

Classification: supplement

Source: TypeThreeAssembly.lean:216-218

Declaration SHA-256: 88694b992a38f1dffa3c67f32d082f4eb742c98f01874e62941887bfa3c13f04d

Mathematical statement. The inner angle satisfies $\sin\beta > 0$.

The same principal-angle bounds give strict positivity.

```
theorem innerSin_pos : 0 < sin a.innerAngle :=  
  sin_pos_of_pos_of_lt_pi a.innerAngle_pos a.innerAngle_lt_pi
```

TypeThreeAssembly.isClosed_bottomSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1589-1594

Declaration SHA-256: 4502cfa8cbb9cfeeb9921da9cbccfe0b4a2a7c969d9473aede4beb6ee9ada554

Mathematical statement. The bottom segment carrier is closed.

It is defined by the equality $y = -1$ and the closed bound $|x| \leq w$.

```
theorem isClosed_bottomSegmentCarrier : IsClosed a.bottomSegmentCarrier := by  
  unfold bottomSegmentCarrier  
  exact (isClosed_eq continuous_snd continuous_const).inter  
    (isClosed_le continuous_fst.abs continuous_const)  
  
/-- The exact complete trace of the type-(iii) boundary. -/
```

TypeThreeAssembly.isClosed_carrier

Classification: supplement

Source: TypeThreeAssembly.lean:1307-1309

Declaration SHA-256: 86d49c14fed38ff9d123ec7002dc853d7ab13d112817308f4d1ef5ac7d463980

Mathematical statement. The complete type-iii carrier is closed.

The core and exterior cap are closed, and their finite union is closed.

```
theorem isClosed_carrier : IsClosed a.carrier := by  
  exact a.isClosed_coreCarrier.union a.outerCap.isClosed_carrier
```

TypeThreeAssembly.isClosed_coreCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1303-1306

Declaration SHA-256: c084bdc57b0172b2f03efa14200c62e0c7ef9b7e0265bac3ea60cec5c75fc390

Mathematical statement. The type-iii core carrier is closed.

It is a finite union of the closed rectangle and the two closed circular segments.

```
theorem isClosed_coreCarrier : IsClosed a.coreCarrier := by  
  exact (a.isClosed_rectangleCarrier.union a.isClosed_leftSegmentCarrier).union  
    a.isClosed_rightSegmentCarrier
```

TypeThreeAssembly.isClosed_leftSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1289-1295

Declaration SHA-256: 9a06aa24438ad23f7c7f01c4c87ea6fc01e48b3ecec4d358799e726bea34abb1

Mathematical statement. The left circular segment carrier is closed.

It is an intersection of a closed disk with closed half-plane and strip constraints.

```
theorem isClosed_leftSegmentCarrier : IsClosed a.leftSegmentCarrier := by
  unfold leftSegmentCarrier
  exact ((isClosed_disk a.leftCenter a.radius).inter
    ((isClosed_le continuous_fst continuous_const).inter
      ((isClosed_le continuous_const continuous_snd).inter
        (isClosed_le continuous_snd continuous_const))))
```

TypeThreeAssembly.isClosed_rectangleCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1282-1288

Declaration SHA-256: a040fbd7853bc5d56529ed0d45def53a17db2084e9e6f891db5067b23f32a560

Mathematical statement. The central rectangle carrier is closed.

All four coordinate inequalities are non-strict, so the rectangle is an intersection of closed half-spaces.

```
theorem isClosed_rectangleCarrier : IsClosed a.rectangleCarrier := by
  unfold rectangleCarrier
  exact ((isClosed_le continuous_const continuous_fst).inter
    (isClosed_le continuous_fst continuous_const)).inter
    ((isClosed_le continuous_const continuous_snd).inter
      (isClosed_le continuous_snd continuous_const))
```

TypeThreeAssembly.isClosed_rightSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1296-1302

Declaration SHA-256: 97ec89af687cf23451a62fdd3fb4816f90150533b2e6c5aca3fc7cb1fe8e300f

Mathematical statement. The right circular segment carrier is closed.

It is the reflected closed disk segment with non-strict coordinate constraints.

```
theorem isClosed_rightSegmentCarrier : IsClosed a.rightSegmentCarrier := by
  unfold rightSegmentCarrier
  exact (isClosed_disk a.rightCenter a.radius).inter
    ((isClosed_le continuous_const continuous_fst).inter
      ((isClosed_le continuous_const continuous_snd).inter
        (isClosed_le continuous_snd continuous_const)))
```

TypeThreeAssembly.isometry_sideArcComplexIsometryMap

Classification: supplement

Source: TypeThreeAssembly.lean:653-664

Declaration SHA-256: 5f9f98564eebaed09c3cd958295e743ceabd92f2b665793225e0fb64c4812ba4

Mathematical statement. For every center c , the side-arc complex map is an isometry from $\mathbb{C}$ to the Euclidean plane.

The coordinate expression preserves Euclidean distance.

```

theorem isometry_sideArcComplexIsometryMap (center : PlanePoint) :
  Isometry (sideArcComplexIsometryMap center) := by
  apply Isometry.of_dist_eq
  intro z w
  rw [WithLp.prod_dist_eq_add (by norm_num)]
  simp only [sideArcComplexIsometryMap]
  norm_num [Real.dist_eq, sq_abs]
  rw [← Real.sqrt_eq_rpow, Complex.dist_eq_re_im]
  congr 1
  ring

/-- Euclidean realization of the left signed-arclength parameter. -/

```

TypeThreeAssembly.leftArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:120-126

Declaration SHA-256: 4f29c060ae66edf060cbefb1375e17d2584dc8c159343c7bce4a1ff009d7632c

Mathematical statement. The left signed-arclength map is

$$s \mapsto (-w + R\sin(s/R), R - 1 - R\cos(s/R)).$$

Its parameter interval later runs from the upper junction through the bottom point, and its image is the clipped left trace.

```

def leftArcParam (s : ℝ) : PlanePoint :=
  (a.leftCenter.1 + a.radius * sin (s / a.radius),
   a.leftCenter.2 - a.radius * cos (s / a.radius))

/-- Signed-arclength parameter for the right strip-side half-circle. The
parameter runs from the bottom point at zero to the upper junction at
`radius * sideArcAngle`. -/

```

TypeThreeAssembly.leftArcParam_image_Icc

Classification: supplement

Source: TypeThreeAssembly.lean:446-546

Declaration SHA-256: a53d3bd58a5cf0ca117ca4045079faea921dc67e709030e1bc93b3bc04963479

Mathematical statement. The left arc parametrization maps $[-R\gamma, 0]$ exactly onto the left arc trace.

The interval covers the upper junction through the bottom point on every branch, including the semicircular transition.

```

theorem leftArcParam_image_Icc :
  a.leftArcParam ''
    Icc (-a.radius * a.sideArcAngle) 0 = a.leftArcTrace := by
  ext p
  constructor
  · rintro ⟨s, hs, rfl⟩
    have hu :
      s / a.radius ∈ Icc (-a.sideArcAngle) 0 := by
      constructor
      · apply (le_div_iff₀ a.radius_pos).2
        nlinarith [hs.1, a.radius_pos]
      · exact div_nonpos_of_nonpos_of_nonneg hs.2 a.radius_pos.le
    have hsin_nonpos : sin (s / a.radius) ≤ 0 :=
      Real.sin_nonpos_of_nonpos_of_neg_pi_le hu.2
      (by linarith [hu.1, a.sideArcAngle_lt_pi])
    have habs : |s / a.radius| ≤ a.sideArcAngle := by
      rw [abs_le]
      exact ⟨hu.1, hu.2.trans a.sideArcAngle_pos.le⟩
    have hcos :
      cos a.sideArcAngle ≤ cos (s / a.radius) := by
      simpa only [Real.cos_abs] using
        (Real.cos_le_cos_of_nonneg_of_le_pi

```

```

      (abs_nonneg (s / a.radius)) a.sideArcAngle_lt_pi.le habs)
have hcircle := Real.sin_sq_add_cos_sq (s / a.radius)
change
  ((a.leftCenter.1 + a.radius * sin (s / a.radius)) -
    a.leftCenter.1) ^ 2 +
    ((a.leftCenter.2 - a.radius * cos (s / a.radius)) -
      a.leftCenter.2) ^ 2 = a.radius ^ 2 ^
  a.leftCenter.1 + a.radius * sin (s / a.radius) ≤
    a.leftCenter.1 ^
  (-1 : ℝ) ≤
    a.leftCenter.2 - a.radius * cos (s / a.radius) ^
  a.leftCenter.2 - a.radius * cos (s / a.radius) ≤ 1
refine (by nlinarith [sq_nonneg a.radius], ?_, ?_, ?_)
· nlinarith [mul_nonpos_of_nonneg_of_nonpos a.radius_pos.le hsin_nonpos]
· change
  (-1 : ℝ) ≤
    (a.radius - 1) - a.radius * cos (s / a.radius)
  nlinarith [mul_le_mul_of_nonneg_left
    (Real.cos_le_one (s / a.radius)) a.radius_pos.le]
· change
  (a.radius - 1) - a.radius * cos (s / a.radius) ≤ 1
  nlinarith [mul_le_mul_of_nonneg_left hcos a.radius_pos.le,
    a.radius_mul_cos_sideArcAngle]
· intro hp
change
  (p.1 - a.leftCenter.1) ^ 2 +
    (p.2 - a.leftCenter.2) ^ 2 = a.radius ^ 2 ^
  p.1 ≤ a.leftCenter.1 ^
  (-1 : ℝ) ≤ p.2 ^ p.2 ≤ 1 at hp
let z : ℂ :=
  ⟨a.leftCenter.2 - p.2, p.1 - a.leftCenter.1⟩
have hnorm_sq : ||z|| ^ 2 = a.radius ^ 2 := by
  rw [Complex.sq_norm, Complex.normSq_apply]
  simp only [z]
  nlinarith [hp.1]
have hnorm : ||z|| = a.radius := by
  nlinarith [norm_nonneg z, a.radius_pos]
have hsin :
  a.radius * sin z.arg = p.1 - a.leftCenter.1 := by
  simpa [hnorm, z] using Complex.norm_mul_sin_arg z
have hcos :
  a.radius * cos z.arg = a.leftCenter.2 - p.2 := by
  simpa [hnorm, z] using Complex.norm_mul_cos_arg z
have hcos_le : cos a.sideArcAngle ≤ cos z.arg := by
  have hmul :
    a.radius * cos a.sideArcAngle ≤ a.radius * cos z.arg := by
    rw [a.radius_mul_cos_sideArcAngle, hcos]
    change a.radius - 2 ≤ (a.radius - 1) - p.2
    linarith [hp.2.2.2]
  nlinarith [hmul, a.radius_pos]
have harg : |z.arg| ≤ a.sideArcAngle := by
  by_contra h
  have hlt : a.sideArcAngle < |z.arg| := lt_of_not_ge h
  have hc := Real.cos_lt_cos_of_nonneg_of_le_pi
    a.sideArcAngle_pos.le (Complex.abs_arg_le_pi z) hlt
  rw [Real.cos_abs] at hc
  linarith
have harg_nonpos : z.arg ≤ 0 := by
  by_contra h
  have harg_pos : 0 < z.arg := lt_of_not_ge h
  have harg_lt_pi : z.arg < π :=
    lt_of_le_of_lt (le_trans (le_abs_self z.arg) harg)
    a.sideArcAngle_lt_pi
  have hsin_pos := Real.sin_pos_of_pos_of_lt_pi harg_pos harg_lt_pi
  nlinarith [a.radius_pos, hp.2.1]
refine (a.radius * z.arg, ?_, ?_)
· constructor
· nlinarith [a.radius_pos, (abs_le.mp harg).1]
· exact mul_nonpos_of_nonneg_of_nonpos a.radius_pos.le harg_nonpos
· have handle : a.radius * z.arg / a.radius = z.arg := by
  field_simp [a.radius_pos.ne']
  apply Prod.ext

```

```

· simp only [leftArcParam, hangle]
  linarith
· simp only [leftArcParam, hangle]
  linarith

```

```

/-- The right signed-arclength parameter fills exactly the right strip-side
trace, on every geometric branch (including the semicircular transition). -/

```

TypeThreeAssembly.leftArcTrace

Classification: supplement

Source: TypeThreeAssembly.lean:104-110

Declaration SHA-256: cfb74d8a54830ee4e695b37318a8d8c8a086063cc569f3e8d4fa167da3d5d0b9

Mathematical statement. The left side trace is $\{(x,y):(x+w)^2 + (y-(R-1))^2 = R^2, x \leq -w, -1 \leq y \leq 1\}$.

This is the exposed circular boundary of the left clipped segment, including its interface endpoints.

```

def leftArcTrace : Set PlanePoint :=
  {p | (p.1 - a.leftCenter.1) ^ 2 + (p.2 - a.leftCenter.2) ^ 2 =
    a.radius ^ 2 ^
    p.1 ≤ a.leftCenter.1 ^
    (-1 : ℝ) ≤ p.2 ^ p.2 ≤ 1}

/-- The exact right circular trace of the type-(iii) strip-side boundary. -/

```

TypeThreeAssembly.leftCenter

Classification: supplement

Source: TypeThreeAssembly.lean:70-72

Declaration SHA-256: ae634544d50a625f2da0ab83dddef536c2004aefef8a002e19ec3ff792534bdf2

Mathematical statement. For a regular type-iii assembly with radius R and half-width w , the left side-circle center is $(-w, R-1)$.

The complete side circle need not lie inside the strip; for example its height is $R-1$, which can exceed 1. Only the selected disk segment is clipped by $-1 \leq y \leq 1$.

```

def leftCenter : PlanePoint := (-a.sideHalfWidth, a.radius - 1)

/-- Source circle center for the right strip arc. -/

```

TypeThreeAssembly.leftSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:85-91

Declaration SHA-256: 7a6b1fb5a536fc1614b926138e279c79369045c2b292e00c145e1a0e699dbced

Mathematical statement. The left core segment is $\{(x,y):(x+w)^2 + (y-(R-1))^2 \leq R^2, x \leq -w, |y| \leq 1\}$.

The set is the left disk clipped both to the exterior side of the rectangle and to the closed strip; the full side circle is not asserted to be contained in the strip.

```

def leftSegmentCarrier : Set PlanePoint :=
  {p | (p.1 - a.leftCenter.1) ^ 2 + (p.2 - a.leftCenter.2) ^ 2 ≤
    a.radius ^ 2} ∩
  ({p | p.1 ≤ a.leftCenter.1} ∩
   ({p | (-1 : ℝ) ≤ p.2} ∩ {p | p.2 ≤ 1}))

/-- Closed right disk segment of `K₃`. -/

```

TypeThreeAssembly.measurableSet_carrier

Classification: supplement

Source: TypeThreeAssembly.lean:1310-1316

Declaration SHA-256: 25696aa660c5c71e8ce3e6a179a069d25633bc99ff664ad8e93ea6e319cb501f

Mathematical statement. The assembled type-iii carrier is measurable.

It is a finite union of closed coordinate-defined pieces, so it supports the weighted-area integral and the Euclidean frontier construction.

```
theorem measurableSet_carrier : MeasurableSet a.carrier :=
  a.isClosed_carrier.measurableSet

/-! ## Carrier-derived weighted area -/

/-- The left half of the standard side cap. Translating it horizontally gives
the actual left strip-side segment. -/
```

TypeThreeAssembly.measurableSet_euclidean_bottomSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:2855-2860

Declaration SHA-256: ac873340c9031353c9d26e2404b771c8a0ff1b638ea75b1a24ed627ed5bc802d

Mathematical statement. The Euclidean realization of the bottom segment is measurable.

It is the image of a compact horizontal interval under the coordinate homeomorphism.

```
theorem measurableSet_euclidean_bottomSegmentCarrier :
  MeasurableSet
    (planeEuclideanHomeomorph '' a.bottomSegmentCarrier) :=
  (planeEuclideanHomeomorph.isClosedMap _
    a.isClosed_bottomSegmentCarrier).measurableSet
```

TypeThreeAssembly.measurableSet_euclidean_leftArcTrace

Classification: supplement

Source: TypeThreeAssembly.lean:732-736

Declaration SHA-256: 791dc1e2fe6e2d0cb10c51a3bd4f4018c5ee53a25eeef24b22bc0d4e6f0f1192

Mathematical statement. The Euclidean image of the left arc trace is measurable.

The exact image is the compact continuous image of a closed bounded parameter interval, hence Borel measurable.

```
theorem measurableSet_euclidean_leftArcTrace :
  MeasurableSet (planeEuclideanHomeomorph '' a.leftArcTrace) := by
  rw [← a.realizedLeftArcParam_image_Icc]
  exact (isCompact_Icc.image a.continuous_realizedLeftArcParam).measurableSet
```

TypeThreeAssembly.measurableSet_euclidean_rightArcTrace

Classification: supplement

Source: TypeThreeAssembly.lean:737-741

Declaration SHA-256: 64197e0a340a1643ad8bc1525fe961f5b8792a0e7cb9321824fa2436e9d02bb4

Mathematical statement. The Euclidean image of the right arc trace is measurable.

The same compact-parametrization argument applies to the right trace.

```
theorem measurableSet_euclidean_rightArcTrace :
  MeasurableSet (planeEuclideanHomeomorph '' a.rightArcTrace) := by
  rw [← a.realizedRightArcParam_image_Icc]
  exact (isCompact_Icc.image a.continuous_realizedRightArcParam).measurableSet
```

TypeThreeAssembly.one_lt_radius

Classification: supplement

Source: TypeThreeAssembly.lean:148-152

Declaration SHA-256: 4e892eef63bc8252d5a243e2b5495e748b55fa956d516b3510c9e5289ec264d9

Mathematical statement. Every regular type-iii assembly has $R > 1$.

The strict curvature bound $h < 1$ gives a radius strictly larger than the strip half-height.

```
theorem one_lt_radius : 1 < a.radius := by
  rw [radius]
  apply (lt_div_iff₀ a.h_pos).2
  nlinarith [a.h_lt_one]
```

TypeThreeAssembly.outerAngle

Classification: supplement

Source: TypeThreeAssembly.lean:59-63

Declaration SHA-256: 0e2ff2070394fdda5b4ec09824f47ed543dabdfef101c076ad378e0c92be2ff47

Mathematical statement. The exterior cap half-angle is $\alpha = \arccos(q/\lambda)$.

This is the principal branch selected by the density-scaled incidence relation.

```
def outerAngle : ℝ := arccos a.outerArgument

/-- Half-central-angle of either strip-side arc. This is the complementary
branch to `innerAngle`, so it remains valid on both sides of `h = 1 / 2`. -/
```

TypeThreeAssembly.outerAngle_lt_pi

Classification: supplement

Source: TypeThreeAssembly.lean:174-177

Declaration SHA-256: ce135b6a3dc191fb33efd487aab9da31365b8ec038647a624b62a436ba911d34

Mathematical statement. The principal exterior angle satisfies $\alpha < \pi$.

The exterior argument is strictly above minus one.

```
theorem outerAngle_lt_pi : a.outerAngle < π :=
  Real.arccos_lt_pi.mpr a.outerArgument_mem.1

/-- The principal-angle definitions recover the source strip angle exactly. -/
```

TypeThreeAssembly.outerAngle_pos

Classification: supplement

Source: TypeThreeAssembly.lean:171-173

Declaration SHA-256: 760f9b90fd549c278aa760178ec027d7c643f8b5bd6d4bf942c86f6a65bb0e28

Mathematical statement. The principal exterior angle $\alpha = \arccos(q/\lambda)$ is positive.

The exterior argument is strictly below one.

```
theorem outerAngle_pos : 0 < a.outerAngle :=  
  Real.arccos_pos.mpr a.outerArgument_mem.2
```

TypeThreeAssembly.outerAngle_semicircular

Classification: supplement

Source: TypeThreeAssembly.lean:374-377

Declaration SHA-256: 9ff26a1092e7089058ecf98f24fecce78972031b475870c0e4d30009e831f8ee

Mathematical statement. If $h = 1/2$, then $\alpha = \pi/2$.

The exterior argument is also zero at the same transition.

```
theorem outerAngle_semicircular (hh : a.h = 1 / 2) :  
  a.outerAngle =  $\pi / 2$  := by  
  simp [outerAngle, outerArgument, shapeParameter, hh]
```

TypeThreeAssembly.outerArgument

Classification: supplement

Source: TypeThreeAssembly.lean:56-58

Declaration SHA-256: d6cfc025aa1368c77ccefa6b050dbf8bfa113c35569b4434625ec2d735b91f41

Mathematical statement. The Snell-adjusted outer cosine argument is q/λ .

The density jump $\lambda > 1$ keeps this argument in $(-1, 1)$ whenever $q \in (-1, 1)$.

```
def outerArgument :  $\mathbb{R}$  := a.shapeParameter / lam  
  
/-- The exterior half-central-angle `a3`.
```

TypeThreeAssembly.outerCap_radius

Classification: supplement

Source: TypeThreeAssembly.lean:419-422

Declaration SHA-256: f107696210fe89f5a746d54584303c70da482f05587bc229f2d37aaa9494b57f

Mathematical statement. The outer cap has radius R .

Its chord is $2R\sin\alpha$, so the cap radius formula cancels the positive sine factor.

```
theorem outerCap_radius : a.outerCap.radius = a.radius := by  
  change (2 * a.radius * sin a.outerAngle) / (2 * sin a.outerAngle) = a.radius  
  field_simp [ne_of_gt a.outerSin_pos]
```

TypeThreeAssembly.outerSin_pos

Classification: supplement

Source: TypeThreeAssembly.lean:213-215

Declaration SHA-256: feedd827e6203dedcb21f4f474b9fea36ae3ea708b33b644b944093c5f958959

Mathematical statement. The exterior angle satisfies $\sin\alpha > 0$.

The angle lies strictly between zero and π .

```
theorem outerSin_pos : 0 < sin a.outerAngle :=
  sin_pos_of_pos_of_lt_pi a.outerAngle_pos a.outerAngle_lt_pi
```

TypeThreeAssembly.pairwiseOverlap_h1_null

Classification: supplement

Source: TypeThreeAssembly.lean:1258-1273

Declaration SHA-256: 545b8d3e4922237126e7364ee9bb07b444cf8cada72b02e8afca89501d2f79b3

Mathematical statement. The Euclidean image of that type-iii pairwise overlap has zero one-dimensional Hausdorff measure.

The inclusion in a finite join set and finiteness of the join set imply the null-measure conclusion.

```
theorem pairwiseOverlap_h1_null :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' pairwiseOverlap
    a.leftArcTrace a.rightArcTrace
    (OneSidedCircularCap.arcTrace a.outerCap)
    a.bottomSegmentCarrier) = 0 := by
  let _ := Measure.nullSingletonClass_hausdorff EuclideanPlane
  (by norm_num : (0 : ℝ) < 1)
  have hjoin :
    (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' a.boundaryJoinSet) = 0 :=
    (a.boundaryJoinSet_finite.image planeEuclideanHomeomorph).measure_zero
    μH[1]
  exact measure_mono_null
    (image_mono a.pairwiseOverlap_subset_boundaryJoinSet) hjoin
```

TypeThreeAssembly.pairwiseOverlap_subset_boundaryJoinSet

Classification: supplement

Source: TypeThreeAssembly.lean:1142-1257

Declaration SHA-256: f8736f4dd5a67fc4149e15b1ed3e3f21030a49e12643cd517b41158566d4664f

Mathematical statement. The pairwise overlap of the left trace, right trace, outer-cap trace, and bottom segment is contained in the boundary join set.

This is an assembly-specific geometric inclusion; it does not turn the unrestricted overlap definition into a finiteness theorem.

```
theorem pairwiseOverlap_subset_boundaryJoinSet :
  pairwiseOverlap a.leftArcTrace a.rightArcTrace
    (OneSidedCircularCap.arcTrace a.outerCap)
    a.bottomSegmentCarrier ⊆ a.boundaryJoinSet := by
  intro p hp
  simp only [pairwiseOverlap, mem_union, mem_inter_iff] at hp
  rcases hp with hLR | hLU | hLB | hRU | hRB | hUB
  · by_cases hw : a.sideHalfWidth = 0
    · have hx : p.1 = 0 := by
        have hl := hLR.1.2.1
        have hr := hLR.2.2.1
        simp only [leftCenter, rightCenter, hw, neg_zero] at hl hr
        linarith
      have hcircle := hLR.1.1
      rw [hx] at hcircle
      simp only [leftCenter, hw, neg_zero, sub_zero] at hcircle
      have hfactor :
        (p.2 + 1) * (p.2 - (2 * a.radius - 1)) = 0 := by
          nlinarith
      rcases mul_eq_zero.mp hfactor with hy | hy
      · have hy' : p.2 = -1 := by linarith
```

```

      have hpEq : p = (0, -1) := Prod.ext hx hy'
      simp [boundaryJoinSet, hpEq, hw]
    · have hy' : p.2 = 2 * a.radius - 1 := by linarith
      have hyUpper := hLR.1.2.2.2
      exfalso
      nlinarith [a.one_lt_radius]
    · have hwpos : 0 < a.sideHalfWidth :=
      lt_of_le_of_ne a.sideHalfWidth_nonneg (Ne.symm hw)
      have hl := hLR.1.2.1
      have hr := hLR.2.2.1
      simp only [leftCenter, rightCenter] at hl hr
      exfalso
      linarith
    · have hyCore : p.2 ≤ 1 := hLU.1.2.2.2
      have hyCap : (1 : ℝ) ≤ p.2 := by
        simp [outerCap] using hLU.2.2
      have hy : p.2 = 1 := le_antisymm hyCore hyCap
      have hdx : p.1 - a.leftCenter.1 ≤ 0 := by
        linarith [hLU.1.2.1]
      have hsq :
        (p.1 - a.leftCenter.1) ^ 2 =
          (a.radius * sin a.innerAngle) ^ 2 := by
        have hpCircle := hLU.1.1
        rw [hy] at hpCircle
        simp only [leftCenter] at hpCircle
        rw [leftCenter]
        nlinarith [a.side_circle_at_top]
      have habs := (sq_eq_sq_iff_abs_eq_abs
        (p.1 - a.leftCenter.1)
        (a.radius * sin a.innerAngle)).mp hsq
      rw [abs_of_nonpos hdx,
        abs_of_pos (mul_pos a.radius_pos a.innerSin_pos)] at habs
      have hdxEq :
        p.1 - a.leftCenter.1 =
          -(a.radius * sin a.innerAngle) := by linarith
      have hx : p.1 = -(a.radius * sin a.outerAngle) := by
        simp only [leftCenter] at hdxEq
        rw [sideHalfWidth] at hdxEq
        linarith
      have hpEq : p = (-a.radius * sin a.outerAngle, 1) := by
        apply Prod.ext
        · simp using hx
        · simp using hy
      simp [boundaryJoinSet, hpEq]
    · have hy : p.2 = -1 := hLB.2.1
      have hbound : |p.1| ≤ a.sideHalfWidth := hLB.2.2
      have hxLower := (abs_le.mp hbound).1
      have hxUpper := hLB.1.2.1
      simp only [leftCenter] at hxUpper
      have hx : p.1 = -a.sideHalfWidth := by linarith
      have hpEq : p = (-a.sideHalfWidth, -1) := Prod.ext hx hy
      simp [boundaryJoinSet, hpEq]
    · have hyCore : p.2 ≤ 1 := hRU.1.2.2.2
      have hyCap : (1 : ℝ) ≤ p.2 := by
        simp [outerCap] using hRU.2.2
      have hy : p.2 = 1 := le_antisymm hyCore hyCap
      have hdx : 0 ≤ p.1 - a.rightCenter.1 := by
        linarith [hRU.1.2.1]
      have hsq :
        (p.1 - a.rightCenter.1) ^ 2 =
          (a.radius * sin a.innerAngle) ^ 2 := by
        have hpCircle := hRU.1.1
        rw [hy] at hpCircle
        simp only [rightCenter] at hpCircle
        rw [rightCenter]
        nlinarith [a.side_circle_at_top]
      have habs := (sq_eq_sq_iff_abs_eq_abs
        (p.1 - a.rightCenter.1)
        (a.radius * sin a.innerAngle)).mp hsq
      rw [abs_of_nonneg hdx,
        abs_of_pos (mul_pos a.radius_pos a.innerSin_pos)] at habs
      have hdxEq :

```

```

    p.1 - a.rightCenter.1 =
      a.radius * sin a.innerAngle := habs
  have hx : p.1 = a.radius * sin a.outerAngle := by
    simp only [rightCenter] at hxEq
    rw [sideHalfWidth] at hxEq
    linarith
  have hpEq : p = (a.radius * sin a.outerAngle, 1) := Prod.ext hx hy
  simp [boundaryJoinSet, hpEq]
  · have hy : p.2 = -1 := hRB.2.1
  have hbound : |p.1| ≤ a.sideHalfWidth := hRB.2.2
  have hxUpper := (abs_le.mp hbound).2
  have hxLower := hRB.1.2.1
  simp only [rightCenter] at hxLower
  have hx : p.1 = a.sideHalfWidth := by linarith
  have hpEq : p = (a.sideHalfWidth, -1) := Prod.ext hx hy
  simp [boundaryJoinSet, hpEq]
  · have hyCap : (1 : ℝ) ≤ p.2 := by
      simpa [outerCap] using hUB.1.2
  have hyBottom : p.2 = -1 := hUB.2.1
  exfalso
  linarith

/-- Pairwise primitive overlaps carry no Euclidean `H¹` mass. -/

```

TypeThreeAssembly.radius

Classification: supplement

Source: TypeThreeAssembly.lean:47-49

Declaration SHA-256: 44bdb144ee33a0fe634670e914d151b3f9a50f2b2f7db8da7c3fad23bdfbf70e

Mathematical statement. The type-iii assembly radius is $R = 1/h$.

The reciprocal-curvature convention is used for every circular component and is strictly positive on the regular domain.

```

def radius : ℝ := 1 / a.h

/-- Source scalar `2h - 1`, equal to `cos b₃`. -/

```

TypeThreeAssembly.radius_pos

Classification: supplement

Source: TypeThreeAssembly.lean:145-147

Declaration SHA-256: a610fc617d9a92f1154e4b9627bb16a0d909464945f9e22ccb900c975acb2c7d

Mathematical statement. Every regular type-iii assembly has $R > 0$.

This follows from the assembly hypothesis $h > 0$ and the reciprocal-radius definition.

```

theorem radius_pos : 0 < a.radius := by
  exact one_div_pos.mpr a.h_pos

```

TypeThreeAssembly.realizedLeftArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:665-668

Declaration SHA-256: 51fbf9517aa10415de53dc106db635c5f70b9c39860ba75778e696cdeb2d6c3

Mathematical statement. The realized left arc parametrization is $J(\text{leftArcParam}(s))$.

This transports the coordinate side trace to the Euclidean metric space.

```

noncomputable def realizedLeftArcParam (s : ℝ) : EuclideanPlane :=
  planeEuclideanHomeomorph (a.leftArcParam s)

/-- Euclidean realization of the right signed-arclength parameter. -/

```

TypeThreeAssembly.realizedLeftArcParam_eq

Classification: supplement

Source: TypeThreeAssembly.lean:672-680

Declaration SHA-256: 538a7925bd9a6977d5e74933bd5f951885abb82dc534e3d102db01bf6aa01a1e

Mathematical statement. The realized left parametrization equals the side-arc complex isometry at the left center applied to $\gamma_R(s)$.

The identity follows from the real and imaginary coordinates of the standard circle.

```

lemma realizedLeftArcParam_eq (s : ℝ) :
  a.realizedLeftArcParam s =
    sideArcComplexIsometryMap a.leftCenter
      (unitCircleArc a.radius s) := by
  rw [show a.realizedLeftArcParam s =
    WithLp.toLp 2 (a.leftArcParam s) by rfl]
  simp [leftArcParam, sideArcComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im]

```

TypeThreeAssembly.realizedLeftArcParam_image_Icc

Classification: supplement

Source: TypeThreeAssembly.lean:710-720

Declaration SHA-256: 06e6662a39cd836487840418c7224db9c1b8779c37aa295aab4e1109503a76c7

Mathematical statement. The realized left arc image on $[-R\gamma, 0]$ equals the Euclidean image of the left arc trace.

This is the homeomorphic transport of the coordinate image theorem.

```

theorem realizedLeftArcParam_image_Icc :
  a.realizedLeftArcParam ''
    Icc (-a.radius * a.sideArcAngle) 0 =
    planeEuclideanHomeomorph '' a.leftArcTrace := by
  calc
  _ = planeEuclideanHomeomorph ''
    (a.leftArcParam '' Icc (-a.radius * a.sideArcAngle) 0) := by
    rw [Set.image_image]
    rfl
  _ = _ := by rw [a.leftArcParam_image_Icc]

```

TypeThreeAssembly.realizedRightArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:669-671

Declaration SHA-256: 8a963395af5bf096a9c5748471f90364728d8fe511c4e2cc0591bd2068d9e016

Mathematical statement. The realized right arc parametrization is $J(\text{rightArcParam}(s))$.

This is the corresponding definition for the right trace.

```

noncomputable def realizedRightArcParam (s : ℝ) : EuclideanPlane :=
  planeEuclideanHomeomorph (a.rightArcParam s)

```

TypeThreeAssembly.realizedRightArcParam_eq

Classification: supplement

Source: TypeThreeAssembly.lean:681-689

Declaration SHA-256: 3339fa9296e8c62289cd021c3334658bb86b73401d06cffbaa790d7423ffd93e

Mathematical statement. The realized right parametrization equals the side-arc complex isometry at the right center applied to $\gamma_R(s)$.

This is the reflected pointwise realization identity.

```
lemma realizedRightArcParam_eq (s : ℝ) :
  a.realizedRightArcParam s =
    sideArcComplexIsometryMap a.rightCenter
      (unitCircleArc a.radius s) := by
  rw [show a.realizedRightArcParam s =
    WithLp.toLp 2 (a.rightArcParam s) by rfl]
  simp [rightArcParam, sideArcComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im]
```

TypeThreeAssembly.realizedRightArcParam_image_Icc

Classification: supplement

Source: TypeThreeAssembly.lean:721-731

Declaration SHA-256: 9469dc298ecad88323016b9907111e322a864f0831a991b637ca11824881af16

Mathematical statement. The realized right arc image on $[0, R\gamma]$ equals the Euclidean image of the right arc trace.

This is the corresponding transported right-trace equality.

```
theorem realizedRightArcParam_image_Icc :
  a.realizedRightArcParam ''
    Icc 0 (a.radius * a.sideArcAngle) =
    planeEuclideanHomeomorph '' a.rightArcTrace := by
  calc
    _ = planeEuclideanHomeomorph ''
      (a.rightArcParam '' Icc 0 (a.radius * a.sideArcAngle)) := by
      rw [Set.image_image]
      rfl
    _ = _ := by rw [a.rightArcParam_image_Icc]
```

TypeThreeAssembly.rectangleCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:80-84

Declaration SHA-256: d451775e4018b5bae444c6a8ed6911c5adcc23db4a0efc68ab71c655f2933477

Mathematical statement. The central rectangular part of the type-iii core is $\{(x, y) : -w \leq x \leq w, -1 \leq y \leq 1\}$.

This is a closed coordinate rectangle between the translated side segments.

```
def rectangleCarrier : Set PlanePoint :=
  ({p | -a.sideHalfWidth ≤ p.1} ∩ {p | p.1 ≤ a.sideHalfWidth}) ∩
  ({p | (-1 : ℝ) ≤ p.2} ∩ {p | p.2 ≤ 1})

/-- Closed left disk segment of `K₃`. -/
```

TypeThreeAssembly.rightArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:127-131

Declaration SHA-256: 74515bd4fc474dfa2c52a14662b8cc34aa0f40ef9d8185819e05d1de94ccf588

Mathematical statement. The right signed-arclength map is

$$s \mapsto (w + R\sin(s/R), R - 1 - R\cos(s/R)).$$

This reflected map runs from the bottom point to the upper right junction.

```
def rightArcParam (s : ℝ) : PlanePoint :=
  (a.rightCenter.1 + a.radius * sin (s / a.radius),
   a.rightCenter.2 - a.radius * cos (s / a.radius))

/-- The possibly degenerate bottom segment. -/
```

TypeThreeAssembly.rightArcParam_image_Icc

Classification: supplement

Source: TypeThreeAssembly.lean:547-648

Declaration SHA-256: d47f88fb5eadc69e4f4bc7ce3bc201dc34d60ad570f34d3d29c4d0184ef0f8f1

Mathematical statement. The right arc parametrization maps $[0, R\gamma]$ exactly onto the right arc trace.

This is the reflected branch-complete parametrization from the bottom point to the upper junction.

```
theorem rightArcParam_image_Icc :
  a.rightArcParam ''
    Icc 0 (a.radius * a.sideArcAngle) = a.rightArcTrace := by
  ext p
  constructor
  · rintro ⟨s, hs, rfl⟩
    have hu :
      s / a.radius ∈ Icc 0 a.sideArcAngle := by
      constructor
      · exact div_nonneg hs.1 a.radius_pos.le
      · apply (div_le_iff a.radius_pos).2
        nlinarith [hs.2, a.radius_pos]
    have hsin_nonneg : 0 ≤ sin (s / a.radius) :=
      Real.sin_nonneg_of_nonneg_of_le_pi hu.1
      (hu.2.trans a.sideArcAngle_lt_pi.le)
    have habs : |s / a.radius| ≤ a.sideArcAngle := by
      rw [abs_le]
      exact ⟨by linarith [hu.1, a.sideArcAngle_pos], hu.2⟩
    have hcos :
      cos a.sideArcAngle ≤ cos (s / a.radius) := by
      simpa only [Real.cos_abs] using
        (Real.cos_le_cos_of_nonneg_of_le_pi
         (abs_nonneg (s / a.radius)) a.sideArcAngle_lt_pi.le habs)
    have hcircle := Real.sin_sq_add_cos_sq (s / a.radius)
    change
      ((a.rightCenter.1 + a.radius * sin (s / a.radius)) -
       a.rightCenter.1) ^ 2 +
      ((a.rightCenter.2 - a.radius * cos (s / a.radius)) -
       a.rightCenter.2) ^ 2 = a.radius ^ 2 ^
      a.rightCenter.1 ≤
      a.rightCenter.1 + a.radius * sin (s / a.radius) ^
      (-1 : ℝ) ≤
      a.rightCenter.2 - a.radius * cos (s / a.radius) ^
      a.rightCenter.2 - a.radius * cos (s / a.radius) ≤ 1
    refine (by nlinarith [sq_nonneg a.radius], ?_, ?_, ?_)
    · nlinarith [mul_nonneg a.radius_pos.le hsin_nonneg]
    · change
      (-1 : ℝ) ≤
```

```

      (a.radius - 1) - a.radius * cos (s / a.radius)
nlinarith [mul_le_mul_of_nonneg_left
  (Real.cos_le_one (s / a.radius)) a.radius_pos.le]
· change
  (a.radius - 1) - a.radius * cos (s / a.radius) ≤ 1
nlinarith [mul_le_mul_of_nonneg_left hcos a.radius_pos.le,
  a.radius_mul_cos_sideArcAngle]
· intro hp
change
  (p.1 - a.rightCenter.1) ^ 2 +
    (p.2 - a.rightCenter.2) ^ 2 = a.radius ^ 2 ∧
    a.rightCenter.1 ≤ p.1 ∧
    (-1 : ℝ) ≤ p.2 ∧ p.2 ≤ 1 at hp
let z : ℂ :=
  (a.rightCenter.2 - p.2, p.1 - a.rightCenter.1)
have hnorm_sq : ||z|| ^ 2 = a.radius ^ 2 := by
  rw [Complex.sq_norm, Complex.normSq_apply]
  simp only [z]
  nlinarith [hp.1]
have hnorm : ||z|| = a.radius := by
  nlinarith [norm_nonneg z, a.radius_pos]
have hsin :
  a.radius * sin z.arg = p.1 - a.rightCenter.1 := by
  simp [hnorm, z] using Complex.norm_mul_sin_arg z
have hcos :
  a.radius * cos z.arg = a.rightCenter.2 - p.2 := by
  simp [hnorm, z] using Complex.norm_mul_cos_arg z
have hcos_le : cos a.sideArcAngle ≤ cos z.arg := by
  have hmul :
    a.radius * cos a.sideArcAngle ≤ a.radius * cos z.arg := by
    rw [a.radius_mul_cos_sideArcAngle, hcos]
    change a.radius - 2 ≤ (a.radius - 1) - p.2
    linarith [hp.2.2.2]
  nlinarith [hmul, a.radius_pos]
have harg : |z.arg| ≤ a.sideArcAngle := by
  by_contra h
  have hlt : a.sideArcAngle < |z.arg| := lt_of_not_ge h
  have hc := Real.cos_lt_cos_of_nonneg_of_le_pi
    a.sideArcAngle_pos.le (Complex.abs_arg_le_pi z) hlt
  rw [Real.cos_abs] at hc
  linarith
have harg_nonneg : 0 ≤ z.arg := by
  by_contra h
  have harg_neg : z.arg < 0 := lt_of_not_ge h
  have harg_gt_neg_pi : -π < z.arg :=
    lt_of_lt_of_le (neg_lt_neg a.sideArcAngle_lt_pi)
      (abs_le.mp harg).1
  have hsin_neg :=
    Real.sin_neg_of_neg_of_neg_pi_lt harg_neg harg_gt_neg_pi
  nlinarith [a.radius_pos, hp.2.1]
refine (a.radius * z.arg, ?_, ?_)
· constructor
· exact mul_nonneg a.radius_pos.le harg_nonneg
· nlinarith [a.radius_pos, (abs_le.mp harg).2]
· have hangle : a.radius * z.arg / a.radius = z.arg := by
  field_simp [a.radius_pos.ne']
  apply Prod.ext
· simp only [rightArcParam, hangle]
  linarith
· simp only [rightArcParam, hangle]
  linarith

```

```

/-- Euclidean isometry that places the standard signed-arclength circle at a
strip-side center, with the orientation used by both side parameters. -/

```

TypeThreeAssembly.rightArcTrace

Classification: supplement

Source: TypeThreeAssembly.lean:111-119

Declaration SHA-256: 828cebab0c56e67a2a3cfaae9e912a8a76f497287dcea6b589e1a22a40edd616

Mathematical statement. The right side trace is $\{(x,y):(x-w)^2 + (y-(R-1))^2 = R^2, w \leq x, -1 \leq y \leq 1\}$.

This is the reflected exposed circular boundary of the right clipped segment.

```
def rightArcTrace : Set PlanePoint :=
  {p | (p.1 - a.rightCenter.1) ^ 2 + (p.2 - a.rightCenter.2) ^ 2 =
    a.radius ^ 2 ^
    a.rightCenter.1 ≤ p.1 ^
    (-1 : ℝ) ≤ p.2 ^ p.2 ≤ 1}

/-- Signed-arclength parameter for the left strip-side half-circle. The
parameter runs from the upper junction at `radius * sideArcAngle` to the
bottom point at zero. -/
```

TypeThreeAssembly.rightCenter

Classification: supplement

Source: TypeThreeAssembly.lean:73-75

Declaration SHA-256: c1d50740a1dfb76a9c424f19f6e78589f8a8ab930ab1727da5282aec42c3f25

Mathematical statement. The right side-circle center is $(w, R - 1)$.

As on the left, the theorem concerns the full circle center; the right selected segment, rather than the full circle, is restricted to the strip.

```
def rightCenter : PlanePoint := (a.sideHalfWidth, a.radius - 1)

/-- Source circle center for the exterior upper arc. -/
```

TypeThreeAssembly.rightSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:92-98

Declaration SHA-256: 889596cedc4d7d957005581540478e3a458f9f3d1f37c9af6b1a713eee05b078

Mathematical statement. The right core segment is $\{(x,y):(x-w)^2 + (y-(R-1))^2 \leq R^2, w \leq x, |y| \leq 1\}$.

This is the reflected right disk segment with the same explicit strip clipping.

```
def rightSegmentCarrier : Set PlanePoint :=
  {p | (p.1 - a.rightCenter.1) ^ 2 + (p.2 - a.rightCenter.2) ^ 2 ≤
    a.radius ^ 2} ∩
  ({p | a.rightCenter.1 ≤ p.1} ∩
   ({p | (-1 : ℝ) ≤ p.2} ∩ {p | p.2 ≤ 1}))

/-- The part of the type-(iii) carrier inside the closed strip. -/
```

TypeThreeAssembly.shapeParameter

Classification: supplement

Source: TypeThreeAssembly.lean:50-52

Declaration SHA-256: 29190b0ec0676db3ee8da889fc7b1e078bce75188f6b4271945e3fee045f2920

Mathematical statement. The type-iii shape parameter is $q = 2h - 1$.

For $0 < h < 1$, it lies in $(-1, 1)$, so the principal angles $\arccos q$ and $\arccos(q/\lambda)$ are defined on their intended branches.

```
def shapeParameter : ℝ := 2 * a.h - 1

/-- The source angle `b₃`, represented by its principal arccosine. -/
```

TypeThreeAssembly.sideArcAngle

Classification: supplement

Source: TypeThreeAssembly.lean:64-65

Declaration SHA-256: c68849312591ff361810874588f817eab8327b61ad9b815244ad1ab61a55597a

Mathematical statement. For a regular type-iii assembly with $\beta = \arccos(2h - 1)$, the side-arc angle is $\gamma = \pi - \beta$.

Here γ is the full central-angle span of each individual clipped strip-side trace. It is the half-central angle of the auxiliary lower cap whose two congruent halves represent the two side traces; it is not the half-angle of an individual side trace.

```
def sideArcAngle : ℝ := π - a.innerAngle
/-- Half the bottom-segment length. -/
```

TypeThreeAssembly.sideArcAngle_lt_pi

Classification: supplement

Source: TypeThreeAssembly.lean:223-226

Declaration SHA-256: 27c4b01d9c8710ab675792367c9a16f3b7952720c452f471c97b46d487e08c08

Mathematical statement. The side angle satisfies $\gamma < \pi$.

The inner angle is strictly positive.

```
theorem sideArcAngle_lt_pi : a.sideArcAngle < π := by
  rw [sideArcAngle]
  linarith [a.innerAngle_pos]
```

TypeThreeAssembly.sideArcAngle_pos

Classification: supplement

Source: TypeThreeAssembly.lean:219-222

Declaration SHA-256: fb3e4cdd6ff568be2285a238c6293705a5d18ec17ca0000cf9fc5fd3be6f53e2

Mathematical statement. The side angle $\gamma = \pi - \beta$ is positive.

The inner angle is strictly below π .

```
theorem sideArcAngle_pos : 0 < a.sideArcAngle := by
  rw [sideArcAngle]
  linarith [a.innerAngle_lt_pi]
```

TypeThreeAssembly.sideArcComplexIsometryMap

Classification: supplement

Source: TypeThreeAssembly.lean:649-652

Declaration SHA-256: 2d2b0ae6442b1aeb7779e0e2c761b53071a3b1948418645b546ba443dcc3b322

Mathematical statement. For a center c , the side-arc complex map is $z \mapsto (c_x + \text{Im}z, c_y - \text{Re}z)$.

The map translates and rotates the standard complex circle into either side trace.

```
noncomputable def sideArcComplexIsometryMap
  (center : PlanePoint) (z : ℂ) : EuclideanPlane :=
  WithLp.toLp 2 (center.1 + z.im, center.2 - z.re)
```

TypeThreeAssembly.sideCap_center

Classification: supplement

Source: TypeThreeAssembly.lean:430-438

Declaration SHA-256: 5c32cc16475c4c5293c8fcbce19421c65fe608e9916172122a7cb5ff16d6b389

Mathematical statement. The side cap center is $(0, R - 1)$.

The lower-cap center formula with base line $y = 1$, angle γ , and radius R gives this coordinate.

```
theorem sideCap_center :
  a.sideCap.center = (0, a.radius - 1) := by
  change
    (0, 1 + a.sideCap.radius * cos a.sideArcAngle) =
      (0, a.radius - 1)
  rw [a.sideCap.radius, a.cos_sideArcAngle, radius, shapeParameter]
  field_simp [ne_of_gt a.h_pos]
  ring
```

TypeThreeAssembly.sideCap_radius

Classification: supplement

Source: TypeThreeAssembly.lean:423-429

Declaration SHA-256: 860b391d38da56536b3934963761766a28238619240605477033823fc900d1af

Mathematical statement. The side cap has radius R .

Its chord is $2R\sin\beta$, and $\sin\gamma = \sin\beta$ makes the cap radius equal to the assembly radius.

```
theorem sideCap_radius : a.sideCap.radius = a.radius := by
  change
    (2 * a.radius * sin a.innerAngle) /
      (2 * sin a.sideArcAngle) = a.radius
  rw [a.sin_sideArcAngle]
  field_simp [ne_of_gt a.innerSin_pos]
```

TypeThreeAssembly.sideHalfWidth

Classification: supplement

Source: TypeThreeAssembly.lean:66-69

Declaration SHA-256: c1a6374b950f6fdf60b58062fd5753e6cafd953a8fbac46cb6164191b66af1b3

Mathematical statement. The half-width of the bottom interface segment is $R(\sin\alpha - \sin\beta)$.

Its nonnegativity and its zero exactly at $h = 1/2$ encode the branch transition and the possible degenerate bottom segment.

```
def sideHalfWidth : ℝ :=
  a.radius * (sin a.outerAngle - sin a.innerAngle)

/-- Source circle center for the left strip arc. -/
```

TypeThreeAssembly.sideHalfWidth_pos

Classification: supplement

Source: TypeThreeAssembly.lean:341-343

Declaration SHA-256: a769284707d6bf25198d747cd5846bae22f2f394cc9d09e601cfcab854579d7a

Mathematical statement. If $h \neq 1/2$, then $w > 0$.

The strict sine comparison and positive radius give a nondegenerate horizontal expansion away from the semicircular transition.

```
theorem sideHalfWidth_pos (hh : a.h ≠ 1 / 2) : 0 < a.sideHalfWidth := by
  exact mul_pos a.radius_pos (sub_pos.mpr (a.sin_inner_lt_sin_outer hh))
```

TypeThreeAssembly.sin_sideArcAngle

Classification: supplement

Source: TypeThreeAssembly.lean:227-230

Declaration SHA-256: 41165bd4a88985079154632e1304282b8868bb98cc6b1d97bb3ea19faaebe0dd

Mathematical statement. $\sin \gamma = \sin \beta$.

This is the sine complementary-angle identity for $\gamma = \pi - \beta$.

```
theorem sin_sideArcAngle :
  sin a.sideArcAngle = sin a.innerAngle := by
  rw [sideArcAngle, Real.sin_pi_sub]
```

TypeThreeAssembly.upperCenter

Classification: supplement

Source: TypeThreeAssembly.lean:76-79

Declaration SHA-256: 6dabb1e2d2dfef1af758b047dfb6728a97191f4c64742b75a35f102c81c1c3c1

Mathematical statement. The upper exterior-circle center is $(0, 1 - R \cos \alpha)$.

This coordinate is determined by the upper cap base line, radius, and exterior half-central angle.

```
def upperCenter : PlanePoint :=
  (0, 1 - a.radius * cos a.outerAngle)

/-- Closed central rectangle of `K₃`. -/
```

area_pos

Classification: supplement

Source: MorgansArcFunction.lean:148-157

Declaration SHA-256: 0af41b50988d0bace92fc2e638571302802d8538245a2c254bedb9f164463ede

Mathematical statement. For $0 < \theta < \pi$, $a(\theta) > 0$.

The circular-segment numerator and squared sine denominator are positive on the full minor-to-major angular interval.

```
lemma area_pos {θ : ℝ} (hθ : 0 < θ) (hπ : θ < π) : 0 < area θ := by
  have hs : 0 < sin θ := sin_pos_of_pos_of_lt_pi hθ hπ
  have hnum : 0 < θ - sin θ * cos θ := by
    nlinarith [sin_lt hθ, neg_one_le_cos θ, cos_le_one θ, sq_nonneg (sin θ)]
  rw [area]
  positivity
```

```

/-- On  $\{0, 1\}$  the area is bounded above by  $\theta$ . The constants come from
 $\sin \theta > \theta - \theta^3/6$ : the numerator is  $< 2\theta^3/3$  and  $4 \sin^2 \theta > 25\theta^2/9$ , so
 $\text{area } \theta < 6\theta/25$ . -/

```

capBoundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:1249-1251

Declaration SHA-256: 5dbd12f9bc57860ad2ce92a638514e0e8d15006685aca7388e2fa2e1689504cd

Mathematical statement. The cap boundary measure is the pushforward of length on $[-R\theta, R\theta]$ by its arclength parameterization.

It measures the arc, not the chord; its relation to Hausdorff measure is supplied by the geometric length results.

```

def capBoundaryMeasure (c : OneSidedCircularCap) : Measure PlanePoint :=
  intervalPushforward (capParam c) (capStart c) (capEnd c)

```

capBoundaryMeasure_integrable

Classification: supplement

Source: CMVGeometry.lean:2260-2267

Declaration SHA-256: 0065afd47de261467dd963dc3f68bd9fc5e9eab2c424cfffb1d00f2e7c2919a7

Mathematical statement. For every cap and real λ , the strip density is integrable against its cap boundary measure.

This establishes the integrability needed to evaluate the cap's weighted arc contribution.

```

lemma capBoundaryMeasure_integrable (lam : ℝ) (c : OneSidedCircularCap) :
  Integrable (StripDensity lam) (capBoundaryMeasure c) := by
  rw [capBoundaryMeasure]
  exact stripDensity_integrable_intervalPushforward lam (by
    apply Continuous.measurable
    unfold capParam OneSidedCircularCap.arcPoint
    split <=> fun_prop)

```

capComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:390-397

Declaration SHA-256: 3ed78deaf1597a91c6218ca8fe10519ba693355e7329e0301f17fcce226f5a43

Mathematical statement. For a cap c , the Euclidean realization map sends $z \in \mathbb{C}$ to $(c_x + \text{Im}z, c_y - R\cos\theta + \text{Re}z)$ on the upper side and to $(c_x + \text{Im}z, c_y + R\cos\theta - \text{Re}z)$ on the lower side.

The two branches combine a translation, coordinate interchange, and optional reflection; the map sends the standard complex circle to the cap coordinates.

```

noncomputable def capComplexIsometryMap (c : OneSidedCircularCap) (z : ℂ) :
  EuclideanPlane :=
  match c.side with
  | .upper => WithLp.toLp 2
    (c.midpointX + z.im, c.baseY - c.radius * Real.cos c.theta + z.re)
  | .lower => WithLp.toLp 2
    (c.midpointX + z.im, c.baseY + c.radius * Real.cos c.theta - z.re)

```

capEnd

Classification: supplement

Source: CMVGeometry.lean:1246-1248

Declaration SHA-256: a140ca726638df619aef507f8019eb9f7ca9503d3231f8bfa8f0a64feb377161

Mathematical statement. The ending arclength parameter of a cap is $R\theta$.

Together with the starting parameter, this gives interval length $2R\theta$.

```
def capEnd (c : OneSidedCircularCap) : ℝ := c.radius * c.theta

/-- Explicit regular boundary measure of a one-sided circular arc. -/
```

capParam

Classification: supplement

Source: CMVGeometry.lean:1241-1243

Declaration SHA-256: d80d6cd5283176fe8b29770f039d1135a8892539144575a4bc1d9962ad427168

Mathematical statement. The cap's arclength parameterization is $s \mapsto c(s/R)$, where c is its angular arc map.

Scaling the angular parameter by the positive radius gives unit speed on the intended arc interval.

```
def capParam (c : OneSidedCircularCap) (s : ℝ) : PlanePoint :=
  c.arcPoint (s / c.radius)
```

capParam_image_Icc_eq_arcTrace

Classification: supplement

Source: FrontierPerimeter.lean:947-1070

Declaration SHA-256: f412e9294ab946fde4ef494447fa75b0b57b0600c2bd61007639df1bb35c0e9e

Mathematical statement. For every one-sided cap c , its cap parametrization maps $[-R\theta, R\theta]$ exactly onto its arc trace.

The closed angular interval covers the selected minor, semicircular, or major arc including both endpoints.

```
theorem capParam_image_Icc_eq_arcTrace (c : OneSidedCircularCap) :
  capParam c '' Icc (capStart c) (capEnd c) = c.arcTrace := by
  apply Set.Subset.antisymm
  · rintro p ⟨s, hs, rfl⟩
    have ht : s / c.radius ∈ Icc (-c.theta) c.theta := by
      have hr := c.radius_pos
      constructor
      · apply (le_div_iff₀ hr).2
        simpa [capStart, mul_comm] using hs.1
      · apply (div_le_iff₀ hr).2
        simpa [capEnd, mul_comm] using hs.2
    have habs : |s / c.radius| ≤ c.theta := (abs_le).2 ht
    constructor
    · cases hside : c.side <=>
      · simp only [capParam, OneSidedCircularCap.radiusSquaredAt,
        OneSidedCircularCap.center, OneSidedCircularCap.arcPoint, hside]
      all_goals
        nlinarith [Real.sin_sq_add_cos_sq (s / c.radius)]
    · exact c.arcPoint_in_closed_side habs
  · intro p hp
    cases hside : c.side with
    | upper =>
      let z : ℂ := ⟨p.2 - (c.baseY - c.radius * Real.cos c.theta),
        p.1 - c.midpointX⟩
      have hcircle :
        (p.1 - c.midpointX) ^ 2 +
```

```

      (p.2 - (c.baseY - c.radius * Real.cos c.theta)) ^ 2 =
      c.radius ^ 2 := by
    simp [OneSidedCircularCap.arcTrace,
      OneSidedCircularCap.radiusSquaredAt,
      OneSidedCircularCap.center, hside] using hp.1
  have hnorm_sq : ||z|| ^ 2 = c.radius ^ 2 := by
    rw [Complex.sq_norm, Complex.normSq_apply]
    simp only [z]
    nlinarith
  have hnorm : ||z|| = c.radius := by
    nlinarith [norm_nonneg z, c.radius_pos]
  have hsin :
    c.radius * Real.sin z.arg = p.1 - c.midpointX := by
    simp [hnorm, z] using Complex.norm_mul_sin_arg z
  have hcos :
    c.radius * Real.cos z.arg =
      p.2 - (c.baseY - c.radius * Real.cos c.theta) := by
    simp [hnorm, z] using Complex.norm_mul_cos_arg z
  have hsidep : c.baseY ≤ p.2 := by
    simp [OneSidedCircularCap.arcTrace, hside] using hp.2
  have hcos_le : Real.cos c.theta ≤ Real.cos z.arg := by
    nlinarith [c.radius_pos]
  have harg : |z.arg| ≤ c.theta := by
    by_contra h
    have hlt : c.theta < |z.arg| := lt_of_not_ge h
    have hc := Real.cos_lt_cos_of_nonneg_of_le_pi
      (le_of_lt c.theta_pos) (Complex.abs_arg_le_pi z) hlt
    rw [Real.cos_abs] at hc
    linarith
  let s := c.radius * z.arg
  have hs : s ∈ Icc (capStart c) (capEnd c) := by
    rcases (abs_le.mp harg) with ⟨hl, hu⟩
    dsimp [s]
    constructor <=> simp only [capStart, capEnd] <=>
      nlinarith [c.radius_pos]
  refine ⟨s, hs, ?_⟩
  have hangle : s / c.radius = z.arg := by
    dsimp [s]
    field_simp [ne_of_gt c.radius_pos]
  apply Prod.ext
  · simp only [capParam, OneSidedCircularCap.arcPoint, hside,
    hangle]
    linarith
  · simp only [capParam, OneSidedCircularCap.arcPoint, hside,
    hangle]
    linarith
| lower =>
  let z : ℂ := ((c.baseY + c.radius * Real.cos c.theta) - p.2,
    p.1 - c.midpointX)
  have hcircle :
    (p.1 - c.midpointX) ^ 2 +
    (p.2 - (c.baseY + c.radius * Real.cos c.theta)) ^ 2 =
    c.radius ^ 2 := by
    simp [OneSidedCircularCap.arcTrace,
      OneSidedCircularCap.radiusSquaredAt,
      OneSidedCircularCap.center, hside] using hp.1
  have hnorm_sq : ||z|| ^ 2 = c.radius ^ 2 := by
    rw [Complex.sq_norm, Complex.normSq_apply]
    simp only [z]
    nlinarith
  have hnorm : ||z|| = c.radius := by
    nlinarith [norm_nonneg z, c.radius_pos]
  have hsin :
    c.radius * Real.sin z.arg = p.1 - c.midpointX := by
    simp [hnorm, z] using Complex.norm_mul_sin_arg z
  have hcos :
    c.radius * Real.cos z.arg =
      (c.baseY + c.radius * Real.cos c.theta) - p.2 := by
    simp [hnorm, z] using Complex.norm_mul_cos_arg z
  have hsidep : p.2 ≤ c.baseY := by
    simp [OneSidedCircularCap.arcTrace, hside] using hp.2
  have hcos_le : Real.cos c.theta ≤ Real.cos z.arg := by

```

```

nlinarith [c.radius_pos]
have harg : |z.arg| ≤ c.theta := by
  by_contra h
  have hlt : c.theta < |z.arg| := lt_of_not_ge h
  have hc := Real.cos_lt_cos_of_nonneg_of_le_pi
    (le_of_lt c.theta_pos) (Complex.abs_arg_le_pi z) hlt
  rw [Real.cos_abs] at hc
  linarith
let s := c.radius * z.arg
have hs : s ∈ Icc (capStart c) (capEnd c) := by
  rcases (abs_le.mp harg) with ⟨hl, hu⟩
  dsimp [s]
  constructor <=> simp only [capStart, capEnd] <=>
    nlinarith [c.radius_pos]
refine ⟨s, hs, ?_⟩
have hangle : s / c.radius = z.arg := by
  dsimp [s]
  field_simp [ne_of_gt c.radius_pos]
apply Prod.ext
· simp only [capParam, OneSidedCircularCap.arcPoint, hside,
  hangle]
  linarith
· simp only [capParam, OneSidedCircularCap.arcPoint, hside,
  hangle]
  linarith

```

capPartitionPoint

Classification: supplement

Source: FrontierPerimeter.lean:154-156

Declaration SHA-256: 49ac6ecbe67b9040843eeee672d7b0f08055016781814c3ce0fa4e7d31ff805a

Mathematical statement. For real r, θ and natural numbers N, i , the signed-arclength partition point is

$$p_{N,i} = -r\theta + i \frac{2r\theta}{N}.$$

This is only the affine partition-point definition; positivity, monotonicity, interval membership, and endpoint identities are established by separate declarations.

```

def capPartitionPoint (r theta : ℝ) (N i : ℕ) : ℝ :=
  -r * theta + (i : ℝ) * (2 * r * theta / (N : ℝ))

```

capPartitionPoint_dist

Classification: supplement

Source: FrontierPerimeter.lean:185-207

Declaration SHA-256: 6b90645e53e16e3855618b966b04af3ee0f15439043926ba1133c583e65b5a00

Mathematical statement. If $r > 0$, $0 < \theta < \pi$, and $N > 0$, then for every natural index i ,

$$\text{dist}(\gamma_r(p_{N,i}), \gamma_r(p_{N,i+1})) = 2r \sin(\theta/N).$$

Adjacent partition points have angular separation θ/N ; the circle chord formula gives the displayed distance. The source theorem has no additional bound on i .

```

lemma capPartitionPoint_dist {r theta : ℝ} (hr : 0 < r)
  (htheta : theta ∈ Ioo 0 Real.pi) {N i : ℕ} (hN : 0 < N) :
  dist (unitCircleArc r (capPartitionPoint r theta N i))
    (unitCircleArc r (capPartitionPoint r theta N (i + 1))) =
    2 * r * Real.sin (theta / (N : ℝ)) := by
  rw [unitCircleArc_dist hr]
  have hNreal : 0 < (N : ℝ) := by positivity
  have harg :

```

```

    ((capPartitionPoint r theta N i - capPartitionPoint r theta N (i + 1)) / r) / 2 =
      -(theta / (N : ℝ)) := by
    unfold capPartitionPoint
    push_cast
    field_simp [hr.ne', hNreal.ne']
    ring
    have hangle_pos : 0 < theta / (N : ℝ) := div_pos htheta.1 hNreal
    have hN_one : (1 : ℝ) ≤ (N : ℝ) := by exact_mod_cast hN
    have hangle_lt : theta / (N : ℝ) < Real.pi := by
      apply (div_lt_iff₀ hNreal).2
      nlinarith [htheta.2, Real.pi_pos]
    rw [harg, Real.sin_neg, abs_mul, abs_of_pos (by norm_num : (0 : ℝ) < 2),
      abs_neg, abs_of_pos (Real.sin_pos_of_pos_of_lt_pi hangle_pos hangle_lt)]
    ring

```

capPartitionPoint_end

Classification: supplement

Source: FrontierPerimeter.lean:161-166

Declaration SHA-256: b5d0e986af4c921461bd13ce89490a41fdc02991eaadda7d26f70db465413519

Mathematical statement. If $N > 0$, then for all real r, θ , $p_{N,N} = r\theta$.

The final partition point equals the right endpoint; the positive natural-number hypothesis makes division by N legitimate.

```

lemma capPartitionPoint_end {r theta : ℝ} {N : ℕ} (hN : 0 < N) :
  capPartitionPoint r theta N N = r * theta := by
  unfold capPartitionPoint
  field_simp [Nat.ne_of_gt hN]
  ring

```

capPartitionPoint_mem

Classification: supplement

Source: FrontierPerimeter.lean:175-184

Declaration SHA-256: 8f26d09621c071ac8a6a1935a21253dfb29cb269e0ec21dc71e59a40f0529e9f

Mathematical statement. If $r > 0, \theta > 0, N > 0$, and $i \leq N$, then

$$p_{N,i} \in [-r\theta, r\theta].$$

Monotonicity together with the two endpoint identities places every indexed point in the closed signed-arclength interval.

```

lemma capPartitionPoint_mem {r theta : ℝ} (hr : 0 < r) (htheta : 0 < theta)
  {N i : ℕ} (hN : 0 < N) (hi : i ≤ N) :
  capPartitionPoint r theta N i ∈ Icc (-r * theta) (r * theta) := by
  have hm := monotone_capPartitionPoint hr htheta hN
  constructor
  · rw [- capPartitionPoint_zero r theta N]
    exact hm (Nat.zero_le i)
  · rw [- capPartitionPoint_end hN]
    exact hm hi

```

capPartitionPoint_zero

Classification: supplement

Source: FrontierPerimeter.lean:157-160

Declaration SHA-256: 1714782661300a21ee2d4243c7e4fbe79e1e97ce545ea187699c13d709db2db9

Mathematical statement. For all real r, θ and every natural number N , $p_{N,0} = -r\theta$.

The initial partition point is the left endpoint without any positivity assumption on the radius, angle, or number of subdivisions.

```
lemma capPartitionPoint_zero (r theta : ℝ) (N : ℕ) :
  capPartitionPoint r theta N 0 = -r * theta := by
  simp [capPartitionPoint]
```

capStart

Classification: supplement

Source: CMVGeometry.lean:1244-1245

Declaration SHA-256: 594eb6e2bbf3cbc7a2397b6af7a93506027168fa75ecf7f70b7012b2fc6e208a

Mathematical statement. The starting arclength parameter of a cap is $-R\theta$.

Dividing it by the radius recovers the angular starting value $-\theta$.

```
def capStart (c : OneSidedCircularCap) : ℝ := -c.radius * c.theta
```

cap_integrableOn

Classification: supplement

Source: CMVGeometry.lean:2303-2308

Declaration SHA-256: bbc4a1a9af0d45bee6aa1c1152d956e0e945a869f3550ca289ef7022556292ef

Mathematical statement. The strip density is integrable over every cap carrier, for every real λ .

The exact cap volume is finite, so the general bounded-density integrability lemma applies.

```
lemma cap_integrableOn (lam : ℝ) (c : OneSidedCircularCap) :
  IntegrableOn (StripDensity lam) c.carrier := by
  apply stripDensity_integrableOn_of_volume_ne_top
  rw [c.volume_carrier]
  exact ENNReal.ofReal_ne_top
```

cap_lower_external_density

Classification: supplement

Source: CMVGeometry.lean:1341-1352

Declaration SHA-256: baaf83c56acf8edc18268269454b4946af9c7098831da004f1da856c3f80d9b2

Mathematical statement. For a lower cap based on $y = -1$, every interior arclength parameter has density λ .

The interior points lie in $y < -1$, whereas the endpoint density is one.

```
theorem cap_lower_external_density (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .lower) (hbase : c.baseY = -1)
  {s : ℝ} (hs : s ∈ Ioo (capStart c) (capEnd c)) :
  StripDensity lam (capParam c s) = lam := by
  have hopen := c.arcPoint_in_open_side (cap_angle_mem_open hs)
  have hy : (capParam c s).2 < -1 := by
    simpa [capParam, hside, hbase] using hopen
  rw [StripDensity, if_neg]
  apply not_le.mpr
  exact (show 1 < -(capParam c s).2 by linarith).trans_le (neg_le_abs _)

/-- Upper external cap: weighted integral equals `lam` times arc length. -/
```

cap_lower_weightedArea

Classification: supplement

Source: CMVGeometry.lean:2360-2389

Declaration SHA-256: af4bcab3c0cdacd24a9ba22f365ac164c96fd2be59faf0a944ca162de8e4ada2

Mathematical statement. For a lower cap based on $y = -1$, the weighted area is λ times its Euclidean area, for every real λ .

The reflected cap has the same exterior-density property up to its planar-null chord.

```
lemma cap_lower_weightedArea (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .lower) (hbase : c.baseY = -1) :
  WeightedArea lam c.carrier = lam * c.euclideanArea := by
  have hne : ∀m p : PlanePoint ∂volume, p.2 ≠ -1 := by
    rw [ae_iff]
    simpa only [not_not] using volume_horizontalLine (-1)
  rw [WeightedArea]
  calc
    (∫ p in c.carrier, StripDensity lam p) =
      ∫ _p in c.carrier, lam := by
        apply integral_congr_ae
        filter_upwards [ae_restrict_mem c.measurableSet_carrier,
          ae_restrict_of_ae hne] with p hp hpne
        have hple : p.2 ≤ (-1 : ℝ) := by
          simpa [OneSidedCircularCap.carrier, hside, hbase] using hp.2
        have hplt : p.2 < (-1 : ℝ) := lt_of_le_of_ne hple hpne
        rw [StripDensity, if_neg]
        apply not_le.mpr
        exact (show 1 < -p.2 by linarith).trans_le (neg_le_abs p.2)
    _ = lam * c.euclideanArea := by
      rw [integral_const, measureReal_restrict_apply_univ]
      have hvolume : volume.real c.carrier = c.euclideanArea := by
        rw [Measure.real, c.volume_carrier,
          ENNReal.toReal_ofReal c.euclideanArea_pos.le]
      rw [hvolume]
      ring

/-- The coordinate four-arc assembly: two side arcs from the strip core and two
congruent reflected exterior caps. -/
```

cap_lower_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1373-1394

Declaration SHA-256: 5575ee03ce1e095004c86962659c2bcf1dde6c0b1ad90b439220173f4e11e7fc

Mathematical statement. For every real λ , a lower cap based on $y = -1$ has weighted arc integral $\lambda L\ell(\theta)$.

The same endpoint-null argument applies to the reflected lower arc.

```
theorem cap_lower_weightedIntegral (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .lower) (hbase : c.baseY = -1) :
  ∫ p, StripDensity lam p ∂capBoundaryMeasure c = lam * c.arcLength := by
  rw [capBoundaryMeasure]
  rw [integral_intervalPushforward (capParam_measurable c) (measurable_stripDensity lam)]
  rw [integral_Icc_eq_integral_Ioo]
  calc
    (∫ s in Ioo (capStart c) (capEnd c), StripDensity lam (capParam c s)) =
      ∫ _s in Ioo (capStart c) (capEnd c), lam := by
        apply integral_congr_ae
        filter_upwards [ae_restrict_mem measurableSet_Ioo] with s hs
        exact cap_lower_external_density lam c hside hbase hs
    _ = lam * (capEnd c - capStart c) := by
      have hlen : 0 ≤ capEnd c - capStart c := by
```

```

    dsimp [capEnd, capStart]
    nlinarith [c.radius_pos, c.theta_pos]
    simp [max_eq_left hlen, mul_comm]
    _ = lam * c.arclength := by rw [cap_arclength_eq c]

/-! ## Horizontal segments -/

/-- Unit-speed left-to-right segment parametrization. -/

```

cap_partition_polygon_lower

Classification: supplement

Source: FrontierPerimeter.lean:208-268

Declaration SHA-256: 4dc7fcd936b6c56a4643f00c8d67c38ce24d6351f625a8873960fc070d61a446

Mathematical statement. If $r > 0$, $0 < \theta < \pi$, and $N > 0$, then

$$\text{ofReal}(N 2r \sin(\theta/N)) \leq \mathcal{H}^1(\gamma_r([-r\theta, r\theta])).$$

The polygonal chain through the N partition points has total chord length on the left and lies in the circular-arc image, yielding a lower bound for its Hausdorff measure.

```

lemma cap_partition_polygon_lower {r theta : ℝ} (hr : 0 < r)
  (htheta : theta ∈ Ioo 0 Real.pi) (N : ℕ) (hN : 0 < N) :
  ENNReal.ofReal ((N : ℝ) * (2 * r * Real.sin (theta / (N : ℝ)))) ≤
    (μH[1] : Measure ℂ)
    (unitCircleArc r '' Icc (-r * theta) (r * theta)) := by
  let p : ℕ → ℝ := capPartitionPoint r theta N
  let A : ℕ → Set ℝ := fun i => Ico (p i) (p (i + 1))
  let S : Set ℝ := Icc (-r * theta) (r * theta)
  let f : ℝ → ℂ := unitCircleArc r
  let μ : Measure ℂ := μH[1]
  have hpmmono : Monotone p := monotone_capPartitionPoint hr htheta.1 hN
  have hinj : Set.InjOn f S := unitCircleArc_injOn hr htheta
  have hpiece_sub (i : ℕ) (hi : i < N) : Icc (p i) (p (i + 1)) ⊆ S := by
    intro x hx
    exact ⟨(capPartitionPoint_mem hr htheta.1 hN hi.le).1.trans hx.1,
      hx.2.trans (capPartitionPoint_mem hr htheta.1 hN (Nat.succ_le_iff.mpr hi)).2⟩
  have hpair : Set.Pairwise (↑(Finset.range N))
    (Function.onFun Disjoint fun i => f '' A i) := by
    intro i hi j hj hij
    change Disjoint (f '' A i) (f '' A j)
    have hsrc : Disjoint (A i) (A j) := hpmmono.pairwise_disjoint_on_Ico_succ hij
    rw [Set.disjoint_left]
    rintro y ⟨x, hxi, rfl⟩ ⟨z, hzj, hzx⟩
    have hiN : i < N := Finset.mem_range.mp hi
    have hjN : j < N := Finset.mem_range.mp hj
    have hxS : x ∈ S := hpiece_sub i hiN ⟨hxi.1, hxi.2.le⟩
    have hzS : z ∈ S := hpiece_sub j hjN ⟨hzj.1, hzj.2.le⟩
    have hxz : x = z := hinj hxS hzS hzx.symm
    exact (Set.disjoint_left.mp hsrc) hxi (hxz ▸ hzj)
  have hmeas (i : ℕ) (hi : i ∈ Finset.range N) : MeasurableSet (f '' A i) := by
    have hiN : i < N := Finset.mem_range.mp hi
    rw [show f '' A i = f '' Icc (p i) (p (i + 1)) \ {f (p (i + 1))} by
      exact image_Ico_eq_image_Icc_diff_endpoint hinj (hpmmono (Nat.le_succ i))
      (hpiece_sub i hiN)]
    exact (isCompact_Icc.image (continuous_unitCircleArc r)).measurableSet.diff
      (measurableSet_singleton _)
  have hunion_sub : (⋃ i ∈ Finset.range N, f '' A i) ⊆ f '' S := by
    intro y hy
    rcases mem_iUnion.mp hy with ⟨i, hy⟩
    rcases mem_iUnion.mp hy with ⟨hi, hy⟩
    rcases hy with ⟨x, hxA, rfl⟩
    exact ⟨x, hpiece_sub i (Finset.mem_range.mp hi) ⟨hxA.1, hxA.2.le⟩, rfl⟩
  calc
    ENNReal.ofReal ((N : ℝ) * (2 * r * Real.sin (theta / (N : ℝ)))) =
      ∑ i ∈ Finset.range N,
        ENNReal.ofReal (2 * r * Real.sin (theta / (N : ℝ))) := by

```

```

      rw [ENNReal.ofReal_mul (by positivity), ENNReal.ofReal_natCast]
      simp [Finset.sum_const, nsmul_eq_mul]
_ ≤ ∑ i ∈ Finset.range N, μ (f '' A i) := by
  apply Finset.sum_le_sum
  intro i hi
  have hiN : i < N := Finset.mem_range.mp hi
  rw [hausdorffMeasure_image_Ico_eq_Icc hinv (hpmono (Nat.le_succ i))
      (hpiece_sub i hiN)]
  simp [f, p, μ, capPartitionPoint_dist hr htheta hN] using
    ofReal_dist_le_hausdorffMeasure_image_Icc (continuous_unitCircleArc r)
    (hpmono (Nat.le_succ i))
_ = μ (U i ∈ Finset.range N, f '' A i) :=
  (measure_biUnion_finset hpair hmeas).symm
_ ≤ μ (f '' S) := measure_mono hunion_sub

```

cap_upper_external_density

Classification: supplement

Source: CMVGeometry.lean:1329-1340

Declaration SHA-256: 0d4d779bde08a636e8536f594b78dfa9705a31925f9d2e9248d0d62fdf5aa11a

Mathematical statement. For an upper cap based on $y = 1$, every interior arclength parameter has density λ .

The strict side inequality places these arc points in $y > 1$; only the two endpoints lie on the density-one interface.

```

theorem cap_upper_external_density (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .upper) (hbase : c.baseY = 1)
  {s : ℝ} (hs : s ∈ Ioo (capStart c) (capEnd c)) :
  StripDensity lam (capParam c s) = lam := by
  have hopen := c.arcPoint_in_open_side (cap_angle_mem_open hs)
  have hy : 1 < (capParam c s).2 := by
    simp [capParam, hside, hbase] using hopen
  rw [StripDensity, if_neg]
  exact not_le.mpr (hy.trans_le (le_abs_self _))

/-- The non-endpoint part of a lower cap based on `y = -1` is in the external
(density `lam`) branch. -/

```

cap_upper_weightedArea

Classification: supplement

Source: CMVGeometry.lean:2334-2359

Declaration SHA-256: 2f2165b544886739ec40b854c29666e67f06c7a423bd046c04eb224a5c6ee0ce

Mathematical statement. For an upper cap based on $y = 1$, the weighted area is λ times its Euclidean area, for every real λ .

Except for the planar-null base line, the whole cap lies in the exterior-density half-plane.

```

lemma cap_upper_weightedArea (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .upper) (hbase : c.baseY = 1) :
  WeightedArea lam c.carrier = lam * c.euclideanArea := by
  have hne : ∀ m p : PlanePoint ∂volume, p.2 ≠ 1 := by
    rw [ae_iff]
    simp only [not_not] using volume_horizontalline 1
  rw [WeightedArea]
  calc
    (f p in c.carrier, StripDensity lam p) =
      f _p in c.carrier, lam := by
        apply integral_congr_ae
        filter_upwards [ae_restrict_mem c.measurableSet_carrier,
          ae_restrict_of_ae hne] with p hp hpne
        have hpge : (1 : ℝ) ≤ p.2 := by
          simp [OneSidedCircularCap.carrier, hside, hbase] using hp.2
        have hpgt : (1 : ℝ) < p.2 := lt_of_le_of_ne hpge (Ne.symm hpne)

```

```

    rw [StripDensity, if_neg]
    exact not_le.mpr (hpgt.trans_le (le_abs_self p.2))
  _ = lam * c.euclideanArea := by
    rw [integral_const, measureReal_restrict_apply_univ]
    have hvolume : volume.real c.carrier = c.euclideanArea := by
      rw [Measure.real, c.volume_carrier,
        ENNReal.toReal_ofReal c.euclideanArea_pos.le]
    rw [hvolume]
  ring

```

cap_upper_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1353-1372

Declaration SHA-256: 45644ff66dcb2be9e3cb6c626527848bf909f532df9be34ffa1977e24d25b8f4

Mathematical statement. For every real λ , an upper cap based on $y = 1$ has weighted arc integral $\lambda L\ell(\theta)$.

The density is λ away from the two parameter endpoints; those endpoints have zero length measure.

```

theorem cap_upper_weightedIntegral (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .upper) (hbase : c.baseY = 1) :
  ∫ p, StripDensity lam p ∂capBoundaryMeasure c = lam * c.arcLength := by
  rw [capBoundaryMeasure]
  rw [integral_intervalPushforward (capParam_measurable c) (measurable_stripDensity lam)]
  rw [integral_Icc_eq_integral_Ioo]
  calc
    (∫ s in Ioo (capStart c) (capEnd c), StripDensity lam (capParam c s)) =
      ∫ _s in Ioo (capStart c) (capEnd c), lam := by
        apply integral_congr_ae
        filter_upwards [ae_restrict_mem measurableSet_Ioo] with s hs
        exact cap_upper_external_density lam c hside hbase hs
  _ = lam * (capEnd c - capStart c) := by
    have hlen : 0 ≤ capEnd c - capStart c := by
      dsimp [capEnd, capStart]
      nlinarith [c.radius_pos, c.theta_pos]
    simp [max_eq_left hlen, mul_comm]
  _ = lam * c.arcLength := by rw [cap_arclength_eq c]

/-- Lower external cap: weighted integral equals `lam` times arc length. -/

```

continuous_realizedCapParam

Classification: supplement

Source: FrontierPerimeter.lean:442-450

Declaration SHA-256: 439602ca8a442d23dec8f858626a27ca71e9723ce90b04b56ee6e94cd89590f1

Mathematical statement. The indicated parametrization or cleared expression is continuous on its stated domain.

Continuity follows from continuity of the elementary coordinate, trigonometric, exponential, or square-root operations and is used for Hausdorff-measure transfer or an intermediate-value argument.

```

lemma continuous_realizedCapParam (c : OneSidedCircularCap) :
  Continuous (realizedCapParam c) := by
  rw [show realizedCapParam c = capComplexIsometryMap c ∘
    unitCircleArc c.radius by
    funext s
    exact realizedCapParam_eq c s]
  exact (isometry_capComplexIsometryMap c).continuous.comp
    (continuous_unitCircleArc c.radius)

```

continuous_realizedLeftCoreParam

Classification: supplement

Source: FrontierPerimeter.lean:727-735

Declaration SHA-256: 4dc1fbf3a43e8117f791c26e28bd5a2a2466716abc2324d51a2522841ccc46bf

Mathematical statement. The indicated parametrization or cleared expression is continuous on its stated domain.

Continuity follows from continuity of the elementary coordinate, trigonometric, exponential, or square-root operations and is used for Hausdorff-measure transfer or an intermediate-value argument.

```
lemma continuous_realizedLeftCoreParam (c : StripCore) :  
  Continuous (realizedLeftCoreParam c) := by  
  rw [show realizedLeftCoreParam c = leftCoreComplexIsometryMap c ◦  
    unitCircleArc c.radius by  
    funext s  
    exact realizedLeftCoreParam_eq c s]  
  exact (isometry_leftCoreComplexIsometryMap c).continuous.comp  
    (continuous_unitCircleArc c.radius)
```

continuous_realizedRightCoreParam

Classification: supplement

Source: FrontierPerimeter.lean:736-744

Declaration SHA-256: db06971d2e0a6e54b3ce1b2cb1ffe296cb5ed7ac07495118f10c8f5f82bfe132

Mathematical statement. The indicated parametrization or cleared expression is continuous on its stated domain.

Continuity follows from continuity of the elementary coordinate, trigonometric, exponential, or square-root operations and is used for Hausdorff-measure transfer or an intermediate-value argument.

```
lemma continuous_realizedRightCoreParam (c : StripCore) :  
  Continuous (realizedRightCoreParam c) := by  
  rw [show realizedRightCoreParam c = rightCoreComplexIsometryMap c ◦  
    unitCircleArc c.radius by  
    funext s  
    exact realizedRightCoreParam_eq c s]  
  exact (isometry_rightCoreComplexIsometryMap c).continuous.comp  
    (continuous_unitCircleArc c.radius)
```

continuous_unitCircleArc

Classification: supplement

Source: FrontierPerimeter.lean:14-17

Declaration SHA-256: 343bc4472a64687ca829ebbf897c060cb6c7c3cad3716259e96fb84cf091470c

Mathematical statement. The indicated parametrization or cleared expression is continuous on its stated domain.

Continuity follows from continuity of the elementary coordinate, trigonometric, exponential, or square-root operations and is used for Hausdorff-measure transfer or an intermediate-value argument.

```
lemma continuous_unitCircleArc (r : ℝ) : Continuous (unitCircleArc r) := by  
  unfold unitCircleArc  
  fun_prop
```

coreArcEnd

Classification: supplement

Source: CMVGeometry.lean:1474-1476

Declaration SHA-256: 7ee06e3535f2fd49529132649f006dc8125adff906e2bb1c8e325a0ca4b835a4

Mathematical statement. A core side arc ends at parameter $R\phi$.

The parameter span is $2R\phi$ and the final vertical coordinate is one.

```
def coreArcEnd (c : StripCore) : ℝ := c.radius * c.sideAngle  
  
/-- Unit-speed parametrization of the left core arc. -/
```

coreArcStart

Classification: supplement

Source: CMVGeometry.lean:1472-1473

Declaration SHA-256: 4c395d23f09d7608737696a9d4043f61933ddaffbb62953e4470c1085d950b4e

Mathematical statement. A core side arc starts at parameter $-R\phi$.

The corresponding circular angle is $-\phi$, with vertical coordinate -1 .

```
def coreArcStart (c : StripCore) : ℝ := -c.radius * c.sideAngle
```

core_integrableOn

Classification: supplement

Source: CMVGeometry.lean:2309-2314

Declaration SHA-256: ebecbc727d05931a9afed5e77aa787703ada65169aa70e3f372bec6e04bddcac

Mathematical statement. The strip density is integrable over every strip-core carrier, for every real λ .

The core's computed finite volume supplies the requisite measure bound.

```
lemma core_integrableOn (lam : ℝ) (c : StripCore) :  
  IntegrableOn (StripDensity lam) c.carrier := by  
  apply stripDensity_integrableOn_of_volume_ne_top  
  rw [c.volume_carrier]  
  exact ENNReal.ofReal_ne_top
```

euclideanJoinSet

Classification: supplement

Source: FrontierPerimeter.lean:1257-1259

Declaration SHA-256: 96d51d86f6c32a7f1c31ecb0290be17bec48e5d48a1aa38fdfbac4457025d389

Mathematical statement. For real q , the Euclidean join set is $J(\text{joinSet}(q))$.

This is the image of the coordinate join set under the Euclidean realization map.

```
def euclideanJoinSet (q : ℝ) : Set EuclideanPlane :=  
  planeEuclideanHomeomorph '' joinSet q
```

euclideanJoinSet_finite

Classification: supplement

Source: FrontierPerimeter.lean:1260-1262

Declaration SHA-256: 9539ffd4336468eab518f7e951c9dc4db7ee2d6bd08f0fa98bbda9dcf0817f38

Mathematical statement. For every real q , the Euclidean join set is finite.

An image of a finite set under the homeomorphism remains finite.

```
theorem euclideanJoinSet_finite (q : ℝ) : (euclideanJoinSet q).Finite :=
  (joinSet_finite q).image planeEuclideanHomeomorph
```

euclideanPairwiseOverlap

Classification: supplement

Source: FrontierPerimeter.lean:1591-1597

Declaration SHA-256: c1559aa1737fb404d99a3376b70fbd4f0ff514e71a947a7d30b01a1ce4c1177e

Mathematical statement. For arbitrary sets A, B, C, D in any type, the Euclidean pairwise-overlap expression is the union of all six pairwise intersections.

This polymorphic definition carries no finiteness, boundary, or nullity assumption.

```
def euclideanPairwiseOverlap {α : Type*}
  (A B C D : Set α) : Set α :=
  (A ∩ B) ∪ ((A ∩ C) ∪ ((A ∩ D) ∪
    ((B ∩ C) ∪ ((B ∩ D) ∪ (C ∩ D)))))

/-- A null union of all pairwise overlaps makes the four sets pairwise
almost disjoint. -/
```

euclideanStripDensity

Classification: supplement

Source: FrontierPerimeter.lean:1389-1391

Declaration SHA-256: 3d074997fd7c2f4c47fc25c7e8a9a39259d6fd4628b103b9b3e4da77e1679848

Mathematical statement. For real λ and Euclidean point z , the transported density is $f_\lambda(J^{-1}z)$.

This definition expresses the coordinate strip density in the Euclidean metric space used by the frontier measure.

```
def euclideanStripDensity (lam : ℝ) (z : EuclideanPlane) : ℝ :=
  StripDensity lam (planeEuclideanHomeomorph.symm z)
```

euclideanStripDensity_integrableOn_of_measure_ne_top

Classification: supplement

Source: FrontierPerimeter.lean:1404-1411

Declaration SHA-256: 509b7e5c9b45ab61b5e958fce825f92bda813640616ddf5c7591ac420818c5d2

Mathematical statement. If a Euclidean set S has finite $\mathcal{H}^1$ -measure, then the transported strip density is integrable on S with respect to $\mathcal{H}^1$, for every real λ .

The transported density is bounded by $1 + |\lambda|$, so finite Hausdorff measure implies integrability.

```
lemma euclideanStripDensity_integrableOn_of_measure_ne_top
  (lam : ℝ) {s : Set EuclideanPlane}
  (hs : (μH[1] : Measure EuclideanPlane) s ≠ ∞) :
```

```

IntegrableOn (euclideanStripDensity lam) s (μH[1] : Measure EuclideanPlane) := by
  apply Measure.integrableOn_of_bounded (M := 1 + |lam|) hs
· exact (measurable_euclideanStripDensity lam).aestronglyMeasurable
· exact Filter.Eventually.of_forall (euclideanStripDensity_bound lam)

```

exact_euclidean_leftCoreParam_hausdorffMeasure

Classification: supplement

Source: FrontierPerimeter.lean:745-772

Declaration SHA-256: e0211c9e8b0941bc0bbeb3bd977afe28d78688025726aac98a4d738bf7f180e9

Mathematical statement. For every strip core c , write $R = c.\text{radius}$, $\phi = c.\text{sideAngle}$, and $s_- = -R\phi$, $s_+ = R\phi$. The Euclidean left arclength image has

$$\mathcal{H}^1(J(\text{leftParam}_c([s_-, s_+])) = \text{ofReal}(s_+ - s_-).$$

The complex arc has exact Hausdorff mass equal to its parameter span, and the left complex map is an isometry. The interval endpoints are the core arc endpoints.

```

theorem exact_euclidean_leftCoreParam_hausdorffMeasure (c : StripCore) :
  (μH[1] : Measure EuclideanPlane)
  (realizedLeftCoreParam c '' Icc (coreArcStart c) (coreArcEnd c)) =
  ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  have hfun : realizedLeftCoreParam c =
    leftCoreComplexIsometryMap c ∘ unitCircleArc c.radius := by
    funext s
  exact realizedLeftCoreParam_eq c s
calc
  _ = (μH[1] : Measure EuclideanPlane)
    (leftCoreComplexIsometryMap c ''
      (unitCircleArc c.radius '' Icc (coreArcStart c) (coreArcEnd c))) := by
    rw [hfun]
    congr 1
  simp only [Function.comp_apply] using (Set.image_image _ _ _).symm
  _ = (μH[1] : Measure ℂ)
    (unitCircleArc c.radius '' Icc (coreArcStart c) (coreArcEnd c)) :=
    (isometry_leftCoreComplexIsometryMap c).hausdorffMeasure_image
    (Or.inl (by norm_num)) _
  _ = ENNReal.ofReal (2 * c.radius * c.sideAngle) := by
    simp [coreArcStart, coreArcEnd] using
      hausdorffMeasure_unitCircleArc_image_eq c.radius_pos
      ⟨c.sideAngle_pos, c.sideAngle_lt_pi⟩
  _ = ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
    congr 1
    simp only [coreArcStart, coreArcEnd]
    ring

```

exact_euclidean_rightCoreParam_hausdorffMeasure

Classification: supplement

Source: FrontierPerimeter.lean:773-800

Declaration SHA-256: 5b9d17b9e5921b7fba027be4ebfa3845e68c3c43a92a67ce9d4854bf1b244db0

Mathematical statement. For every strip core c , write $R = c.\text{radius}$, $\phi = c.\text{sideAngle}$, and $s_- = -R\phi$, $s_+ = R\phi$. The Euclidean right arclength image has

$$\mathcal{H}^1(J(\text{rightParam}_c([s_-, s_+])) = \text{ofReal}(s_+ - s_-).$$

The right isometry preserves the exact Hausdorff mass of the same signed-arclength interval.

```

theorem exact_euclidean_rightCoreParam_hausdorffMeasure (c : StripCore) :
  (μH[1] : Measure EuclideanPlane)
  (realizedRightCoreParam c '' Icc (coreArcStart c) (coreArcEnd c)) =

```

```

    ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
have hfun : realizedRightCoreParam c =
  rightCoreComplexIsometryMap c ∘ unitCircleArc c.radius := by
  funext s
  exact realizedRightCoreParam_eq c s
calc
  _ = (μH[1] : Measure EuclideanPlane)
    (rightCoreComplexIsometryMap c ''
      (unitCircleArc c.radius '' Icc (coreArcStart c) (coreArcEnd c))) := by
    rw [hfun]
    congr 1
    simp only [Function.comp_apply] using (Set.image_image _ _ _).symm
  _ = (μH[1] : Measure ℂ)
    (unitCircleArc c.radius '' Icc (coreArcStart c) (coreArcEnd c)) :=
    (isometry_rightCoreComplexIsometryMap c).hausdorffMeasure_image
    (Or.inl (by norm_num)) _
  _ = ENNReal.ofReal (2 * c.radius * c.sideAngle) := by
    simp [coreArcStart, coreArcEnd] using
      hausdorffMeasure_unitCircleArc_image_eq c.radius_pos
      ⟨c.sideAngle_pos, c.sideAngle_lt_pi⟩
  _ = ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
    congr 1
    simp only [coreArcStart, coreArcEnd]
    ring

```

fourArc_pairwiseOverlap_h1_null

Classification: supplement

Source: FrontierPerimeter.lean:1269-1279

Declaration SHA-256: 01eb58d468da4075ed5a4b3c4808bf8deddca98be0f05ef720c74c5a64fd9f4f

Mathematical statement. The pairwise overlaps of the displayed boundary pieces are contained in a finite set and therefore have zero Euclidean one-dimensional Hausdorff measure.

This prevents double counting at junctions while retaining all endpoints in the set-theoretic frontier.

```

theorem fourArc_pairwiseOverlap_h1_null (a : FourArcAssembly) :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' pairwiseOverlap
      (StripCore.leftArcTrace a.core)
      (StripCore.rightArcTrace a.core)
      (OneSidedCircularCap.arcTrace a.upperCap)
      (OneSidedCircularCap.arcTrace a.lowerCap)) = 0 := by
  exact measure_mono_null
    (image_mono (fourArc_pairwiseOverlap_subset_joinSet a))
    (hausdorffMeasure_one_euclideanJoinSet a.core.chord)

```

fourArc_pairwiseOverlap_subset_joinSet

Classification: supplement

Source: FrontierPerimeter.lean:1142-1195

Declaration SHA-256: eea91f9cc72f6aabac972107df3b4a4acd7547fe5fa392689f59302f202356a4

Mathematical statement. The pairwise overlaps of the displayed boundary pieces are contained in a finite set and therefore have zero Euclidean one-dimensional Hausdorff measure.

This prevents double counting at junctions while retaining all endpoints in the set-theoretic frontier.

```

theorem fourArc_pairwiseOverlap_subset_joinSet (a : FourArcAssembly) :
  pairwiseOverlap
    (StripCore.leftArcTrace a.core)
    (StripCore.rightArcTrace a.core)
    (OneSidedCircularCap.arcTrace a.upperCap)
    (OneSidedCircularCap.arcTrace a.lowerCap) ⊆

```

```

      joinSet a.core.chord := by
intro p hp
simp only [pairwiseOverlap, mem_union, mem_inter_iff] at hp
rcases hp with hLR | hLU | hLL | hRU | hRL | hUL
· exact False.elim (by
  have h1 := hLR.1.2.1
  have hr := hLR.2.2.1
  linarith [a.core.chord_pos])
· have hyCore : p.2 ≤ 1 := (le_abs_self p.2).trans hLU.1.2.2
  have hyCap : (1 : ℝ) ≤ p.2 := by
    simp [FourArcAssembly.upperCap] using hLU.2.2
  have hy : p.2 = 1 := le_antisymm hyCore hyCap
  have hxBound := four_upper_arc_x_bound a hLU.2 hy
  have hx := eq_left_of_bounds hLU.1.2.1 hxBound
  have hpEq : p = (-a.core.chord / 2, 1) := Prod.ext hx hy
  simp [joinSet, hpEq]
· have hyCore : -1 ≤ p.2 := by
  linarith [(neg_le_abs p.2).trans hLL.1.2.2]
  have hyCap : p.2 ≤ (-1 : ℝ) := by
    simp [FourArcAssembly.lowerCap] using hLL.2.2
  have hy : p.2 = -1 := le_antisymm hyCap hyCore
  have hxBound := four_lower_arc_x_bound a hLL.2 hy
  have hx := eq_left_of_bounds hLL.1.2.1 hxBound
  have hpEq : p = (-a.core.chord / 2, -1) := Prod.ext hx hy
  simp [joinSet, hpEq]
· have hyCore : p.2 ≤ 1 := (le_abs_self p.2).trans hRU.1.2.2
  have hyCap : (1 : ℝ) ≤ p.2 := by
    simp [FourArcAssembly.upperCap] using hRU.2.2
  have hy : p.2 = 1 := le_antisymm hyCore hyCap
  have hxBound := four_upper_arc_x_bound a hRU.2 hy
  have hx := eq_right_of_bounds hRU.1.2.1 hxBound
  have hpEq : p = (a.core.chord / 2, 1) := Prod.ext hx hy
  simp [joinSet, hpEq]
· have hyCore : -1 ≤ p.2 := by
  linarith [(neg_le_abs p.2).trans hRL.1.2.2]
  have hyCap : p.2 ≤ (-1 : ℝ) := by
    simp [FourArcAssembly.lowerCap] using hRL.2.2
  have hy : p.2 = -1 := le_antisymm hyCap hyCore
  have hxBound := four_lower_arc_x_bound a hRL.2 hy
  have hx := eq_right_of_bounds hRL.1.2.1 hxBound
  have hpEq : p = (a.core.chord / 2, -1) := Prod.ext hx hy
  simp [joinSet, hpEq]
· have hyUpper : (1 : ℝ) ≤ p.2 := by
  simp [FourArcAssembly.upperCap] using hUL.1.2
  have hyLower : p.2 ≤ (-1 : ℝ) := by
    simp [FourArcAssembly.lowerCap] using hUL.2.2
  linarith

```

hausdorffMeasure_image_Ico_eq_Icc

Classification: supplement

Source: FrontierPerimeter.lean:142-153

Declaration SHA-256: 1c850c9b7ca6e169808ebfb87cac4902c44102258f4c3ba0c9b6433987f2a5de

Mathematical statement. If f is injective on S , $u \leq v$, and $[u, v] \subseteq S$, then

$$\mathcal{H}^1(f([u, v])) = \mathcal{H}^1(f([u, v])).$$

The two images differ by the singleton $\{f(v)\}$, which has zero one-dimensional Hausdorff measure. The theorem equates measures, not the underlying half-open and closed sets.

```

lemma hausdorffMeasure_image_Ico_eq_Icc
  {E : Type*} [EMetricSpace E] [MeasurableSpace E] [BorelSpace E]
  {f : ℝ → E} {S : Set ℝ} (hf : Set.InjOn f S)
  {u v : ℝ} (huv : u ≤ v) (hsub : Icc u v ⊆ S) :
  (μH[1] : Measure E) (f '' Ico u v) =
  (μH[1] : Measure E) (f '' Icc u v) := by
  rw [image_Ico_eq_image_Icc_diff_endpoint hf huv hsub]

```

```

letI : NullSingletonClass (μH[1] : Measure E) :=
  Measure.nullSingletonClass_hausdorff E (by norm_num)
exact measure_sdiff_null (measure_singleton (f v))

/-- Equally spaced partition points on the signed-arclength interval. -/

```

hausdorffMeasure_one_euclideanJoinSet

Classification: supplement

Source: FrontierPerimeter.lean:1263-1268

Declaration SHA-256: 7ebe8fae03ad531268ff769afa248cd1067ee674eba1a7f8bc77f36d3ab49ebc

Mathematical statement. For every real q , the Euclidean join set has zero one-dimensional Hausdorff measure:

$$\mathcal{H}^1(J(\text{joinSet}(q))) = 0.$$

Finite sets are null for one-dimensional Hausdorff measure.

```

theorem hausdorffMeasure_one_euclideanJoinSet (q : ℝ) :
  (μH[1] : Measure EuclideanPlane) (euclideanJoinSet q) = 0 := by
  let _ := Measure.nullSingletonClass_hausdorff EuclideanPlane
  (by norm_num : (0 : ℝ) < 1)
  exact (euclideanJoinSet_finite q).measure_zero μH[1]

```

hausdorffMeasure_unitCircleArc_image_le

Classification: supplement

Source: FrontierPerimeter.lean:93-104

Declaration SHA-256: 07ee598762aa4a7964c4db549c22833ebf32ee2d325dcc190dd9a72212015319

Mathematical statement. If $r > 0$, then for all real a, b ,

$$\mathcal{H}^1(\gamma_r([a, b])) \leq \max\{b - a, 0\}.$$

This is an upper bound for the Hausdorff measure of the parametrized image; it is not an equality with the interval or with a carrier.

```

lemma hausdorffMeasure_unitCircleArc_image_le {r a b : ℝ} (hr : 0 < r) :
  (μH[1] : Measure ℂ) (unitCircleArc r '' Icc a b) ≤
    ENNReal.ofReal (b - a) := by
  calc
    (μH[1] : Measure ℂ) (unitCircleArc r '' Icc a b)
      ≤ (μH[1] : Measure ℝ) (Icc a b) := by
        simpa using (lipschitzWith_unitCircleArc hr).hausdorffMeasure_image_le
          (d := (1 : ℝ)) (by norm_num) (Icc a b)
    _ = ENNReal.ofReal (b - a) := by
      rw [hausdorffMeasure_real, Real.volume_Icc]

/-- Every continuous curve image has  $H^1$  mass at least the distance between its endpoints. -/

```

image_Ico_eq_image_Icc_diff_endpoint

Classification: supplement

Source: FrontierPerimeter.lean:124-141

Declaration SHA-256: c4fb0e64c0405d8f9fa3cb2429d9a9bc01ac95be8aba1e08560d1aeae3f939e9

Mathematical statement. If f is injective on S , $u \leq v$, and $[u, v] \subseteq S$, then

$$f([u, v]) = f([u, v]) \setminus \{f(v)\}.$$

Injectivity removes precisely the final endpoint from the image of the half-open interval; the initial endpoint remains because it belongs to both intervals.

```
lemma image_Ico_eq_image_Icc_diff_endpoint
  {α : Type*} {f : ℝ → α} {S : Set ℝ} (hf : Set.InjOn f S)
  {u v : ℝ} (huv : u ≤ v) (hsub : Icc u v ⊆ S) :
  f '' Ico u v = f '' Icc u v \ {f v} := by
  ext y
  constructor
  · rintro ⟨x, hx, rfl⟩
    refine ⟨⟨x, ⟨hx.1, hx.2.le⟩, rfl⟩, ?_⟩
    simp only [mem_singleton_iff]
    intro hfv
    have hxv := hf (hsub ⟨hx.1, hx.2.le⟩) (hsub ⟨huv, le_rfl⟩) hfv
    exact hx.2.ne hxv
  · rintro ⟨⟨x, hx, rfl⟩, hne⟩
    refine ⟨x, ⟨hx.1, lt_of_le_of_ne hx.2 ?_⟩, rfl⟩
    intro hxv
    subst x
    exact hne (mem_singleton (f v))
```

integral_four_union

Classification: supplement

Source: FrontierPerimeter.lean:1562-1590

Declaration SHA-256: 8b3fbc02eae74d84636263a1b0bedcffb91c58bcc3410ac92807b51d54cc9ea4

Mathematical statement. Let A, B, C, D be Euclidean sets and f a function. If all six pairs are almost disjoint for $\mathcal{H}^1$, B, C, D are measurable, and f is integrable on each of A, B, C, D , then

$$\int_{A \cup (B \cup (C \cup D))} f d\mathcal{H}^1 = \int_A f d\mathcal{H}^1 + \int_B f d\mathcal{H}^1 + \int_C f d\mathcal{H}^1 + \int_D f d\mathcal{H}^1.$$

The six pairwise almost-disjoint hypotheses permit successive union-splitting, while the stated measurability and integrability hypotheses justify each set integral.

```
theorem integral_four_union
  {f : EuclideanPlane → ℝ} {A B C D : Set EuclideanPlane}
  (hAB : AEDisjoint (μH[1] : Measure EuclideanPlane) A B)
  (hAC : AEDisjoint (μH[1] : Measure EuclideanPlane) A C)
  (hAD : AEDisjoint (μH[1] : Measure EuclideanPlane) A D)
  (hBC : AEDisjoint (μH[1] : Measure EuclideanPlane) B C)
  (hBD : AEDisjoint (μH[1] : Measure EuclideanPlane) B D)
  (hCD : AEDisjoint (μH[1] : Measure EuclideanPlane) C D)
  (hB : MeasurableSet B) (hC : MeasurableSet C) (hD : MeasurableSet D)
  (hfA : IntegrableOn f A (μH[1] : Measure EuclideanPlane))
  (hfB : IntegrableOn f B (μH[1] : Measure EuclideanPlane))
  (hfC : IntegrableOn f C (μH[1] : Measure EuclideanPlane))
  (hfD : IntegrableOn f D (μH[1] : Measure EuclideanPlane)) :
  (∫ z in A ∪ (B ∪ (C ∪ D)), f z ∂(μH[1] : Measure EuclideanPlane)) =
    (∫ z in A, f z ∂(μH[1] : Measure EuclideanPlane)) +
      ((∫ z in B, f z ∂(μH[1] : Measure EuclideanPlane)) +
        ((∫ z in C, f z ∂(μH[1] : Measure EuclideanPlane)) +
          ∫ z in D, f z ∂(μH[1] : Measure EuclideanPlane))) := by
  rw [setIntegral_union₀
    (hAB.union_right (hAC.union_right hAD))
    (hB.union (hC.union hD)).nullMeasurableSet
    hfA (hfB.union (hfC.union hfD)),
    setIntegral_union₀
    (hBC.union_right hBD)
    (hC.union hD).nullMeasurableSet
    hfB (hfC.union hfD),
    setIntegral_union₀ hCD hD.nullMeasurableSet hfC hfD]

/-- The union of all six pairwise overlaps among four Euclidean sets. -/
```

integral_semicircle_scaled

Classification: supplement

Source: CMVGeometry.lean:1759-1791

Declaration SHA-256: fe66bbeed922271b7cf0e4043f58e6b4f6651310c0d02aa776383ac42b18240d

Mathematical statement. For $r > 0$ and $0 \leq \theta \leq \pi$, $\int_{r \cos \theta}^r 2\sqrt{r^2 - y^2} \, dy = r^2(\theta - \sin \theta \cos \theta)$.

Scaling the unit-circle integral produces the Euclidean area of the circular segment.

```
lemma integral_semicircle_scaled {r θ : ℝ} (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  (∫ y in r * cos θ..r, 2 * √(r ^ 2 - y ^ 2)) =
    r ^ 2 * (θ - sin θ * cos θ) := by
  have hcomp := intervalIntegral.integral_comp_mul_deriv
  (a := cos θ) (b := 1) (f := fun x : ℝ ↦ r * x) (f' := fun _ ↦ r)
  (g := fun y : ℝ ↦ 2 * √(r ^ 2 - y ^ 2))
  (fun x _ ↦ by simp using (hasDerivAt_id x).const_mul r)
  continuousOn_const (by fun_prop)
  simp only [Function.comp_apply, mul_one] at hcomp
  rw [← hcomp]
  calc
    (∫ x in cos θ..1, 2 * √(r ^ 2 - (r * x) ^ 2) * r) =
      ∫ x in cos θ..1, (2 * r ^ 2) * √(1 - x ^ 2) := by
        apply intervalIntegral.integral_congr
        intro x hx
        dsimp
        have hx' : x ∈ Icc (-1 : ℝ) 1 := by
          have hcos : -1 ≤ cos θ := neg_one_le_cos θ
          have hcos' : cos θ ≤ 1 := cos_le_one θ
          have : x ∈ Icc (cos θ) 1 := by
            simp [uIcc, hcos'] using hx
          exact ⟨hcos.trans this.1, this.2⟩
        have hnonneg : 0 ≤ 1 - x ^ 2 := by nlinarith [hx'.1, hx'.2]
        rw [show r ^ 2 - (r * x) ^ 2 = r ^ 2 * (1 - x ^ 2) by ring,
          Real.sqrt_mul (sq_nonneg r), Real.sqrt_sq hr.le]
        ring
    _ = (2 * r ^ 2) * (∫ x in cos θ..1, √(1 - x ^ 2)) := by
      rw [intervalIntegral.integral_const_mul]
    _ = r ^ 2 * (θ - sin θ * cos θ) := by
      rw [integral_sqrt_one_sub_sq_from_cos hθ0 hθπ]
      ring
```

integral_semicircle_translated

Classification: supplement

Source: CMVGeometry.lean:1792-1800

Declaration SHA-256: 06721e5dc911d180205234ff69b820e369a8d82b66f142e17c0a1377b9b27f8d

Mathematical statement. For real $a, r > 0$, and $0 \leq \theta \leq \pi$, $\int_{a+r \cos \theta}^{a+r} 2\sqrt{r^2 - (y-a)^2} \, dy = r^2(\theta - \sin \theta \cos \theta)$.

Translation moves the circle center without changing the segment-area integral.

```
lemma integral_semicircle_translated {a r θ : ℝ} (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  (∫ y in a + r * cos θ..a + r, 2 * √(r ^ 2 - (y - a) ^ 2)) =
    r ^ 2 * (θ - sin θ * cos θ) := by
  rw [← intervalIntegral.integral_comp_add_left
    (f := fun y : ℝ ↦ 2 * √(r ^ 2 - (y - a) ^ 2)) a]
  simp only [add_sub_cancel_left]
  exact integral_semicircle_scaled hr hθ0 hθπ
```

integral_sqrt_one_sub_sq_from_cos

Classification: supplement

Source: CMVGeometry.lean:1729-1758

Declaration SHA-256: e4a969c6c1b67c6f267db8fb9582c7cd96174fa24489265c840c646c3085a52a

Mathematical statement. For $0 \leq \theta \leq \pi$, $\int_{\cos \theta}^1 \sqrt{1-x^2} \, dx = (\theta - \sin \theta \cos \theta)/2$.

The circular-slice integral is evaluated on the full half-central-angle range, including the degenerate endpoint values.

```
lemma integral_sqrt_one_sub_sq_from_cos {θ : ℝ} (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  (∫ x in cos θ..1, √(1 - x ^ 2)) =
    (θ - sin θ * cos θ) / 2 := by
  have hcomp := intervalIntegral.integral_comp_mul_deriv
    (a := θ) (b := 0) (f := cos) (f' := fun x ↦ -sin x)
    (g := fun x : ℝ ↦ √(1 - x ^ 2))
    (fun x _ ↦ Real.hasDerivAt_cos x)
    (continuous_sin.neg.continuous0n) (by fun_prop)
  simp only [Function.comp_apply, cos_zero] at hcomp
  rw [← hcomp]
  calc
    (∫ x in θ..0, √(1 - cos x ^ 2) * -sin x) =
      ∫ x in θ..0, -(sin x ^ 2) := by
        apply intervalIntegral.integral_congr
        intro x hx
        dsimp
        have hx' : x ∈ Icc 0 θ := by
          simpa [uIcc, hθ0] using hx
        rw [← Real.sin_eq_sqrt_one_sub_cos_sq]
        · ring
        · exact hx'.1
        · exact hx'.2.trans hθπ
  _ = ∫ x in 0..θ, sin x ^ 2 := by
    rw [intervalIntegral.integral_symm]
    simp
  _ = (θ - sin θ * cos θ) / 2 := by
    rw [integral_sin_sq]
    simp
    ring
```

intervalPushforward

Classification: supplement

Source: CMVGeometry.lean:1200-1202

Declaration SHA-256: 366d70f2307ddf912f8fe8519c486284fdacdab2c9bb76f371f9a65dd8ee807d

Mathematical statement. For a curve c and real endpoints a, b , the interval pushforward is $c_*(\mathcal{L}^1 \text{ restricted to } [a, b])$.

The definition is meaningful even when the interval is empty; arclength curves make it a boundary-length measure.

```
def intervalPushforward (curve : ℝ → PlanePoint) (a b : ℝ) : Measure PlanePoint :=
  Measure.map curve (volume.restrict (Icc a b))
```

isometry_capComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:398-408

Declaration SHA-256: 5d4136606591a26f6abf24f8ba286ddba7154e77e9f96c81673196c1854009b0

Mathematical statement. The indicated coordinate-to-complex or rotation map preserves Euclidean distance.

Translations, reflections, rotations, and the coordinate realization preserve the metric, allowing Hausdorff-measure results for the standard circular parametrization to transfer to the modeled traces.

```

lemma isometry_capComplexIsometryMap (c : OneSidedCircularCap) :
  Isometry (capComplexIsometryMap c) := by
  apply Isometry.of_dist_eq
  intro z w
  rw [WithLp.prod_dist_eq_add (by norm_num)]
  simp only [capComplexIsometryMap]
  split <;>
  norm_num [Real.dist_eq, sq_abs] <;>
  rw [← Real.sqrt_eq_rpow, Complex.dist_eq_re_im] <;>
  congr 1 <;> ring

```

isometry_leftCoreComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:685-695

Declaration SHA-256: a61c4ab4c222e88c43376bf46386ccf7697c506b7a23f81458ff102288a9e14a

Mathematical statement. The indicated coordinate-to-complex or rotation map preserves Euclidean distance.

Translations, reflections, rotations, and the coordinate realization preserve the metric, allowing Hausdorff-measure results for the standard circular parametrization to transfer to the modeled traces.

```

lemma isometry_leftCoreComplexIsometryMap (c : StripCore) :
  Isometry (leftCoreComplexIsometryMap c) := by
  apply Isometry.of_dist_eq
  intro z w
  rw [WithLp.prod_dist_eq_add (by norm_num)]
  simp only [leftCoreComplexIsometryMap]
  norm_num [Real.dist_eq, sq_abs]
  rw [← Real.sqrt_eq_rpow, Complex.dist_eq_re_im]
  congr 1
  ring

```

isometry_rightCoreComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:696-704

Declaration SHA-256: 2857926ac4225e60a51ede217224fdFDA63879d202e0be14411ea5c13b2fc6bf

Mathematical statement. The indicated coordinate-to-complex or rotation map preserves Euclidean distance.

Translations, reflections, rotations, and the coordinate realization preserve the metric, allowing Hausdorff-measure results for the standard circular parametrization to transfer to the modeled traces.

```

lemma isometry_rightCoreComplexIsometryMap (c : StripCore) :
  Isometry (rightCoreComplexIsometryMap c) := by
  apply Isometry.of_dist_eq
  intro z w
  rw [WithLp.prod_dist_eq_add (by norm_num)]
  simp only [rightCoreComplexIsometryMap]
  norm_num [Real.dist_eq, sq_abs]
  rw [← Real.sqrt_eq_rpow, Complex.dist_eq_re_im]

```

isometry_unitCircleArcRotation

Classification: supplement

Source: FrontierPerimeter.lean:315-322

Declaration SHA-256: 0390b164e94132c53e908a7960b31f68dc786e7272a9a9f6482fa79cade88362

Mathematical statement. The indicated coordinate-to-complex or rotation map preserves Euclidean distance.

Translations, reflections, rotations, and the coordinate realization preserve the metric, allowing Hausdorff-measure results for the standard circular parametrization to transfer to the modeled traces.

```
lemma isometry_unitCircleArcRotation (r m : ℝ) :
  Isometry (unitCircleArcRotation r m) := by
  apply Isometry.of_dist_eq
  intro z w
  rw [Complex.dist_eq, Complex.dist_eq]
  simp only [unitCircleArcRotation, ← sub_mul, norm_mul, Complex.norm_exp]
  norm_num
```

joinSet

Classification: supplement

Source: FrontierPerimeter.lean:1115-1117

Declaration SHA-256: 613dcfda543c5255d80175237d7db38b3386ce55bc3215682252569ce26c42d5

Mathematical statement. For real q , the join set is the four-point set

$$\{(-q/2, 1), (q/2, 1), (-q/2, -1), (q/2, -1)\}.$$

This finite coordinate set records the possible junction points of the four primitive boundary traces.

```
def joinSet (q : ℝ) : Set PlanePoint :=
  {(-q / 2, 1), (q / 2, 1), (-q / 2, -1), (q / 2, -1)}
```

joinSet_finite

Classification: supplement

Source: FrontierPerimeter.lean:1253-1256

Declaration SHA-256: c95df1795c640672a88890bc658373f829154b7fa3a2361c967c14e17f22b551

Mathematical statement. For every real q , the join set is finite.

The conclusion concerns the explicitly defined four-point set, not arbitrary overlaps.

```
theorem joinSet_finite (q : ℝ) : (joinSet q).Finite := by
  simp [joinSet]

/-- The join set realized in the metric where Euclidean  $H^1$  is formed. -/
```

leftCoreBoundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:1487-1490

Declaration SHA-256: e243bfd8c440c79be3e701f3e1ba7f0b4af716d3ad6d1c3de2eb88023b1a30c8

Mathematical statement. The left core boundary measure pushes interval length along the left core arclength map.

The interval is $[-R\phi, R\phi]$; this measure counts only the left curved side.

```
def leftCoreBoundaryMeasure (c : StripCore) : Measure PlanePoint :=
  intervalPushforward (leftCoreParam c) (coreArcStart c) (coreArcEnd c)

/-- Explicit regular boundary measure of the right core arc. -/
```

leftCoreBoundaryMeasure_integrable

Classification: supplement

Source: CMVGeometry.lean:2275-2281

Declaration SHA-256: efd79d96b8f43a7c478608fb7242070e102fd586af27d2826a566f07542e0a9

Mathematical statement. For every core and real λ , the strip density is integrable against the left core boundary measure.

The curve is measurable and its parameter interval has finite length.

```
lemma leftCoreBoundaryMeasure_integrable (lam : ℝ) (c : StripCore) :  
  Integrable (StripDensity lam) (leftCoreBoundaryMeasure c) := by  
  rw [leftCoreBoundaryMeasure]  
  exact stripDensity_integrable_intervalPushforward lam (by  
    unfold leftCoreParam  
    fun_prop)
```

leftCoreComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:677-680

Declaration SHA-256: c0385e604fc62ea1f5b752f17a4538ba2edeedf20e52cbfb6848b3b966b2ab57

Mathematical statement. For a strip core c , the left complex-to-Euclidean map is $z \mapsto (c_L - \operatorname{Re} z, \operatorname{Im} z)$.

The map places the standard circle at the left side-circle center and reverses its horizontal coordinate.

```
noncomputable def leftCoreComplexIsometryMap (c : StripCore) (z : ℂ) :  
  EuclideanPlane :=  
  WithLp.toLp 2 (c.leftCenterX - z.re, z.im)
```

leftCoreParam

Classification: supplement

Source: CMVGeometry.lean:1477-1481

Declaration SHA-256: 80f940fe52c510e69a283605275f10b49a7ca369836a80a40ece896dbbf606db

Mathematical statement. The left core arclength map is $(c_L - R \cos(s/R), R \sin(s/R))$.

It traces the left outer side between the two interfaces on $[-R\phi, R\phi]$.

```
def leftCoreParam (c : StripCore) (s : ℝ) : PlanePoint :=  
  (c.leftCenterX - c.radius * cos (s / c.radius),  
   c.radius * sin (s / c.radius))  
  
/-- Unit-speed parametrization of the right core arc. -/
```

leftCoreParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:517-596

Declaration SHA-256: 01c2af95d70e04d0bac530bc681fda34196e33e841f34e26466b4a3a9c6ecd18

Mathematical statement. For every strip core c , write $R = c.\text{radius}$, $\phi = c.\text{sideAngle}$, $s_- = -R\phi$, and $s_+ = R\phi$. Then the left arclength map satisfies

$$\text{leftParam}_c([s_-, s_+]) = c.\text{leftArcTrace}.$$

Here s_- and s_+ are the core arc endpoints. The theorem identifies the complete clipped circular trace, not the full side circle.

```

theorem leftCoreParam_image_Icc (c : StripCore) :
  leftCoreParam c '' Icc (coreArcStart c) (coreArcEnd c) =
    c.leftArcTrace := by
  ext p
  constructor
  · rintro ⟨s, hs, rfl⟩
    have hu := core_angle_mem' hs
    have htrig := Real.sin_sq_add_cos_sq (s / c.radius)
    have hcircle :
      ((leftCoreParam c s).1 - c.leftCenterX) ^ 2 +
      (leftCoreParam c s).2 ^ 2 = c.radius ^ 2 := by
      simp only [leftCoreParam, Prod.fst, Prod.snd]
      nlinarith [sq_nonneg c.radius]
    have habs : |s / c.radius| ≤ c.sideAngle := abs_le.mpr hu
    have hcos : Real.cos c.sideAngle ≤ Real.cos (s / c.radius) := by
      simp only [Real.cos_abs] using
      (Real.cos_le_cos_of_nonneg_of_le_pi (abs_nonneg (s / c.radius))
        c.sideAngle_lt_pi.le habs)
    have hx : (leftCoreParam c s).1 ≤ -c.chord / 2 := by
      simp only [leftCoreParam, Prod.fst]
      unfold StripCore.leftCenterX
      nlinarith [mul_le_mul_of_nonneg_left hcos c.radius_pos.le]
    exact ⟨hcircle, hx, coreParam_y_bound' c hs⟩
  · intro hp
    change (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2 ^
      p.1 ≤ -c.chord / 2 ^ |p.2| ≤ 1 at hp
    let u : ℝ := Real.arcsin (p.2 / c.radius)
    have hp2 := abs_le.mp hp.2.2
    have hlo : -c.curvature ≤ p.2 / c.radius := by
      apply (le_div_iff₀ c.radius_pos).2
      nlinarith [core_radius_mul_curvature' c]
    have hhi : p.2 / c.radius ≤ c.curvature := by
      apply (div_le_iff₀ c.radius_pos).2
      nlinarith [core_radius_mul_curvature' c]
    have hu : u ∈ Icc (-c.sideAngle) c.sideAngle := by
      constructor
      · simp [u, StripCore.sideAngle] using Real.arcsin_le_arcsin hlo
      · simp [u, StripCore.sideAngle] using Real.arcsin_le_arcsin hhi
    have hratio : p.2 / c.radius ∈ Icc (-1 : ℝ) 1 :=
      ⟨(by linarith [c.curvature_le_one] : (-1 : ℝ) ≤ -c.curvature) |>.trans hlo,
        hhi.trans c.curvature_le_one⟩
    have hsinu : Real.sin u = p.2 / c.radius :=
      Real.sin_arcsin hratio.1 hratio.2
    have hy : c.radius * Real.sin u = p.2 := by
      rw [hsinu]
      field_simp [ne_of_gt c.radius_pos]
    have hcostheta : 0 ≤ Real.cos c.sideAngle :=
      Real.cos_nonneg_of_mem_Icc
      ⟨by linarith [c.sideAngle_pos, Real.pi_pos],
        c.sideAngle_le_pi_div_two⟩
    have hxu : p.1 - c.leftCenterX ≤ 0 := by
      unfold StripCore.leftCenterX
      nlinarith [mul_nonneg c.radius_pos.le hcostheta]
    have hcosu : 0 ≤ Real.cos u :=
      Real.cos_nonneg_of_mem_Icc
      ⟨(neg_le_neg c.sideAngle_le_pi_div_two).trans hu.1,
        hu.2.trans c.sideAngle_le_pi_div_two⟩
    have htrig := Real.sin_sq_add_cos_sq u
    have hsquares :
      (p.1 - c.leftCenterX) ^ 2 = (c.radius * Real.cos u) ^ 2 := by
      calc
      _ = c.radius ^ 2 - p.2 ^ 2 := by nlinarith [hp.1]
      _ = (c.radius * Real.cos u) ^ 2 := by rw [← hy]; nlinarith
    have hcosprod : 0 ≤ c.radius * Real.cos u :=
      mul_nonneg c.radius_pos.le hcosu
    have hx : p.1 = c.leftCenterX - c.radius * Real.cos u := by
      rcases sq_eq_sq_iff_eq_or_eq_neg.mp hsquares with hpos | hneg
      · have hz : p.1 - c.leftCenterX = 0 := by nlinarith

```

```

      nlinarith
    · linarith
  refine (c.radius * u, ?_, ?_)
  · change -c.radius * c.sideAngle ≤ c.radius * u ∧
    c.radius * u ≤ c.radius * c.sideAngle
  constructor <=> nlinarith [c.radius_pos, hu.1, hu.2]
  · have hdiv : c.radius * u / c.radius = u := by
    field_simp [ne_of_gt c.radius_pos]
    apply Prod.ext
  · simp [leftCoreParam, hdiv, hx]
  · simp [leftCoreParam, hdiv, hy]

```

leftCore_strip_density

Classification: supplement

Source: CMVGeometry.lean:1662-1667

Declaration SHA-256: 3222d64a655ce3af15a638606821b923e89c31ae75558031c08763aeb76b72b6

Mathematical statement. Every point on the closed parameter interval of the left core arc has density one.

Its vertical coordinate stays within the closed strip, including both interfaces.

```

theorem leftCore_strip_density (lam : ℝ) (c : StripCore) {s : ℝ}
  (hs : s ∈ Icc (coreArcStart c) (coreArcEnd c)) :
  StripDensity lam (leftCoreParam c s) = 1 := by
  rw [StripDensity, if_pos]
  simp [leftCoreParam] using coreParam_y_bound c hs

```

leftCore_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1675-1694

Declaration SHA-256: e333f475432d30c40456ab56a9f5187cebdb61288c7e07d5eec293804ac77f49

Mathematical statement. The weighted integral along the left core boundary measure is $2\ell(\phi)$, for every real λ .

The density is constantly one and the parameter interval has length $2R\phi = 2\ell(\phi)$.

```

theorem leftCore_weightedIntegral (lam : ℝ) (c : StripCore) :
  ∫ p, StripDensity lam p ∂leftCoreBoundaryMeasure c = 2 * ell c.sideAngle := by
  rw [leftCoreBoundaryMeasure]
  rw [integral_intervalPushforward (leftCoreParam_measurable c)
    (measurable_stripDensity lam)]
  calc
    (∫ s in Icc (coreArcStart c) (coreArcEnd c),
      StripDensity lam (leftCoreParam c s)) =
      ∫ _s in Icc (coreArcStart c) (coreArcEnd c), 1 := by
    apply integral_congr_ae
    filter_upwards [ae_restrict_mem measurableSet_Icc] with s hs
    exact leftCore_strip_density lam c hs
  _ = coreArcEnd c - coreArcStart c := by
    have horder : coreArcStart c ≤ coreArcEnd c := by
      dsimp [coreArcStart, coreArcEnd]
      nlinarith [c.radius_pos, c.sideAngle_pos]
    simp [horder]
  _ = 2 * ell c.sideAngle := core_single_arc_length c

/-- Weighted integral of the right core arc equals its Euclidean length. -/

```

lintegral_semicircle_translated

Classification: supplement

Source: CMVGeometry.lean:1801-1819

Declaration SHA-256: 3c8e90cd635d133e832b64fe2731d46db4f68d0a1a341d292df3904cf66e67e8

Mathematical statement. The nonnegative integral of $2\sqrt{r^2 - (y - a)^2}$ over $[a + r\cos\theta, a + r]$ equals $r^2(\theta - \sin\theta\cos\theta)$, for $r > 0$ and $0 \leq \theta \leq \pi$.

Both sides are interpreted as extended nonnegative reals; the equality converts the real slice integral into the form used by volume integration.

```
lemma lintegral_semicircle_translated {a r θ : ℝ} (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  (∫ y in Icc (a + r * cos θ) (a + r),
    ENNReal.ofReal (2 * √(r ^ 2 - (y - a) ^ 2))) =
    ENNReal.ofReal (r ^ 2 * (θ - sin θ * cos θ)) := by
  have hcos : cos θ ≤ 1 := cos_le_one θ
  have hab : a + r * cos θ ≤ a + r := by nlinarith
  have hcont : Continuous (fun y : ℝ ↦ 2 * √(r ^ 2 - (y - a) ^ 2)) := by
    fun_prop
  have hint :
    IntegrableOn (fun y : ℝ ↦ 2 * √(r ^ 2 - (y - a) ^ 2))
      (Icc (a + r * cos θ) (a + r)) :=
    hcont.integrableOn_Icc
  rw [← MeasureTheory.ofReal_integral_eq_lintegral_ofReal hint
    (Filter.Eventually.of_forall fun _ ↦ by positivity)]
  congr 1
  rw [integral_Icc_eq_integral_Ioc, ← intervalIntegral.integral_of_le hab]
  exact integral_semicircle_translated hr hθ0 hθπ
```

lipschitzWith_unitCircleArc

Classification: supplement

Source: FrontierPerimeter.lean:33-54

Declaration SHA-256: 6ddc026dd017d7b20d15e385a48bc34f5bcbfcad3796885aedaeb1fd06730313

Mathematical statement. If $r > 0$, the map $s \mapsto \gamma_r(s)$ is globally 1-Lipschitz from $\mathbb{R}$ to $\mathbb{C}$.

The chord distance between two parametrized points is bounded by the signed-arclength parameter difference, providing the Hausdorff-measure upper bound for every interval.

```
lemma lipschitzWith_unitCircleArc {r : ℝ} (hr : 0 < r) :
  LipschitzWith 1 (unitCircleArc r) := by
  rw [lipschitzWith_iff_dist_le_mul]
  intro s t
  simp only [NNReal.coe_one, one_mul, Complex.dist_eq]
  rw [unitCircleArc_sub_factor hr.ne', norm_mul, norm_mul]
  have hexp : ||Complex.exp (Complex.I * (t / r : ℝ))|| = 1 := by
    rw [Complex.norm_exp]
    simp
  rw [hexp, mul_one]
  have hnormr : ||(r : ℂ)|| = r := by simp [abs_of_pos hr]
  rw [hnormr]
  calc
    r * ||Complex.exp (Complex.I * ((s - t) / r : ℝ)) - 1||
      ≤ r * ||(s - t) / r|| := by
        gcongr
        exact Real.norm_exp_I_mul_ofReal_sub_one_le
    _ = ||s - t|| := by
      rw [norm_div, Real.norm_eq_abs, Real.norm_eq_abs, abs_of_pos hr]
      field_simp
    _ = dist s t := by simp only [Real.norm_eq_abs, Real.dist_eq]
```

measurableSet_euclidean_capTrace

Classification: supplement

Source: FrontierPerimeter.lean:1104-1109

Declaration SHA-256: 345ff49d99187e89c9b6a7558ffa41e164108dfd32b3dc525b699de014a17b5a

Mathematical statement. The indicated Euclidean trace is measurable.

It is a continuous image of a compact interval or a closed semialgebraic trace, so it is measurable in the Euclidean realization.

```
theorem measurableSet_euclidean_capTrace (c : OneSidedCircularCap) :  
  MeasurableSet (planeEuclideanHomeomorph '' c.arcTrace) := by  
  rw [← realizedCapParam_image_Icc]  
  exact (isCompact_Icc.image (continuous_realizedCapParam c)).measurableSet  
  
/-- Union of all six pairwise intersections of four primitive traces. -/
```

measurableSet_euclidean_leftArcTrace

Classification: supplement

Source: FrontierPerimeter.lean:1089-1093

Declaration SHA-256: a31ed55013e95ae690ffcc5bab6587c2933302f4970b8b673a167e6c53e1b7f8

Mathematical statement. The indicated Euclidean trace is measurable.

It is a continuous image of a compact interval or a closed semialgebraic trace, so it is measurable in the Euclidean realization.

```
theorem measurableSet_euclidean_leftArcTrace (c : StripCore) :  
  MeasurableSet (planeEuclideanHomeomorph '' c.leftArcTrace) := by  
  rw [← realizedLeftCoreParam_image_Icc]  
  exact (isCompact_Icc.image (continuous_realizedLeftCoreParam c)).measurableSet
```

measurableSet_euclidean_rightArcTrace

Classification: supplement

Source: FrontierPerimeter.lean:1094-1098

Declaration SHA-256: 771198ca90f5054f065ccd16a46e41b5fce0886173f7b3c654e6b61824be845e

Mathematical statement. The indicated Euclidean trace is measurable.

It is a continuous image of a compact interval or a closed semialgebraic trace, so it is measurable in the Euclidean realization.

```
theorem measurableSet_euclidean_rightArcTrace (c : StripCore) :  
  MeasurableSet (planeEuclideanHomeomorph '' c.rightArcTrace) := by  
  rw [← realizedRightCoreParam_image_Icc]  
  exact (isCompact_Icc.image (continuous_realizedRightCoreParam c)).measurableSet
```

measurable_stripDensity

Classification: supplement

Source: CMVGeometry.lean:1234-1240

Declaration SHA-256: b6fc908d8bba478440a5c35f7c75a94366dc691bdf7d2125848e2a8f512284af

Mathematical statement. The strip density is measurable for every real λ .

The strip is a measurable set and the density takes only two constant values.

```
theorem measurable_stripDensity (lam : ℝ) : Measurable (StripDensity lam) := by
  unfold StripDensity
  exact Measurable.ite (by measurability) measurable_const measurable_const

/-! ## One-sided circular caps -/

/-- The cap arc, parametrized by signed arclength. -/
```

monotone_capPartitionPoint

Classification: supplement

Source: FrontierPerimeter.lean:167-174

Declaration SHA-256: 62e77199a2eb7f24e5f93307536675c49f368eab2d84be96574f544f2152a857

Mathematical statement. If $r > 0$, $\theta > 0$, and $N > 0$, the map $i \mapsto p_{N,i}$ on natural indices is monotone.

The increment per index is $2r\theta/N \geq 0$, so the partition is ordered from left to right.

```
lemma monotone_capPartitionPoint {r theta : ℝ} (hr : 0 < r) (htheta : 0 < theta)
  {N : ℕ} (hN : 0 < N) : Monotone (capPartitionPoint r theta N) := by
  intro i j hij
  unfold capPartitionPoint
  have hden : 0 < (N : ℝ) := by positivity
  have hstep : 0 ≤ 2 * r * theta / (N : ℝ) := (div_pos (by positivity) hden).le
  gcongr
```

ofReal_dist_le_hausdorffMeasure_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:105-123

Declaration SHA-256: 272ec4c298bc0f3582f8dd04d91ee8113b10ec4ec1c37538e792ec5391cc0936

Mathematical statement. For a continuous map $f: \mathbb{R} \rightarrow E$ into a metric Borel space and $u \leq v$,

$$\text{dist}(f(u), f(v)) \leq \mathcal{H}^1(f([u, v])).$$

The image of the interval contains a connected path joining the endpoint images, so its one-dimensional Hausdorff measure dominates their distance. The assertion is a measure inequality, not a set equality.

```
lemma ofReal_dist_le_hausdorffMeasure_image_Icc
  {E : Type*} [MetricSpace E] [MeasurableSpace E] [BorelSpace E]
  {f : ℝ → E} (hf : Continuous f) {u v : ℝ} (huv : u ≤ v) :
  ENNReal.ofReal (dist (f u) (f v)) ≤
    (μH[1] : Measure E) (f '' Icc u v) := by
  let g : E → ℝ := fun z => dist (f u) z
  have hg : LipschitzWith 1 g := LipschitzWith.dist_right (f u)
  have hI : Icc 0 (dist (f u) (f v)) ⊆ g '' (f '' Icc u v) := by
    have hIV := intermediate_value_Icc huv (hg.continuous.comp hf).continuousOn
    simpa only [g, dist_self, Set.image_image, Function.comp_apply] using hIV
  calc
    ENNReal.ofReal (dist (f u) (f v)) =
      (μH[1] : Measure ℝ) (Icc 0 (dist (f u) (f v))) := by
        rw [hausdorffMeasure_real, Real.volume_Icc, sub_zero]
    _ ≤ (μH[1] : Measure ℝ) (g '' (f '' Icc u v)) := measure_mono hI
    _ ≤ (μH[1] : Measure E) (f '' Icc u v) := by
      simpa using hg.hausdorffMeasure_image_le (d := (1 : ℝ)) (by norm_num)
      (f '' Icc u v)
```

pairwiseOverlap

Classification: supplement

Source: FrontierPerimeter.lean:1110-1114

Declaration SHA-256: f2051dcd764bb55c1f4c980a58c56ba3f51d928d6141df3262dbdd919f1b5fbd

Mathematical statement. For arbitrary sets A, B, C, D , the pairwise-overlap set is the union of all six intersections $A \cap B$, $A \cap C$, $A \cap D$, $B \cap C$, $B \cap D$, and $C \cap D$.

This is a definition for unrestricted sets. It asserts neither finiteness nor nullity; those properties require separate assembly-specific hypotheses or theorems.

```
def pairwiseOverlap (A B C D : Set PlanePoint) : Set PlanePoint :=
  (A ∩ B) ∪ ((A ∩ C) ∪ ((A ∩ D) ∪
    ((B ∩ C) ∪ ((B ∩ D) ∪ (C ∩ D)))))

/-- The four possible primitive joins in either assembly. -/
```

pairwise_aedisjoint_of_overlap_null

Classification: supplement

Source: FrontierPerimeter.lean:1598-1616

Declaration SHA-256: 4b46ea8bfc070cb38c17e38f03c2486a1eb119281a4706f7a55c5da843e21d49

Mathematical statement. If the one-dimensional Hausdorff measure of the union of all six pairwise intersections of Euclidean sets A, B, C, D is zero, then every one of the six pairs is almost disjoint for that measure.

The null-overlap hypothesis is an assumption and the conclusion is almost-disjointness. No finiteness conclusion about the overlap set follows from this declaration.

```
theorem pairwise_aedisjoint_of_overlap_null
  {A B C D : Set EuclideanPlane}
  (hnull : (μH[1] : Measure EuclideanPlane)
    (euclideanPairwiseOverlap A B C D) = 0) :
  AEDisjoint (μH[1] : Measure EuclideanPlane) A B ∧
  AEDisjoint (μH[1] : Measure EuclideanPlane) A C ∧
  AEDisjoint (μH[1] : Measure EuclideanPlane) A D ∧
  AEDisjoint (μH[1] : Measure EuclideanPlane) B C ∧
  AEDisjoint (μH[1] : Measure EuclideanPlane) B D ∧
  AEDisjoint (μH[1] : Measure EuclideanPlane) C D := by
  refine ⟨measure_mono_null (fun _ hz => Or.inl hz) hnull,
    measure_mono_null (fun _ hz => Or.inr (Or.inl hz)) hnull,
    measure_mono_null (fun _ hz => Or.inr (Or.inr (Or.inl hz))) hnull,
    measure_mono_null
      (fun _ hz => Or.inr (Or.inr (Or.inr (Or.inl hz)))) hnull,
    measure_mono_null
      (fun _ hz => Or.inr (Or.inr (Or.inr (Or.inr hz)))) hnull⟩
```

planeEuclideanHomeomorph

Classification: supplement

Source: CMVGeometry.lean:38-42

Declaration SHA-256: 0cae97f48ab4c061d7f08a907381bf52e9560477df33ffb993c23c2cbbf11049

Mathematical statement. The map J identifies the coordinate plane homeomorphically with its Euclidean realization.

It sends each coordinate pair to the identical pair with the Euclidean norm; it is a homeomorphism, not a claim that every product metric agrees with that norm.

```

noncomputable def planeEuclideanHomeomorph : PlanePoint ≈t EuclideanPlane :=
  (WithLp.homeomorphProd 2 ℝ ℝ).symm

/-- CMV's strip density. The closed interfaces belong to the density-one
branch. -/

```

realizedCapParam

Classification: supplement

Source: FrontierPerimeter.lean:430-433

Declaration SHA-256: 95b1661a30d7b5d3f17854c6eabbe1a4da9d944d9ad542e596607c6cb4fd12b0

Mathematical statement. For every cap c and real s , the Euclidean cap parametrization is $J(\text{capParam}_c(s))$.

This is a definition by composition with the coordinate-to-Euclidean homeomorphism, not an existence theorem for a cap.

```

noncomputable def realizedCapParam (c : OneSidedCircularCap) (s : ℝ) :
  EuclideanPlane :=
  planeEuclideanHomeomorph (capParam c s)

```

realizedCapParam_eq

Classification: supplement

Source: FrontierPerimeter.lean:434-441

Declaration SHA-256: 9c421cce21d93c49e9d419ecc30c8d31de995d391da25f034d29cf3f0bd85599

Mathematical statement. For every cap c and real s , the Euclidean cap parametrization equals the cap isometry applied to the standard circle point:

$$J(\text{capParam}_c(s)) = \Phi_c(\gamma_R(s)).$$

The equality identifies the coordinate parametrization with the explicit upper/lower Euclidean isometry and supplies the realization bridge for Hausdorff measure.

```

lemma realizedCapParam_eq (c : OneSidedCircularCap) (s : ℝ) :
  realizedCapParam c s = capComplexIsometryMap c (unitCircleArc c.radius s) := by
  rw [show realizedCapParam c s = WithLp.toLp 2 (capParam c s) by rfl]
  cases hside : c.side <|>
  simp [capParam, OneSidedCircularCap.arcPoint, capComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im, hside] <|>
  ring

```

realizedCapParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:1071-1080

Declaration SHA-256: a37b40496e1e5c2ae0b778fe01e7fee7a91557ce9473bf6e0b7755afebbd941b

Mathematical statement. For every one-sided cap c ,

$$J(\text{capParam}_c([-R\theta, R\theta])) = J(c.\text{arcTrace}).$$

This is the Euclidean transport of the exact coordinate image equality.

```

theorem realizedCapParam_image_Icc (c : OneSidedCircularCap) :
  realizedCapParam c '' Icc (capStart c) (capEnd c) =
  planeEuclideanHomeomorph '' c.arcTrace := by
  calc
  _ = planeEuclideanHomeomorph ''
    (capParam c '' Icc (capStart c) (capEnd c)) := by

```

```

    rw [Set.image_image]
    rfl
  _ = _ := by rw [capParam_image_Icc_eq_arcTrace]

```

realizedLeftCoreParam

Classification: supplement

Source: FrontierPerimeter.lean:705-708

Declaration SHA-256: 854e6d1ea17e2b2d6576f273610fd43051ac61c0bb7e75c185b3d3aa821f7da4

Mathematical statement. For every strip core c and real s , the realized left core parametrization is $J(\text{leftParam}_c(s))$.

This definition transports the coordinate left-arc map to the Euclidean metric space used for Hausdorff measure.

```

noncomputable def realizedLeftCoreParam (c : StripCore) (s : ℝ) :
  EuclideanPlane :=
  planeEuclideanHomeomorph (leftCoreParam c s)

```

realizedLeftCoreParam_eq

Classification: supplement

Source: FrontierPerimeter.lean:713-719

Declaration SHA-256: e2c0026193d6fa0dd58c252b75c6df1a108616049c60130059ddb93311c0b2a0

Mathematical statement. For every strip core c and real s , the realized left parametrization equals the left complex isometry applied to $\gamma_R(s)$.

The pointwise equality combines the real and imaginary coordinate formulas with the explicit left isometry.

```

lemma realizedLeftCoreParam_eq (c : StripCore) (s : ℝ) :
  realizedLeftCoreParam c s =
  leftCoreComplexIsometryMap c (unitCircleArc c.radius s) := by
  rw [show realizedLeftCoreParam c s = WithLp.toLp 2 (leftCoreParam c s) by rfl]
  simp [leftCoreParam, leftCoreComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im]

```

realizedLeftCoreParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:801-810

Declaration SHA-256: 43b7188db3ec73adee2747ff0dc65e5368084e5e17965fa7fdaa8f3c9301ccd7

Mathematical statement. For every strip core c , with $s_- = -c.\text{radius } c.\text{sideAngle}$ and $s_+ = c.\text{radius } c.\text{sideAngle}$,

$$J(\text{leftParam}_c([s_-, s_+])) = J(c.\text{leftArcTrace}).$$

Transporting the coordinate image equality through the homeomorphism gives the Euclidean image equality.

```

theorem realizedLeftCoreParam_image_Icc (c : StripCore) :
  realizedLeftCoreParam c '' Icc (coreArcStart c) (coreArcEnd c) =
  planeEuclideanHomeomorph '' c.leftArcTrace := by
  calc
  _ = planeEuclideanHomeomorph ''
    (leftCoreParam c '' Icc (coreArcStart c) (coreArcEnd c)) := by
    rw [Set.image_image]
    rfl
  _ = _ := by rw [leftCoreParam_image_Icc]

```

realizedRightCoreParam

Classification: supplement

Source: FrontierPerimeter.lean:709-712

Declaration SHA-256: 4f8de565538cdbc799e2faf0a2d6452b4030fcfacd5b43a7347015b8e46924a9

Mathematical statement. For every strip core c and real s , the realized right core parametrization is $J(\text{rightParam}_c(s))$.

This is the corresponding Euclidean realization of the right coordinate arc.

```
noncomputable def realizedRightCoreParam (c : StripCore) (s : ℝ) :  
  EuclideanPlane :=  
  planeEuclideanHomeomorph (rightCoreParam c s)
```

realizedRightCoreParam_eq

Classification: supplement

Source: FrontierPerimeter.lean:720-726

Declaration SHA-256: 4aab5f14bc5465771d4021be2ff0ba59e74b69eeaba0b9a9c834bd0bf72ed625

Mathematical statement. For every strip core c and real s , the realized right parametrization equals the right complex isometry applied to $\gamma_R(s)$.

The pointwise equality is the reflected counterpart of the left realization identity.

```
lemma realizedRightCoreParam_eq (c : StripCore) (s : ℝ) :  
  realizedRightCoreParam c s =  
    rightCoreComplexIsometryMap c (unitCircleArc c.radius s) := by  
  rw [show realizedRightCoreParam c s = WithLp.toLp 2 (rightCoreParam c s) by rfl]  
  simp [rightCoreParam, rightCoreComplexIsometryMap,  
    unitCircleArc_re, unitCircleArc_im]
```

realizedRightCoreParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:811-820

Declaration SHA-256: e28432904f0d4f1c849ed6ed56e30bdae00185a8e05f01ccceab8ef89d16f29b

Mathematical statement. For every strip core c , with $s_- = -c.\text{radius } c.\text{sideAngle}$ and $s_+ = c.\text{radius } c.\text{sideAngle}$,

$$J(\text{rightParam}_c([s_-, s_+])) = J(c.\text{rightArcTrace}).$$

This is the corresponding transported equality for the right clipped arc.

```
theorem realizedRightCoreParam_image_Icc (c : StripCore) :  
  realizedRightCoreParam c '' Icc (coreArcStart c) (coreArcEnd c) =  
    planeEuclideanHomeomorph '' c.rightArcTrace := by  
  calc  
  _ = planeEuclideanHomeomorph ''  
    (rightCoreParam c '' Icc (coreArcStart c) (coreArcEnd c)) := by  
    rw [Set.image_image]  
    rfl  
  _ = _ := by rw [rightCoreParam_image_Icc]
```

realizedSegmentParam

Classification: supplement

Source: FrontierPerimeter.lean:864-867

Declaration SHA-256: 4c1728c18c2df9a1da871e357fce4f6439c6307fe654c71c273fe06b8caaa0fd

Mathematical statement. For every horizontal segment g and real s , the Euclidean segment parametrization is $J(\text{segmentParam}_g(s))$.

This is a definition transporting the coordinate affine segment into the Euclidean realization.

```
noncomputable def realizedSegmentParam (seg : HorizontalSegment) (s : ℝ) :  
  EuclideanPlane :=  
  planeEuclideanHomeomorph (segmentParam seg s)
```

realizedSegmentParam_image_affineSegment

Classification: supplement

Source: FrontierPerimeter.lean:873-912

Declaration SHA-256: c3be9a5bc2d447d5c5665690a46411881b4f0933eb5c3cf50deaa41a88946c72

Mathematical statement. For every horizontal segment g , the realized parameter image equals the Euclidean affine segment joining the realized left and right endpoints.

The exact closed parameter interval is carried affinely between the two endpoint images; this is a set equality in the Euclidean plane.

```
theorem realizedSegmentParam_image_affineSegment (seg : HorizontalSegment) :  
  realizedSegmentParam seg '' Icc (segmentStart seg) (segmentEnd seg) =  
  affineSegment ℝ  
    (planeEuclideanHomeomorph seg.leftEndpoint)  
    (planeEuclideanHomeomorph seg.rightEndpoint) := by  
  rw [affineSegment]  
  ext p  
  constructor  
  · rintro (s, hs, rfl)  
    have hs' : -seg.chord / 2 ≤ s ∧ s ≤ seg.chord / 2 := by  
      simpa only [segmentStart, segmentEnd, mem_Icc] using hs  
    let t : ℝ := (s + seg.chord / 2) / seg.chord  
    refine ⟨t, ?_, ?_⟩  
    · constructor  
      · exact div_nonneg (by linarith) seg.chord_pos.le  
      · exact (div_le_one seg.chord_pos).2 (by linarith)  
    · rw [AffineMap.lineMap_apply_module, WithLp.ext_iff]  
      apply Prod.ext  
      · simp [realizedSegmentParam, planeEuclideanHomeomorph_apply, segmentParam,  
        HorizontalSegment.leftEndpoint, HorizontalSegment.rightEndpoint, t]  
        field_simp [ne_of_gt seg.chord_pos]  
        ring  
      · simp [realizedSegmentParam, planeEuclideanHomeomorph_apply, segmentParam,  
        HorizontalSegment.leftEndpoint, HorizontalSegment.rightEndpoint]  
        ring  
    · rintro (t, ht, rfl)  
      let s : ℝ := -seg.chord / 2 + t * seg.chord  
      refine ⟨s, ?_, ?_⟩  
      · change -seg.chord / 2 ≤ s ∧ s ≤ seg.chord / 2  
        dsimp [s]  
        constructor <=> nlinarith [seg.chord_pos, ht.1, ht.2]  
      · rw [AffineMap.lineMap_apply_module, WithLp.ext_iff]  
        apply Prod.ext  
        · simp [realizedSegmentParam, planeEuclideanHomeomorph_apply, segmentParam,  
          HorizontalSegment.leftEndpoint, HorizontalSegment.rightEndpoint, s]  
          ring  
        · simp [realizedSegmentParam, planeEuclideanHomeomorph_apply, segmentParam,
```

```
HorizontalSegment.leftEndpoint, HorizontalSegment.rightEndpoint]
ring
```

realizedSegmentParam_image_carrier

Classification: supplement

Source: FrontierPerimeter.lean:913-922

Declaration SHA-256: fc21a65690cb2d00a27cb6aebde20046f1dffbc15b7f30d3caf58ac1aefd1725

Mathematical statement. For every horizontal segment g , the realized parametrization image equals $J(g.\text{carrier})$.

The coordinate parametrization-carrier equality is transported through the homeomorphism.

```
theorem realizedSegmentParam_image_carrier (seg : HorizontalSegment) :
  realizedSegmentParam seg '' Icc (segmentStart seg) (segmentEnd seg) =
    planeEuclideanHomeomorph '' seg.carrier := by
  calc
  _ = planeEuclideanHomeomorph ''
    (segmentParam seg '' Icc (segmentStart seg) (segmentEnd seg)) := by
    rw [Set.image_image]
    rfl
  _ = _ := by rw [segmentParam_image_Icc]
```

rightCoreBoundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:1491-1493

Declaration SHA-256: 33cbc13fa6fcd71207ac7e5dfb0579966744c8887bb1c9af1b95e6c688f55ea8

Mathematical statement. The right core boundary measure pushes interval length along the right core arclength map.

It is the analogous measure on the right curved side, not on either chord.

```
def rightCoreBoundaryMeasure (c : StripCore) : Measure PlanePoint :=
  intervalPushforward (rightCoreParam c) (coreArcStart c) (coreArcEnd c)
```

rightCoreBoundaryMeasure_integrable

Classification: supplement

Source: CMVGeometry.lean:2282-2288

Declaration SHA-256: 63cd5ea28e72c7cf39aef8bfd2e698b16940d7ae74acc20f074d60c55994913c

Mathematical statement. For every core and real λ , the strip density is integrable against the right core boundary measure.

This is the corresponding finiteness statement for the reflected arc measure.

```
lemma rightCoreBoundaryMeasure_integrable (lam : ℝ) (c : StripCore) :
  Integrable (StripDensity lam) (rightCoreBoundaryMeasure c) := by
  rw [rightCoreBoundaryMeasure]
  exact stripDensity_integrable_intervalPushforward lam (by
    unfold rightCoreParam
    fun_prop)
```

rightCoreComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:681-684

Declaration SHA-256: 24206b125ec8e6de86a054126e34b32568c5cb9c365728de42d394ff9cc0ede6

Mathematical statement. For a strip core c , the right complex-to-Euclidean map is $z \mapsto (c_R + \text{Re}z, \text{Im}z)$.

The map places the standard circle at the right side-circle center without horizontal reflection.

```
noncomputable def rightCoreComplexIsometryMap (c : StripCore) (z : ℂ) :  
  EuclideanPlane :=  
  WithLp.toLp 2 (c.rightCenterX + z.re, z.im)
```

rightCoreParam

Classification: supplement

Source: CMVGeometry.lean:1482-1486

Declaration SHA-256: 38ea0d845e704b5d320adc3936a1b6a5fba52e17ce5fb320da7c20200afa9e4d

Mathematical statement. The right core arclength map is $(c_R + R\cos(s/R), R\sin(s/R))$.

It traces the reflected right outer side over the same parameter interval.

```
def rightCoreParam (c : StripCore) (s : ℝ) : PlanePoint :=  
  (c.rightCenterX + c.radius * cos (s / c.radius),  
   c.radius * sin (s / c.radius))  
  
/-- Explicit regular boundary measure of the left core arc. -/
```

rightCoreParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:597-676

Declaration SHA-256: 78bb53ade5c270538e82ae914ea314e16857498de0ee3dfc73dde7468e65c773

Mathematical statement. For every strip core c , write $R = c.\text{radius}$, $\phi = c.\text{sideAngle}$, $s_- = -R\phi$, and $s_+ = R\phi$. Then the right arclength map satisfies

$$\text{rightParam}_c([s_-, s_+]) = c.\text{rightArcTrace}.$$

The reflected parametrization covers precisely the clipped right circular trace on the same closed parameter interval.

```
theorem rightCoreParam_image_Icc (c : StripCore) :  
  rightCoreParam c '' Icc (coreArcStart c) (coreArcEnd c) =  
    c.rightArcTrace := by  
  ext p  
  constructor  
  · rintro ⟨s, hs, rfl⟩  
    have hu := core_angle_mem' hs  
    have htrig := Real.sin_sq_add_cos_sq (s / c.radius)  
    have hcircle :  
      ((rightCoreParam c s).1 - c.rightCenterX) ^ 2 +  
      (rightCoreParam c s).2 ^ 2 = c.radius ^ 2 := by  
      simp only [rightCoreParam, Prod.fst, Prod.snd]  
      nlinarith [sq_nonneg c.radius]  
    have habs : |s / c.radius| ≤ c.sideAngle := abs_le.mpr hu  
    have hcos : Real.cos c.sideAngle ≤ Real.cos (s / c.radius) := by  
      simp only [Real.cos_abs] using  
      (Real.cos_le_cos_of_nonneg_of_le_pi (abs_nonneg (s / c.radius))  
       c.sideAngle_lt_pi.le habs)  
    have hx : c.chord / 2 ≤ (rightCoreParam c s).1 := by
```

```

    simp only [rightCoreParam, Prod.fst]
    unfold StripCore.rightCenterX
    nlinarith [mul_le_mul_of_nonneg_left hcos c.radius_pos.le]
exact {hcircle, hx, coreParam_y_bound' c hs}
· intro hp
change (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2 ^
  c.chord / 2 ≤ p.1 ^ |p.2| ≤ 1 at hp
let u : ℝ := Real.arcsin (p.2 / c.radius)
have hp2 := abs_le.mp hp.2.2
have hlo : -c.curvature ≤ p.2 / c.radius := by
  apply (le_div_iff₀ c.radius_pos).2
  nlinarith [core_radius_mul_curvature' c]
have hhi : p.2 / c.radius ≤ c.curvature := by
  apply (div_le_iff₀ c.radius_pos).2
  nlinarith [core_radius_mul_curvature' c]
have hu : u ∈ Icc (-c.sideAngle) c.sideAngle := by
  constructor
  · simpa [u, StripCore.sideAngle] using Real.arcsin_le_arcsin hlo
  · simpa [u, StripCore.sideAngle] using Real.arcsin_le_arcsin hhi
have hratio : p.2 / c.radius ∈ Icc (-1 : ℝ) 1 :=
  ⟨(by linarith [c.curvature_le_one] : (-1 : ℝ) ≤ -c.curvature) |>.trans hlo,
    hhi.trans c.curvature_le_one⟩
have hsinu : Real.sin u = p.2 / c.radius :=
  Real.sin_arcsin hratio.1 hratio.2
have hy : c.radius * Real.sin u = p.2 := by
  rw [hsinu]
  field_simp [ne_of_gt c.radius_pos]
have hcostheta : 0 ≤ Real.cos c.sideAngle :=
  Real.cos_nonneg_of_mem_Icc
  ⟨by linarith [c.sideAngle_pos, Real.pi_pos],
    c.sideAngle_le_pi_div_two⟩
have hxu : 0 ≤ p.1 - c.rightCenterX := by
  unfold StripCore.rightCenterX
  nlinarith [mul_nonneg c.radius_pos.le hcostheta]
have hcosu : 0 ≤ Real.cos u :=
  Real.cos_nonneg_of_mem_Icc
  ⟨(neg_le_neg c.sideAngle_le_pi_div_two).trans hu.1,
    hu.2.trans c.sideAngle_le_pi_div_two⟩
have htrig := Real.sin_sq_add_cos_sq u
have hsquares :
  (p.1 - c.rightCenterX) ^ 2 = (c.radius * Real.cos u) ^ 2 := by
  calc
  _ = c.radius ^ 2 - p.2 ^ 2 := by nlinarith [hp.1]
  _ = (c.radius * Real.cos u) ^ 2 := by rw [← hy]; nlinarith
have hcosprod : 0 ≤ c.radius * Real.cos u :=
  mul_nonneg c.radius_pos.le hcosu
have hx : p.1 = c.rightCenterX + c.radius * Real.cos u := by
  rcases sq_eq_sq_iff_eq_or_eq_neg.mp hsquares with hpos | hneg
  · linarith
  · have hz : p.1 - c.rightCenterX = 0 := by nlinarith
    nlinarith
refine (c.radius * u, ?_, ?_)
· change -c.radius * c.sideAngle ≤ c.radius * u ^
  c.radius * u ≤ c.radius * c.sideAngle
  constructor <|> nlinarith [c.radius_pos, hu.1, hu.2]
· have hdiv : c.radius * u / c.radius = u := by
  field_simp [ne_of_gt c.radius_pos]
  apply Prod.ext
  · simp [rightCoreParam, hdiv, hx]
  · simp [rightCoreParam, hdiv, hy]

```

rightCore_strip_density

Classification: supplement

Source: CMVGeometry.lean:1668-1674

Declaration SHA-256: d879e18591ac7ff00f0a228fad33d85d1e290fb3c77820aaf80c672ec9411662

Mathematical statement. Every point on the closed parameter interval of the right core arc has density one.

The statement uses the same strip bound for the reflected arc.

```
theorem rightCore_strip_density (lam : ℝ) (c : StripCore) {s : ℝ}
  (hs : s ∈ Icc (coreArcStart c) (coreArcEnd c)) :
  StripDensity lam (rightCoreParam c s) = 1 := by
  rw [StripDensity, if_pos]
  simpa [rightCoreParam] using coreParam_y_bound c hs

/-- Weighted integral of the left core arc equals its Euclidean length. -/
```

rightCore_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1695-1714

Declaration SHA-256: 12c65212e5417b38acb8560f789f61c31ede351c3de6efe2b847438f7d6cf718

Mathematical statement. The weighted integral along the right core boundary measure is $2\ell(\phi)$, for every real λ .

This is the equal contribution of the reflected right curved side.

```
theorem rightCore_weightedIntegral (lam : ℝ) (c : StripCore) :
  ∫ p, StripDensity lam p ∂rightCoreBoundaryMeasure c = 2 * ell c.sideAngle := by
  rw [rightCoreBoundaryMeasure]
  rw [integral_intervalPushforward (rightCoreParam_measurable c)
    (measurable_stripDensity lam)]
  calc
    (∫ s in Icc (coreArcStart c) (coreArcEnd c),
      StripDensity lam (rightCoreParam c s)) =
      ∫ _s in Icc (coreArcStart c) (coreArcEnd c), 1 := by
      apply integral_congr_ae
      filter_upwards [ae_restrict_mem measurableSet_Icc] with s hs
      exact rightCore_strip_density lam c hs
  _ = coreArcEnd c - coreArcStart c := by
  have horder : coreArcStart c ≤ coreArcEnd c := by
    dsimp [coreArcStart, coreArcEnd]
    nlinarith [c.radius_pos, c.sideAngle_pos]
  simp [horder]
  _ = 2 * ell c.sideAngle := core_single_arc_length c

/-- The two arc masses add to the declared strip-core boundary-arc length. -/
```

segmentEnd

Classification: supplement

Source: CMVGeometry.lean:1400-1402

Declaration SHA-256: 161abc8655ce59212c511d028e178416823adf64c523c73dae6b889ae03ed1f4

Mathematical statement. The segment's ending parameter is $L/2$.

The parameter interval therefore has length equal to the chord.

```
def segmentEnd (seg : HorizontalSegment) : ℝ := seg.chord / 2

/-- Explicit regular boundary measure of a horizontal segment. -/
```

segmentEndpoint_edist

Classification: supplement

Source: FrontierPerimeter.lean:923-936

Declaration SHA-256: 667e4fe55a19b902ccd7cc29782bd67f737d862a1bddf625b46565ed0a83a1cd

Mathematical statement. For every horizontal segment g , the extended distance between its realized endpoints is

$$\text{edist}(J(g.\text{leftEndpoint}), J(g.\text{rightEndpoint})) = \text{ofReal}(L).$$

The Euclidean endpoint distance equals the positive segment length.

```
lemma segmentEndpoint_edist (seg : HorizontalSegment) :
  edist (planeEuclideanHomeomorph seg.leftEndpoint)
    (planeEuclideanHomeomorph seg.rightEndpoint) =
    ENNReal.ofReal seg.euclideanLength := by
  rw [edist_dist, WithLp.prod_dist_eq_add (by norm_num)]
  simp only [planeEuclideanHomeomorph_apply]
  norm_num [Real.dist_eq, HorizontalSegment.leftEndpoint,
    HorizontalSegment.rightEndpoint, HorizontalSegment.euclideanLength]
  congr 1
  have h : seg.midpointX -
    (seg.midpointX + seg.chord / 2 + seg.chord / 2) = -seg.chord := by ring
  rw [h, neg_sq, + Real.sqrt_eq_rpow, Real.sqrt_sq_eq_abs,
    abs_of_pos seg.chord_pos]
```

segmentParam

Classification: supplement

Source: CMVGeometry.lean:1395-1397

Declaration SHA-256: ba8ed6bbc5cd19b9c405036bea7c221ae24e5c76ea0f6ef5cc2d62c327ae1dc5

Mathematical statement. A horizontal segment has arclength map $s \mapsto (m + s, b)$.

It is intended for $-L/2 \leq s \leq L/2$ and has unit horizontal speed.

```
def segmentParam (seg : HorizontalSegment) (s : ℝ) : PlanePoint :=
  (seg.midpointX + s, seg.baseY)
```

segmentParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:835-863

Declaration SHA-256: 78cfdac8328c8d0308b83fec45a7d72c3a0a830edb6a3f41f9bd2da54db0f76

Mathematical statement. For every horizontal segment with chord length L , its affine parametrization maps the closed interval $[-L/2, L/2]$ exactly onto its carrier.

The closed interval contains both segment endpoints and the carrier contains exactly the same horizontal points.

```
theorem segmentParam_image_Icc (seg : HorizontalSegment) :
  segmentParam seg '' Icc (segmentStart seg) (segmentEnd seg) = seg.carrier := by
  ext p
  constructor
  · rintro ⟨s, hs, rfl⟩
    have hs' : -seg.chord / 2 ≤ s ∧ s ≤ seg.chord / 2 := by
      simpa only [segmentStart, segmentEnd, mem_Icc] using hs
    have habs : |s| ≤ seg.chord / 2 := by
      rw [abs_le]
      constructor <=> linarith [hs'.1]
    change seg.baseY = seg.baseY ∧
      |seg.midpointX + s - seg.midpointX| ≤ seg.chord / 2
    rw [add_sub_cancel_left]
    exact ⟨rfl, habs⟩
  · intro hp
    change p.2 = seg.baseY ∧ |p.1 - seg.midpointX| ≤ seg.chord / 2 at hp
    refine ⟨p.1 - seg.midpointX, ?_, ?_⟩
    · have hb := abs_le.mp hp.2
      simpa only [segmentStart, segmentEnd, mem_Icc]
        using (show -seg.chord / 2 ≤ p.1 - seg.midpointX ∧
          p.1 - seg.midpointX ≤ seg.chord / 2 by
            constructor <=> linarith [hb.1])
```

```

· apply Prod.ext
· simp [segmentParam]
· simpa [segmentParam] using hp.1.symm

```

```

@[simp] lemma planeEuclideanHomeomorph_apply (p : PlanePoint) :
  planeEuclideanHomeomorph p = WithLp.toLp 2 p := rfl

```

segmentStart

Classification: supplement

Source: CMVGeometry.lean:1398-1399

Declaration SHA-256: e90adf884523aab9d2e3af449adbc9d655d375051a612c0b4970a212a5dca61f

Mathematical statement. The segment's starting parameter is $-L/2$.

Substitution in the parameterization yields its left endpoint.

```

def segmentStart (seg : HorizontalSegment) : ℝ := -seg.chord / 2

```

stripDensity_integrableOn_of_volume_ne_top

Classification: supplement

Source: CMVGeometry.lean:2289-2302

Declaration SHA-256: d26957a576fdcc692079b92ac5054cbbeda2a914af15d77e6d70ad1d418d20fd

Mathematical statement. On any coordinate set of finite planar measure, the strip density is integrable for every real λ .

No additional measurability assumption on the set is explicitly required by this lemma; boundedness and the measurable density give integrability for the restricted measure.

```

lemma stripDensity_integrableOn_of_volume_ne_top (lam : ℝ)
  {s : Set PlanePoint} (hs : volume s ≠ 0) :
  IntegrableOn (StripDensity lam) s := by
  apply Measure.integrableOn_of_bounded (M := 1 + |lam|) hs
  · exact (measurable_stripDensity lam).aestronglyMeasurable
  · apply Filter.Eventually.of_forall
    intro p
    change |StripDensity lam p| ≤ 1 + |lam|
    unfold StripDensity
    split_ifs
    · simp only [abs_one]
      nlinarith [abs_nonneg lam]
    · nlinarith [abs_nonneg lam]

```

stripDensity_integrable_intervalPushforward

Classification: supplement

Source: CMVGeometry.lean:2243-2259

Declaration SHA-256: 6315c9471ea2031d23e0cb4552b23f2812fe90938358c371a246f04d7f2d8095

Mathematical statement. For every measurable curve and every real λ , a , b , the strip density is integrable against the interval pushforward on $[a, b]$.

A bounded measurable two-valued function is integrable against this finite pushforward measure.

```

lemma stripDensity_integrable_intervalPushforward (lam : ℝ)
  {curve : ℝ → PlanePoint} {a b : ℝ} (hcurve : Measurable curve) :
  Integrable (StripDensity lam) (intervalPushforward curve a b) := by
  rw [intervalPushforward, integrable_map_measure
    (measurable_stripDensity lam).aestronglyMeasurable hcurve.aemeasurable]
  apply Measure.integrableOn_of_bounded (M := 1 + |lam|)

```

```

    measure_Icc_lt_top.ne
  · exact ((measurable_stripDensity lam).comp hcurve).aestronglyMeasurable
  · apply Filter.Eventually.of_forall
    intro x
    change |StripDensity lam (curve x)| ≤ 1 + |lam|
    unfold StripDensity
    split_ifs
    · simp only [abs_one]
      nlinarith [abs_nonneg lam]
    · nlinarith [abs_nonneg lam]

```

tendsto_cap_polygon_length

Classification: supplement

Source: FrontierPerimeter.lean:269-293

Declaration SHA-256: 32d7803e3d290677cdac6cd7bd331241d0a1e99888d3a8665e326de2ca241a11

Mathematical statement. For every real r and every $\theta > 0$, the sequence

$$(n+1)2r\sin\left(\frac{\theta}{n+1}\right)$$

converges to $2r\theta$ as $n \rightarrow \infty$.

This limit uses the continuity of the normalized sinc function at zero and does not require a positive-radius hypothesis.

```

lemma tendsto_cap_polygon_length {r theta : ℝ} (htheta : 0 < theta) :
  Tendsto
    (fun n : ℕ => ((n + 1 : ℕ) : ℝ) *
      (2 * r * Real.sin (theta / ((n + 1 : ℕ) : ℝ))))
    atTop (ℳ (2 * r * theta)) := by
  have hden : Tendsto (fun n : ℕ => ((n + 1 : ℕ) : ℝ)) atTop atTop :=
    (tendsto_natCast_atTop_iff (R := ℝ)).2 (tendsto_add_atTop_nat 1)
  have hquot : Tendsto (fun n : ℕ => theta / ((n + 1 : ℕ) : ℝ)) atTop (ℳ 0) :=
    hden.const_div_atTop theta
  have hsinc : Tendsto (fun n : ℕ => Real.sinc (theta / ((n + 1 : ℕ) : ℝ)))
    atTop (ℳ 1) := by
    convert (Real.continuous_sinc.tendsto 0).comp hquot using 1 <.>
    simp [Function.comp_def]
  have hscaled : Tendsto
    (fun n : ℕ => (2 * r * theta) * Real.sinc (theta / ((n + 1 : ℕ) : ℝ)))
    atTop (ℳ (2 * r * theta)) := by
    simpa using hsinc.const_mul (2 * r * theta)
  apply hscaled.congr'
  filter_upwards [] with n
  have hN : (((n + 1 : ℕ) : ℝ)) ≠ 0 := by positivity
  have hq : theta / (((n + 1 : ℕ) : ℝ)) ≠ 0 := div_ne_zero htheta.ne' hN
  rw [Real.sinc_of_ne_zero hq]
  field_simp

/-- Exact H¹ mass of every injective circular arc with half-angle in `(0, π)`. -/

```

unitCircleArc

Classification: supplement

Source: FrontierPerimeter.lean:11-13

Declaration SHA-256: 6865b0b68c07403ee39252efa35feb9b50850a4644c2bf915fb506723c7a07c7

Mathematical statement. For all real numbers r and s , the complex parametrization is $\gamma_r(s) = \text{rexp}(is/r)$.

This is a total definition on $\mathbb{R}^2$; later geometric theorems impose $r > 0$ or $r \neq 0$ when division or a genuine circle is required.

```
def unitCircleArc (r s : ℝ) : ℂ :=
  (r : ℂ) * Complex.exp (Complex.I * (s / r : ℝ))
```

unitCircleArcRotation

Classification: supplement

Source: FrontierPerimeter.lean:312-314

Declaration SHA-256: cfdc45400ed0d08a69b42a215854d1f81999a500771f48a7f18476aafe48ef

Mathematical statement. For real r, m , rotation by phase m/r is the map $z \mapsto z \exp(im/r)$ on $\mathbb{C}$.

This definition is total in r ; later shift identities assume $r \neq 0$.

```
noncomputable def unitCircleArcRotation (r m : ℝ) (z : ℂ) : ℂ :=
  z * Complex.exp (Complex.I * (m / r : ℝ))
```

unitCircleArcRotation_apply

Classification: supplement

Source: FrontierPerimeter.lean:323-331

Declaration SHA-256: dcedc37c9428c0a722c5bf130023e2389a83c7faa4036432847247c3401f6b9b

Mathematical statement. If $r \neq 0$, then for all real m, s , rotating $\gamma_r(s)$ by phase m/r gives

$$\gamma_r(s + m).$$

The nonzero-radius hypothesis permits the exponential-addition calculation that translates the signed-arclength parameter by m .

```
lemma unitCircleArcRotation_apply {r : ℝ} (hr : r ≠ 0) (m s : ℝ) :
  unitCircleArcRotation r m (unitCircleArc r s) =
    unitCircleArc r (s + m) := by
  unfold unitCircleArcRotation unitCircleArc
  rw [mul_assoc, ← Complex.exp_add]
  congr 2
  push_cast
  field_simp
```

unitCircleArcRotation_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:332-349

Declaration SHA-256: 6bbfe5bd6af2f20e278e2e30ff5adc9fe7c0761359113d169ea5baae1f20d6a2

Mathematical statement. If $r \neq 0$, then for all real m, u, v ,

$$R_{r,m}(\gamma_r([u, v])) = \gamma_r([u + m, v + m]).$$

The rotation identity is applied pointwise and the affine parameter translation carries the closed interval exactly to its translated interval.

```
lemma unitCircleArcRotation_image_Icc {r : ℝ} (hr : r ≠ 0)
  (m u v : ℝ) :
  unitCircleArcRotation r m '' (unitCircleArc r '' Icc u v) =
    unitCircleArc r '' Icc (u + m) (v + m) := by
  ext z
  constructor
  · rintro ⟨_, ⟨s, hs, rfl⟩, rfl⟩
    refine ⟨s + m, ⟨by linarith [hs.1], by linarith [hs.2]⟩, ?_⟩
    exact (unitCircleArcRotation_apply hr m s).symm
```

```

· rintro (t, ht, rfl)
· refine (unitCircleArc r (t - m), ?_, ?_)
· exact (t - m, ⟨by linarith [ht.1], by linarith [ht.2]⟩, rfl)
· rw [unitCircleArcRotation_apply hr]
  congr 1
  ring

```

-- Exact H^1 mass of an arbitrary injective signed-arclength interval on a circle. The strict span bound rules out wrapping through a full revolution. -/

unitCircleArc_dist

Classification: supplement

Source: FrontierPerimeter.lean:55-66

Declaration SHA-256: 1c583965a8663dcd3d0483716b4801199d867b2ff4cbe96f04d4cbeb57ff9c08

Mathematical statement. If $r > 0$, then for all real s, t ,

$$\text{dist}(\gamma_r(s), \gamma_r(t)) = r \left| 2 \sin \left(\frac{s-t}{2r} \right) \right|.$$

This is the exact chord-distance formula for the radius- r circle parametrized by signed arclength.

```

lemma unitCircleArc_dist {r s t : ℝ} (hr : 0 < r) :
  dist (unitCircleArc r s) (unitCircleArc r t) =
    r * |2 * Real.sin (((s - t) / r) / 2)| := by
  rw [Complex.dist_eq, unitCircleArc_sub_factor hr.ne', norm_mul, norm_mul]
  have hexp : ‖Complex.exp (Complex.I * (t / r : ℝ))‖ = 1 := by
    rw [Complex.norm_exp]
  simp
  rw [hexp, mul_one]
  have hnormr : ‖(r : ℂ)‖ = r := by simp [abs_of_pos hr]
  rw [hnormr, Complex.norm_exp_I_mul_ofReal_sub_one]
  simp only [Real.norm_eq_abs]

```

unitCircleArc_eq

Classification: supplement

Source: FrontierPerimeter.lean:409-415

Declaration SHA-256: 9342d6a8ed8b7dcd1fbce36251dddf6b271aeea6ec0298f5d467e178c8fbf9cec

Mathematical statement. For all real r, s ,

$$\gamma_r(s) = r(\cos(s/r) + \sin(s/r)i).$$

This is the Euler-form expansion of the complex exponential used to read off real and imaginary coordinates.

```

lemma unitCircleArc_eq (r s : ℝ) :
  unitCircleArc r s =
    (r : ℂ) * ((Real.cos (s / r) : ℂ) + (Real.sin (s / r) : ℂ) * Complex.I) := by
  unfold unitCircleArc
  rw [show Complex.I * (s / r : ℝ) = ((s / r : ℝ) : ℂ) * Complex.I by ring]
  rw [Complex.exp_ofReal_mul_I]

```

unitCircleArc_im

Classification: supplement

Source: FrontierPerimeter.lean:423-429

Declaration SHA-256: 0666e8d860656cd1bc24ade42af99acdafd9034497a0ba0f27acf44cddcd77e5

Mathematical statement. For all real r, s ,

$$\text{Im}\gamma_r(s) = r\sin(s/r).$$

The identity is obtained by taking the imaginary coordinate of the exponential expansion.

```
lemma unitCircleArc_im (r s : ℝ) :
  (unitCircleArc r s).im = r * Real.sin (s / r) := by
  rw [unitCircleArc_eq]
  simp only [Complex.mul_im, Complex.add_im, Complex.ofReal_re, Complex.ofReal_im,
    Complex.I_re, Complex.I_im]
  ring
```

unitCircleArc_injOn

Classification: supplement

Source: FrontierPerimeter.lean:67-92

Declaration SHA-256: ceabbb1bd00ed7b3d5a248b0d9d8b4d41f8f1fc0af8194bd56a8092deed637e9

Mathematical statement. If $r > 0$ and $0 < \theta < \pi$, the map γ_r is injective on $[-r\theta, r\theta]$.

The angular span is strictly less than one full revolution, so no two distinct parameters in the stated closed interval have the same complex point.

```
lemma unitCircleArc_injOn {r theta : ℝ} (hr : 0 < r)
  (htheta : theta ∈ Ioo 0 Real.pi) :
  Set.InjOn (unitCircleArc r) (Icc (-r * theta) (r * theta)) := by
  intro s hs t ht hst
  have hsdiv : s / r ∈ Icc (-theta) theta := by
    constructor
    · apply (le_div_iff₀ hr).2
      nlinarith [hs.1]
    · apply (div_le_iff₀ hr).2
      nlinarith [hs.2]
  have htdiv : t / r ∈ Icc (-theta) theta := by
    constructor
    · apply (le_div_iff₀ hr).2
      nlinarith [ht.1]
    · apply (div_le_iff₀ hr).2
      nlinarith [ht.2]
  rcases hsdiv with ⟨hsl, hsr⟩
  rcases htdiv with ⟨htl, htr⟩
  have hdist : |s / r - t / r| < 2 * Real.pi := by
    rw [abs_lt]
    constructor <|> nlinarith [htheta.2]
  have hcircle : circleMap 0 r (s / r) = circleMap 0 r (t / r) := by
    simpa [unitCircleArc, circleMap_zero, mul_comm] using hst
  have hang := eq_of_circleMap_eq hr.ne' hdist hcircle
  exact (div_left_inj' hr.ne').mp hang
```

unitCircleArc_re

Classification: supplement

Source: FrontierPerimeter.lean:416-422

Declaration SHA-256: 238aa74976048276575d89eb69760ceb968a224b5358a1775e61818ec60df916

Mathematical statement. For all real r, s ,

$$\text{Re}\gamma_r(s) = r\cos(s/r).$$

The identity is obtained by taking the real coordinate of the exponential expansion.

```
lemma unitCircleArc_re (r s : ℝ) :
  (unitCircleArc r s).re = r * Real.cos (s / r) := by
  rw [unitCircleArc_eq]
  simp only [Complex.mul_re, Complex.add_re, Complex.ofReal_re, Complex.ofReal_im,
```

```
Complex.I_re, Complex.I_im]
ring
```

unitCircleArc_sub_factor

Classification: supplement

Source: FrontierPerimeter.lean:18-32

Declaration SHA-256: d6c59e7f498e1352e21969a1c8084fa6600038a792db1172f4e5bd3b2836eece

Mathematical statement. If $r \neq 0$, then for all real s, t ,

$$\gamma_r(s) - \gamma_r(t) = \text{rexp}(it/r) (\exp(i(s-t)/r) - 1).$$

The factorization is an exact complex identity under the nonzero-radius hypothesis; it isolates the phase at t and the increment $s - t$.

```
lemma unitCircleArc_sub_factor {r s t : ℝ} (_hr : r ≠ 0) :
  unitCircleArc r s - unitCircleArc r t =
    (r : ℂ) * Complex.exp (Complex.I * (t / r : ℝ)) *
      (Complex.exp (Complex.I * ((s - t) / r : ℝ)) - 1) := by
  have he : Complex.exp (Complex.I * (s / r : ℝ)) =
    Complex.exp (Complex.I * (t / r : ℝ)) *
      Complex.exp (Complex.I * ((s - t) / r : ℝ)) := by
    rw [- Complex.exp_add]
    congr 2
    push_cast
    field_simp
    ring
  rw [unitCircleArc, unitCircleArc, he]
  ring
```

upper_cap_integral

Classification: supplement

Source: FrontierPerimeter.lean:1545-1552

Declaration SHA-256: 447beffa9af9c0047e982aa709b3b243852770c1fde70cc3c6676215458bf

Mathematical statement. For every real λ and every cap based at $y = 1$ on the upper side,

$$\int_{J(\text{arcTrace } c)} f_\lambda \circ J^{-1} d\mathcal{H}^1 = \lambda \text{arcLength}(c).$$

The interior of the upper arc lies in the exterior-density region; the two endpoints are Hausdorff-null, so the integral is the exterior density times the exact arc length.

```
theorem upper_cap_integral (lam : ℝ)
  (c : OneSidedCircularCap) (hside : c.side = .upper)
  (hbase : c.baseY = 1) :
  (∫ z in planeEuclideanHomeomorph '' c.arcTrace,
    euclideanStripDensity lam z ∂(μH[1] : Measure EuclideanPlane)) =
    lam * c.arcLength :=
  cap_integral lam c (cap_upper_external_density lam c hside hbase)
```

volume_horizontalLine

Classification: supplement

Source: CMVGeometry.lean:2029-2036

Declaration SHA-256: ea745db5e4b12e1d05fe0934ca3c81eb24b84ea87dd522e3aa6824c86d1a2dce

Mathematical statement. Every horizontal line has zero planar Lebesgue measure.

The statement holds at every real height and removes area contributions from interfaces and joining chords.

```
lemma volume_horizontalline (b : ℝ) :
  volume {p : PlanePoint | p.2 = b} = 0 := by
  have hset : {p : PlanePoint | p.2 = b} = Set.univ ×ˢ ({b} : Set ℝ) := by
    ext p
    simp
  rw [hset, Measure.volume_eq_prod, Measure.prod_prod]
  simp
```

volume_lower_circular_segment

Classification: supplement

Source: CMVGeometry.lean:1945-1995

Declaration SHA-256: 2301e25d7ced8d1d9e2c23a9b63e02ba7c6e8f9ea615112d4c41d15aeb777045

Mathematical statement. For $r > 0$ and $0 \leq \theta \leq \pi$, the disk centered at (m, c) , restricted to $y \leq c - r \cos \theta$, has the same circular-segment area.

Reflection changes the upper slice condition into the lower one and preserves planar volume.

```
lemma volume_lower_circular_segment (mx cy r θ : ℝ) (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  volume {p : ℝ × ℝ |
    (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
    p.2 ≤ cy - r * cos θ} =
  ENNReal.ofReal (r ^ 2 * (θ - sin θ * cos θ)) := by
  let U : Set (ℝ × ℝ) :=
    {p | (p.1 - mx) ^ 2 + (p.2 - (-cy)) ^ 2 ≤ r ^ 2 ∧
    -cy + r * cos θ ≤ p.2}
  have hU : MeasurableSet U := by
    have hcircle : Measurable (fun p : ℝ × ℝ ↦
      (p.1 - mx) ^ 2 + (p.2 - (-cy)) ^ 2) := by fun_prop
    have hside : Measurable (fun p : ℝ × ℝ ↦ p.2) := measurable_snd
    exact (measurableSet_le hcircle measurable_const).inter
      (measurableSet_le measurable_const hside)
  let F : ℝ × ℝ → ℝ × ℝ := Prod.map id (fun y : ℝ ↦ -y)
  have hF :
    MeasurePreserving F (volume : Measure (ℝ × ℝ)) volume := by
    change MeasurePreserving (Prod.map id (fun y : ℝ ↦ -y))
      ((volume : Measure ℝ).prod volume) ((volume : Measure ℝ).prod volume)
    exact (MeasurePreserving.id (volume : Measure ℝ)).prod
      (Measure.measurePreserving_neg (volume : Measure ℝ))
  have hpre :
    F ⁻¹' U =
    {p : ℝ × ℝ |
      (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
      p.2 ≤ cy - r * cos θ} := by
    ext p
    change ((p.1 - mx) ^ 2 + (-p.2 - -cy) ^ 2 ≤ r ^ 2 ∧
    -cy + r * cos θ ≤ -p.2) ↔
      ((p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
      p.2 ≤ cy - r * cos θ)
  constructor <=> rintro (hcircle, hside)
  · constructor
    · nlinarith
    · linarith
  · constructor
    · nlinarith
    · linarith
  calc
    volume {p : ℝ × ℝ |
      (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
      p.2 ≤ cy - r * cos θ} =
      volume (F ⁻¹' U) := by rw [hpre]
    _ = volume U := hF.measure_preimage hU.nullMeasurableSet
    _ = ENNReal.ofReal (r ^ 2 * (θ - sin θ * cos θ)) :=
```

```
volume_upper_circular_segment mx (-cy) r θ hr h0 hπ
```

```
namespace OneSidedCircularCap
```

volume_upper_circular_segment

Classification: supplement

Source: CMVGeometry.lean:1820-1944

Declaration SHA-256: d23f6efcc432aaa58959c64bfa3f6f589362c33546093657b1717ec5dbfb1b34

Mathematical statement. For $r > 0$ and $0 \leq \theta \leq \pi$, the disk centered at (m, c) , restricted to $y \geq c + r \cos \theta$, has area $r^2(\theta - \sin \theta \cos \theta)$.

Integrating horizontal slices derives the area of this actual upper circular segment for arbitrary center coordinates.

```
lemma volume_upper_circular_segment (mx cy r θ : ℝ) (hr : 0 < r)
  (h0 : 0 ≤ θ) (hπ : θ ≤ π) :
  volume {p : ℝ × ℝ |
    (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
    cy + r * cos θ ≤ p.2} =
    ENNReal.ofReal (r ^ 2 * (θ - sin θ * cos θ)) := by
let S : Set (ℝ × ℝ) :=
  {p | p.1 ∈ Icc (cy + r * cos θ) (cy + r) ∧
    p.2 ∈ Icc
      (mx - √(r ^ 2 - (p.1 - cy) ^ 2))
      (mx + √(r ^ 2 - (p.1 - cy) ^ 2))}
have hS : MeasurableSet S := by
  exact measurableSet_region_between_cc
  (f := fun y : ℝ ↦ mx - √(r ^ 2 - (y - cy) ^ 2))
  (g := fun y : ℝ ↦ mx + √(r ^ 2 - (y - cy) ^ 2))
  (s := Icc (cy + r * cos θ) (cy + r))
  (by fun_prop) (by fun_prop) measurableSet_Icc
have hpre :
  Prod.swap ⁻¹' S =
  {p : ℝ × ℝ |
    (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
    cy + r * cos θ ≤ p.2} := by
ext p
constructor
· intro hp
  rcases hp with ⟨hy, hx⟩
  change p.2 ∈ Icc (cy + r * cos θ) (cy + r) at hy
  change p.1 ∈ Icc
    (mx - √(r ^ 2 - (p.2 - cy) ^ 2))
    (mx + √(r ^ 2 - (p.2 - cy) ^ 2)) at hx
  have hcosmul : -r ≤ r * cos θ := by
    simp using mul_le_mul_of_nonneg_left (neg_one_le_cos θ) hr.le
  have hymin : cy - r ≤ p.2 := by
    linarith [hy.1]
  have hprod :
    0 ≤ (r - (p.2 - cy)) * (r + (p.2 - cy)) :=
    mul_nonneg (by linarith [hy.2]) (by linarith)
  have hD : 0 ≤ r ^ 2 - (p.2 - cy) ^ 2 := by
    nlinarith
  have hsqrt :
    (√(r ^ 2 - (p.2 - cy) ^ 2)) ^ 2 =
    r ^ 2 - (p.2 - cy) ^ 2 :=
    Real.sq_sqrt hD
  have hxl :
    -(√(r ^ 2 - (p.2 - cy) ^ 2)) ≤ p.1 - mx := by
    linarith [hx.1]
  have hxu :
    p.1 - mx ≤ √(r ^ 2 - (p.2 - cy) ^ 2) := by
    linarith [hx.2]
  have hxprod :
    0 ≤ (√(r ^ 2 - (p.2 - cy) ^ 2) - (p.1 - mx)) *
    (√(r ^ 2 - (p.2 - cy) ^ 2) + (p.1 - mx)) :=
```

```

mul_nonneg (by linarith [h xu]) (by linarith [h x1])
constructor
· nlinarith
· exact hy.1
· rintro ⟨hcircle, hbase⟩
have hyUpper : p.2 ≤ cy + r := by
  nlinarith [sq_nonneg (p.1 - mx), sq_nonneg (p.2 - cy + r)]
have hD : 0 ≤ r ^ 2 - (p.2 - cy) ^ 2 := by
  nlinarith [sq_nonneg (p.1 - mx)]
have hsqrt :
  (√(r ^ 2 - (p.2 - cy) ^ 2)) ^ 2 =
  r ^ 2 - (p.2 - cy) ^ 2 :=
  Real.sq_sqrt hD
have hsqrt0 : 0 ≤ √(r ^ 2 - (p.2 - cy) ^ 2) :=
  Real.sqrt_nonneg _
have hdist :
  (p.1 - mx) ^ 2 ≤ (√(r ^ 2 - (p.2 - cy) ^ 2)) ^ 2 := by
  nlinarith
change p.2 ∈ Icc (cy + r * cos θ) (cy + r) ^
p.1 ∈ Icc
  (mx - √(r ^ 2 - (p.2 - cy) ^ 2))
  (mx + √(r ^ 2 - (p.2 - cy) ^ 2))
constructor
· exact ⟨hbase, hyUpper⟩
· constructor <|> nlinarith
have hsec (y : ℝ) :
  volume (Prod.mk y ^ -1' S) =
  (Icc (cy + r * cos θ) (cy + r)).indicator
  (fun y ↦ ENNReal.ofReal
    (2 * √(r ^ 2 - (y - cy) ^ 2))) y := by
by_cases hy : y ∈ Icc (cy + r * cos θ) (cy + r)
· simp only [Set.indicator_of_mem hy]
  have heq :
    Prod.mk y ^ -1' S =
    Icc (mx - √(r ^ 2 - (y - cy) ^ 2))
    (mx + √(r ^ 2 - (y - cy) ^ 2)) := by
  ext x
  change (y ∈ Icc (cy + r * cos θ) (cy + r) ^
    x ∈ Icc (mx - √(r ^ 2 - (y - cy) ^ 2))
    (mx + √(r ^ 2 - (y - cy) ^ 2))) ↔
  x ∈ Icc (mx - √(r ^ 2 - (y - cy) ^ 2))
  (mx + √(r ^ 2 - (y - cy) ^ 2))
  simp only [hy, true_and]
  rw [heq, Real.volume_Icc]
  congr 1
  ring
· simp only [Set.indicator_of_notMem hy]
  have heq : Prod.mk y ^ -1' S = ∅ := by
  ext x
  change (y ∈ Icc (cy + r * cos θ) (cy + r) ^
    x ∈ Icc (mx - √(r ^ 2 - (y - cy) ^ 2))
    (mx + √(r ^ 2 - (y - cy) ^ 2))) ↔ False
  constructor
  · intro h
    exact (hy h.1).elim
  · intro h
    exact h.elim
  rw [heq, measure_empty]
calc
volume {p : ℝ × ℝ |
  (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ^
  cy + r * cos θ ≤ p.2} =
  volume (Prod.swap ^ -1' S) := by rw [hpre]
_ = volume S := by
  exact Measure.measurePreserving_swap.measure_preimage hS.nullMeasurableSet
_ = ∫ y in Icc (cy + r * cos θ) (cy + r),
  ENNReal.ofReal (2 * √(r ^ 2 - (y - cy) ^ 2)) := by
  rw [Measure.volume_eq_prod, Measure.prod_apply hS]
  simp_rw [hsec]
  rw [lintegral_indicator measurableSet_Icc]

```

```
_ = ENNReal.ofReal (r ^ 2 * (θ - sin θ * cos θ)) :=  
  lintegral_semicircle_translated hr h00 h0π
```

volume_verticalLine

Classification: supplement

Source: CMVGeometry.lean:2037-2046

Declaration SHA-256: b32e9d96eca7ded1e8b5354b6d117e61bcda7b34c1fe056587cd14e1871da61b

Mathematical statement. Every vertical line has zero planar Lebesgue measure.

This permits addition of rectangle and side-piece areas despite their shared vertical boundaries.

```
lemma volume_verticalLine (a : ℝ) :  
  volume {p : PlanePoint | p.1 = a} = 0 := by  
  have hset : {p : PlanePoint | p.1 = a} = ({a} : Set ℝ) ×ˢ Set.univ := by  
    ext p  
    simp  
  rw [hset, Measure.volume_eq_prod, Measure.prod_prod]  
  simp  
  
namespace StripCore
```

Generated certificate families

External references

- **CMV2010** — Antonio Cañete, Michele Miranda Jr., and Davide Vittone, *Some Isoperimetric Problems in Planes with Density*, §3.2, Lemma 3.8, Conjecture 3.12, and equations (26)–(27). <https://arxiv.org/abs/0906.1256>
- **Lean4CADE2021** — Leonardo de Moura and Sebastian Ullrich, *The Lean 4 Theorem Prover and Programming Language*, CADE 28 (2021), pp. 625–635. https://doi.org/10.1007/978-3-030-79876-5_37
- **mathlib2020** — The mathlib Community, *The Lean Mathematical Library*, CPP 2020, pp. 367–381. <https://doi.org/10.1145/3372885.3373824>